

ANÁLISE E PROJETO DE SISTEMAS DE INFORMAÇÃO ORIENTADOS A OBJETOS



Preencha a **ficha de cadastro** no final deste livro e receba gratuitamente informações sobre os lançamentos e as promoções da Elsevier.

Consulte também nosso catálogo completo, últimos lançamentos e serviços exclusivos no site **www.elsevier.com.br**

Raul Sidnei Wazlawick

ANÁLISE E PROJETO DE SISTEMAS DE INFORMAÇÃO ORIENTADOS A OBJETOS

2ª edição revista e atualizada



© 2011, Elsevier Editora Ltda.

Todos os direitos reservados e protegidos pela Lei nº 9.610, de 19/02/1998.

Nenhuma parte deste livro, sem autorização prévia por escrito da editora, poderá ser reproduzida ou transmitida sejam quais forem os meios empregados: eletrônicos, mecânicos, fotográficos, gravação ou quaisquer outros.

Copidesque: Ivone Teixeira

Revisão: Bruno Barrio

Editoração Eletrônica: SBNIGRI Artes e Textos Ltda.

Elsevier Editora Ltda.

Conhecimento sem Fronteiras

Rua Sete de Setembro, 111 – 16ª andar

20050-006 – Centro – Rio de Janeiro – RJ – Brasil

Rua Quintana, 753 – 8ª andar

04569-011 – Brooklin – São Paulo – SP – Brasil

Serviço de Atendimento ao Cliente

0800-0265340

sac@elsevier.com.br

ISBN 978-85-352-3916-4

Nota: Muito zelo e técnica foram empregados na edição desta obra. No entanto, podem ocorrer erros de digitação, impressão ou dúvida conceitual. Em qualquer das hipóteses, solicitamos a comunicação ao nosso Serviço de Atendimento ao Cliente, para que possamos esclarecer ou encaminhar a questão.

Nem a editora nem o autor assumem qualquer responsabilidade por eventuais danos ou perdas a pessoas ou bens, originados do uso desta publicação.

CIP-Brasil. Catalogação-na-fonte.
Sindicato Nacional dos Editores de Livros, RJ

W372a Wazlawick, Raul Sidnei

2.ed. Análise e projeto de sistemas de informação orientados a objetos
/ Raul Sidnei Wazlawick. – 2.ed. – Rio de Janeiro: Elsevier, 2011.
(Série SBC, Sociedade Brasileira de Computação)

Apêndice: Sumário OCL

Inclui bibliografia e índice

ISBN 978-85-352-3916-4

1. Métodos orientados a objetos (Computação). 2. UML (Computação). 3. Análise de sistemas. 4. Projeto de sistemas. 5. Software – Desenvolvimento. I. Sociedade Brasileira de Computação. II. Título. III. Série.

10-2632.

CDD: 005.117

CDU: 004.414.2

Dedicatória

Este livro é dedicado aos meus pais e antepassados,
sem os quais eu não existiria.

Agradecimentos

Desejo agradecer a várias pessoas que, de uma forma ou outra, tornaram este livro possível: ao mestre Luiz Fernando Bier Melgarejo, por apresentar as ideias de orientação a objetos já em 1987; ao colega Marcos Eduardo Casa, por todos os trabalhos desenvolvidos em conjunto nos tempos em que orientação a objetos era “coisa de outro mundo”; ao colega Antônio Carlos Mariani, pelo Mundo dos Atores, ferramenta que tanto usamos para ensinar programação orientada a objetos; ao ex-aluno Leonardo Ataíde Minora, por inicialmente me chamar a atenção para o livro de Larman; às empresas e órgãos públicos que possibilitaram a implantação dessas técnicas em ambientes reais de produção de software e especialmente ao engenheiro de software Gilmar Purim, pelas interessantes discussões que muito contribuíram para dar a forma final a este livro; aos ex-alunos Everton Luiz Vieira e Kuesley Fernandes do Nascimento, por terem ajudado a consolidar algumas das técnicas quando da aplicação delas a um interessante sistema Web; ao Departamento de Informática e Estatística da UFSC, pela oportunidade de concretizar este trabalho; e a Dayane Montagna, por digitar o primeiro rascunho deste livro a partir das gravações das minhas aulas.

Agradeço também aos mais de mil ex-alunos, vítimas da minha disciplina de Análise e Projeto de Sistemas Orientados a Objetos – suas dúvidas e dificuldades me fizeram pesquisar e aprender muito mais; ao colega Rogério Cid Bastos, por muitas orientações recebidas; e, finalmente, aos amigos e irmãos, pelos momentos de descontração e higiene mental.

Prefácio

Este livro apresenta, de maneira didática e aprofundada, elementos de análise e projeto de sistemas de informação orientados a objetos.

A área de desenvolvimento de software tem se organizado nos últimos anos em torno da linguagem de modelagem UML (*Unified Modeling Language*) e do processo UP (*Unified Process*), transformados em padrão internacional pela OMG (*Object Management Group*).

Não se procura aqui realizar um trabalho enciclopédico sobre UP ou UML, mas uma apresentação de cunho estritamente prático, baseada em mais de vinte anos de experiência em ensino, prática e consultoria em análise, projeto e programação orientada a objetos.

Este livro diferencia-se da maioria de outros livros da área por apresentar em detalhes as técnicas de construção de contratos de operação e consulta de sistema de forma que esses contratos possam ser usados para efetiva geração de código.

Novos padrões e técnicas de modelagem conceitual são detalhadamente apresentados, técnicas estas também adequadas para uso com contratos e diagramas de comunicação, de forma a garantir geração automática de código; não apenas de esqueletos, mas de código final executável.

Em relação aos casos de uso de análise, o livro apresenta, em detalhes, técnicas para ajudar a decidir o que considerar efetivamente como caso de uso. Essa dificuldade tem sido sistematicamente relatada por analistas de várias partes do Brasil, a partir do contato obtido em cursos ministrados pelo autor.

Ao contrário de outros livros da área, que se organizam em torno da apresentação dos diagramas UML e procuram explicar todos os seus possí-

veis usos, este livro se concentra nas *atividades* com as quais o analista e o projetista de software possivelmente vão se deparar e sugere quais diagramas poderiam ajudá-los e de que forma.

Algumas empresas brasileiras ainda têm dificuldade em conseguir exportar software devido à falta de flexibilidade e manutenibilidade dos sistemas gerados. Este livro apresenta um conjunto de informações e técnicas que pode suprir essa carência. As técnicas em questão foram implementadas com êxito pelo autor na empresa TEClógica Ltda., em Blumenau, no desenvolvimento de um projeto de grande porte em 2004. Posteriormente, as técnicas foram aplicadas e aperfeiçoadas nos departamentos de tecnologia de informação do Ministério Público de Santa Catarina, Tribunal Regional do Trabalho do Mato Grosso e Justiça Federal de Santa Catarina, contendo agora ainda mais orientações e detalhes do que na primeira edição deste livro.

O livro é direcionado a profissionais de computação (analistas, projetistas e programadores) e a estudantes de graduação e pós-graduação das disciplinas de Análise e Projeto de Sistemas e Engenharia de Software. Como conhecimentos prévios são recomendados rudimentos sobre orientação a objetos, notação UML e fundamentos de banco de dados.

Para que o livro pudesse aprofundar ainda mais as informações sobre análise e projeto orientados a objetos sem se tornar demasiadamente longo, foram suprimidas nesta segunda edição algumas informações referentes ao *processo* de Engenharia de Software que estavam presentes na primeira edição. Esses processos serão descritos de forma detalhada pelo autor em um novo livro sobre Engenharia de Software a ser lançado brevemente. Além disso, para ganhar espaço e dinamismo, os exercícios, anteriormente incluídos no livro, passam a estar disponíveis apenas na Internet (www.elsevier.com.br/wazlawick ou www.inf.ufsc.br/~raul/).

Raul Sidnei Wazlawick

Florianópolis, 19 de fevereiro de 2010.

Introdução

O principal objetivo deste livro é apresentar um conjunto de informações práticas que possibilite aos desenvolvedores de software a compreensão e utilização da *orientação a objetos* de forma consciente e eficaz.

A justificativa deste trabalho parte da observação de que há uma vasta literatura que visa apenas a apresentar os diagramas da UML (OMG, 2009) de forma sintática (por exemplo, Erickson & Penker, 1998), mas poucos livros que ofereçam informações suficientes para viabilizar a aplicação eficaz da orientação a objetos no desenvolvimento de software no mundo real.

Neste livro, são feitas uma interpretação e um detalhamento de partes do método de análise e projeto apresentado por Larman (2002), o qual é baseado no *Processo Unificado* ou *UP* (Jacobson, Booch & Rumbaugh, 1999; Scott, 2001).

A motivação para o uso do método de Larman como base para este trabalho deve-se ao fato de que Larman apresenta uma abordagem concisa e eficiente para análise e projeto de sistemas orientados a objetos. Nessa abordagem, cada artefato (documento ou diagrama) tem uma razão muito clara para existir, e as conexões entre os diferentes artefatos são muito precisas.

Pode-se até dizer que o método seria inspirado em *Extreme Programming* ou XP (Beck, 2004) no qual, em vez de usar uma linguagem de progra-

mação (como Java ou PHP), utilizam-se diagramas e outros artefatos. Dentro dessa proposta, diagramas e artefatos só fazem sentido se contribuem diretamente para a geração automática de código. Não são usados, portanto, como mera documentação, mas como programação em nível muito alto.

Em relação ao processo descrito por Larman, este livro aprofunda e apresenta conceitos originais em vários tópicos, como, por exemplo, o uso de *Object Constraint Language* ou OCL (OMG, 2006) para construção de contratos de operação de sistema, a discussão sobre quais passos são realmente obrigatórios em casos de uso expandidos, a noção de contratos de consultas de sistema, a interconexão entre os contratos e os diagramas de comunicação ou sequência e o projeto da camada de interface com o uso de WebML (Ceri *et al.*, 2003).

Desde o início, o uso dessas práticas vai levando sistematicamente à produção de software de boa qualidade, isto é, bem organizado, baseado em uma arquitetura multicamadas e com possibilidade de incluir novos requisitos e modificar os requisitos existentes.

1.1. Desenvolvimento de Sistemas Orientados a Objetos

Em primeiro lugar, deve-se discutir o que é realmente desenvolver sistemas orientados a objetos. Ao observar a forma como a análise e o projeto de sistemas vêm sendo ensinados e praticados em certos lugares, pode-se verificar que muitos profissionais simplesmente adotam uma linguagem orientada a objetos ou até algum fragmento de processo orientado a objetos, mas sem ter realmente muita noção do que estão fazendo.

O problema de fazer os profissionais migrarem de paradigmas mais antigos para orientação a objetos apresenta situações caricatas. Em determinada ocasião, durante uma palestra, alguém comentou que programava há muitos anos usando a linguagem C e que havia resolvido começar a trabalhar com C++, mas que após alguns meses não notou absolutamente nenhuma vantagem nessa migração. Essa pessoa realmente não viu diferença entre as linguagens porque faltou a ela saber o que havia por trás da nova abordagem, e que a linguagem C++ é mais interessante do que a linguagem C não porque tem mais recursos ou eficiência, mas porque traz consigo uma maneira muito mais sensata de se pensar e organizar sistemas.

O seguinte ditado tem relação com essa situação: “Comprar um martelo não transforma você em um arquiteto; pode ser necessário, mas não suficiente”.

Também não é suficiente organizar o sistema em classes e hierarquias se o código implementado nos métodos é desorganizado. Alguns programadores organizam o sistema adequadamente em classes e pacotes, mas fazem o código dos métodos tão desorganizado como uma macarronada. Outros ainda aplicam técnicas de decomposição *top-down* que não são apropriadas quando se trata de desenvolvimento orientado a objetos (para isso existe a programação estruturada).

Para a correta construção de código orientado a objetos deve-se conhecer as técnicas de delegação e distribuição de responsabilidades, que levam a código reusável e baixo acoplamento, de acordo com padrões de projeto. Essas técnicas são explicadas ao longo deste livro.

De nada adianta realizar pesados investimentos em ferramentas CASE orientadas a objetos sem que se compreenda a forma de *pensar* orientada a objetos.

O uso de diagramas não vai melhorar necessariamente a qualidade do software produzido.

Para que um profissional possa chegar a ser um arquiteto de software, existe uma série de conhecimentos que precisam ser compreendidos, e espera-se que este livro possa dar uma boa contribuição para que esses tópicos sejam abordados com profundidade em todas as fases do processo de desenvolvimento de software.

1.2. Linguagem de Modelagem Unificada — UML

Algumas pessoas menos informadas acreditam que a UML é uma metodologia, talvez por causa do “M” na sigla. Mas não é. A letra mais importante nessa sigla é o L, de linguagem. UML quer dizer *Unified Modeling Language* (Linguagem de Modelagem Unificada) e é, portanto, uma *linguagem* que pode ser usada para descrever coisas.

Porém, conhecer uma linguagem não implica a habilidade de saber usá-la para produzir artefatos úteis. Por exemplo, a língua portuguesa é uma linguagem, mas uma pessoa que sabe escrever em português não sabe necessariamente fazer bons discursos ou boa poesia. Existem, por trás da linguagem, técnicas e conhecimento de melhores práticas, que auxiliam os grandes

oradores e poetas a colocar os elementos da linguagem na ordem e estrutura adequadas para produzir um efeito esperado.

A UML foi sendo gradativamente definida a partir de 1994 quando James Rumbaugh e Grady Booch criaram a empresa Rational e unificaram suas já conhecidas linguagens de diagramas. Um ano depois, Ivar Jacobson entrou na parceria e adicionou seus casos de uso e outras notações ao sistema de diagramas que vinha sendo definido.

A UML vem sendo constantemente revisada e, correntemente, tem três famílias de diagramas:

- a) *Diagramas estruturais*, compreendendo os diagramas de pacotes, classes, objetos, estrutura composta, componentes e distribuição.
- b) *Diagramas comportamentais*, compreendendo os diagramas de casos de uso, atividades e máquina de estados.
- c) *Diagramas de interação*, compreendendo os diagramas de comunicação, sequência, tempo e visão geral de integração.

Nem todos os diagramas precisam ser usados durante o desenvolvimento de um sistema. Usam-se apenas aqueles que possam apresentar alguma informação útil para o processo. Neste livro é enfatizado o uso dos diagramas de atividades, estados, caso de uso, sequência, classes e comunicação, para modelar sistemas de informação. Em alguns momentos, porém, outros diagramas poderão ser necessários, conforme as características do sistema. Para mais informações sobre os diferentes diagramas da UML em português, pode-se consultar o livro de Pereira e Silva (2007).

1.3. Processo Unificado — UP

As técnicas apresentadas neste livro são adequadas para uso com o processo unificado que, da forma como foi proposto, está fortemente associado, embora não exclusivamente, com a notação UML.

O UP também foi proposto pelos três gurus da orientação a objetos: Grady Booch, James Rumbaugh e Ivar Jacobson (Jacobson, Booch & Rumbaugh, 1999), sendo o resultado de mais de trinta anos de experiência acumulada.

O processo se fundamenta em três valores:

- a) *é dirigido por casos de uso*: o planejamento do desenvolvimento é feito em função dos casos de uso identificados, tratando-se prioritariamente os mais complexos;

- b) *é centrado na arquitetura*: o processo de desenvolvimento prioriza a construção de uma arquitetura de sistema que permita a realização dos requisitos. Essa arquitetura baseia-se na identificação de uma estrutura de classes, produzida a partir de um modelo conceitual;
- c) *é iterativo e incremental*: a cada ciclo de trabalho realizado, novas características são adicionadas à arquitetura do sistema, deixando-a mais completa e mais próxima do sistema final.

O UP comporta, em suas disciplinas as atividades de estudo de viabilidade, análise de requisitos, análise de domínio, projeto etc. Porém, essas atividades aparecem no UP associadas, com maior ou menor ênfase, às quatro grandes fases do UP, que são: concepção, elaboração, construção e transição.

A fase de *concepção* incorpora o estudo de viabilidade, o levantamento dos requisitos e uma parte da sua análise. A fase de *elaboração* incorpora o detalhamento da análise de requisitos, a modelagem de domínio e o projeto. A fase de *construção* corresponde à programação e testes, e a fase de *transição* consiste na instalação do sistema e migração de dados.

A fase de concepção, denominada *inception* em inglês, é a primeira fase do processo unificado, na qual se procura levantar os principais requisitos e compreender o sistema de forma abrangente. Os resultados dessa fase usualmente são um documento de requisitos e riscos, uma listagem de casos de uso de alto nível e um cronograma de desenvolvimento baseado nesses casos de uso.

As fases de elaboração e construção ocorrem em ciclos iterativos. A elaboração incorpora a maior parte da análise e projeto, e a construção incorpora a maior parte da implementação e testes. É durante os ciclos iterativos propriamente ditos que acontece a análise detalhada do sistema, a modelagem de domínio e o projeto do sistema usando os *padrões de projeto*.

Na fase de transição, o sistema, depois de pronto, será implantado substituindo o sistema atual, seja ele manual ou computadorizado.

Apesar de ser um processo prescritivo, o UP pode também ser realizado como um processo ágil, com poucos artefatos e burocracia, permitindo o desenvolvimento de software de forma eficiente, uma vez que o que interessa ao cliente é o software pronto e não uma pilha de documentos justificando por que não ficou pronto.

Para que isso seja obtido, a documentação deve ser dirigida à produção do software. Cada atividade realizada pelo desenvolvedor deve ter um obje-

tivo muito claro e uma utilização precisa visando sempre à produção de um código que atenda aos requisitos da melhor forma possível no menor tempo.

1.4. As Atividades de Análise e Projeto no Contexto do Processo Unificado

As diferentes atividades de análise e projeto não ocorrem de forma estanque em cada uma das fases do processo unificado, mas podem ocorrer com maior ou menor ênfase nas diferentes fases. A Figura 1.1 é a representação clássica da distribuição das atividades de desenvolvimento de sistemas e sua ênfase nas diferentes fases da implementação mais conhecida do UP, denominada RUP, ou *Rational Unified Process* (Kruchten, 2003).

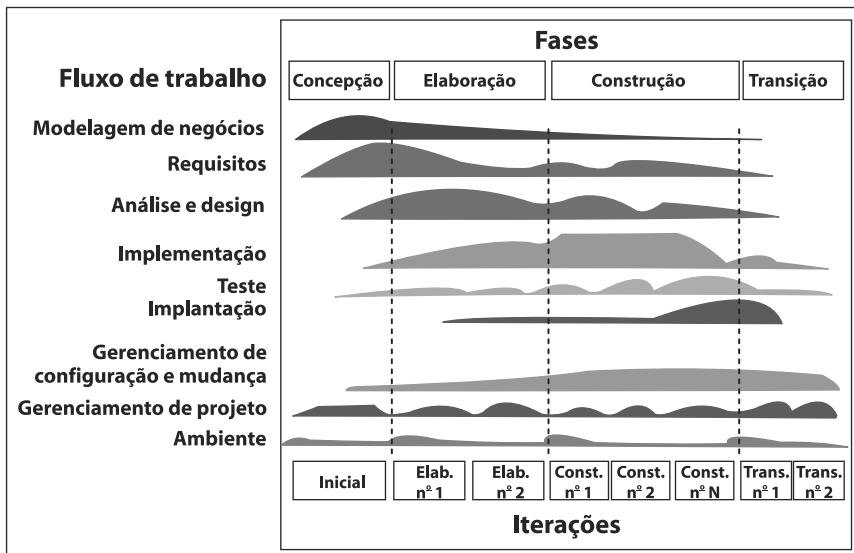


Figura 1.1: As diferentes ênfases das atividades de desenvolvimento ao longo das quatro fases do Processo Unificado (fonte: IBM).

Este livro vai abordar com maior ênfase as atividades típicas de *análise* e *projeto*. Para uma visão maior sobre gerenciamento de projeto e processo deve-se consultar um livro de engenharia de software, como, por exemplo, o de Pressman (2010). Já as atividades de programação são orientadas de acordo com a linguagem escolhida.

Então, com o foco nas atividades de análise e projeto, a fase de concepção vai exigir do analista uma visão inicial e geral do sistema a ser desenvolvido (Capítulo 2). Essa visão pode ser obtida a partir de entrevistas, documentos e sistemas. Para apoiar a modelagem dessa visão geral pode-se usar

diagramas de máquina de estados ou diagramas de atividades da UML, que correspondem, nessa fase, à modelagem de negócios.

A partir dessa compreensão do negócio pode-se analisar mais aprofundadamente cada uma das atividades ou estados para obter os requisitos funcionais e não funcionais do sistema (Capítulo 3).

Ainda na fase de concepção pode-se elaborar com o diagrama de classes um modelo conceitual preliminar (Capítulo 7) para compreensão da estrutura da informação a ser gerenciada pelo sistema.

Esse modelo conceitual preliminar e os requisitos já levantados ajudarão a compreender quais são os processos de negócio e processos complementares da empresa, obtendo-se assim os casos de uso de alto nível (Capítulo 4). Esses casos de uso deverão ser usados como base para planejar o restante do desenvolvimento.

A fase de elaboração inicia com a expansão dos casos de uso de alto nível (Capítulo 5) e posterior representação de seus fluxos através dos diagramas de sequência de sistema (Capítulo 6), quando são descobertas as operações e consultas de sistema.

Na fase de elaboração, o modelo conceitual poderá ser refinado, e mais informações agregadas a ele a partir das descobertas feitas durante a expansão dos casos de uso.

Ainda nessa fase poderão ser feitos os contratos de operação e consulta de sistema (Capítulo 8) que definem a funcionalidade, ou seja, os resultados de cada operação e consulta realizadas.

Posteriormente, com os contratos e modelo conceitual à mão, podem ser criados os diagramas de comunicação ou sequência (Capítulo 9), que, seguindo critérios de delegação e distribuição de responsabilidades, vão mostrar quais métodos devem ser criados em cada classe e como esses métodos devem ser implementados, produzindo assim o diagrama de classes de projeto, ou DCP.

Ainda na fase de elaboração, pode-se passar ao projeto da interface (Capítulo 10). Como grande parte dos sistemas de informação são desenvolvidos para Web ou interfaces compatíveis com modelos Web, este livro apresenta o WebML (Ceri *et al.*, 2003) como opção para a modelagem para esse aspecto do sistema.

A fase de construção inclui a geração de bancos de dados (Capítulo 11) e a geração de código e testes (Capítulo 12). A persistência dos dados usual-

mente não precisa ser modelada, pois pode ser gerada automaticamente. Assim mesmo, o livro mostra como ela ocorre quando se usa um *framework* orientado a objetos para persistência. Também a geração de código é apresentada como um conjunto de regras que podem ser automatizadas. As atividades de teste de software são adaptadas neste livro para as características peculiares da orientação a objetos.

Visão Geral do Sistema

A *fase de concepção* do UP consiste em uma etapa em que o analista vai buscar as primeiras informações sobre o sistema a ser desenvolvido. Nessa etapa, assume-se que o analista possui pouco conhecimento sobre o sistema e que a interação com o usuário e cliente será grande. O objetivo dessa fase é descobrir se vale a pena fazer a análise, mas sem fazer a análise propriamente dita.

Os artefatos dessa fase não precisam ser ainda perfeitamente estruturados, ou seja, eles não são necessariamente completos e organizados. A cada reunião realizada com o usuário ou cliente, um registro (uma ata simplificada) deve ser produzido, o qual possivelmente apresentará várias ideias sobre o sistema sem ter necessariamente uma organização sistemática.

O levantamento da *visão geral do sistema* deve durar poucos dias. Nesse período, deve-se levantar todas as informações possíveis sobre o negócio da empresa, a partir de entrevistas com os usuários e clientes, bem como através de exame de documentos, relatórios, sistemas e bibliografia.

A fase de concepção inclui o primeiro contato do analista com o cliente, no qual o analista vai descobrir o que o cliente quer. Essa fase tem de ser rápida e deve produzir um relatório sucinto do sistema a ser desenvolvido.

A maioria dos projetos exige que o analista responda primeiro qual é a *visão da empresa* para o projeto, ou seja, o que a empresa quer com o projeto, por que ele está sendo proposto e por que a empresa vai gastar dinheiro com ele.

Essas são as questões que devem ser trabalhadas no primeiro momento. Nessa fase, também surge outra questão que os analistas frequentemente esquecem: comprar ou construir? Muitas vezes, o produto que o cliente quer já está pronto, podendo-se comprar um pacote e adaptá-lo para as necessidades da empresa, em vez de construir a partir do zero.

Essas questões devem ser respondidas em um tempo relativamente curto. Por isso, sugere-se que a fase de concepção não dure muito tempo. Por que motivo? Porque, nessa fase, normalmente, o analista e o cliente ainda não têm um contrato fechado; ela é, do ponto de vista do analista, um investimento no futuro.

A *visão geral do sistema*, ou *sumário executivo*, é um documento em formato livre, no qual o analista deve escrever o que ele conseguiu descobrir de relevante sobre o sistema após as conversas iniciais com os clientes e usuários.

Aqui não são propostas regras sobre como deve ser escrito esse documento. Mas sugere-se que ele não seja longo demais. Uma ou duas páginas de texto e alguns diagramas parece ser suficiente para descrever, de forma resumida, a maioria dos sistemas. Com mais do que isso, possivelmente estarão sendo incluídos detalhes que não são relevantes nesse resumo e que deveriam ser tratados em outros documentos, como análise de riscos, requisitos ou casos de uso.

Na Figura 2.1 é apresentada a visão geral de um sistema de livreria virtual fictício, que será usado como exemplo para as técnicas de modelagem ao longo do livro.

Sistema Livir: Livreria Virtual

Visão Geral do Sistema

O sistema deve gerenciar todos os processos de uma livreria virtual, desde a aquisição até a venda dos livros. O acesso dos compradores e gerentes deve ser feito através de um site Web e possivelmente com outras tecnologias. Os compradores fazem as transações pagando com cartão de crédito.

Existem promoções eventuais pelas quais os livros podem ser comprados com desconto.

De início, a livreria vai trabalhar apenas com livros novos a serem adquiridos de editoras que tenham sistema automatizado de aquisição. O sistema a ser desenvolvido deve conectar-se aos sistemas das editoras para efetuar as compras.

O sistema deve calcular o custo de entrega baseado no peso dos livros e na distância do ponto de entrega. Eventualmente pode haver promoções do tipo “entrega gratuita” para determinadas localidades.

O sistema deve permitir a um gerente emitir relatórios de livros mais vendidos e de compradores mais assíduos, bem como sugerir compras para compradores baseadas em seus interesses anteriores.

Quando um livro é pedido, se existe em estoque, é entregue imediatamente, senão o livro é solicitado ao fornecedor, e um prazo compatível é informado ao comprador.

Figura 2.1: Sumário executivo do sistema Livir.

Para permitir a apresentação do sistema no curto espaço disponível neste livro, várias simplificações foram consideradas. A descrição e a modelagem de um sistema real poderiam ser bem mais extensas.

Observa-se que a visão geral do sistema é apenas uma descrição desestruturada. Existem aqui informações de nível gerencial e de nível operacional. Muitas vezes, até detalhes sobre tecnologias a serem empregadas também são descritos.

O que o analista deve ter em mente é que esse documento descreve as principais preocupações do cliente, as quais serão mais bem estruturadas nas fases posteriores do processo de análise.

2.1. Modelagem de Negócio com Diagrama de Atividades

Para melhor compreensão do funcionamento da empresa, pode-se criar um ou mais modelos das atividades de negócio. Para tal, pode ser usado o diagrama de atividades da UML. Os diagramas de atividades podem ser usados para representar processos em nível organizacional, ou seja, de forma muito mais ampla do que a mera visão de requisitos de um sistema informatizado.

O diagrama pode ser dividido em *raias* (*swimlanes*), de forma que cada raia represente um ator ou sistema que participa de um conjunto de atividades. O ator pode ser um ser humano, um departamento ou mesmo uma organização completa.

O *processo*, esquematizado no diagrama da Figura 2.2 deve ter uma pseudoatividade inicial representada por um círculo preto e uma pseudoatividade final representada por um círculo preto dentro de outro círculo.

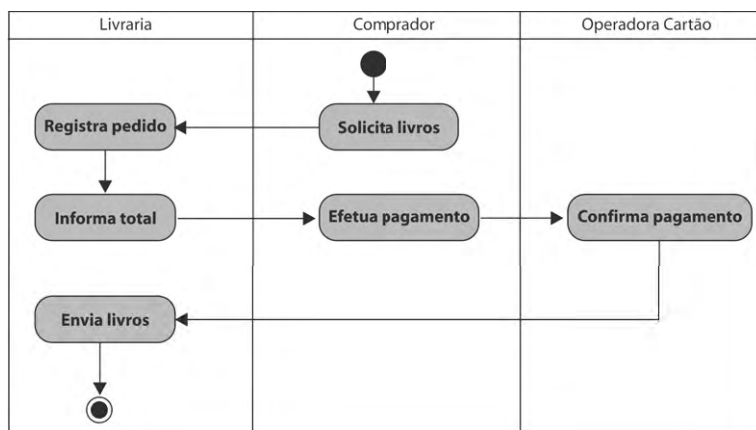


Figura 2.2: Primeira versão de um diagrama de atividades para modelar o processo de venda de livros.

As *atividades* são representadas por figuras oblongas. Quando uma atividade é representada dentro de uma determinada raia, isso significa que o ator ou sistema correspondente àquela raia é o responsável pela sua execução.

Fluxos ou *dependências* entre atividades são representados por setas. Os fluxos normalmente ligam duas atividades indicando precedência entre elas.

Um *caminho* é uma sequência de atividades ligadas por fluxos.

O diagrama de atividades pode ser então usado como ferramenta de visualização do negócio da livraria virtual. A modelagem apresentada na Figura 2.2 mostra uma primeira aproximação do processo de venda de livros. Três entidades participam do processo: a livraria virtual, o comprador (um ator humano) e a empresa de cartão de crédito.

Esse diagrama não tem a intenção de ser um modelo do sistema a ser construído e não deve ser pensado dessa forma. Sua função é ajudar o analista a entender quais são as atividades e os atores envolvidos nos principais processos de negócio da empresa, para que, a partir dessas informações, ele possa efetuar uma captura de requisitos mais eficaz. Assim, cada uma das atividades descritas no diagrama deve estar corretamente encadeada para que o caminho lógico delas seja efetivamente executado. Posteriormente, o analista vai examinar em detalhe como cada uma das atividades deve ser realizada. Se a atividade em questão envolver o sistema a ser desenvolvido, então um levantamento de requisitos mais detalhado deve ser feito referente à atividade.

Duas estruturas de controle de fluxo são usuais nesse diagrama:

- a) a estrutura de seleção (*branch* e *merge*), representada por losangos. Do nó *branch* saem fluxos com *condições de guarda* (expressões lógicas entre colchetes). Todos os caminhos devem voltar a se encontrar em um nó *merge*. Dois ou mais fluxos podem sair de uma estrutura de seleção, mas é importante que as condições de guarda sejam mutuamente excluentes, ou seja, exatamente uma delas pode ser verdadeira de cada vez;
- b) a estrutura de paralelismo (*fork* e *join*), representada por barras pretas. Caminhos independentes entre os nó *fork* e *join* podem ser executados em paralelo, ou seja, sem dependências entre suas atividades.

O diagrama da Figura 2.3 ainda é um esboço muito rudimentar do real processo de venda de livros. Apenas para ilustrar uma possível evolução desse diagrama, pode-se analisar o que aconteceria se algum dos livros em questão não estivesse em estoque. Seria necessário solicitá-lo a uma das editoras e adicioná-lo ao pedido quando chegasse.

A Figura 2.3 mostra essa situação indicando que, se nem todos os livros estão disponíveis, então, antes de o pedido ser entregue, alguns livros devem ser solicitados às editoras, e apenas após a chegada desses livros é que o pedido é atendido.

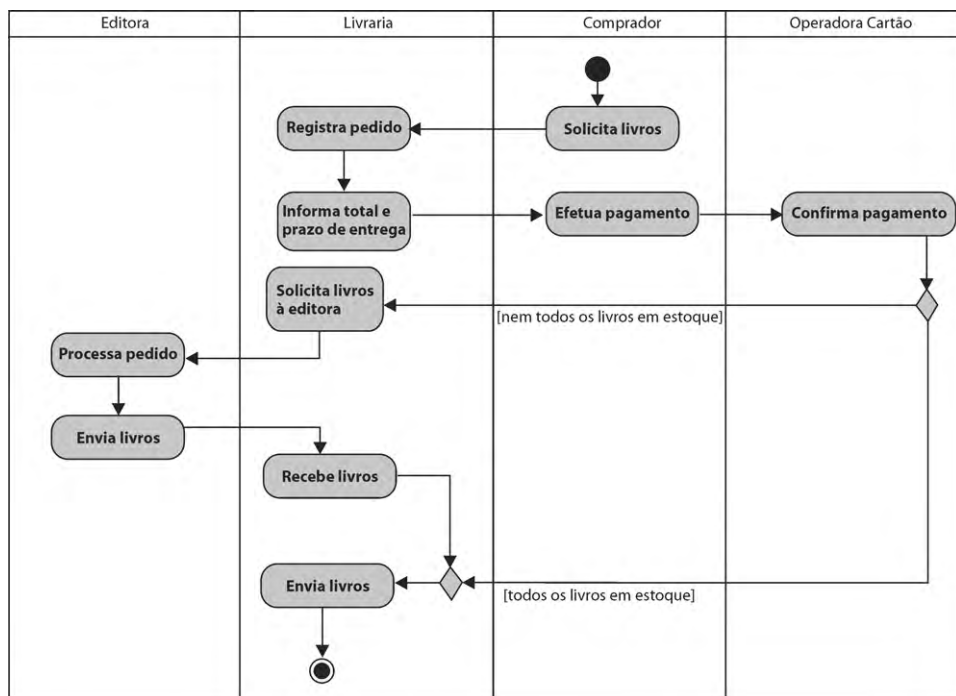


Figura 2.3: O processo de venda considerando a necessidade de comprar livros que não estejam em estoque.

Logo abaixo da atividade Confirma pagamento, há um nó *branch* representado pelo losango. Como foi dito, o nó *branch* permite que apenas um dos fluxos de saída seja executado. As condições de guarda colocadas ([nem todos os livros em estoque] e [todos livros em estoque]) são, portanto, mutuamente excludentes. A partir desse ponto, as atividades vão por um ou outro caminho até chegarem ao nó *merge*, que aparece ao lado da atividade Envia livros.

Porém, o analista poderia descobrir que esse modelo ainda não é satisfatório. Por exemplo, na falta de livros em estoque, o pedido poderia ser enviado em dois lotes. Assim, dois caminhos poderiam ocorrer em paralelo: o envio dos livros em estoque e, se necessário, a encomenda e posterior envio dos demais livros, como ilustrado na Figura 2.4.

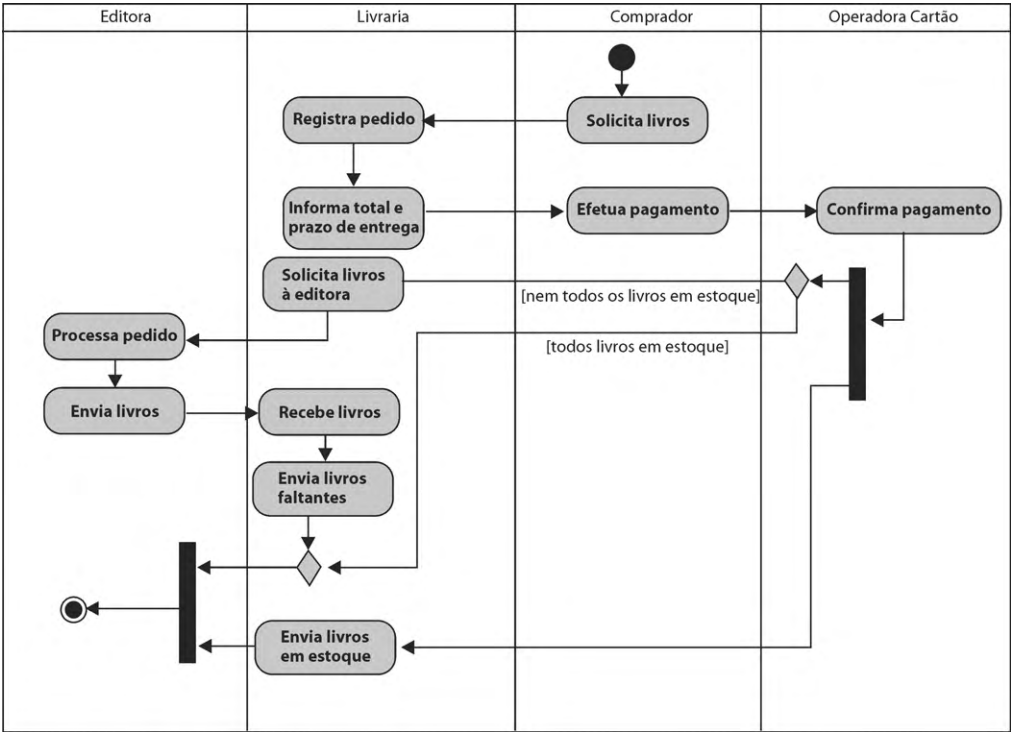


Figura 2.4: Processo de venda com envio em dois lotes.

A barra preta superior (à direita), no diagrama da Figura 2.4, representa um nó *fork*, porque ela inicia dois caminhos paralelos, e a barra preta inferior (à esquerda) representa um nó *join* porque ela novamente sincroniza os caminhos paralelos.

Depois do nó *join*, o processo só pode prosseguir se todos os caminhos que entram nele tiverem sido executados. Em função dos nós *branch* e *merge* no modelo, ainda se pode verificar que, se todos os livros estiverem em estoque, apenas

um dos caminhos paralelos será efetivamente executado (o que tem a ação Envia livros em estoque), já que o outro finaliza imediatamente no nó *merge*.

Para o diagrama estar sintaticamente correto, deve-se observar que:

- a) a cada nó *branch* deve corresponder um nó *merge*;
- b) a cada nó *fork* deve corresponder um nó *join*;
- c) os nós *branch*, *merge*, *fork* e *join* devem estar perfeitamente aninhados (ou seja, um *branch* não pode terminar com *join* e um *fork* não pode terminar com *merge* nem podem estar entrelaçados);
- d) só pode existir um nó inicial;
- e) só pode existir um nó final;
- f) cada atividade só pode ter um único fluxo de entrada e um único fluxo de saída (isso não vale para os nós *join*, *fork*, *merge* e *branch*, que não são atividades).

Os nós *fork*, *join*, *branch* e *merge* podem estar em qualquer uma das raíais, pois seu significado não é afetado por elas.

2.2. Modelagem de Aspectos de Negócio com Diagrama de Máquina de Estados

Outro diagrama que pode ser útil ao analista para entender o negócio da empresa é o *diagrama de máquina de estados*. Assim como o diagrama de atividades, esse é um diagrama comportamental, mas, em vez de modelar atividades e processos, ele apresenta estados de um sistema, ator ou entidade que se deseja estudar.

Um diagrama de máquina de estados também tem um nó (ou estado) inicial. Mas, ao contrário do diagrama de atividades, ele pode ter mais de um estado final. Em outras palavras, ele pode finalizar em diferentes estados, cada um dos quais com um significado diferente.

Outra diferença entre esses diagramas é que, no caso do diagrama de atividades, os fluxos representam apenas as condições para a realização das atividades. Assim, se existe um fluxo de A para B, a atividade B só poderá ser executada depois que a atividade A for executada. Mas o diagrama não estabelece *quando* essas atividades serão executadas.

Já o diagrama de máquina de estados especifica, nos seus fluxos, um *evento* que provoca a mudança de estado. Ou seja, os fluxos são rotulados com eventos que, se ocorrerem, fazem com que a entidade passe de um estado a outro.

Essas ocorrências podem ser, porém, restringidas por condições de guarda. Por exemplo, o evento login só pode levar o sistema de um estado ini-

cial a um estado logado se a senha do usuário estiver correta. Graficamente, os eventos são representados nos fluxos por expressões simples e as condições de guarda, por expressões entre colchetes.

Os diagramas de máquina de estado também podem representar ações que são executadas durante uma transição de estado. Por exemplo, a transição ativada pelo evento login e guardada pela condição [senha correta] pode ainda ativar a ação /autorizar acesso, a qual é uma *consequência* da transição (mas não a sua causa nem sua condição). As ações associadas às transições devem ser representadas por expressões iniciadas por uma barra (/).

Para exemplificar, pode-se considerar que o analista interessado em entender melhor o ciclo de vida de um livro na livraria virtual resolve estudar os estados pelos quais ele passa. Assim, ele descobre que, inicialmente, um livro é disponibilizado no catálogo de um fornecedor. A livraria pode então encomendar um conjunto de cópias desse livro. Quando a encomenda chega, o livro é adicionado ao estoque e disponibilizado para venda. Uma vez vendido, o livro é baixado do estoque e vai ao departamento de remessa. Depois de enviado, o livro é considerado definitivamente vendido. Eventualmente, livros enviados podem retornar, por exemplo, em função de endereço incorreto ou ausência de uma pessoa para receber a encomenda. Nesse caso, a livraria entra em contato com o comprador e, conforme for, cancela a venda ou reenvia o material. O livro é considerado definitivamente entregue apenas quando o correio confirma a entrega. Todas essas mudanças de estado (transições) estão representadas na Figura 2.5.

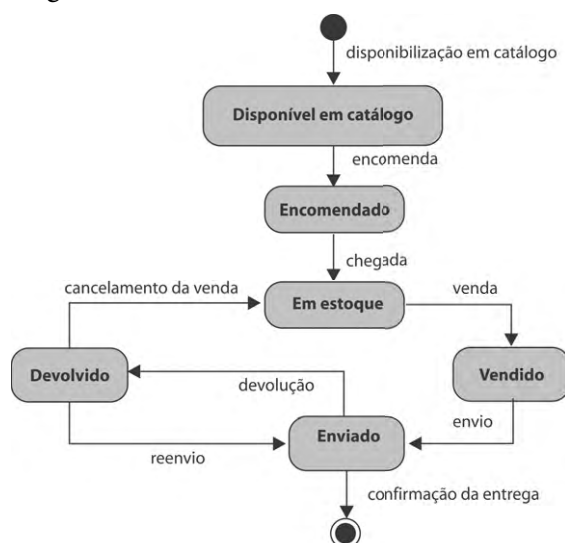


Figura 2.5: Uma primeira modelagem do ciclo de vida de um livro no sistema Livir como máquina de estados.

Porém, o modelo representado na Figura 2.5 ainda é incompleto em relação às possíveis transições de estados de um livro. Por exemplo, um livro enviado pode retornar danificado devido ao manuseio e, nesse caso, não pode ser reenviado. Isso pode ser representado pela adição de uma *condição de guarda* à transição que vai do estado Devolvido para Enviado, e com a adição de um novo estado final em que o livro é descartado, conforme a Figura 2.6.

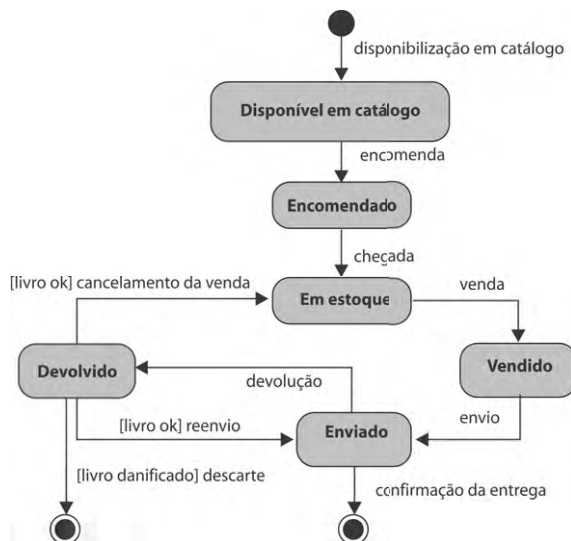


Figura 2.6: Diagrama de máquina de estados com condições de guarda.

Porém, essas condições de guarda ainda são bastante informais. Para ter condições de guarda efetivamente formais seria necessário usar uma linguagem formal como a OCL (*Object Constraint Language*), que será apresentada mais adiante neste livro.

Em algumas situações, é possível que um evento ocorra em mais de um estado, levando a uma transição para outro estado. No exemplo da Figura 2.6 pode-se imaginar que um livro pode ser danificado não apenas a partir do estado Devolvido, mas também a partir dos estados Em estoque e Vendido, pois, estando no depósito da livraria, é possível que venha a ser danificado também. Nos três casos, cabe uma transição para o estado final através do evento descarte. É possível representar um conjunto de transições com o mesmo evento de/ou para o mesmo estado utilizando o conceito de *superestado*.

Na Figura 2.7, qualquer transição de/ou para o superestado No depósito corresponde ao conjunto de transições de cada um de seus três subestados. No caso de transições *para* o superestado, é necessário estabelecer entre seus subestados qual seria o inicial. Para isso, basta incluir um pseudoestado inicial dentro do superestado ou fazer as transições diretamente para um dos subestados, como na Figura 2.7.

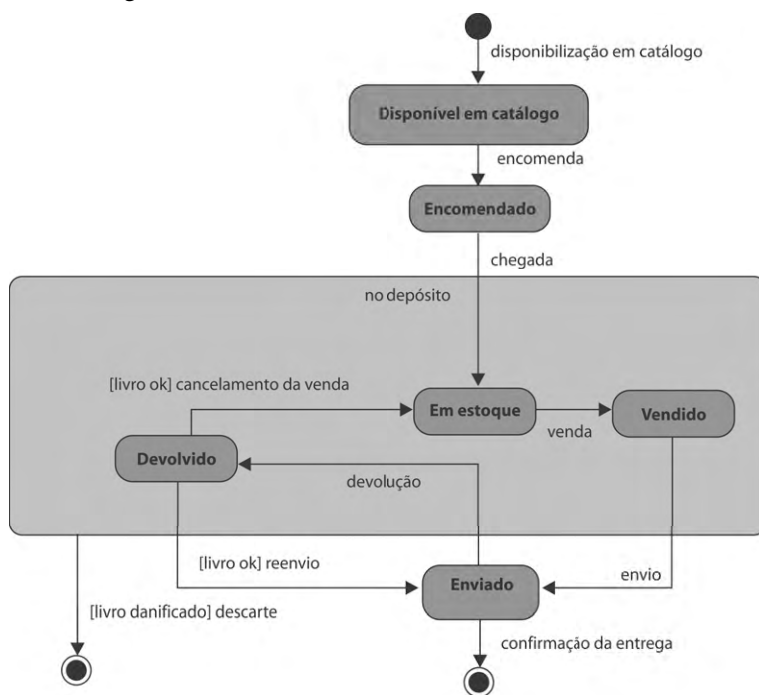


Figura 2.7: Diagrama de máquina de estados com um superestado.

Um objeto pode estar simultaneamente em mais de um estado. Por exemplo, um livro pode estar em oferta ou não desde o momento em que é encomendado até o momento em que seja descartado ou que sua entrega ao comprador seja confirmada. Assim, pode-se modelar um livro com dois estados paralelos: o primeiro estabelece se o livro está em oferta ou não, e o segundo estabelece seu *status* em relação à venda. No diagrama da Figura 2.8 os estados paralelos são representados dentro de *regiões concorrentes* de um superestado. As transições para dentro de regiões concorrentes devem ocorrer através de um nó *fork*, e as transições para fora das regiões concorrentes, através de um nó *join*.

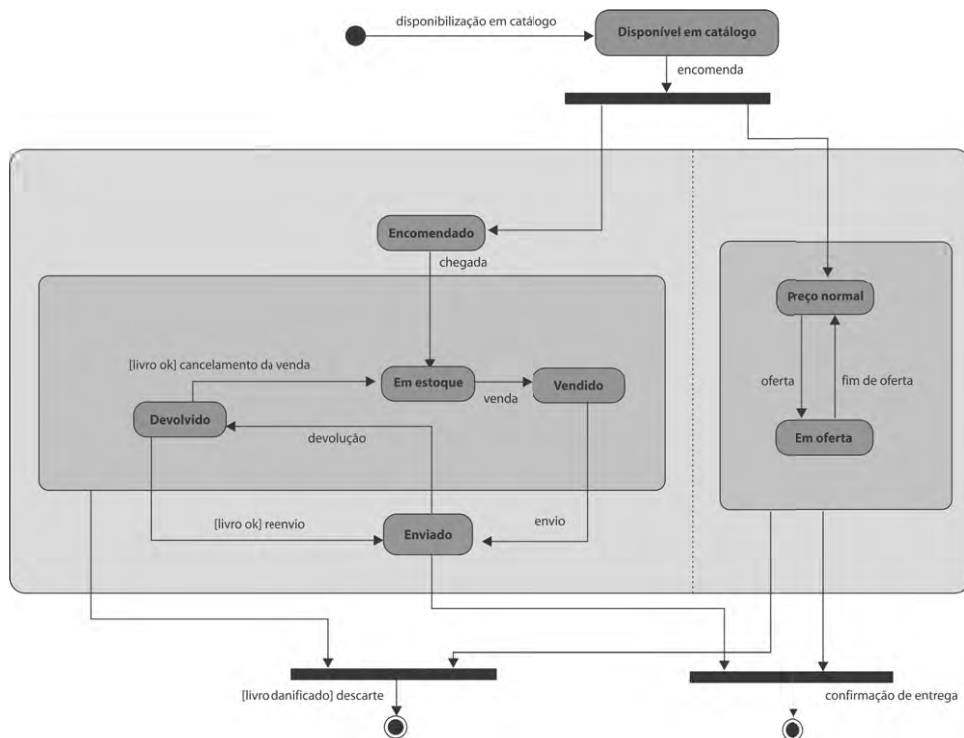


Figura 2.8: Diagrama de máquina de estados com subestados paralelos.

As transições divididas por *fork* e unidas por *join* devem ser rotuladas apenas na parte em que são de fluxo único, ou seja, na entrada, no caso de *fork*, e na saída, no caso de *join*.

Continuando a modelagem, ainda seria possível estabelecer que um livro no subestado vendido, devolvido ou enviado não pode mudar do subestado preço normal para em oferta ou vice-versa. Isso pode ser modelado adicionando-se uma condição de guarda às transições dos subestados preço normal e em oferta: [não está em vendido, devolvido ou enviado].

2.3. Comentários

É importante frisar que o objetivo dessa etapa da análise é obter uma *visão geral do sistema*, e não uma especificação detalhada do seu funcionamento. Então, na maioria dos casos, a preocupação do analista deve se centrar em descobrir informações sobre o sistema e não, ainda, especificar formalmente o seu funcionamento.

Fica a pergunta: para quais elementos do sistema deve-se fazer diagramas de máquina de estados ou de atividades durante essa fase? Não é recomendável criar diagramas para todo e qualquer elemento do futuro sistema porque essa fase demoraria demais e sua objetividade seria prejudicada, já que há pouco conhecimento sobre o sistema para poder realizar tal modelagem. Nesse ponto, é necessário a modelagem de alguns *elementos-chave* para que se possa entender melhor seu funcionamento.

Uma pista para identificar esses elementos-chave é verificar *qual o objeto do negócio*. No caso da livraria, são os livros; no caso de um hotel, são as hospedagens; no caso de um sistema de controle de processos, são os próprios processos. Então, são esses elementos que devem ser modelados para serem mais bem compreendidos nessa fase.

A segunda pergunta seria: devo usar um diagrama de máquina de estados ou um de atividades? A resposta depende da natureza do que vai ser modelado. Observa-se que um estado não comporta necessariamente uma atividade. Uma TV, por exemplo, pode estar no estado *desligada* quando não está fazendo nada. Um diagrama de atividades é útil quando se trata de pessoas ou sistemas fazendo coisas, como numa linha de produção ou na execução de um processo. Já o diagrama de máquina de estados é mais útil quando a entidade em questão passa por diferentes estados nos quais não está necessariamente realizando alguma atividade.

Além disso, o diagrama de máquina de estados usualmente apresenta diferentes estados de uma única entidade, enquanto o diagrama de atividades apresenta atividades realizadas por um conjunto de pessoas, sistemas ou organizações representados nas *swimlanes*.

Requisitos

O levantamento e a análise de requisitos compõem uma parte significativa da fase de concepção dentro do UP. O analista pode e deve utilizar todas as informações disponíveis para identificar as fontes de requisitos (departamentos, pessoas, clientes, interfaces, sistemas etc.) e, para cada fonte, identificar as funções que o sistema deverá disponibilizar.

No caso da existência de diagramas de atividades ou de estados para entidades-chave do sistema, o levantamento de requisitos deve identificar quais as funções necessárias para realizar as atividades previstas ou as mudanças de estado.

A etapa de *levantamento de requisitos* corresponde a buscar todas as informações possíveis sobre as funções que o sistema deve executar e as restrições sobre as quais o sistema deve operar. O produto dessa etapa será o *documento de requisitos*, principal componente do anteprojeto de software.

A etapa de *análise de requisitos* serve para estruturar e detalhar os requisitos de forma que eles possam ser abordados na fase de elaboração para o desenvolvimento de outros elementos como casos de uso, classes e interfaces.

3.1. Levantamento de Requisitos

O levantamento de requisitos é o processo de descobrir quais são as *funções* que o sistema deve realizar e quais são as *restrições* que existem sobre essas funções. No caso do sistema Livir, por exemplo, o levantamento de requisitos vai permitir descobrir que o sistema deve controlar a compra e venda de livros, calcular automaticamente os pagamentos, permitir o registro de danos aos livros, gerar relatórios de vendas, verificar a disponibilidade de livros em estoque etc. Essas operações e muitas outras virão a constituir a *funcionalidade* do sistema, e por isso são chamadas também de *requisitos funcionais*.

As *restrições* sobre essas funções têm a ver com a questão: de que forma essas operações se realizam? Ou, ainda, quando, como, onde, para quem, por quem, por quanto tempo etc. essas operações se realizam? Essas restrições são também conhecidas como *requisitos não funcionais*.

3.1.1. Levantar Requisitos não é Projeto!

Um sistema a ser analisado é como uma floresta. Para explorar uma floresta desconhecida não é possível, em um primeiro momento, conhecer cada planta e inseto. Apenas no final da implantação do sistema é que a equipe poderá dizer que adquiriu conhecimento sobre cada uma das suas diminutas partes. Porém, no primeiro momento do processo de análise, não se pode entrar na floresta para estudar planta por planta porque assim não se irá adquirir uma visão do todo. Há um ditado que diz que *os detalhistas não conseguem enxergar a floresta devido ao excesso de árvores*.

Então, a fase de concepção deve fornecer a visão do todo, para que se possa ver o que é mais importante, e depois dividir o todo em partes nas quais os detalhes possam ser analisados e finalmente uma solução possa ser projetada. A organização de *ciclos iterativos* nas fases de elaboração e construção corresponde a subdividir a floresta em setores para olhar um setor de cada vez e, dessa forma, poder trabalhar com a complexidade inerente. Então, um dos objetivos finais da fase de concepção é justamente organizar a divisão do trabalho em casos de uso, que serão associados aos diferentes ciclos iterativos.

Na fase de concepção, o levantamento de requisitos é rápido e genérico. Ele é feito em extensão e não em profundidade. O analista deve entender a extensão do que o sistema deve fazer, mas sem detalhar *como* ele vai fazer. Somente na fase de elaboração, a análise dos requisitos será aprofundada.

A etapa de levantamento de requisitos deve ser de *descoberta* e não de invenção. Nela, a equipe de analistas, juntamente com o cliente e usuários, vai procurar listar o maior número possível de capacidades e restrições, mas sem se preocupar demasiadamente em ter uma lista completa por enquanto, o que normalmente é impossível. Os requisitos não descobertos nessa etapa deverão ser convenientemente acomodados ao longo do resto do processo de desenvolvimento.

Deve ficar claro para o analista que requisitos são coisas que o cliente ou usuário *solicita*, e não coisas que ele, como analista, *planeja*. Alguns analistas confundem o registro dos requisitos, que são a memória das solicitações do cliente, com o início do projeto do sistema. Um exemplo desse tipo de confusão consiste em colocar projetos de tabelas do banco de dados no documento de requisitos. Como justificar que um determinado projeto de tabelas relacionais seja uma *solicitação* do cliente? Isso eventualmente pode ser possível com clientes mais sofisticados ou que tenham sistemas legados, mas não é a regra geral. As tabelas de banco de dados são parte do *domínio da solução* (projeto), e não da análise. O analista deve buscar os requisitos que correspondem às informações que o cliente precisa que sejam gerenciadas. Depois, ele pode decidir se armazena essa informação em um banco de dados ou em alguma outra estrutura.

3.1.2. Desafios dos Requisitos

O documento ou diagrama de requisitos deve registrar as capacidades do sistema e as condições às quais ele deve se conformar. Os desafios ligados a essas informações são os seguintes:

- a) como *descobrir* os requisitos;
- b) como *comunicar* os requisitos para as outras fases ou equipes do projeto;
- c) como *lembrar* dos requisitos durante o desenvolvimento e verificar se foram todos atendidos;
- d) como *gerenciar* a mudança dos requisitos.

Não adiantaria escrever um belo documento de requisitos e depois não saber se os requisitos foram ou não atendidos pelo projeto. É importante que se tenham mecanismos para fazer sistematicamente essa verificação. Por isso, é importante manter relações de *rastreabilidade* entre os requisitos e outras partes do projeto do software.

Deve-se ter em mente também que os requisitos inevitavelmente mudam durante o desenvolvimento do projeto. Deve-se esperar, então, poder gerenciar a mudança dos requisitos nas demais fases de desenvolvimento.

Outras vezes, os requisitos mudam depois do desenvolvimento. Podem mudar as condições de contexto ou a forma de trabalho ou as políticas da empresa. Embora essas mudanças não possam ser previstas pelo analista, podem ser criados mecanismos que as acomodem ao sistema quando surgirem. Existem padrões de projeto específicos para tratar essas instabilidades do sistema (como, por exemplo, o padrão *Estratégia*).

Essas mudanças são totalmente imprevisíveis. Se o sistema não estiver estruturado para acomodar mudanças nos requisitos, haverá excesso de trabalho para implementá-las.

Esse tipo de situação faz com que os processos de análise e projeto dirigidos por requisitos (Alford, 1991) sejam inadequados para muitos sistemas. Fundamentar a arquitetura de um sistema em seus requisitos é como construir um prédio sobre areia movediça. Quando os requisitos mudam, a estrutura do sistema muda. O UP, porém, propõe que a arquitetura do sistema seja fundamentada em elementos muito mais estáveis: as *classes* (e *componentes*) que encapsulam informação e comportamento.

Essas classes, mais adiante, vão implementar as funcionalidades que, combinadas, permitem a realização dos requisitos. Mudando-se os requisitos, mudam-se as combinações, mas não a estrutura básica. Essa estrutura usualmente segue o *princípio aberto-fechado* (Meyer, 1988), no sentido de que está sempre pronta para funcionar (fechada), mas aberta para incorporar novas funcionalidades.

3.1.3. Requisitos Funcionais

Os requisitos funcionais devem conter basicamente os seguintes elementos:

- a) a *descrição* de uma função a ser executada pelo sistema (usualmente entrada, saída ou transformação da informação);
- b) a *origem* do requisito (quem solicitou) e/ou quem vai executar a função em questão (usuário);
- c) quais as *informações* que são passadas do sistema para o usuário e vice-versa quando a função for executada;
- d) quais *restrições lógicas* (regras de negócio) ou *tecnológicas* se aplicam à função.

Cada requisito funcional deve conter, portanto, uma *função*, que pode ser uma entrada (exemplo, “cadastrar comprador”) ou saída (exemplo, “emitir relatório de vendas no mês”).

É importante identificar a *origem* ou *usuário* do requisito, pois sempre é necessário validar os requisitos com essas fontes, verificando se estão bem escritos, completos e consistentes.

Algumas vezes também acontece de pessoas ou departamentos diferentes apresentarem o mesmo requisito de forma discrepante. Nesse caso, é necessário realizar um acordo ou identificar quem teria autoridade para determinar a forma aceitável do requisito.

As *informações de entrada e saída* são importantíssimas para que a análise de requisitos ocorra de forma sistemática. Sem essas informações, os requisitos ficam muito vagos e pouco é aprendido sobre o sistema nessa fase. Dizer simplesmente que “o sistema deve permitir o cadastro de compradores” é muito vago como requisito. O analista deve sempre procurar saber quais movimentos de informação essas funções envolvem. Por exemplo, o cadastro do comprador envolve apenas nome, endereço e telefone? No caso do sistema Livir, não. Deve incluir com certeza dados de um ou mais cartões de crédito, pois serão necessários para efetuar os pagamentos. Essa verificação de informações que entram e saem do sistema é que vai permitir ao analista descobrir a maioria dos conceitos e funções, realizando a pesquisa em extensão no espaço de requisitos para ter uma visão abrangente do todo. No exemplo visto, fica patente, após o detalhamento das informações que entram e saem, a necessidade de definir um requisito funcional para cadastrar cartões de crédito adicionais para o comprador.

3.1.4. Requisitos Não Funcionais

Os requisitos não funcionais aparecem sempre ligados a requisitos funcionais e podem ser basicamente de dois tipos: lógicos ou tecnológicos.

As *restrições lógicas* são as regras de negócio relacionadas à função em questão. Por exemplo, no registro de uma venda, uma série de restrições lógicas poderia ser considerada, como por exemplo: não efetuar a venda caso a operadora de cartão não autorize o pagamento ou não efetuar a venda caso a venda anterior tenha sido cancelada devido a um endereço inválido que ainda não foi corrigido.

As *restrições tecnológicas* dizem respeito à tecnologia para a realização da função, como, por exemplo, a interface (Web, por exemplo), o tipo de protocolo de comunicação, restrições de segurança ou tolerância a falhas etc.

Por exemplo, registrar a venda de um conjunto de livros é um requisito funcional. Estabelecer que o tipo de interface para efetuar uma venda deve seguir um padrão de interface de fluxo sequencial de telas é uma restrição tecnológica (de interface) sobre a forma como essa função é realizada.

Outro exemplo de requisito não funcional seria “a autorização de débito no cartão de crédito não deve levar mais do que 5 segundos”. Isso seria uma restrição tecnológica de desempenho e afetaria a forma como o projetista iria pensar no mecanismo de acesso ao sistema da operadora de cartões. Nesse caso, o projeto do sistema teria de considerar seriamente o uso de conexões fixas com as operadoras de cartão em vez de linhas discadas.

3.1.5. Requisitos Suplementares

Os *requisitos suplementares* são todo tipo de restrição tecnológica ou lógica que se aplica ao sistema como um todo e não apenas a funções individuais. Por exemplo, um requisito suplementar pode estabelecer que o sistema deva ser compatível com um banco de dados legado ou ser implementado em uma determinada linguagem de programação, ou ainda, seguir um determinado padrão de interface ou *look and feel*.

Deve-se tomar certo cuidado no enunciado dos requisitos suplementares. Um requisito como “o sistema deve ser fácil de usar” é muito pouco esclarecedor para que possa ser útil. Melhor seria dizer: “o sistema terá ajuda *on-line* em todas as telas e para todas as funções”. Isso dá uma ideia mais precisa sobre o que realmente deve ser realizado pelo sistema.

3.1.6. Documento de Requisitos

O *documento de requisitos* registra todos os tópicos relativos ao que o sistema deve fazer e sob quais condições. Esse documento ainda não precisa ser totalmente estruturado, e assume-se que não será completo em profundidade, embora se possa esperar que esteja razoavelmente completo em extensão. Eventuais lacunas desse documento serão preenchidas durante a fase de elaboração.

O documento de requisitos pode ser organizado de forma textual ou na forma de um diagrama (o diagrama de requisitos existente em algumas ferramentas CASE, porém, não pertence à especificação da UML).

Os requisitos funcionais podem ainda ser subdivididos em subsistemas, especialmente se a quantidade de funções for muito grande. Essa é uma primeira forma de organização do sistema.

Sugere-se que o documento de requisitos tenha um índice consistindo no nome da função ou requisito suplementar e que o corpo do documento contenha os detalhes mencionados na Seção 3.1.3.

A Figura 3.1 apresenta um exemplo de documento de requisitos para o sistema Livir. Se houvesse subsistemas, eles seriam o primeiro nível de divisão dentro de “Requisitos funcionais”, contendo cada divisão um conjunto de requisitos numerados.

Sistema Livir – Documento de Requisitos

Requisitos funcionais

1. Registrar novos títulos a partir do catálogo das editoras.
2. Registrar vendas de livros.
3. Realizar encomendas de livros.
4. Registrar e autorizar pagamentos com cartão de crédito.
5. Registrar e aplicar promoções.
6. Calcular custos de entrega.
7. Emitir relatório de livros mais vendidos.
8. Emitir relatório de compradores mais assíduos.
9. Emitir sugestões de compra para compradores baseadas em compras anteriores.
10. ...

Requisitos suplementares

1. O sistema deve operar via interface Web.
2. Todos os controles de interface devem ter um campo de ajuda associado.
3. ...

Figura 3.1: O índice de um documento de requisitos para o sistema Livir.

O detalhamento de cada requisito (especialmente os funcionais) deve aparecer no corpo do documento. Esse detalhamento deve conter as partes mencionadas na Seção 3.1.3. A Figura 3.2 apresenta um exemplo.

1. Registrar novos títulos a partir do catálogo das editoras

Descrição:

O gerente seleciona as editoras para as quais pretende fazer a atualização. O processo é automático. O sistema consulta os ISBN disponibilizados e os compara com os existentes na base. Havendo novos ISBN, o sistema atualiza a base com as novas informações.

Fontes:

Sr. Fulano de Tal (gerente) e manual técnico da interface de catálogo das editoras.

Usuário:

O próprio gerente.

Informações de entrada:

O gerente informa quais são as editoras para as quais pretende fazer a atualização a partir de uma lista fornecida pelo sistema.

Informações de saída:

- A lista de editoras (nome).
- O relatório de atualizações efetuadas (uma lista contendo: nome da editora, ISBN, título e preço de compra).

Restrições lógicas:

Não há.

Restrições tecnológicas:

1. Deve ser usado o sistema de interface com as editoras, de acordo com o manual XYZ.1234.
2. A seleção feita pelo gerente se dá através de uma lista de seleção múltipla, a qual deve ser ordenada de forma alfabética.
3. ...

Figura 3.2: Detalhamento de um requisito.

Observa-se que o detalhamento das informações que entram e saem do sistema é fundamental para a compreensão mais detalhada dessa função. Através desse detalhamento é que a verdadeira dimensão do sistema vai sendo descoberta.

Sem esse detalhamento, o sistema pode parecer mais simples do que realmente é, o que explica por que, em muitos casos, os analistas esperam desenvolver um sistema em determinado tempo mas levam muito mais tempo, estourando prazos e orçamentos.

3.2. Análise de Requisitos

Na análise de requisitos, o analista vai procurar caracterizar certas propriedades dos requisitos já levantados, conforme as subseções seguintes.

3.2.1. Permanência e Transitoriedade

Uma primeira caracterização considerada importante na análise de requisitos é decidir se determinado requisito não funcional ou suplementar é *permanente* ou *transitório*.

Requisitos não funcionais podem ser considerados permanentes (não vão mudar) ou transitórios (podem mudar) de acordo com uma decisão tomada pelo analista em conjunto com o cliente. O requisito não tem a propriedade de permanência ou transitoriedade por si: ela é decidida de acordo com a conveniência. Um mesmo requisito pode ser considerado permanente ou transitório dependendo do que se queira em relação ao tempo de desenvolvimento ou custo da manutenção do software.

Por exemplo, um requisito suplementar poderia estabelecer que o sistema Livir trabalhe com uma única moeda: o real. Se esse requisito suplementar for considerado *permanente*, todo o sistema será construído de forma que o real seja a única moeda. Se o requisito for considerado *transitório*, o sistema deverá ser construído de forma a poder acomodar futuramente outras moedas ou, ainda, mais de uma moeda de cada vez.

As consequências de decidir que um requisito é permanente são as seguintes:

- a) fica mais barato e rápido desenvolver o sistema em si;
- b) fica mais caro e demorado mudar o sistema caso o requisito, por algum motivo, mude no futuro (com a implantação de uma nova moeda, por exemplo).

Por outro lado, decidir que um requisito é transitório tem como consequências:

- a) fica mais caro e complexo desenvolver o sistema (ele deverá acomodar funcionalidades para a mudança da moeda);

- b) fica mais barato e rápido fazer a manutenção no sistema (caso a moeda mude, o sistema já está preparado para acomodar esse fato com uma simples reconfiguração).

Então, a natureza dos requisitos não funcionais não vai decidir se eles são permanentes ou transitórios. O analista é que tem de tomar essa decisão (com o aval do cliente). O ideal seria elencar aqueles requisitos de maior importância (que se espera que possam *mesmo* mudar num futuro próximo e cuja mudança tenha maior *impacto* no sistema) e considerá-los transitórios, deixando os demais como permanentes.

3.2.2. Requisitos Evidentes e Ocultos

Os requisitos funcionais podem ser opcionalmente classificados em evidentes ou ocultos:

- a) os *requisitos funcionais evidentes* são funções efetuadas com conhecimento do usuário. Esses requisitos usualmente correspondem a trocas de informação, como consultas e entrada de dados, que ocorrem com o meio exterior através da interface do sistema;
- b) os *requisitos funcionais ocultos* são funções efetuadas pelo sistema sem o conhecimento explícito do usuário. Usualmente, são cálculos ou atualizações feitas pelo sistema sem a solicitação explícita do usuário, mas como consequência de outras funções solicitadas por ele.

É importante classificar os requisitos dessa forma porque, posteriormente, eles serão associados aos casos de uso através de *relações de rastreabilidade*. Apenas os requisitos evidentes corresponderão aos passos do caso de uso expandido porque são executados com o conhecimento explícito do usuário. Os requisitos ocultos são executados internamente pelo sistema. Então, embora não apareçam explicitamente nos passos de um caso de uso expandido, esses precisam ser adequadamente associados a aqueles para ser lembrados no momento de projetar e implementar as operações do caso de uso.

Um exemplo de requisito evidente é emitir um relatório de livros mais vendidos por requisição do gerente. Um exemplo de requisito oculto seria aplicar uma política de desconto, se ela existir. Nesse caso, nenhum usuário solicita explicitamente ao sistema para fazer essa aplicação. É uma atividade que o sistema executa independentemente dos usuários e, portanto, de forma oculta.

3.2.3. Requisitos Obrigatórios e Desejados

Os requisitos ainda podem ser classificados em *obrigatórios* e *desejados*, ou seja, aqueles que devem ser obtidos de qualquer maneira e aqueles que podem ser obtidos caso isso não cause maiores transtornos no processo de desenvolvimento.

No caso dos requisitos funcionais, essa classificação indica uma priorização de desenvolvimento. Um bom analista tomaria as funções como base para calcular o tempo de desenvolvimento. Assim, não faria muito sentido falar em funções obrigatórias e desejáveis, mas sim em quanto tempo levaria para desenvolver esse ou aquele conjunto de funções.

Entretanto, nem sempre a coisa funciona assim. No caso de alguns sistemas pode-se querer trabalhar com prioridades, desenvolvendo inicialmente determinadas funções consideradas obrigatórias e, posteriormente (se sobrar tempo), outras funções consideradas desejadas.

Mas os requisitos não funcionais e suplementares são bem mais imprevisíveis do que os funcionais para efeito de estimativa de esforço. Assim, em alguns casos, pode ser necessário efetivamente classificar esses requisitos por graus de prioridade.

Definem-se algumas restrições que devem ser obtidas a qualquer custo e outras que seria desejável obter, desde que isso não extrapole o tempo ou recursos disponibilizados para o projeto.

Por exemplo, no caso do sistema Livir, o requisito de que a interface seja Web poderia ser considerado obrigatório. Nesse caso, não se aceita outra coisa. Porém, o acesso através de telefone celular poderia ser um requisito desejável, já que não é absolutamente necessário para o efetivo funcionamento do sistema.

Com a formalização dos contratos de desenvolvimento de software, porém, cada vez menos flexibilidade se tem em relação a requisitos desejáveis. O desenvolvedor deve dizer claramente quais requisitos vai implementar, quanto tempo vai levar e quanto vai custar. Qualquer desvio dessas previsões pode implicar multas ou cancelamento de contratos.

3.2.4. Classificação de Requisitos Não Funcionais e Suplementares

Os requisitos não funcionais e suplementares podem ser classificados por atributo, ou seja, se são requisitos de interface, de implementação, de efi-

ciência, de tolerância a falhas etc. A finalidade principal das classificações de requisitos em categorias é a organização. Algumas sugestões de possíveis categorias são:

- a) *usabilidade*: quais fatores humanos estão envolvidos no sistema? Que tipo de ajuda o sistema vai prover? Quais as formas de documentação ou manuais estarão disponíveis? Como esses manuais vão ser produzidos? Que tipo de informação eles vão conter? Seria interessante definir esses tópicos na fase de concepção, visto que o contrato com o cliente deve especificar muitas dessas questões;
- b) *confiabilidade*: que tipo de tratamento de falhas o sistema vai ter? O analista não é obrigado a produzir um sistema totalmente tolerante a falhas, mas deve estabelecer que tipo de falhas o sistema será capaz de gerenciar: falta de energia, falha de comunicação, falha na mídia de gravação etc. Não se deve confundir *falha* com erro de programação, visto que erros de programação são elementos que nenhum software deveria conter. As falhas são situações anormais que podem ocorrer mesmo para um software implementado sem nenhum erro de programação;
- c) *desempenho*: que tipo de eficiência e precisão o sistema será capaz de apresentar? Pode-se estabelecer, por exemplo, como requisito de eficiência, que nenhuma consulta à base de dados de compradores vai demorar mais de cinco segundos. Na fase de concepção não se define *como* o sistema fará para cumprir o requisito, apenas se diz que de alguma forma ele terá de ser cumprido no projeto. Cabe ao projetista e programador garantir que o requisito seja satisfeito. Se o analista por algum motivo conclui que o requisito dificilmente poderá ser implementado, o requisito passa a ser um *risco* do sistema e eventualmente necessitará de um estudo mais aprofundado ainda na fase de concepção, para verificar a possibilidade de sua realização;
- d) *configurabilidade*: o que pode ser configurado no sistema? Deve-se definir os elementos que poderão ser configurados pelo usuário sem que seja necessário recompilar o sistema. Exemplos de itens configuráveis são: o tipo de impressoras, a moeda do país, políticas da empresa, fontes e cores da interface, idioma etc.;
- e) *segurança*: quais são os tipos de usuários e que funções cada um pode executar? Sugere-se a implantação de um sistema de permissões, de for-

ma que possam ser criados dinamicamente perfis de usuários com diferentes conjuntos de permissões;

- f) *implementação*: qual linguagem deve ser usada? Por que motivo? Que bibliotecas estarão disponíveis? Quais bancos de dados serão acessíveis? Há necessidade de comunicação com sistemas legados?;
- g) *interface*: como deve ser a interface? Vai ser seguida alguma norma ergonômica?;
- h) *empacotamento*: de que forma o software deve ser entregue ao usuário final?;
- i) *legais*: muitas vezes, uma equipe de desenvolvimento deve contar com uma assessoria jurídica para saber se está infringindo direitos autorais ou normas específicas da área para a qual o software está sendo desenvolvido.

Embora essa lista seja extensa, o analista deve ter em mente que se trata apenas de uma forma de classificação para que ele possa mais facilmente avaliar quais requisitos são relevantes para cada um dos tipos listados. Não há necessidade de procurar requisitos que não existem, por exemplo, estabelecendo complicados requisitos de empacotamento para um cliente para o qual não faz a menor diferença a forma como o software será entregue.

Também não se deve perder tempo discutindo se um requisito é desse ou daquele tipo. Mais importante do que classificar é reconhecer que o requisito existe. Esse tipo de discussão, que não acrescenta conhecimento ao estudo do problema, deixa a análise travada, ou seja, não se anda mais para frente.

Casos de Uso de Alto Nível

Uma vez que os requisitos tenham sido levantados, cabe agora organizá-los em grupos correlacionados, de forma a abordá-los nos ciclos iterativos. Essa organização ainda deve ocorrer na fase de concepção do UP.

Na fase de concepção, é necessário identificar os principais casos de uso do sistema. Um caso de uso difere dos processos modelados no Capítulo 2 porque, ao contrário daqueles, que podem se estender ao longo de dias ou mesmo anos, os casos de uso são processos de interação com o sistema que têm início e fim em tempo contíguo (sem interrupções), ou seja, são executados, normalmente, em minutos.

Os casos de uso devem cobrir as principais atividades de negócio ligadas ao sistema que será desenvolvido. Um *caso de uso de alto nível* corresponde a apenas um nome (representado dentro de uma elipse no diagrama), possivelmente associado a um ou mais atores (pessoas ou sistemas que interagem com o sistema, sendo analisado através do caso de uso), conforme se pode ver na Figura 4.1.

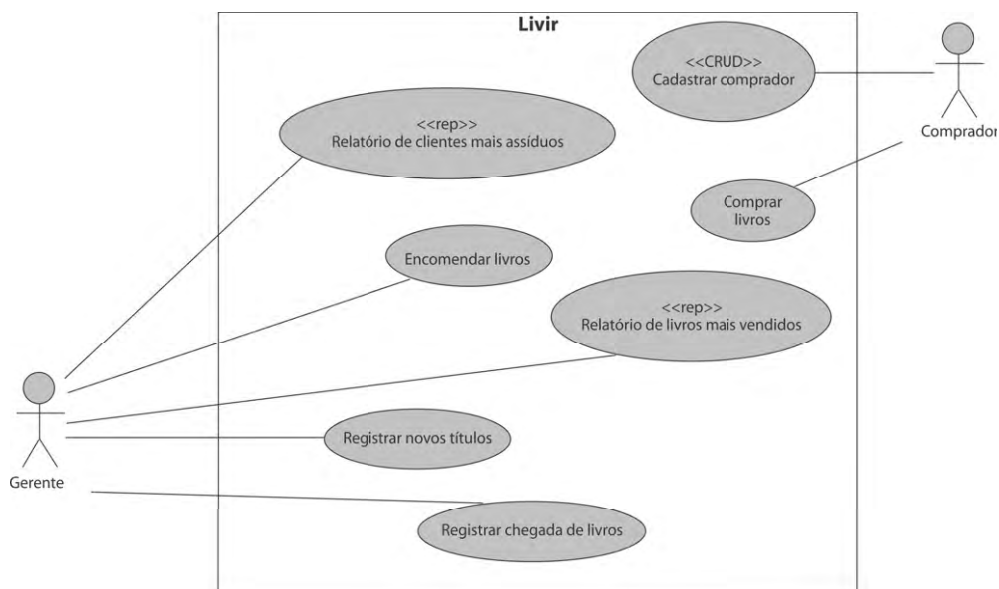


Figura 4.1: Diagrama de casos de uso da UML.

O objetivo de listar os casos de uso é levantar informações sobre como o sistema interage com possíveis usuários e quais consultas e transformações da informação são necessárias para que processos completos de interação sejam executados. É, portanto, uma forma de sistematizar e organizar os requisitos.

Por exemplo, no caso do sistema Livir, os principais casos de uso (entre outros) são: Comprar livros, Encomendar livros, Registrar novos títulos etc. Funções mais simples (requisitos funcionais), como Calcular custos de entrega e Registrar e autorizar pagamentos com cartão de crédito, possivelmente vão ocorrer apenas como parte dos processos maiores, nunca isoladamente. Casos de uso são processos que podem ocorrer isoladamente. Processos que só podem ocorrer juntamente com outros processos são apenas partes de casos de uso, mas não são casos de uso por si.

No caso do sistema Livir, o cálculo de custos de entrega acontecerá durante o processo de venda de livros, nunca como um caso de uso isolado. Já a venda de livros pode ser considerada como um caso de uso porque tem início e fim bem definidos, ocorre em um intervalo de tempo contíguo (sem interrupções) e produz um resultado consistente (a venda registrada).

Cada caso de uso será associado a um conjunto de requisitos funcionais do sistema. Algumas ferramentas *CASE* fornecem recursos para representar essas dependências. Normalmente isso se faz através de relações de *rastreabilidade* ou matriz de relacionamento. Na falta de uma ferramenta desse tipo, basta que o analista procure listar os casos de uso anotando ao lado os códigos dos requisitos funcionais associados. Usualmente, vários requisitos associam-se a um caso de uso, especialmente quando se tratar de um caso de uso complexo. Alguns requisitos, porém, podem estar associados a vários casos de uso. Em alguns casos, também é possível que um requisito corresponda a um único caso de uso e vice-versa.

Para descobrir os casos de uso, deve-se identificar os atores envolvidos com o sistema (funcionários, gerentes, compradores, fornecedores etc.). Após as entrevistas com esses atores, para descobrir seus objetivos, o analista deve descobrir quais os principais processos de negócio de que eles participam. A cada processo possivelmente corresponderá um ou mais casos de uso.

Os casos de uso de alto nível da fase de concepção não têm como intenção definir perfis de segurança para acessar funções do sistema. Portanto, a definição de diferentes atores tem apenas papel ilustrativo.

4.1. Caracterização de Casos de Uso

Existe um diagrama na UML para representar casos de uso e seus atores (Figura 4.1). Nesse diagrama, as elipses representam casos de uso, os bonecos representam atores (usuários) e o retângulo representa a fronteira do sistema ou subsistemas. Não é um dos diagramas mais úteis, mas é bastante popular para mostrar quais funções um sistema efetivamente executa.

Deve-se evitar que o diagrama tenha um conjunto muito grande de elipses, pois, nesse caso, fica inviável compreendê-lo. Assim, deve-se caracterizar muito bem o que são casos de uso para evitar que esse diagrama tenha, por um lado, processos demais, excessivamente detalhados, ou, por outro lado, processos de menos, faltando funcionalidades importantes do sistema.

Via de regra, a solução é representar no diagrama apenas os processos que podem ser executados isoladamente. Processos parciais que são executados necessariamente dentro de outros processos não devem ser representados nesse diagrama.

4.1.1. Monossessão

Um bom caso de uso deve ser *monossessão*. Isso significa que ele deve iniciar e terminar sem ser interrompido. Por exemplo, o registro de uma encomenda de livros é feito em uma única sessão de uso do sistema.

O pedido e a entrega de livros encomendados, embora ocorram necessariamente um após o outro, devem ser considerados casos de uso independentes, pois acontecem em momentos diferentes, havendo uma interrupção da interação com o sistema entre esses dois processos.

Por outro lado, o caso da venda de livros deve ser considerado como um processo ininterrupto. Então, como tratar a situação em que o carrinho de compras é “guardado” pelo comprador para continuar outro dia? Essa situação pode ser pensada assim: o caso de uso de venda de livros pode terminar de duas maneiras alternativas – com a consumação da venda ou com o carrinho sendo guardado. Em uma próxima oportunidade, o caso de uso é iniciado novamente (possivelmente com alguns livros já no carrinho) e novamente pode ser concluído de uma das duas maneiras mencionadas.

4.1.2. Interativo

Um caso de uso também deve ser *interativo*, o que significa que necessariamente deve existir um ator interagindo com o sistema. Processos internos do sistema não são casos de uso.

Por exemplo, “fazer *backup* automático dos dados” não pode ser considerado caso de uso porque é algo que o sistema faz internamente, sem repassar necessariamente informações aos atores ou receber informações deles.

4.1.3. Resultado Consistente

Um caso de uso deve produzir resultado consistente, seja um registro completo produzido ou uma consulta realizada. Ele não pode terminar deixando a informação em estado inconsistente. Por exemplo, um registro de uma venda não pode deixar de identificar o comprador e os livros solicitados, caso contrário a informação ficará inconsistente com as regras de negócio. Não se poderia cobrar o total da venda se não se sabe quais livros foram comprados nem quem os solicitou.

Pode-se pensar assim: somente será um caso de uso um processo completo, no sentido de que um usuário iria ao computador, ligaria o sistema, executaria o processo e em seguida poderia desligar o computador porque o processo estaria completo.

Isso exclui fragmentos como “Calcular custos de entrega” no caso do sistema Livir, porque esses custos são calculados dentro do processo de venda de livros e não como um processo isolado. Isso também exclui operações como *login*, visto que ir ao sistema fazer *login* e em seguida desligar o computador não pode ser visto como um processo completo que produz resultado consistente.

Por outro lado, é possível que casos de uso completos ocorram dentro de outros casos de uso. Por exemplo, o processo de cadastramento de um comprador pode ser considerado um caso de uso completo, e esse processo pode ocorrer dentro do caso de uso de venda de livros quando for a primeira vez que o comprador usa o sistema.

4.2. Complexidade de Casos de Uso

Os casos de uso podem ser classificados de acordo com sua complexidade da seguinte forma:

- a) *processos de negócio*. Os principais processos de negócio da empresa que não se encaixam em nenhum dos padrões a seguir possivelmente abarcarão um número considerável de requisitos funcionais. Esses processos são desconhecidos e apresentam alto risco na modelagem de um sistema, pois usualmente são os mais complexos;
- b) *CRUD*. A sigla *CRUD* vem do inglês: *Create, Retrieve, Update e Delete*, ou seja, criar, consultar, atualizar e remover, as quatro operações básicas sobre unidades de informação ou conceitos. Assim, em vez de definir cada uma dessas operações como um caso de uso individual, elas devem ser agrupadas em casos de uso do tipo “manter” ou “gerenciar”. Essas operações seguem um padrão bem definido, variando apenas as regras de negócio que são específicas do conceito sendo gerenciado ou mantido. Portanto, são casos de uso de médio risco e média complexidade;
- c) *relatórios*. Um relatório é um acesso à informação presente no sistema que não altera essa informação (não tem efeito colateral). A consulta (*retrieve*) de *CRUD* apenas apresenta dados sobre um conceito (p. ex.,

nome, endereço e telefone de um comprador). Mas um relatório deve ter pelo menos uma *totalização* ou *filtro* para ser considerado um caso de uso à parte (p. ex., quantidade de livros vendidos para um comprador). Como os relatórios implicam apresentar informação que já está disponível, são considerados casos de uso de baixo risco e baixa complexidade.

Apenas os casos de uso que correspondem a processos de negócio precisam ser expandidos na fase de elaboração, visto que os outros dois tipos seguem padrões que permitem que as operações envolvidas sejam especificadas diretamente, sem necessidade de um estudo sobre a interatividade do caso de uso.

Alguns casos de uso da Figura 4.1 estão estereotipados, o que é uma forma de caracterizar tipos especiais com UML. O estereótipo <<CRUD>> indica que o caso de uso em questão representa as quatro operações CRUD mencionadas. O estereótipo <<report>> ou simplesmente <<rep>> indica que o caso de uso consiste em um único relatório sem efeitos colaterais, mas com totalizações ou filtros.

4.3. Priorização de Casos de Uso

Uma vez que os casos de uso tenham sido identificados, deve-se verificar se todos os requisitos funcionais do sistema se associam a pelo menos um caso de uso. Se isso não acontecer, é possível que ainda esteja faltando algum caso de uso.

O UP é dirigido por casos de uso e centrado na arquitetura. Isso significa que as próximas fases desse processo consistirão em detalhar cada vez mais uma arquitetura de sistema que permita que os casos de uso identificados sejam executados pelos atores.

O UP é também um processo iterativo e incremental. Então, não se espera que o desenvolvimento seja feito por fases, como no caso do modelo Waterfall (Royce, 1970), mas que, a cada ciclo de desenvolvimento, um conjunto tratável de casos de uso seja considerado para estudo e incorporação de funcionalidade na arquitetura.

A principal técnica de redução de risco a aplicar nesse momento é buscar tratar *prioritariamente* os processos de negócio, pois são mais complexos e é com eles que o analista pode aprender mais sobre o sistema real. Deixa-se para uma segunda etapa os casos de uso padronizados CRUD e bem para o

final os relatórios, que vão apenas tabular e totalizar informação que já deve estar disponível.

4.4. Fronteira do Sistema

Uma das decisões que o analista precisa tomar quando está projetando casos de uso é qual a *fronteira do sistema*. Gráficamente, a fronteira é apenas um retângulo que aparece no diagrama (ver Figura 4.1), dentro do qual estão os casos de uso e fora do qual estão os atores.

Mas, na prática, decidir sobre essa fronteira nem sempre é tão simples. Um exemplo seria um posto de gasolina onde há um frentista que usa a bomba a pedido do cliente. O ator é o frentista ou o cliente? Ou ambos? O frentista é um ator ou é parte do sistema? Isso o analista deve resolver e permanecer consistente com sua decisão.

Para que um caso de uso seja o mais essencial possível, recomenda-se, porém, que apenas os atores efetivamente interessados no caso de uso sejam considerados. O frentista, no exemplo anterior, é um mero instrumento de ação do cliente. O frentista não tem interesse pessoal no processo de encher o tanque. Ele atua simplesmente como uma ferramenta do sistema ou como parte da tecnologia. Se a bomba fosse operada pelo próprio cliente, como acontece em alguns países, o caso de uso essencial ainda seria o mesmo, pois as mesmas informações seriam trocadas com o sistema, mudando apenas o personagem. Então, nesse caso, recomenda-se que o ator seja o cliente e que o frentista sequer apareça como ator.

Dessa forma, a análise vai produzir casos de uso que são independentes da tecnologia de interface. No caso do sistema Livir, por exemplo, se o ator do caso de uso de venda de livros for apenas o comprador, a descrição do caso de uso pode ser a mesma, quer seja uma livraria virtual ou uma livraria presencial, onde há funcionários atendendo os clientes e operando o sistema.

Alguém poderá perguntar: mas, e se o funcionário deve indicar que foi ele quem fez a venda para receber uma comissão sobre ela? Nesse caso, trata-se de *outro* sistema, com outro caso de uso, que difere do mencionado para o sistema Livir. E, nesse caso, tanto o comprador quanto o funcionário seriam atores com interesse na informação trocada com o sistema.

Casos de Uso Expandidos

A fase de *elaboração* do UP comporta as atividades de análise e projeto do sistema. A análise, nessa fase, por sua vez, comporta três subatividades distintas:

- a) expansão dos *casos de uso* (capítulo presente) e determinação dos *eventos e respostas de sistema* (Capítulo 6);
- b) construção ou refinamento do *modelo conceitual* (Capítulo 7);
- c) elaboração dos *contratos das operações e consultas de sistema* (Capítulo 8).

A *expansão dos casos de uso* pode ocorrer em primeiro lugar porque é uma atividade que toma como entradas apenas o caso de uso de alto nível identificado na fase de concepção e o documento de requisitos.

O refinamento do *modelo conceitual* pode ser feito depois disso porque as informações explicitamente trocadas entre o sistema e o mundo externo, conforme a expansão do caso de uso, serão usadas como base para construir e aprimorar o modelo conceitual.

A atividade de *elaboração dos contratos* deve ser realizada por último, já que ela depende da descoberta das operações e consultas de sistema, bem como do modelo conceitual.

A expansão dos casos de uso corresponde ao aprofundamento da análise de requisitos. Já a modelagem conceitual corresponde à análise de domínio

em seus aspectos estáticos. Finalmente, a elaboração dos contratos corresponde à modelagem funcional de domínio, ou seja, ela mostra como a informação é transformada pelo sistema.

Quando se está expandindo um caso de uso de análise, deve-se proceder a um exame detalhado do processo envolvido. Deve-se descrever o caso de uso passo a passo: como ele ocorre e como é a interação entre os atores e o sistema. Deve-se evitar mencionar interfaces ou tecnologia, mas apenas dizer quais informações os atores passam ao sistema e quais informações o sistema passa aos atores.

Essa descrição passo a passo, a princípio, não deve ser estruturada com desvios. Ela deve ser baseada em uma sequência *default*, ou *fluxo principal*, na qual se descreve o que acontece quando tudo dá certo na interação. Esse fluxo também é chamado de “*caminho feliz*”, pois nele não se deve prever erros ou exceções.

Depois de descrever o fluxo principal do caso de uso, analisa-se criticamente cada passo e procura-se verificar o que poderia dar errado. A partir da identificação de uma possível exceção, deve-se construir uma descrição de procedimentos para resolver o problema. O caso de uso então passa a possuir *fluxos alternativos*, semelhantes aos *handlers* dos métodos de tratamento de exceções.

Essa descrição do caso de uso na atividade de análise é feita sem considerar a tecnologia de interface. É, portanto, uma descrição *essencial*. Nesse nível da análise, não interessa a forma das interfaces do sistema, mas *quais* informações serão trocadas entre o sistema e o ambiente externo. Deve-se evitar, portanto, mencionar “menus”, “janelas” etc. Apenas a *informação* efetivamente trocada deve ser mencionada.

5.1. Caso de Uso Essencial Versus Caso de Uso Real

Todos os casos de uso de análise são do tipo *essencial*. Essencial, nesse contexto, significa que ele é descrito em um nível de discurso em que apenas a “essência” das operações é apresentada, em oposição à sua realização concreta. Em outras palavras, o analista deve descrever “o que” acontece entre o usuário e o sistema, sem, entretanto, informar sobre “como” essa interação ocorre. O analista não deve, portanto, na atividade de análise, tentar descrever a tecno-

logia de interface entre o sistema e o usuário. Isso será feito na atividade de projeto, na qual serão construídos casos de uso *reais*.

Uma dúvida frequente refere-se a *o que* descrever no caso de uso: o sistema atual ou o sistema como vai ficar depois de pronto? Se, no sistema atual, as operações são feitas manualmente e depois serão feitas no computador, qual deve ser a descrição produzida pelo caso de uso? A resposta é: nem uma nem outra. O caso de uso de análise deve descrever a *essência* das operações e não a sua realização concreta. Assim, o analista deve procurar sempre abstrair a tecnologia empregada no processo e se concentrar nas informações trocadas. Em vez de dizer “o funcionário preenche os dados do comprador numa ficha de papel”, correspondendo a uma tecnologia manual, empregada normalmente em sistemas não informatizados ou “o comprador preenche seus dados na tela XYZ”, que corresponde a uma tecnologia informatizada, o analista deve registrar no caso de uso simplesmente “o comprador informa seus dados”. Esta última forma é independente de tecnologia ou interface e representa, portanto, a descrição da operação no seu nível essencial.

Porém, recomenda-se também que *sempre* seja deixado explícito quais dados são informados ou recebidos, para maior clareza do caso de uso. Assim, a forma final desse passo seria, por exemplo, “o comprador informa seu nome, CPF, telefone e endereço”.

Então, como descrever, por exemplo, o caso de uso “sacar dinheiro” de um caixa automático, sem entrar no nível da tecnologia de interface? Em vez de dizer que o comprador *passa o cartão magnético*, diz-se que ele *informa sua identificação*. Em vez de dizer que o sistema *imprime o extrato*, diz-se apenas que o sistema *apresenta o extrato*. Assim, eliminando as referências à tecnologia, fica-se apenas com a essência das informações. Isso abre caminho para que na atividade de projeto seja possível pensar em diferentes alternativas para implementar as operações e a interface que permitirá realizar um caso de uso.

Cabe ao analista, portanto, estudar os processos correntes da empresa e produzir uma versão essencial deles, correspondendo ao caso de uso expandido essencial. Depois, o projetista vai apresentar uma solução real para essa especificação baseada em uma ou mais tecnologias existentes.

5.2. Fluxo Principal

O *fluxo principal* é a principal seção de um caso de uso expandido. Ele é a descrição do processo quando tudo dá certo, ou seja, quando não ocorre nenhuma exceção. No caso do sistema de caixa automático, seria a situação em que o comprador informa corretamente sua identificação e senha, tem saldo na conta, há dinheiro suficiente na máquina etc. Já os fluxos alternativos, que serão discutidos mais adiante, correspondem ao tratamento das possíveis exceções e variantes identificadas pelo analista.

Uma das grandes dúvidas dos analistas que trabalham com casos de uso costuma ser decidir exatamente o que pode e/ou deve ser colocado como passo dos fluxos de um caso de uso expandido.

De fato, duas pessoas que descrevam o mesmo processo, se não utilizarem um bom padrão, quase sempre vão gerar uma sequência de passos diferente. Em uma livraria tradicional, um analista poderia fazer constar um passo em que o funcionário pergunta o nome do cliente. Outro analista poderia excluir esse passo e dizer que o cliente simplesmente chega ao balcão e se identifica.

A pergunta é: qual das duas abordagens está correta? Se ambas estão corretas, existem descrições incorretas?

Ambas estão corretas, mas uma delas é mais útil. O objetivo do padrão é incentivar os analistas a construírem versões corretas e semelhantes entre si, além de evitar as versões incorretas. Os conceitos de passos obrigatórios, complementares e impróprios existem para ajudar os analistas a estabelecerem essas distinções.

Todo caso de uso tem passos que são *obrigatórios*. Esses passos envolvem informações que passam dos atores para o sistema e do sistema para os atores; sem essas trocas de informação, o caso de uso não faz sentido ou não pode prosseguir. Esses passos devem constar em qualquer caso de uso expandido, e a falta deles faz com que o caso de uso esteja incorretamente descrito.

Outros passos, como “perguntar o nome do comprador” são opcionais ou *complementares*. Eles servem para contextualizar o caso de uso, mas não são fundamentais porque não passam nenhuma informação para dentro ou para fora do sistema.

Além disso, deve-se ter em mente que, como o caso de uso é uma descrição da interação entre os atores e o sistema, deve-se evitar descrever quaisquer processos internos que ocorram no sistema, como “o sistema armazena a informação no banco de dados”. Esses passos são considerados como *impróprios* ou não recomendados para a descrição do caso de uso.

5.2.1. Passos Obrigatórios

Na categoria de passos *obrigatórios* estão incluídos todos os passos que indicam de alguma forma que foi trocada informação entre o sistema e um ou mais atores (usuários), conforme ilustrado na Figura 5.1.

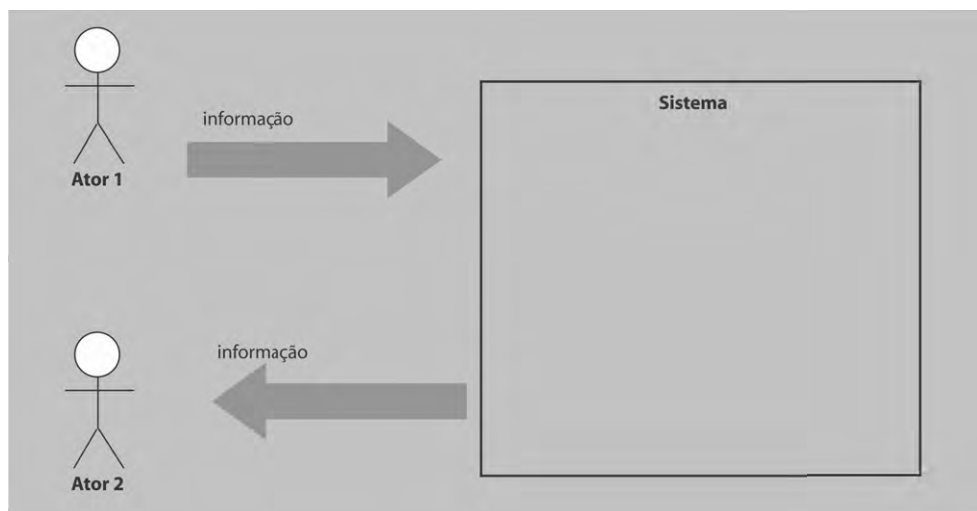


Figura 5.1: Passos obrigatórios implicam trocas de informação entre o sistema e os atores.

Assim, um caso de uso expandido para comprar livros deve obrigatoriamente conter os passos que indicam que o comprador se identifica, identifica os livros que deseja comprar, e o sistema informa o valor dos livros. Por que esses passos são obrigatórios? Porque sem essas informações nenhum sistema seria capaz de registrar adequadamente uma venda. Certamente não seria de grande ajuda um sistema que registrasse uma venda sem indicar quem foi o comprador ou quais foram os livros vendidos. Visto que essas informações são tão importantes para a conclusão do caso de uso, os passos da interação em que os atores passam essa informação ao sistema devem ser obrigatoriamente citados no caso de uso expandido. Também não seria

aceitável se o sistema não informasse o valor dos livros, pois, nesse caso, o comprador poderia pagar aquilo que bem entendesse, já que não haveria o “contrato” de venda.

Em uma descrição de caso de uso, a informação não surge do nada. Ela é transmitida dos atores para o sistema e vice-versa. A ausência de passos de interação que registrem essa troca de informação das fontes para os destinatários deixa os casos de uso sem sentido. A Figura 5.2 mostra um exemplo em que um caso de uso foi mal construído porque uma informação obrigatória foi omitida.

Caso de Uso (mal construído): Comprar Livros

1. O comprador informa seu CPF.
2. O sistema confirma a venda informando o valor total.

Figura 5.2: Um caso de uso em que falta pelo menos um passo obrigatório.

Esse caso de uso está incompleto porque uma venda necessitaria de mais informações do que as que foram trocadas entre o comprador e o sistema. Como o sistema poderia saber quais os livros a serem comprados se o comprador, que é quem detém essa informação, não a transmitiu?

Por outro lado, como o comprador poderia saber quais livros podem ser comprados se o sistema não lhe apresentar as possibilidades? Então, deve também haver um passo em que o sistema apresenta os livros que podem ser comprados, e o comprador seleciona um ou mais dentre eles.

Uma versão melhor desse caso de uso é descrita na Figura 5.3.

Caso de Uso: Comprar livros

1. O comprador informa sua identificação.
2. O sistema informa os livros disponíveis para venda (título, capa e preço).
3. O comprador seleciona os livros que deseja comprar.
4. O sistema informa o valor total dos livros e apresenta as opções de endereço cadastradas.

5. O comprador seleciona um endereço para entrega.
6. O sistema informa o valor do frete e total geral, bem como a lista de cartões de crédito já cadastrados para pagamento.
7. O comprador seleciona um cartão de crédito.
8. O sistema envia os dados do cartão e valor da venda para a operadora.
9. A operadora autoriza a venda.
10. O sistema informa o prazo de entrega.

Figura 5.3: Um caso de uso com fluxo principal completo.

Esse caso de uso também está bem construído porque não entra em detalhes sobre a tecnologia de interface. Quando o caso de uso diz que o sistema informa os livros disponíveis, há inúmeras maneiras tecnológicas para realizar essa tarefa: menus, listas, campos autocompletar, campos de pesquisa, sequências de consultas etc. Um bom caso de uso essencial não precisa mencionar essas opções. Elas são deixadas para a atividade de projeto.

Há um motivo muito claro para que o analista se preocupe em fazer constar no caso de uso toda troca de informação obrigatória: é que essa informação será usada mais adiante para estabelecer quais são as operações de sistema e consultas de sistema, ou seja, quais métodos devem ser implementados pelo sistema para realizar sua funcionalidade. Se essas informações não forem corretamente identificadas, métodos incompletos poderão ser implementados posteriormente, o que dará origem à necessidade de refazê-los mais adiante, quando o problema for identificado.

Além disso, essas informações servirão de base para a construção do modelo conceitual do sistema, a partir do qual toda a arquitetura do software vai ser definida.

Os passos obrigatórios em um caso de uso podem ser de dois tipos:

- a) *eventos de sistema*: são passos que indicam que alguma informação é passada dos atores para o sistema;
- b) *respostas de sistema*: são passos que indicam que alguma informação é passada do sistema para os atores.

Deve-se tomar especial cuidado com as respostas de sistema. Esses passos, para serem realmente obrigatórios, devem passar alguma informação do

sistema para os atores. Essa informação deve ser algo que os atores, a princípio, não tenham ou não possam inferir necessariamente sem consultar o sistema.

Por exemplo, o sistema tem de informar o valor total no caso de uso “Comprar livros”, caso contrário o comprador não saberá quanto vai pagar. Essa informação até poderia ser calculada pelo comprador se ele somasse os valores individuais de cada livro, mas o comprador não poderia ter certeza sobre quanto seria cobrado, devido a descontos e frete, sem que a livraria explicitamente o informasse disso; a responsabilidade de fornecer essa informação é do *sistema*.

Por outro lado, quando um comprador envia dados ao sistema, a interface pode emitir algum tipo de *feedback* para informar que os dados foram recebidos e corretamente processados. Mas esse retorno (normalmente uma mensagem do tipo “ok” ou “operação confirmada”) não constitui nova informação sobre livros, cartões ou endereços. É apenas um *feedback* de interface e, portanto, refere-se à tecnologia usada. Não sendo uma informação propriamente dita, não deve ser considerada como passo obrigatório.

Será interessante, para efeito de identificação de operações e consultas de sistema, que os passos do caso de uso que correspondem a eventos e respostas sejam claramente marcados. Sugere-se o marcador [IN] para eventos de sistema e [OUT] para respostas de sistema. Esses marcadores podem ser colocados logo após o número da linha do passo no caso de uso, como na Figura 5.4.

Caso de Uso: Comprar livros

1. [IN] O comprador informa sua identificação.
2. [OUT] O sistema informa os livros disponíveis para venda (título, capa e preço).
3. [IN] O cliente seleciona os livros que deseja comprar.
4. [OUT] O sistema informa o valor total dos livros e apresenta as opções de endereço cadastradas.
5. [IN] O cliente seleciona um endereço para entrega.
6. [OUT] O sistema informa o valor do frete e total geral, bem como a lista de cartões de crédito já cadastrados para pagamento.
7. [IN] O cliente seleciona um cartão de crédito.

8. [OUT]	O sistema envia os dados do cartão e valor da venda para a operadora.
9. [IN]	A operadora informa o código de autorização da venda.
10. [OUT]	O sistema informa o prazo de entrega.

Figura 5.4: Um caso de uso com eventos e respostas de sistema marcados nos respectivos passos.

Outra opção é escrever o caso de uso com colunas, onde uma coluna (mais à direita) representa as ações do sistema e as demais colunas representam as ações dos atores. O mesmo caso de uso da Figura 5.4 pode ser representado como na Figura 5.5. A tabela deve ser lida de cima para baixo e da esquerda para a direita. A coluna com o sistema deve ficar sempre na extremidade direita.

Caso de Uso: Comprar livros			
Passo	Operadora	Comprador	Sistema
1		Informa ao sistema sua identificação	Informa ao comprador os livros disponíveis para venda (título, capa e preço)
2		Seleciona os livros que deseja comprar	Informa ao comprador o valor total dos livros e apresenta as opções de endereço cadastradas
3		Seleciona um endereço para entrega	Informa ao comprador o valor do frete e total geral, bem como a lista de cartões de crédito já cadastrados para pagamento

4		Seleciona um cartão de crédito	Envia os dados do cartão e valor da venda para a operadora
5	Informa ao sistema o código de autorização da venda		Informa ao comprador o prazo de entrega

Figura 5.5: Um caso de uso em múltiplas colunas.

Nesse tipo de notação, fica mais claro que o sistema apenas reage às ações dos atores. Inclusive o sistema da operadora de cartão, que aqui é considerado um ator, age de forma autônoma em relação ao sistema Livir.

5.2.2. Passos Complementares

A segunda categoria de passos citada é a dos passos *complementares*, a qual consiste em todos aqueles passos que não apresentam informações trocadas entre o sistema e os atores, mas que ajudam a entender o contexto do caso de uso.

Esse tipo de passo corresponde, normalmente, à comunicação entre os atores (comunicação que não envolve o sistema, conforme ilustrado na Figura 5.6) ou à descrição de suas ações ou atitudes, que também não se configuram como envio de informação para o sistema, como “o comprador acessa o site da livraria”, “o comprador decide se compra os itens”, “o sistema pede ao comprador que se identifique” ou “o sistema informa que a venda foi concluída com sucesso”.

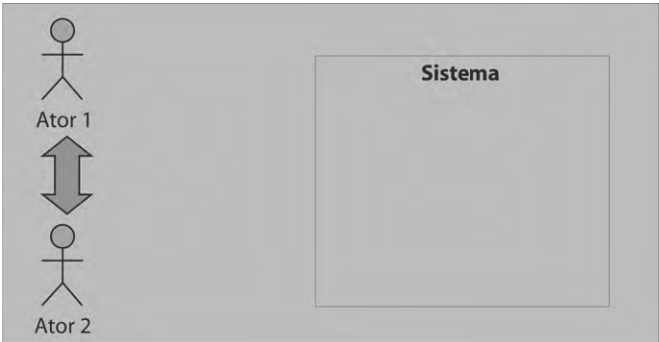


Figura 5.6: Passos complementares em um caso de uso referem-se a ações dos atores que não afetam o sistema.

Os passos complementares não são fundamentais no caso de uso essencial porque não correspondem a eventos nem respostas do sistema, já que não passam informação através da fronteira do sistema.

Alguns deles até poderão, na atividade de projeto, ser mapeados em operações de navegação na interface. Por exemplo, um caso de uso real de projeto poderia ter um passo como “o comprador seleciona a opção ‘iniciar compra’ no menu de opções”. Essa linha não seria necessariamente uma operação do sistema, pois, nesse momento, o comprador ainda não passou nenhuma informação ao sistema (como nome, CPF, título de livro etc.). A ação consiste, então, apenas em uma mudança de estado, que possivelmente corresponderá a uma navegação na interface (uma nova tela é aberta para que o comprador possa iniciar sua compra, por exemplo).

5.2.3. Passos Impróprios

A terceira categoria de passos refere-se aos passos *impróprios*, ou não recomendados. Nessa categoria incluem-se todos os processos considerados internos ao sistema (Figura 5.7).



Figura 5.7: Passos não recomendados em um caso de uso são aqueles que descrevem processamento interno do sistema.

O caso de uso é uma ferramenta para descrever a interação entre usuários e um sistema. Então, não é com esta ferramenta que se deve descrever o processamento interno do sistema.

Esses aspectos serão mais bem descritos na atividade de projeto com ferramentas adequadas (diagramas de comunicação ou sequência). Assim, não seria recomendado, por exemplo, a construção de um caso de uso que tivesse passos como “o sistema registra o nome do comprador no banco de dados” ou “o sistema calcula a média das vendas”, pois esses passos correspondem a processamento interno. Porém, um passo como “o sistema *apresenta* a

média das vendas” seria aceitável, pois indica troca de informação do sistema com o mundo exterior, e não apenas um processamento interno.

Na descrição dos casos de uso, o analista deve se concentrar em descrever a informação que é passada dos usuários para o sistema (por exemplo, o usuário *informa* os livros) e do sistema para os usuários (por exemplo, o sistema *apresenta* o valor total da compra). Deve-se, portanto, omitir quaisquer aspectos sobre o funcionamento interno do sistema. Pode-se dizer que o sistema exibe o valor total da venda, mas não se deve descrever como foi que ele calculou esse total, pois este é um processo interno. Em resumo, o sistema deve ser visto nessa fase ainda como uma caixa-preta.

Opcionalmente, o analista poderá fazer anotações sobre a forma como determinados cálculos são feitos (regras de negócio), mas espera-se que tais descrições estejam no documento de requisitos e não no caso de uso expandido.

A Figura 5.8 apresenta uma situação em que a descrição do caso de uso vai além do que seria recomendado.

Caso de Uso (mal construído): Encomendar livros

1. O sistema apresenta a lista de editoras.
2. O gerente seleciona uma editora.
3. O sistema calcula a média mensal de venda de cada livro disponibilizado.
4. O sistema apresenta a lista de livros disponíveis (ISBN, autor, título, preço, quantidade em estoque, e média mensal de venda).
5. O gerente seleciona os livros que deseja comprar na lista.
6. O sistema soma o preço de todos os livros para obter o total.
7. O sistema apresenta o preço total.
8. O gerente confirma a encomenda informando o código de acesso.
9. O sistema envia o pedido à editora.
10. A editora envia o número do pedido e o prazo de entrega.

Figura 5.8: Caso de uso com passos não recomendados.

O caso de uso contém todos os passos obrigatórios que deveriam existir, mas acrescenta dois passos impróprios (passos 3 e 6), que correspondem a processos internos e não a envio ou recebimento de informações.

O fato de que o sistema “calcula a média mensal” não afeta a interação com o usuário. Isso é processamento interno. Ao usuário interessa apenas saber que essa média será apresentada quando necessário. A forma como o sistema vai processar isso é um problema do sistema, não do usuário. Portanto, essa questão será definida mais adiante, nas atividades de projeto e não na atividade de análise.

5.3. Estilos de Escrita

O estilo de escrita dos passos de um caso de uso deve, sempre que possível, seguir um padrão do tipo “ator informa.../sistema informa...”. Evita-se escrever “o sistema solicita...” porque se deseja representar, nos passos, apenas fluxos de informação e não as eventuais solicitações que deram origem a esses fluxos, já que estas nem sempre existem, sendo passos *complementares* e não obrigatórios.

Deve-se tomar cuidado para não colocar, no fluxo principal do caso de uso, testes para verificar exceções. Evita-se, portanto, o uso de seletores como “se o usuário está com o cadastro em dia, *então* o sistema apresenta...”. Esse tipo de teste é desnecessário porque, como será visto adiante, os tratadores de exceção já estarão associados aos passos do fluxo principal. Então, se uma exceção como essa puder ocorrer, isso estará explícito no fluxo alternativo que corresponde ao tratador da exceção.

Evita-se colocar esses testes no fluxo principal para que ele não acabe se transformando em um complexo fluxograma em vez de uma lista simples de passos. Muitos “se/então” em um fluxo poderão deixá-lo obscuro, e o analista poderá não saber mais qual é exatamente a situação *esperada* (caminho feliz) do processo.

A única situação na qual um teste seria aceitável no fluxo é quando se tratar de um passo opcional, como, por exemplo, “se o usuário desejar, informa também seu celular”.

Outra situação a ser observada é evitar, sempre que não houver justificativa, a inclusão de eventos de sistema em sequência ([IN] seguidos de [IN]) ou respostas de sistema em sequência ([OUT] seguidas de [OUT]). A ideia, nesse caso, é normalizar a quantidade de passos que diferentes analistas poderiam atribuir a um caso de uso. Sem essa regra, um analista poderia escrever:

1. [IN] O comprador informa seu nome, CPF e telefone.

E outro analista poderia escrever:

1. [IN] O comprador informa seu nome.
2. [IN] O comprador informa seu CPF.
3. [IN] O comprador informa seu telefone.

A opção mais útil e correta sob esse ponto e vista é a primeira, até porque, no segundo caso, a sequência estrita exigiria que cada uma das informações fosse apresentada na ordem dos passos; uma vez que uma informação entrou, não se pode mais voltar atrás, a não ser que alguma estrutura explícita permita isso. Então, a primeira opção é mais compatível com a realidade da maioria dos sistemas de informação, que apresentam interfaces em que várias informações podem ser passadas de uma única vez e editadas enquanto não forem definitivamente repassadas ao sistema.

Justificam-se dois passos [IN] em sequência apenas se o primeiro passo puder causar uma exceção que, se ocorrer, deve impedir a execução do segundo passo. Um exemplo disso seria:

1. [IN] O comprador informa seu CPF.
2. [IN] O comprador informa o número, validade e bandeira de seu cartão de crédito.

Nesse exemplo, quando o comprador informa o CPF pode haver uma exceção, caso o CPF informado esteja incorreto ou se não houver ainda cadastro desse comprador no sistema. Então, a informação dos dados do cartão de crédito deve ser postergada até que o comprador se cadastre. Esse cadastramento pode ser feito na sequência alternativa que trata essa exceção, como será visto na seção seguinte.

5.4. Tratamento de Exceções em Casos de Uso

Depois de descrever o fluxo principal do caso de uso, o analista deve imaginar o que poderia dar errado em cada um dos passos descritos, gerando, dessa forma, os fluxos alternativos que tratam as exceções.

Uma *exceção* (no sentido usado em computação) não é necessariamente um evento que ocorra muito raramente, mas sim um evento que, se não for devidamente tratado, impede o prosseguimento do caso de uso.

Por exemplo, quando uma pessoa vai pagar uma conta, ela pode usar cheque, cartão ou dinheiro. Mesmo que apenas 1% das contas sejam recebidas em dinheiro contra 99% sendo pagas em cheque ou cartão, isso não significa

que o pagamento em dinheiro seja uma exceção, mas apenas uma opção pouco frequente. Porém, o fato de o comprador não ter meios para pagar a conta constitui uma exceção, pois isso impede que o caso de uso seja concluído (independentemente da frequência com que isso ocorre).

A Figura 5.9 apresenta um exemplo de caso de uso em que possíveis exceções foram identificadas.

Caso de Uso: Comprar livros

1. [IN] O comprador informa sua identificação.
2. [OUT] O sistema informa os livros disponíveis para venda (título, capa e preço).
3. [IN] O cliente seleciona os livros que deseja comprar.
4. [OUT] O sistema informa o valor total dos livros e apresenta as opções de endereço cadastradas.
5. [IN] O cliente seleciona um endereço para entrega.
6. [OUT] O sistema informa o valor do frete e total geral, bem como a lista de cartões de crédito já cadastrados para pagamento.
7. [IN] O cliente seleciona um cartão de crédito.
8. [OUT] O sistema envia os dados do cartão e valor da venda para a operadora.
9. [IN] A operadora informa o código de autorização.
10. [OUT] O sistema informa o prazo de entrega.

Exceção 1a: Comprador não cadastrado

- 1a.1 [IN] O comprador informa seu CPF, nome, endereço e telefone.

Retorna ao passo 1.

Exceção 5a: Endereço consta como inválido.

- 5a.1 [IN] O comprador atualiza o endereço.

Avança para o passo 6.

Exceção 9a: A operadora não autoriza a venda.

- 9a.1 [OUT] O sistema apresenta outras opções de cartão ao cliente.

- 9a.2 [IN] O cliente seleciona outro cartão.

Retorna ao passo 8.

Figura 5.9: Um caso de uso com eventos e respostas de sistema marcados nos respectivos passos.

Nota-se que a exceção em um caso de uso não é necessariamente algo que impeça que o caso de uso seja iniciado, mas normalmente algo que impede que ele seja concluído. No exemplo da Figura 5.9, o fato de o comprador não ter cadastro válido não o impediu de acessar o site do sistema e a tela na qual a identificação (CPF) é solicitada. Porém, a conclusão do caso de uso dependeria de ele ter um cadastro válido, o que não ocorre. Portanto, embora o caso de uso tenha iniciado, ele não pode ser concluído a não ser que essa exceção seja tratada.

Observa-se, na prática, que as exceções ocorrem apenas nos passos que correspondem a eventos de sistema [IN] porque, quando uma informação é passada ao sistema, ele, em muitos casos, realiza certas validações (O comprador é cadastrado? O endereço é válido? O limite de crédito não foi excedido?), que correspondem às restrições lógicas estabelecidas nos requisitos ou regras de negócio. A cada uma dessas regras corresponde uma exceção.

Cada exceção deve ser *tratada* por um fluxo alternativo, que corresponde a uma ramificação do fluxo principal. Um *tratador de exceção* tem pelo menos quatro partes:

- a) *identificador*, que consiste em: (1) o número da linha do fluxo principal (ou, eventualmente, de algum outro fluxo alternativo) em que a exceção ocorreu e (2) uma letra para identificar a própria exceção na linha. Por exemplo, na linha 1 do fluxo principal poderia haver exceções identificadas como 1a, 1b, 1c etc. Para a linha 2, as exceções seriam 2a, 2b, 2c etc.;
- b) *exceção*, que consiste em uma frase que explica qual regra foi violada, pois em uma mesma linha podem ocorrer diferentes tipos de exceções. Por exemplo, “comprador sem cadastro”, “comprador sem crédito” etc.;
- c) *ações corretivas*, que consistem em um fluxo alternativo, ou seja, uma sequência de ações que deveriam ser executadas para corrigir a exceção. As ações corretivas são numeradas sequencialmente e cada passo é prefixado pelo identificador da exceção. Por exemplo, a exceção 2a terá seus passos numerados como 2a.1, 2a.2 etc.;
- d) *finalização*, que consiste na última linha do fluxo alternativo que indica se e como o caso de uso retorna ao fluxo principal depois das ações corretivas.

Existem quatro formas básicas para finalizar a sequência de ações corretivas:

- a) *voltar ao início do caso de uso*, o que não é muito comum nem muito prático, na maioria das vezes, a não ser em sistemas que precisam receber uma sequência de dados em tempo real;
- b) *retornar ao início do passo que causou a exceção* e executá-lo novamente, o que é mais comum. Deve-se optar por essa forma quando o passo que causou a exceção eventualmente causar outras exceções diferentes, mesmo que uma delas já tenha sido tratada;
- c) *avançar para algum passo posterior*. Isso pode ser feito quando as ações corretivas realizam a operação que o passo ou a sequência de passos posterior deveria ter executado. Porém, deve-se verificar se novas exceções não poderiam ainda ocorrer no passo do fluxo anterior que originou a exceção;
- d) *abortar o caso de uso*. Nesse caso, não se retorna ao fluxo principal. O caso de uso não atinge seus objetivos. Se for necessário fazer alguma ação corretiva no sentido de desfazer registros intermediários, isso deve ser indicado nos passos do fluxo alternativo (essa forma de tratar uma exceção é conhecida como “pânico organizado”).

No caso de uso “Comprar livros” (Figura 5.9), no passo 1, quando o comprador informa o seu identificador (CPF), o sistema pode verificar que ele não possui cadastro (exceção 1a). Nessa eventualidade, o caso de uso não pode prosseguir a não ser que as ações corretivas sejam executadas.

Como foi dito, não se deve colocar essa verificação como uma condicional no fluxo principal (por exemplo, não se deve escrever “2. Se o comprador possui cadastro, *então* o sistema informa os livros disponíveis...”), mas como um fluxo alternativo.

Para cada exceção, deve-se tentar identificar possíveis ações corretivas. Não havendo possibilidade ou interesse em efetuar ações corretivas por parte do ator, o caso de uso será abortado.

Exceções genéricas, que podem ocorrer em qualquer passo de qualquer caso de uso, indiferentemente, como, por exemplo, o ator cancelar o processo ou faltar energia no sistema, não devem ser tratadas como exceções nos passos. Esse tipo de situação é tratada através de mecanismos genéricos. No caso do cancelamento, o sistema deverá ter um mecanismo de *rollback* geral.

Considerar essas exceções genéricas em cada passo de um caso de uso criaria um conjunto enorme de informações cujo resultado final é inócuo. Se, em qualquer passo de qualquer transação, o usuário pode efetuar um cancela-

mento, não é necessário dizer isso em cada passo de cada transação (que poderão ser centenas ou milhares), mas diz-se apenas uma vez de forma geral no documento de requisitos (seria um requisito suplementar do tipo: “A cada instante, qualquer transação em andamento pode ser cancelada pelo usuário”).

5.5. Variantes do Fluxo Principal

Admite-se, por princípio, que o fluxo principal seja uma sequência não ramificada e não aninhada de passos de interação. Porém, algumas vezes, pode ser útil representar o caso de uso de uma forma não tão plana.

O caso de uso “Comprar livros”, por exemplo, pode ter dois finais opcionais: no primeiro, a compra é finalizada, e, no segundo, o carrinho é guardado para que a compra seja continuada em outro momento.

Pode-se até pensar que seriam dois casos de uso distintos: preencher carrinho de compras e finalizar compra, em que cada caso de uso teria sua descrição específica. Porém, essa divisão soa um tanto artificial visto que o processo completo consiste em escolher os produtos e pagar por eles.

A opção de guardar o carrinho é apenas uma variante desse processo e não uma situação de interação diferente e desconexa. Guardar o carrinho para continuar uma compra em outro momento também não é uma exceção, mas uma opção do comprador. Não é uma condição que impeça que o caso de uso seja concluído com alguma informação produzida, pois o caso de uso apresenta resultado: o carrinho fica armazenado para uso posterior. Então, não se trata de exceção, mas de uma opção. Pode-se dizer que o fluxo principal desse caso de uso possui dois fluxos variantes. A Figura 5.10 ilustra essa situação.

Caso de Uso: Comprar livros

1. [IN] O comprador informa sua identificação.
2. [OUT] O sistema informa os livros disponíveis para venda (título, capa e preço) e o conteúdo atual do carrinho de compras (título, capa, preço e quantidade).
3. [IN] O comprador seleciona os livros que deseja comprar.
4. O comprador decide se finaliza a compra ou se guarda o carrinho:

4.1 Variante: Finalizar a compra.

4.2 Variante: Guardar carrinho.

Variante 4.1: Finalizar a compra

4.1.1. [OUT] O sistema informa o valor total dos livros e apresenta as opções de endereço cadastradas.

4.1.2. [IN] O comprador seleciona um endereço para entrega.

4.1.3. [OUT] O sistema informa o valor do frete e total geral, bem como a lista de cartões de crédito já cadastrados para pagamento.

4.1.4. [IN] O comprador seleciona um cartão de crédito.

4.1.5. [OUT] O sistema envia os dados do cartão e valor da venda para a operadora.

4.1.6. [IN] A operadora informa o código de autorização.

4.1.7. [OUT] O sistema informa o prazo de entrega.

Variante 4.2: Guardar carrinho

4.2.1 [OUT] O sistema informa o prazo (dias) em que o carrinho será mantido.

Exceção 1a: Comprador não cadastrado

1a.1 [IN] O comprador informa seu CPF, nome, endereço e telefone.

Retorna ao passo 1.

Exceção 4.1.2a: Endereço consta como inválido.

4.1.2a.1 [IN] O comprador atualiza o endereço.

Avança para o passo 4.1.2.

Exceção 4.1.6a: A operadora não autoriza a venda.

4.1.6a.1 [OUT] O sistema apresenta outras opções de cartão ao comprador.

4.1.6a.2 [IN] O comprador seleciona outro cartão.

Retorna ao passo 4.1.5.

Figura 5.10: Um caso de uso com variantes.

Como se pode notar, o passo 4 é complementar, porque o comprador não repassa ao sistema nenhuma informação nova, apenas decide sobre como

será a continuidade do caso de uso. Isso se traduzirá, na atividade de projeto de interface, em uma operação de controle de navegação e não em uma operação de sistema.

5.6. Casos de Uso Incluídos

Pode ser possível também que dois casos de uso ou mais tenham partes coincidentes. Por exemplo, vários casos de uso podem comportar uma subsequência de pagamento ou um caso de uso pode incluir outro caso de uso completo, como é o caso do cadastramento do comprador, que deve ser feito como sequência alternativa no caso de uso “Comprar livros”, caso o comprador não tenha cadastro.

Casos de uso completos podem então ser referenciados dessa forma. No caso dos CRUD, convém informar qual a operação indicada, quando for o caso. Por exemplo, o passo 1a.1 na Figura 5.10 poderia ser escrito como na Figura 5.11.

1a.1 Inclui <<CRUD>> Gerenciar Comprador.

Figura 5.11: Redação alternativa para um dos passos do caso de uso da Figura 5.10.

Nesse caso, apenas a operação de inserção do caso de uso CRUD pode ser executada para que o comprador insira seus dados.

Tratadores de exceção também podem ser referenciados dessa forma, desde que a ação de finalização seja coerente.

Quando um caso de uso inclui outro, pode-se relacioná-los com a associação de dependência estereotipada por <<include>>, como na Figura 5.12.

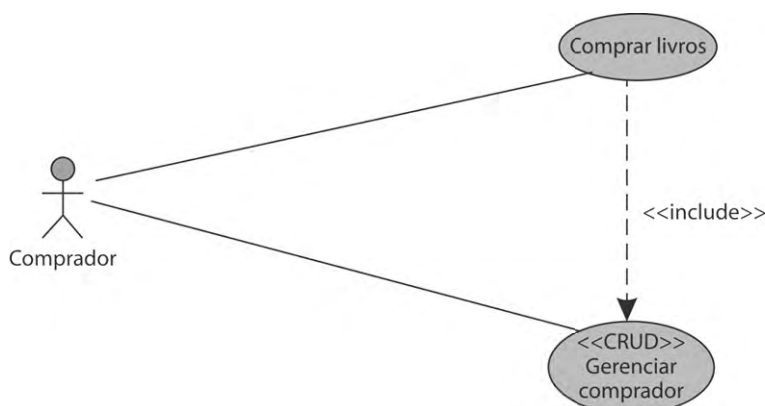


Figura 5.12: Um caso de uso que inclui outro.

Porém, as variantes e tratadores de exceção não têm o *status* de caso de uso, embora sejam representadas pelo mesmo símbolo nos diagramas de caso de uso da UML. Eles não são casos de uso porque não são processos completos, mas partes de outro processo. A rigor, nem deveriam aparecer nos diagramas de caso de uso de análise, mas se for absolutamente necessário mencioná-las, pode-se estereotipar as variantes com <<variant>> e os tratadores de exceção com <<exception>>, para que fique claro que não são processos autônomos, mas apenas partes reusáveis de outros processos.

Deve-se ter em conta, sempre, que o objetivo do caso de uso expandido na análise é o estudo do problema de interação dos atores com o sistema e não a estruturação de um algoritmo para descrever essa estrutura.

5.7. Cenários e Casos de Uso

Um caso de uso pode ser compreendido como uma descrição ou especificação geral que comporta um conjunto de diferentes *cenários*. Cada cenário é uma realização particular ou instância do caso de uso. Usualmente, considera-se que o caso de uso comporta um cenário principal (fluxo principal) e cenários alternativos. Porém, a noção de variantes de fluxo principal normalmente dá margem a dúvidas sobre o que deveria realmente ser um caso de uso. Por exemplo, existe um caso de uso “Comprar livros” com duas variantes em relação à finalização (guardar carrinho ou pagar compra) ou existem dois casos de uso?

Na verdade, o que importa nessa fase da análise é descobrir quais são as informações trocadas entre os atores e o sistema. Assim, não faz muita diferença em relação ao resultado final da análise se as operações são descobertas ou descritas em um único caso de uso com dois cenários alternativos ou em dois casos de uso com um único cenário cada.

A vantagem de unir cenários variantes em um único caso de uso é que, assim, não se precisa repetir a descrição daqueles passos que são coincidentes nos diferentes cenários. É a mesma vantagem que se tem ao fatorar as propriedades de suas classes em uma superclasse pelo uso de herança (Capítulo 7).

Porém, antes de decidir pela criação de casos de uso com variantes a partir da união de dois cenários semelhantes, o analista deve verificar se as sequências variantes efetivamente apresentam passos obrigatórios. Apenas as variantes que contêm passos obrigatórios devem ser consideradas.

5.8. Expansão de Casos de Uso Padrão

Durante o processo de análise não é necessário expandir os casos de uso estereotipados cujo padrão já é conhecido.

Os padrões *relatório* e *CRUD*, identificados pelos respectivos estereótipos <<rep>> e <<CRUD>>, são construções que terão sempre a mesma forma expandida, sendo necessário, portanto, apenas saber de antemão que forma é essa e usá-la nas fases posteriores do projeto, quando for o momento de gerar interfaces e código para essas construções.

Então, os modelos de expansão dos casos de uso padrão apresentados nas subseções seguintes servem apenas para que se tenha uma referência de um possível formato para tais casos de uso. Pequenas variações podem ser possíveis dependendo da interpretação que se tenha das operações ou das decisões sobre *look-and-feel* do sistema.

5.8.1. Relatório Expandido

Um relatório é um caso de uso simples que comporta um acesso a dados já armazenados no sistema. As informações passadas pelo ator no início do caso de uso são apenas parâmetros para filtragem do relatório. A Figura 5.13 apresenta o formato geral de um relatório expandido.

```
Caso de Uso: <<rep>> Emitir relatório de ...  
1. O usuário informa os parâmetros x, y, z, ...  
2. O sistema apresenta os dados d1, d2, d3, agrupados  
   por, a1, a2, a3, e ordenados por o1, o2, o3, ...
```

Figura 5.13: Um *template* para relatório expandido.

A Figura 5.14 apresenta um exemplo concreto, ou seja, uma instanciação para o *template* da Figura 5.13 aplicado ao exemplo do sistema Livir.

```
Caso de Uso: <<rep>> Emitir relatório de vendas por título  
1. O usuário informa mês e ano.  
2. O sistema informa os títulos vendidos no mês com a  
   quantidade de livros vendidos para cada título em or-  
   dem decrescente pela quantidade.
```

Figura 5.14: Um relatório expandido.

Praticamente todas as informações sobre como gerar os relatórios já devem aparecer nos requisitos, de onde se conclui não ser produtivo apenas repeti-las aqui sob outro formato. Porém, se o analista quiser padronizar a apresentação das funções do sistema na forma de casos de uso, poderá fazê-lo, caso tenha tempo.

A quantidade de casos de uso <<rep>> que o sistema vai ter depende de quais são os relatórios. Sempre que a parametrização permitir, deve-se agrupar relatórios. Não faz sentido, por exemplo, ter um caso de uso para vendas em janeiro e outro para vendas em fevereiro. É um relatório só. O mês de referência deve ser um parâmetro.

Por outro lado, não é recomendável agrupar relatórios de natureza ou formato diferentes, como por exemplo, relatório de vendas por título e relatório de vendas por comprador. Os formatos de saída e parâmetros de entrada são distintos nos dois casos. Dessa forma, esses dois relatórios devem ser representados efetivamente por dois casos de uso <<rep>> distintos.

5.8.2. CRUD Expandido

Da mesma forma que os relatórios, os CRUD também constituem casos de uso que têm uma forma padrão. Esse caso de uso corresponde à execução opcional de qualquer uma das quatro operações de gerenciamento ou cadastro, que são: inclusão, exclusão, consulta e alteração.

Deve-se considerar que se trata de quatro variantes, já que não há operação dentre essas quatro que possa ser considerada “caminho feliz” ou “exceção”. São operações distintas, agrupadas mais por conveniência por atuarem sobre uma mesma entidade do que por semelhança nos seus passos. Daí o fluxo principal consistir apenas em uma decisão sobre qual variante usar. A Figura 5.15 apresenta um possível *template* para esse tipo de caso de uso.

Caso de Uso: <<CRUD>> Gerenciar ...

1. O usuário escolhe a operação:
 - 1.1 Variante “inserir”.
 - 1.2 Variante “consultar”.
 - 1.3 Variante “alterar”.
 - 1.4 Variante “excluir”.

Variante 1.1: Inserir

1.1.1 O usuário informa: ...

Variante 1.2: Consultar

1.2.1 O sistema apresenta uma lista de

1.2.2 O usuário seleciona um elemento da lista.

1.2.3 O sistema apresenta ... do elemento selecionado.

Variante 1.3: Alterar

1.3.1 Inclui Variante 1.2

1.3.2 O usuário informa novos valores para ...

Variante 1.4: Excluir

1.4.1 O sistema apresenta uma lista de ...

1.4.2 O usuário seleciona um elemento da lista para excluir.

Exceção 1.1.1a Inclusão fere regra de negócio.

1.1.1a.1 O sistema informa a regra que impede a inclusão.

1.1.1a.2 Retorna ao passo 1.1.1 informando novos dados.

Exceção 1.3.2a Alteração fere regra de negócio.

1.3.2a.1 O sistema informa a regra que impede a alteração.

1.3.2a.2 Retorna ao passo 1.3.2 informando novos dados.

Exceção 1.4.2a Exclusão fere regra estrutural ou de negócio.

1.4.2a.1 O sistema informa a regra que impede a exclusão.

1.4.2a.2 Retorna ao passo 1.4.2 para selecionar um novo elemento.

Figura 5.15: Um *template* para CRUD expandido.

Observa-se nesse *template* que a variante 1.3 inclui os passos da variante 1.2. Isso significa que a variante 1.3 na verdade tem quatro passos, identificados no *template* por 1.2.1, 1.2.2, 1.2.3 e 1.3.2, ou seja, o passo 1.3.1 expande-se em 1.2.1, 1.2.2 e 1.2.3.

A forma de tratamento da exceção 1.4.2a (exclusão) abordada aqui é a de impedir que a ação seja executada. No Capítulo 8 esse tema será retomado, e outras duas abordagens possíveis para tratar a exceção de exclusão serão discutidas.

A Figura 5.16 apresenta um exemplo concreto de caso de uso CRUD expandido.

Caso de Uso: <<CRUD>> Gerenciar comprador.

1. O usuário escolhe a operação:

- 1.1 Variante "inserir".
- 1.2 Variante "consultar".
- 1.3 Variante "alterar".
- 1.4 Variante "excluir".

Variante 1.1: Inserir

1.1.1 O usuário informa: nome, CPF, endereço e telefone do comprador.

Variante 1.2: Consultar

- 1.2.1 O sistema apresenta uma lista de CPF e nome ordenada pelo nome.
- 1.2.2 O usuário seleciona um elemento da lista.
- 1.2.3 O sistema apresenta nome, CPF, endereço e telefone do comprador selecionado.

Variante 1.3: Alterar

- 1.3.1 Inclui Variante 1.2
- 1.3.2 O usuário informa novos valores para nome, CPF, endereço e telefone.

Variante 1.4: Excluir

- 1.4.1 O sistema apresenta uma lista de CPF e nome ordenada pelo nome.
- 1.4.2 O usuário seleciona um elemento da lista para excluir.

Exceção 1.1.1a e 1.3.2a CPF já cadastrado

- 1.1.1a.1 O sistema informa que o CPF já está cadastrado.
- 1.1.1a.2 Retorna ao passo 1.

Exceção 1.4.2a O comprador tem compras cadastradas em seu nome.

- 1.4.2a.1 O sistema informa que é impossível excluir o comprador, pois ele já tem compras em seu nome.
- 1.4.2a.2 O caso de uso é abortado.

Figura 5.16: Um caso de uso CRUD expandido.

Nota-se na figura 5.16 que a exceção “CPF já cadastrado” serve a dois passos: 1.1.1 e 1.3.2.

5.9. Outras Seções de um Caso de Uso Expandido

Desde que Jacobson (1992) criou o conceito de casos de uso, várias versões de formato têm sido apresentadas. Cada uma das propostas apresenta diferentes elementos. Pode-se considerar que as seções “fluxo principal” e “fluxos alternativos” são fundamentais em qualquer descrição de caso de uso expandido. Porém, outras seções podem ser incluídas se o analista sentir necessidade: atores, interessados, precondições, pós-condições de sucesso, requisitos correlacionados, variações tecnológicas e questões em aberto, apenas para citar alguns exemplos.

5.9.1. Atores

A seção “atores” lista quais os tipos de entidades do mundo real que interagem com o sistema através do caso de uso. Atores podem ser tipos de pessoas como compradores, fornecedores, vendedores, operadores etc.

Atores também podem ser classes de sistemas externos ao sistema sendo desenvolvido, mas que interagem com ele. Por exemplo, se for necessário autorizar pagamento com cartão de crédito através do sistema da empresa emissora do cartão, esse sistema externo poderá ser considerado como um ator.

Atores humanos ou sistemas externos interferem no sistema e recebem informações dele através de uma interface.

No caso de atores humanos, as informações são passadas para o sistema através de dispositivos de entrada de dados, como teclado, mouse ou leitores especiais. O recebimento de informações do sistema pode se dar através de interfaces, como monitor de vídeo, impressora ou outros dispositivos especializados.

A comunicação com atores que são sistemas externos se dá usualmente através da rede. Nesse caso, a interface de comunicação é a rede, através da qual o sistema envia informações aos atores e aguarda que esses atores respondam à comunicação, o que corresponde à ativação de alguma operação de sistema no sistema original.

Não se deve confundir a ideia de sistemas externos (atores) com sistemas internos usados como módulos na implementação do sistema de informação. Um sistema gerenciador de banco de dados, usado para implementar a persistência das classes do sistema sendo desenvolvido, por exemplo, não é um ator, mas um módulo dentro da arquitetura interna do sistema. As regras a seguir podem ajudar a identificar apropriadamente os sistemas externos que seriam efetivamente atores:

- a) *sistemas atores são sistemas de informação completos* e não apenas bibliotecas de classes ou módulos de programas, como sistemas gerenciadores de banco de dados ou bibliotecas de classes de interface. Esses sistemas detêm algum tipo de informação que pode ser trocada com o sistema sendo desenvolvido;
- b) *sistemas atores estão fora do escopo de desenvolvimento do sistema atual*, ou seja, o analista e sua equipe não terão necessariamente acesso ao projeto interno desses sistemas nem a possibilidade de alterar suas funções, devendo adequar a comunicação entre o sistema em desenvolvimento e o sistema ator às características do sistema ator, visto que este não pode, a princípio, ser modificado.

5.9.2. Interessados

Nem sempre apenas os atores são interessados em um caso de uso. Outros setores da empresa poderão ter interesse nos resultados produzidos pelo caso de uso. Por exemplo, em um caso de uso de venda de livros, os atores são o comprador e a operadora de cartão. Mas os resultados desse caso de uso poderão interessar ao departamento de estoque e ao departamento de contabilidade da empresa, pois os resultados de uma venda afetam tanto a quantidade de livros em estoque quanto a quantidade de dinheiro em caixa ou a receber. Assim, mesmo que esses departamentos não sejam participantes diretos no caso de uso, podem ser listados como interessados.

A utilidade de listar tais elementos em um caso de uso reside no fato de que um caso de uso deve procurar satisfazer todos os interessados. Assim, essa documentação poderá ser útil para lembrar ao analista algumas informações que precisam ser armazenadas, processadas ou transmitidas, para que essas expectativas possam ser satisfeitas.

5.9.3. Precondições

Por definição, precondições são fatos considerados verdadeiros antes do início do caso de uso. Não se deve confundir as precondições com as exceções, visto que estas últimas não são necessariamente verdadeiras antes do início do caso de uso. As exceções podem ocorrer durante a execução justamente porque não se pode garantir que elas não ocorram. Não é possível, por exemplo, garantir que o comprador terá dinheiro para pagar a dívida antes de iniciar o caso de uso. Portanto, isso é uma exceção. Entretanto, é possível assumir que um comprador não poderá em hipótese alguma comprar um livro que a livraria não tenha disponibilizado no catálogo. Essa disponibilização pode ser então considerada como uma precondição.

Como as pre-condições são dadas como verdadeiras antes do início do caso de uso, resulta que elas não serão testadas durante a execução do caso de uso. Ou seja, as precondições, dessa forma, não gerariam exceções. Simplesmente seria impossível iniciar o caso de uso se a precondição fosse falsa.

5.9.4. Pós-condições de Sucesso

As pós-condições estabelecem normalmente os resultados do caso de uso, ou seja, o que será verdadeiro após sua execução. Por exemplo, o caso de uso “Comprar livros” pode ter como pós-condições os seguintes resultados: “foi criado um registro da venda dos livros para o comprador” e “o setor de entregas foi notificado da venda”.

Os resultados de um caso de uso podem ser bem variados em sua natureza, o que difere bastante das pós-condições de operações de sistema, que serão estudadas no Capítulo 7, e que são bem mais formais.

5.9.5. Requisitos Correlacionados

Quando a análise produz um documento estruturado de requisitos (iniciado normalmente na fase de concepção e incrementado ao longo da fase de elaboração), pode ser útil correlacionar esses requisitos aos casos de uso.

A correlação entre requisitos e casos de uso permite ao analista perceber se ainda existem requisitos não abordados.

Para simplificar o processo de associar um requisito a um caso de uso, usualmente coloca-se o código alfanumérico de cada requisito na seção cor-

respondente do caso de uso ou usam-se relações de rastreabilidade (setas tracejadas com o estereótipo <<trace>>).

5.9.6. Variações Tecnológicas

Um caso de uso de análise deve ser descrito no nível essencial e, portanto, não deve tratar de aspectos tecnológicos. Porém, algumas vezes, pode ser interessante registrar, para a atividade de projeto, possíveis variações tecnológicas que poderiam ser utilizadas para realizar o caso de uso.

Por exemplo, o passo do caso de uso Comprar livros, que corresponde à identificação do comprador, pode ter como variações tecnológicas a digitação do CPF ou do nome do comprador em um campo apropriado ou outro código qualquer. Se essas possibilidades estiverem sendo consideradas para o desenvolvimento do sistema, então podem ser listadas na seção “variações tecnológicas”.

5.9.7. Questões em Aberto

Muitas vezes, o analista, trabalhando sem a presença do cliente, não sabe como decidir sobre determinado assunto que pode depender de políticas da empresa. Por exemplo, se o usuário pode pagar a dívida a prazo ou se existem promoções para usuários que compram certa quantidade de livros, e assim por diante.

Essas dúvidas devem ser documentadas na seção “questões em aberto” para serem resolvidas no momento em que o cliente estiver disponível. No final da atividade de análise, espera-se que todas as questões em aberto tenham sido resolvidas e incorporadas à descrição do caso de uso expandido.

Diagramas de Sequência de Sistema

Na atividade de análise, o texto dos casos de uso expandidos terá basicamente duas utilizações:

- a) como fonte de informação para encontrar conceitos para o modelo conceitual (Capítulo 7);
- b) como fonte de informação para encontrar as *operações e consultas de sistema*, que darão origem aos métodos que fazem a interface do sistema com o mundo externo (Capítulo 8).

Operações de sistema são métodos que são ativados a partir de um *evento de sistema*, ou seja, como resposta a uma ação de um usuário. As operações de sistema, por definição, indicam um fluxo de informações do exterior para o interior do sistema e, portanto, de alguma forma, elas alteram as informações gerenciadas pelo sistema.

Consultas de sistema são métodos que correspondem à simples verificação de informação já armazenada. Essa informação pode ser apresentada exatamente como está ou modificada pela aplicação de funções (p. ex., média, total etc.). Mas, por definição, uma consulta de sistema não deve ser responsável por inserir, remover ou alterar informações armazenadas.

Essa separação entre consulta e operação é um princípio antigo em engenharia de software (Meyer, 1988) e justifica-se por permitir melhor reusabi-

lidade do código. Uma consulta que altera dados é menos coesa do que uma consulta sem efeitos colaterais e uma operação que não retorna dados.

Pode-se definir que as operações e consultas de sistema, em conjunto, correspondem à totalidade das funções possíveis do sistema, ou seja, à funcionalidade efetiva total do sistema.

Os casos de uso são excelentes fontes para encontrar operações e consultas de sistema. Nos casos de uso encontram-se operações de sistema a partir da observação de ações do usuário que produzem modificações no sistema (possivelmente estarão marcadas com [IN]), ou seja, as ações que levam informação dos atores para o sistema. Já as consultas de sistema são identificadas por passos que trazem informação do sistema para os atores (possivelmente marcadas por [OUT]).

6.1. Elementos do Diagrama de Sequência

A UML possui um diagrama que pode ser útil para representar a sequência dos eventos do sistema em um cenário de um caso de uso (Figura 6.1). O diagrama de sequência tem como elementos *instâncias de atores*, representados por figuras humanas esquematizadas, e instâncias que representam elementos do sistema. Nessa primeira versão do diagrama, apenas a interface do sistema (camada de aplicação) estará representada.

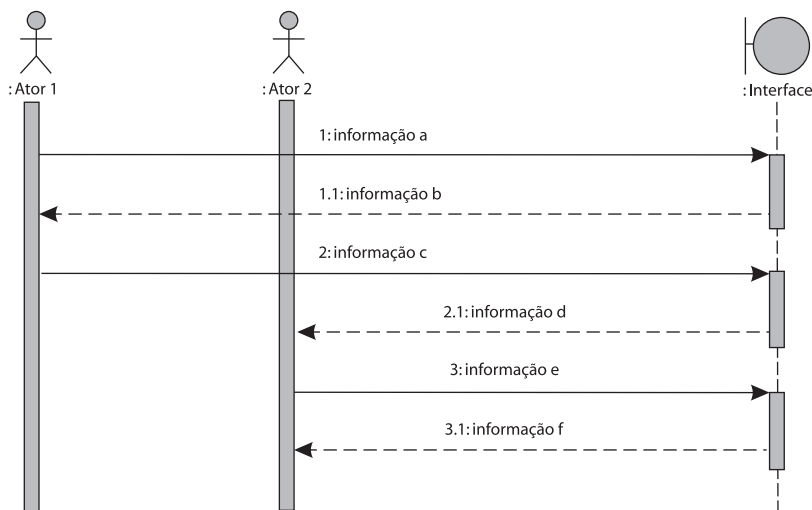


Figura 6.1: Diagrama de sequência de sistema.

As mensagens retornadas pelo sistema são tracejadas porque o sistema apenas reage aos atores, e a mensagem tracejada representa então esse mero retorno de informação a partir de um estímulo provocado por um dos atores. Da mesma forma, a numeração das mensagens pode ser diferente da numeração do caso de uso, visto que os retornos são subordinados à mensagem original. Assim, se o caso de uso numera os passos como 1, 2, 3, 4... , o diagrama de sequência poderá ter os passos equivalentes numerados com 1, 1.1, 2, 2.1....

Em relação à numeração, o caso de uso multicolumnas pode ser mais apropriado, pois a cada linha dele corresponde uma mensagem de um ator para o sistema, e a resposta do sistema vem subordinada na mesma linha. Assim, a correspondência entre números de linha do caso de uso multicolumnas e do diagrama de sequência de sistema será mais direta.

Como a atividade de análise não considera ainda os objetos internos ao sistema, será necessário representar o sistema como um único objeto, do tipo caixa-preta. Nesse caso, usa-se o símbolo de interface (Figura 6.1), conforme proposto por Jacobson *et al.* (1992). Um ator só pode se comunicar diretamente com a aplicação através de sua interface.

Atores, interfaces e outros elementos possuem, no diagrama de sequência, uma linha de tempo, representada pelas linhas verticais, onde os eventos podem ocorrer. Quando a linha está tracejada, o ator ou sistema está inativo. Quando a linha está cheia, isso significa que o ator ou sistema está ativo (operando ou aguardando o resultado de alguma operação). Atores humanos estão sempre ativos.

As linhas horizontais representam o fluxo de informação. Existem três tipos de envio de informação nesse diagrama:

- a) entre atores (comunicação entre atores, correspondendo a passos complementares do caso de uso expandido);
- b) dos atores para o sistema (*eventos de sistema* do caso de uso expandido);
- c) do sistema para os atores (*respostas do sistema* do caso de uso expandido).

Os envios de informação do tipo “a” não pertencem ao escopo do sistema e apenas são úteis para ilustrar como a informação é trocada entre os atores.

Uma coisa para ter em mente quando se constrói esses diagramas é que a informação normalmente não é criada durante esses processos, mas apenas transferida ou transformada. Um ator ou sistema detém alguma informação

e, para realizar o processo, ele terá de passar essa informação adiante. Para realizar a venda de um livro, é necessário que o sistema tenha o registro da identificação do comprador que está efetuando a compra e o identificador do livro que está sendo vendido. Porém, quem detém essa informação é o comprador. O sistema detém o cadastro de todos os compradores, mas não sabe, até que ele se identifique, quem é o comprador que está nesse momento comprando um livro.

O diagrama de sequência pode ser construído para o fluxo principal do caso de uso e também para alguns fluxos alternativos com passos obrigatórios. Porém, o mais importante nesse momento ainda não é especificar as sequências, mas saber *quais* são as informações repassadas dos atores para o sistema e vice-versa. O analista deve então construir um catálogo com todas as operações e consultas de sistema identificadas nos fluxos principais e nos fluxos alternativos. Mais adiante, ainda no processo de análise, essas informações serão usadas para definir os contratos de operação e consulta de sistema que indicam como o sistema transforma a informação. Ferramentas CASE poderão construir esse catálogo automaticamente indicando a implementação de métodos em uma classe chamada *controladora de sistema*.

6.2. Representação de Casos de Uso Expandidos como Diagramas de Sequência de Sistema

O diagrama de sequência de sistema é uma forma de sistematizar o caso de uso expandido e, assim, refiná-lo para obter mais detalhes sobre o funcionamento do sistema.

A representação do caso de uso em um diagrama de sequência de sistema é feita em duas etapas:

- a) representação dos passos do caso de uso como troca de informações entre os atores e a interface do sistema;
- b) representação de operações e consultas de sistema como troca de mensagens entre a interface e a *controladora-fachada* da camada de domínio do sistema.

A primeira etapa é simples: a cada passo identificado com [IN] equivale um envio de informação de um ator para a interface do sistema, e a cada passo [OUT] equivale um envio de informação do sistema para um ator (Figura 6.2).

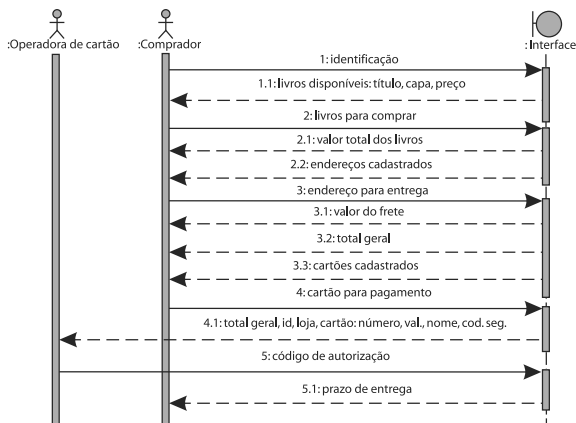
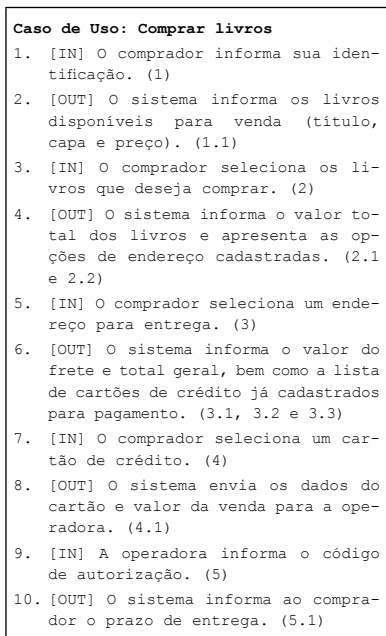


Figura 6.2: Um caso de uso e sua representação como diagrama de sequência de sistema (os números equivalentes do diagrama de sequência estão marcados entre parênteses no caso de uso para facilitar sua identificação).

Ao sistematizar os passos do caso de uso como envios de informação de atores para o sistema e vice-versa, o analista poderá, entre outras coisas, se dar conta de informações faltantes no caso de uso, como, por exemplo, o envio da identificação da loja no passo 8, conforme a Figura 6.2.

6.3. Ligação da Interface com o Domínio

Os eventos de sistema representam ações que o ator efetua contra a interface do sistema. Quando se usa uma interface Web, por exemplo, essas ações consistem no preenchimento de formulários e apertar de botões em páginas Web. Não são, ainda, operações propriamente ditas.

As *operações e consultas de sistema* são procedimentos computacionais, que são executados em função de um evento ou resposta de sistema. Trata-se agora de um componente do sistema que chama outro. No caso, é a interface que envia uma solicitação de execução de operação ou consulta de sistema para a camada de domínio, a qual é responsável pela execução de toda a lógica de acesso e transformação dos dados.

A camada de domínio é representada por sua controladora-fachada, uma instância de uma classe que implementa todas as operações e consultas de sistema que devem ser acessíveis a partir de uma determinada interface ou um conjunto de interfaces.

Os envios de mensagem entre a interface e a controladora são determinados a partir de um exame dos eventos e respostas de sistema existentes entre os atores e a interface. Assim:

- um evento de sistema, quando informa dados que o sistema deverá *armazenar*, corresponde inicialmente a uma operação de sistema, conforme a Figura 6.3;
- um evento de sistema que apenas envia dados que servirão de *parâmetro* para uma resposta de sistema não gera necessariamente operação de sistema, conforme a Figura 6.4, mas os parâmetros de uma consulta de sistema;
- uma resposta de sistema, para ser obtida, necessita que tenha sido executada (antes) uma consulta de sistema, conforme a Figura 6.4.

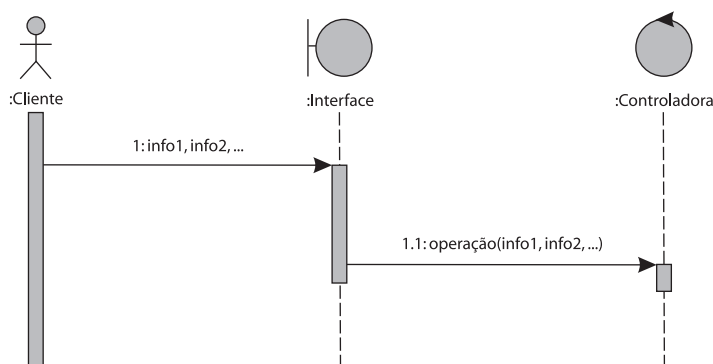


Figura 6.3: Um evento de sistema que tem como consequência uma chamada de operação de sistema na controladora.

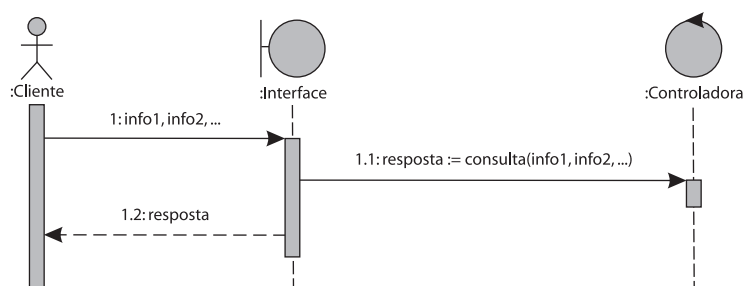


Figura 6.4: Uma resposta de sistema (passo 1.2) exige uma consulta (passo 1.1) à controladora para ser obtida, possivelmente com parâmetros (obtidos no passo 1).

Nem sempre uma resposta de sistema exige parâmetros. Nesse caso, o passo 1 da Figura 6.4 não existiria. Mas a consulta de sistema seria ativada por algum outro evento de interface (qualquer ação do ator, passagem de tempo ou mesmo a inicialização da interface).

Essas regras de derivação de operações e consultas de sistema a partir de eventos e respostas de sistema são apenas uma primeira aproximação do projeto da camada de interface. Posteriormente, técnicas de modelagem e decisões de projeto poderão alterar a forma como essas operações são chamadas.

Existem, portanto, nos diagramas de sequência de sistema, quatro tipos de envio de mensagens:

- a) *evento de sistema*: é uma ação realizada por um ator que envia alguma informação ao sistema. No diagrama é representado por uma seta do ator para a interface;
- b) *resposta do sistema*: informação que o sistema repassa aos atores, representada no diagrama como uma seta tracejada da interface para os atores;
- c) *operação do sistema*: uma chamada de método que o sistema executa internamente em resposta a um evento do sistema. A operação do sistema deve, por definição, alterar alguma informação armazenada. No diagrama é representada por uma seta da interface para a controladora rotulada com uma chamada de operação;
- d) *consulta do sistema*: é uma chamada de método cuja execução faz com que o sistema retorne alguma informação que interessa aos atores. As consultas não devem alterar os dados armazenados no sistema, mas apenas retornar dados de uma forma apropriada ao usuário. No diagrama, as consultas são representadas por setas da interface para a controladora rotuladas com uma chamada de função e com valor de retorno explícito. Também seria possível representar os retornos como setas tracejadas da controladora para a interface, mas essa opção gera mais elementos no diagrama de sequência, deixando-o mais complexo, e deve ser evitada.

Também é possível que os diagramas apresentem comunicação entre os atores, representada por setas de um ator para outro, mas essas informações não geram nenhum tipo de consequência direta no sistema.

A regra que exige que as operações não retornem dados (o que equivaleria a uma consulta com efeito colateral) tem uma exceção aceitável consagrada pelo uso e pela praticidade. Se usada de forma controlada, essa exceção não

prejudica a coesão dos métodos. Trata-se da *criação de elementos conceituais*, correspondendo, por exemplo, ao *create* do padrão *CRUD*. Quando uma operação de sistema *cria* um elemento conceitual, admite-se que ela retorne uma *referência* para o elemento criado, conforme pode ser visto na primeira operação de sistema (1.1) da Figura 6.5, que retorna o identificador de uma compra recém-criada.

6.4. Estratégias Statefull e Stateless

Quando se faz o projeto do diagrama de sequência, cada informação é repassada pelos atores para a interface apenas uma vez. Porém, no nível seguinte, várias operações e consultas de sistema podem necessitar da mesma informação. Nesse ponto, o projetista deve decidir se vai considerar que a controladora possui memória temporária para essas informações (estratégia *statefull*) ou se ela é desprovida de memória (estratégia *stateless*), situação na qual cada vez que uma operação ou consulta necessitar de uma informação deverá recebê-la explicitamente da interface.

A Figura 6.5 mostra como ficaria o diagrama completo, feito a partir do exemplo da Figura 6.2, com o uso da estratégia *stateless*. A informação é passada pelo ator à interface apenas uma vez, mas, cada vez que uma operação de sistema necessita da informação, ela deve ser enviada novamente.

Em relação à Figura 6.2, esse diagrama também apresenta mais algumas diferenças. Em primeiro lugar, os descritores das informações passadas dos atores para a interface e vice-versa passaram a ser apresentados como identificadores. Cada informação individual corresponde a um identificador (uma expressão alfanumérica iniciando com letra e sem espaços). Em segundo lugar, o envio de uma sequência de informações (identificadores dos livros) passa a ocorrer dentro de uma estrutura de repetição (*loop*), de forma que cada chamada de operação e consulta de sistema corresponda a uma operação individual que possa ser efetivamente programada.

Como se pode ver nesse diagrama, *idComprador* e *idCompra* são passados explicitamente a todas as operações e consultas que precisam desses parâmetros.

No caso da estratégia *statefull*, assume-se que a controladora de alguma maneira possa lembrar os parâmetros já passados. Não se trata de informação persistente, ou seja, informação que vai ser armazenada no banco de dados ou estrutura equivalente, mas de informações temporárias (por exemplo, qual o

comprador corrente, qual a compra atual etc.) que ficam disponíveis apenas durante a execução da transação representada pelo caso de uso. A Figura 6.6 apresenta a mesma situação da Figura 6.5, mas desta vez com a estratégia *statefull*. Observa-se que, entre outras coisas, não é mais necessário retornar o identificador da nova compra, pois a controladora vai lembrar qual é.

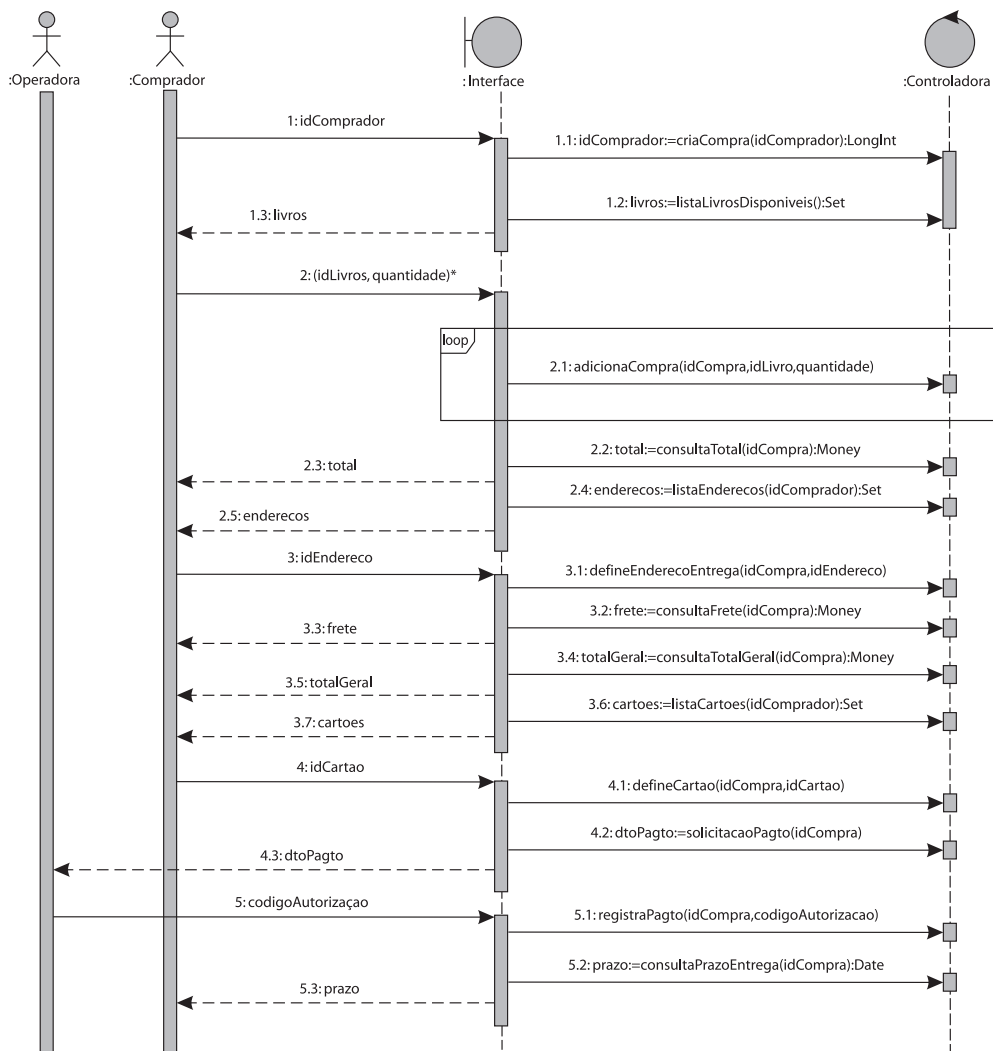


Figura 6.5: Diagrama de sequência completo com estratégia *stateless*.¹

¹ O significado da sigla “dto” nesta figura será explicado na seção 6.6.

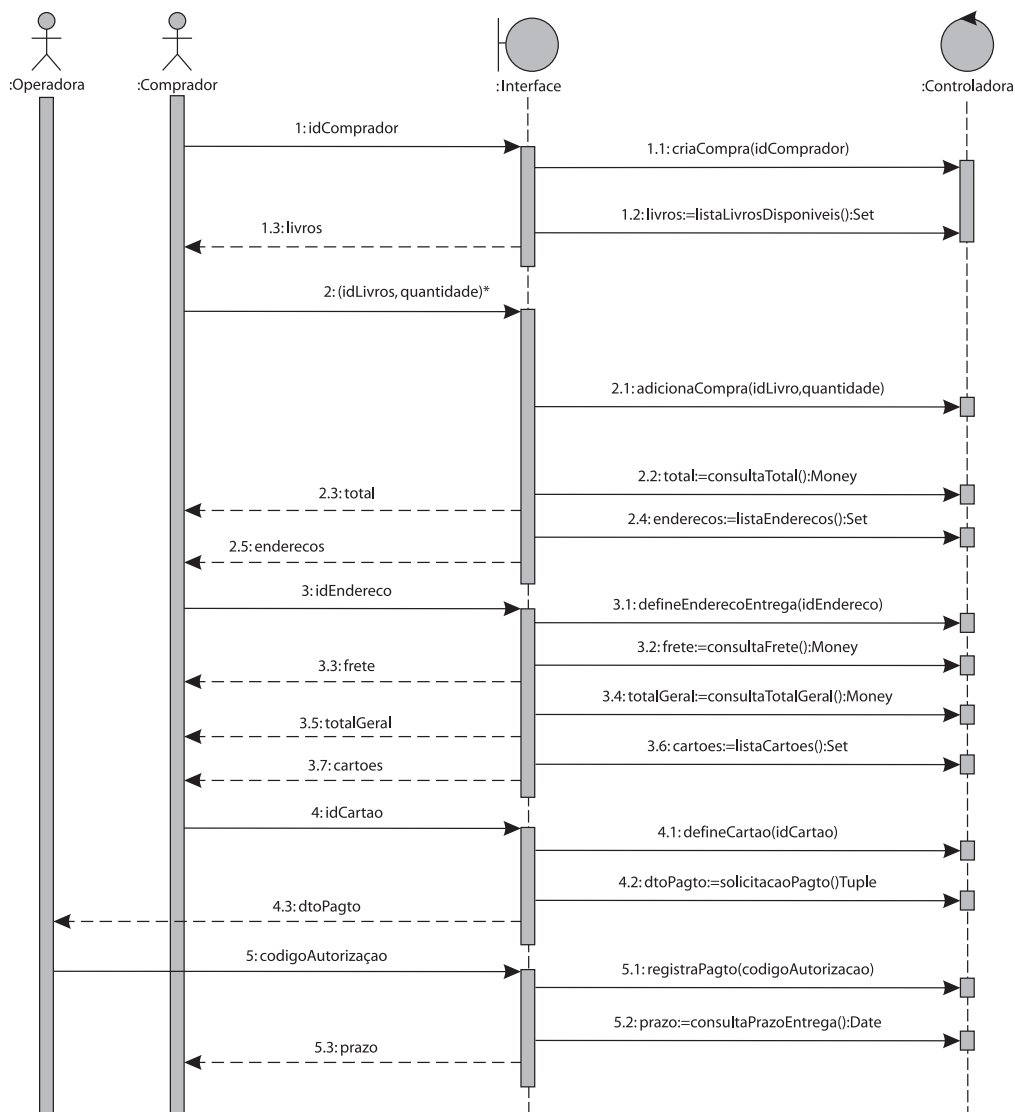


Figura 6.6: Diagrama de sequência completo com estratégia *statefull*.

Cabe ao projetista decidir qual estratégia usar. Os prós e contras de cada uma são os seguintes:

- a) a estratégia *statefull* exige a implementação de um mecanismo de memória temporária (não persistente) para lembrar alguns parâmetros (uso de associações temporárias no modelo conceitual, por exemplo). A estratégia *stateless* não exige esse tipo de mecanismo;

- b) a estratégia *stateless* exige maior passagem de parâmetros entre a interface e a controladora. Quando se trata de envio de informações pela rede, isso pode ser inconveniente. Com a estratégia *statefull*, cada informação é transmitida uma única vez.

6.5. Exceções em Diagramas de Sequência

Como visto no capítulo anterior, passos em casos de uso, especialmente eventos de sistema, podem ter exceções associadas, cujo tratamento é descrito em um fluxo alternativo do caso de uso.

Uma exceção pode ser modelada no diagrama de sequência como um evento condicional sinalizado que aborta a operação que está sendo tentada. (Figura 6.7).

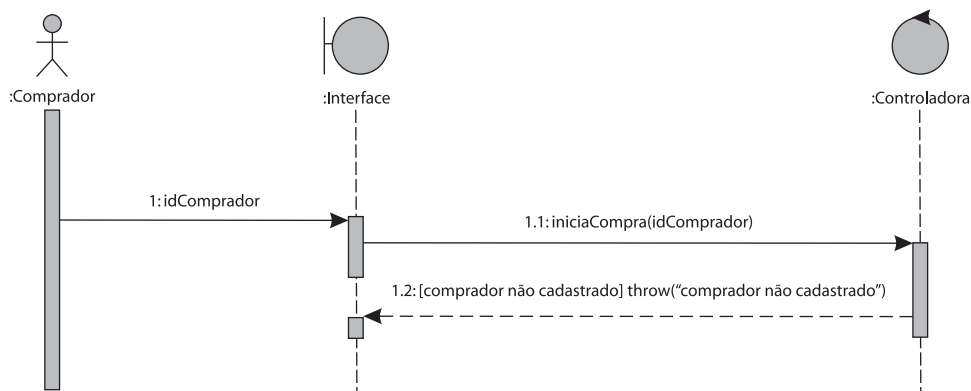


Figura 6.7: Uma operação de sistema com exceção.

As exceções possíveis são aquelas identificadas no caso de uso, ocorrendo nas operações de sistema correspondentes. Usualmente, não ocorrem exceções em consultas porque estas sempre retornam algum tipo de resultado.

Uma vez identificada a exceção, há pelo menos duas formas de tratá-la:

- pode-se tratar a exceção na interface, emitindo algum tipo e mensagem ao ator e realizando o fluxo alternativo;
- pode-se também tentar transformar a exceção em uma precondição, evitando que o erro detectado ocorra na operação, mas que seja evitado antes da operação ser tentada.

No primeiro caso, pode-se representar o fluxo alternativo como uma chamada à operação de sistema *criaComprador*, que é parte do caso de uso <<CRUD>> “Gerenciar comprador”. O resultado é mostrado na Figura 6.8.

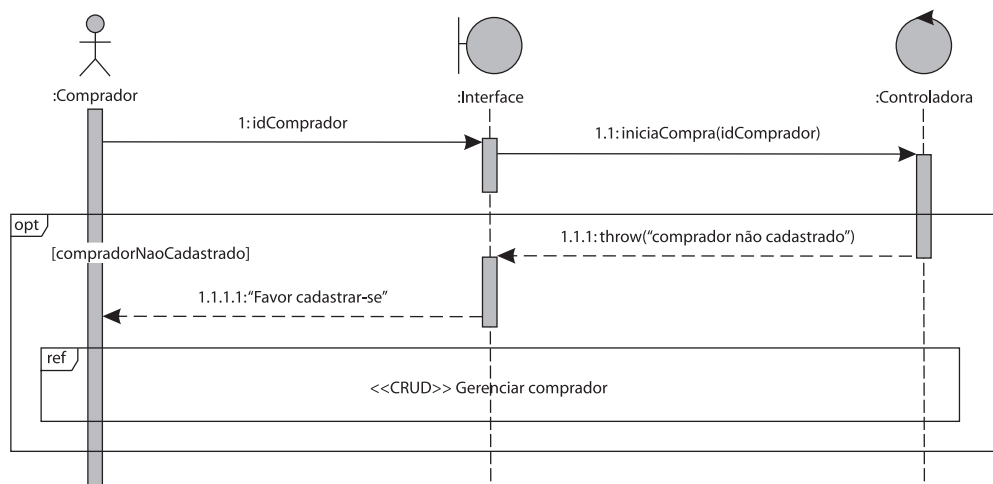


Figura 6.8: Uma exceção tratada em um diagrama de sequência.

Conforme estipulado na Figura 5.10 e complementado na Figura 5.11, o tratamento da exceção 1a do caso de uso Comprar livros exige o cadastramento do comprador. Na Figura 6.8 são usadas duas estruturas de *fragmento* típicas do diagrama de sequência para modelar situações que justamente fogem de uma sequência estrita.

O primeiro fragmento *opt* indica que os elementos incluídos nele só são executados quando a condição *[compradorNaoCadastrado]* for verdadeira. Essa exceção, aqui apenas mencionada, pode ser formalmente especificada como uma expressão OCL no contrato da operação de sistema *iniciaCompra*, conforme será visto no Capítulo 8.

O segundo fragmento, *ref*, indica uma referência a outro diagrama de sequência cujo nome é dado dentro do fragmento. Ou seja, os passos desse outro diagrama seriam expandidos no local onde a referência aparece.

É recomendável, para que os diagramas de sequência não fiquem demasiadamente complexos, que todas as sequências alternativas sejam definidas como diagramas separados (subordinados ao diagrama do fluxo principal se a ferramenta *CASE* assim o permitir) e referenciados no diagrama do fluxo principal, com fragmentos *ref*.

Outra maneira de tratar a exceção é evitar que ela ocorra na operação de sistema, transformando-a em pré-condição. Isso pode ser feito se uma consulta verificasse a condição antes de tentar a operação de sistema. Essa opção é mostrada na Figura 6.9.

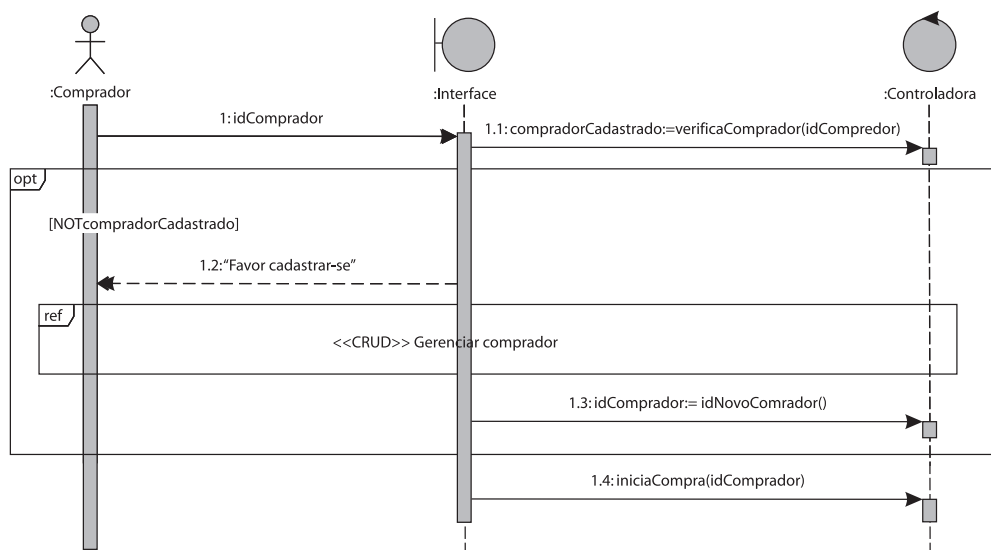


Figura 6.9: Uma exceção transformada em pré-condição em um diagrama de sequência.

Nesse caso, a operação de sistema *iniciaCompra*, em vez de ter uma exceção, como no caso da Figura 6.8, terá uma pré-condição: o *idComprador* sempre será um identificador válido.

A mensagem 1.3 nesse diagrama será desnecessária se a variante *create* do *CRUD* for definida de forma a já retornar à interface o *idComprador*.

Uma terceira forma seria possível caso o *idComprador* fosse selecionado de uma lista de identificadores válidos em vez de ser meramente digitado. Nesse caso, a exceção simplesmente não ocorre mais no caso de uso, sendo transformada em uma pré-condição do próprio caso de uso. A Figura 6.10 representa essa situação.

Na mensagem 3, a expressão entre chaves $\{ids \rightarrow \text{includes}(\text{idComprador})\}$ corresponde a uma condição escrita em OCL que exige que *idComprador* seja um elemento da lista *ids* retornada pela controladora. Matematicamente, essa expressão poderia ser escrita como $\text{idComprador} \in \text{ids}$ ou, ainda, como $\text{ids} \ni \text{idComprador}$.

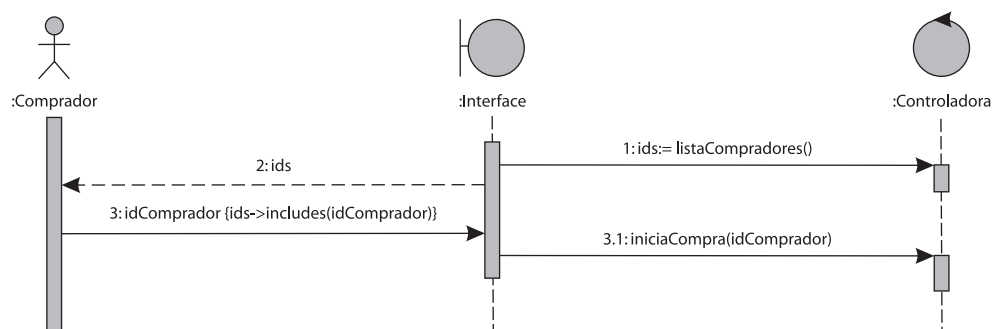


Figura 6.10: Uma exceção eliminada pela sua transformação em pré-condição.

Porém, pode não ser desejável que a operação seja executada dessa forma nesse caso, pois, assim, qualquer usuário poderia saber quais são os identificadores de compradores válidos. Esse tipo de modelagem seria adequado, entretanto, para selecionar livros, porque nesse caso não há problema de quebra de segurança caso um usuário possa acessar todos os códigos válidos.

6.6. Padrão DTO — Data Transfer Object

Na Figura 6.5, foi usado o conceito de DTO (*Data Transfer Object*). Muitas vezes pode ser inconveniente rotular transições de um diagrama de sequência (ou outros) com uma série de informações ou parâmetros, como nome, endereço, CPF, telefone etc. Assume-se, então, que uma entidade mais complexa pode representar esse conjunto de atributos. No caso da Figura 6.5, usou-se o termo `dtoPagto` para referenciar uma série de informações alfanuméricas sobre pagamentos.

Mas qual a diferença entre um DTO e as classes do modelo conceitual que serão discutidas no capítulo seguinte? A diferença reside no fato de que uma classe ou conceito tem uma estrutura semântica complexa, com associações, restrições e, mais adiante, quando transformada em classe de implementação, também métodos que serão responsáveis pela transformação e acesso à informação.

Essas classes pertencem à camada de domínio da aplicação, e sua funcionalidade deve ser encapsulada pela controladora-fachada. Não é possível, portanto, que os atores ou a interface (no diagrama de sequência de sistema) façam uso dessas classes.

Assim, a classe DTO é uma espécie de *registro* ou *tupla*, ou seja, uma classe que representa um conjunto coeso de informações alfanuméricas (ou atributos)

e que é capaz apenas de acessar e modificar essas mesmas informações (através de métodos *get* e *set*), não tendo nenhum outro tipo de funcionalidade.

Essas classes servem exatamente como estruturas de dados (registros) que permitem acesso e modificação, e um conjunto de informações alfanuméricas que, de outra forma, consistiria em longas listas de atributos. Um DTO, a princípio, deve ser definido como uma classe estereotipada em um diagrama específico (fora do modelo conceitual): o pacote de DTOs. A Figura 6.11 apresenta um exemplo.

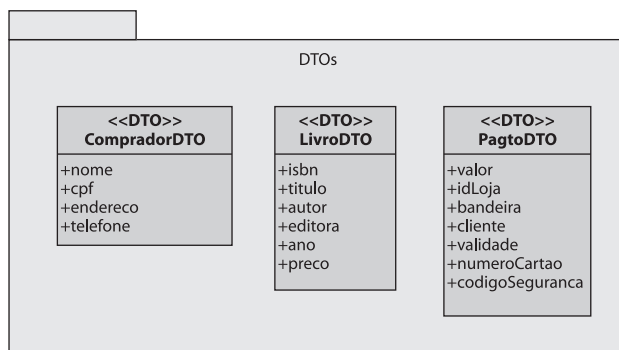


Figura 6.11: Um pacote com DTOs.

Uma vez definido o DTO, seu nome pode ser usado nos diagramas de sequência para representar a lista de atributos que ele define. Sugere-se o uso do sufixo (ou prefixo) DTO sempre para evitar confusão com as classes conceituais que serão estudadas no capítulo seguinte.

Para referenciar um valor específico dentro de um DTO, como, por exemplo, o CPF de um comprador, pode-se escrever: CompradorDTO.cpf .

A vantagem de se ter DTOs paralelamente às classes conceituais é que, dessa forma, diferentes visões de interface sobre a informação gerenciada pelo sistema não afetarão diretamente a organização interna do sistema. Pode-se traçar um paralelismo com a área de banco de dados: as classes conceituais correspondem às tabelas, que são os repositórios persistentes da informação, enquanto os DTOs correspondem às visões, que são diferentes maneiras como esses mesmos dados podem ser vistos e acessados.

Uma forma eficiente de implementar DTOs é pelo uso do padrão *Protection Proxy*, que consiste em utilizar um objeto com uma interface simples (consistindo apenas em *getters* e *setters*) encapsulando um ou mais objetos conceituais.

Modelagem Conceitual

A *análise de domínio* está relacionada à descoberta das informações que são gerenciadas no sistema, ou seja, à representação e transformação da informação. Ela ocorre em pelo menos duas fases do processo unificado. Na fase de concepção, pode-se fazer um modelo conceitual preliminar. Na fase de elaboração, esse modelo é refinado e complementado.

No sistema de informações de uma livraria virtual, as informações descobertas na análise de domínio possivelmente seriam relativas aos compradores, livros, editoras, autores, pagamentos, vendas etc.

As informações têm dois aspectos analisados: o *estático* (também denominado *estrutural*, que será estudado neste capítulo) e o *funcional* (estudado no Capítulo 8). O aspecto estático pode ser representado no *modelo conceitual*, e o aspecto funcional pode ser representado através dos *contratos de operações e consultas de sistema*.

Na atividade de análise não existe *modelo dinâmico*, visto que a análise considera apenas a descrição da visão *externa* do sistema. O modelo dinâmico, consistindo nas colaborações entre objetos, é reservado à atividade de projeto (Capítulo 9), pois apenas nessa fase é que se vai tratar dos aspectos internos do sistema. Assim, o modelo funcional da análise apenas especifica o que entra e o que sai do sistema, sem indicar como as transformações ocorrem. Já o mo-

delo dinâmico da atividade de projeto vai ter de mostrar claramente como as transformações ocorrem através das colaborações entre objetos.

O modelo conceitual deve descrever a informação que o sistema vai gerenciar. Trata-se de um artefato do domínio do problema e não do domínio da solução. Portanto, o modelo conceitual não deve ser confundido com a arquitetura do software (representada no DCP, ou diagrama de classes de projeto do Capítulo 9) porque esta, embora inicialmente derivada do modelo conceitual, pertence ao domínio da solução e, portanto, serve a um objetivo diferente.

O modelo conceitual também não deve ser confundido com o modelo de dados (Capítulo 11), pois o modelo de dados enfatiza a representação e a organização dos dados armazenados, enquanto o modelo conceitual visa a representar a compreensão da informação e não a sua representação física. Assim, um modelo de dados relacional é apenas uma possível representação física de um modelo conceitual mais essencial.

O analista deve lembrar que a atividade de análise de domínio considera apenas o mundo exterior ao sistema. Por isso, não faz sentido falar em modelo de dados nessa fase porque os dados estarão representados no interior do sistema. Uma maneira interessante de compreender o modelo conceitual é imaginar que os elementos descritos nele correspondem a informações que inicialmente existem apenas na mente do usuário, como na Figura 7.1, e não em um sistema físico de armazenamento.

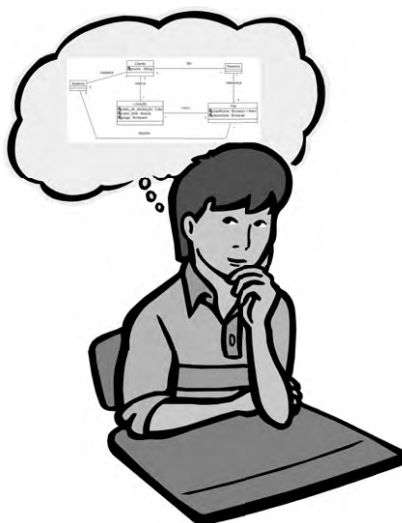


Figura 7.1: O modelo conceitual é uma representação da visão que o usuário tem das informações gerenciadas pelo sistema.

O usuário, através das operações e consultas de sistema (Capítulo 6), passa informações ao sistema e recupera informações do sistema. O sistema nem sequer precisa ser considerado como um sistema *computacional* nesse momento. Ou seja, essas informações existem independentemente de um computador para armazená-las.

O objetivo da análise é estudar o problema. Mas o sistema computacional seria uma solução para o problema e, portanto, objeto da atividade de projeto. O sistema-solução poderia também não ser computacional. Seria possível analisar todo um sistema e propor uma solução manual para implementá-lo, na qual os dados são armazenados em fichas de papel e as operações são efetuadas por funcionários da empresa usando lápis, borracha e grampeador.

Assim como os casos de uso essenciais, o modelo conceitual deve ser independente da solução física que virá a ser adotada e deve conter apenas elementos referentes ao domínio do problema em questão, ficando relegados à atividade de projeto os elementos da solução, ou seja, todos os conceitos que se referem à tecnologia, como interfaces, formas de armazenamento (banco de dados), segurança de acesso, comunicação etc.

O modelo conceitual representa somente o aspecto estático da informação. Portanto, não podem existir no modelo conceitual referências a operações ou aspectos dinâmicos dos sistemas. Então, embora o modelo conceitual seja representado pelo diagrama de classes da UML, o analista não deve ainda adicionar métodos a essas classes (o Capítulo 9 vai mostrar como fazer isso de forma adequada na atividade de projeto).

Quando se trabalha modelagem conceitual com o diagrama de classes da UML, existem precisamente três tipos de elementos para modelar a informação:

- a) *atributos*, que são informações alfanuméricas simples, como números, textos, datas etc. Exemplos de atributos no sistema Livir são: nome do comprador, data do pagamento, título do livro e valor da venda. Observe-se que um atributo sempre está ligado a um elemento mais complexo: o conceito;
- b) *conceitos*, que são a representação da informação complexa que agrega atributos e que não pode ser descrita meramente por tipos alfanuméricos. Exemplos de conceitos no sistema Livir são: livro, comprador, venda e pagamento;

- c) *associações*, que consistem em um tipo de informação que liga diferentes conceitos entre si. Porém, a associação é mais do que uma mera ligação: ela própria é um tipo de informação. No sistema Livir, devem existir associações entre uma venda e seus itens e entre a venda e seu comprador.

Nas próximas seções, esses elementos serão detalhados. É praticamente impossível falar de um sem mencionar os outros, pois os três se inter-relacionam fortemente.

7.1. Atributos

Atributos são, no modelo conceitual, os tipos escalares, textos e outros formatos derivados populares, como data, moeda, intervalo etc.

Não devem ser consideradas como atributos as estruturas de dados com múltiplos valores como listas, conjuntos, árvores etc. Essas estruturas devem ser modeladas como associações, conforme será visto mais adiante.

Conceitos complexos (classes) também não devem ser modelados como atributos. Por exemplo, um livro não pode ser atributo de uma venda. Pode existir, se for o caso, uma *associação* entre um livro e uma venda, pois ambos são conceitos complexos.

Atributos são sempre representados no seio de uma classe, conforme a Figura 7.2, em que a classe Comprador tem os atributos nome, cpf, endereço e telefone.

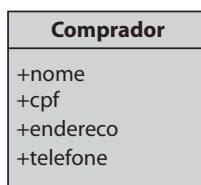


Figura 7.2: Atributos representados dentro de uma classe.

7.1.1. Tipagem

Atributos podem ser tipados, embora o modelo conceitual não exija isso explicitamente. A Figura 7.3 mostra uma versão da mesma classe da Figura 7.2 com atributos tipados.

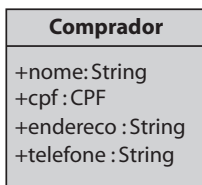


Figura 7.3: Atributos tipados.

O significado dos tipos é o mesmo correntemente atribuído pelas linguagens de programação. Observa-se, na Figura 7.3, a existência de um tipo clássico (String) e um tipo primitivo definido (CPF). Quando um atributo é definido por regras de formação, como é o caso do CPF, convém que se defina um tipo primitivo especialmente para ele, como foi feito na Figura 7.3.

O atributo telefone também poderia ser definido por um tipo especial, já que admite uma formatação específica.

Por outro lado, seria possível argumentar que um telefone é um número. Isso é verdade. Mas dificilmente alguém faria operações matemáticas com números de telefones, a não ser nos cálculos de improbabilidade da “Coração de Ouro” (Adams, 2005), mas isso já é ficção. No mundo real, embora um telefone seja composto apenas por números, ele se comporta mais como uma *string*. É mais comum extrair uma *substring* (código DDD, por exemplo) do que somar um telefone com outro.

Já o endereço é um caso à parte. Trata-se de um atributo ou um conceito complexo? Afinal, um endereço é composto por logradouro, CEP, cidade etc. Esse caso, como muitos outros, define-se pela necessidade de informação do sistema. Se endereços são usados apenas para gerar etiquetas de envelopes, então eles se comportam como atributos e podem ser representados por uma simples *string*. Porém, se endereços são usados para calcular distâncias entre diferentes pontos ou para agrupar compradores por localidade ou proximidade, então eles se comportam como conceitos complexos e devem ser modelados como uma classe com atributos e associações próprias.

7.1.2. Valores Iniciais

Um atributo pode ser declarado com um valor inicial, ou seja, sempre que uma instância do conceito (classe) for criada, aquele atributo receberá o valor inicial definido, que posteriormente poderá ser mudado, se for o caso.

Uma venda, por exemplo, pode ser criada com um valor total que inicialmente (antes que haja algum livro na venda) será zero. Isso pode ser definido no próprio diagrama de classes do modelo conceitual, como mostrado na Figura 7.4.

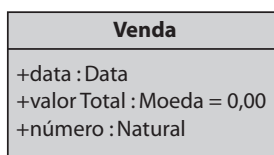


Figura 7.4: Um atributo com valor inicial.

Pode-se usar a linguagem OCL também para definir o valor inicial de um atributo de forma textual. Para isso, é necessário primeiro declarar o contexto da expressão (no caso, o atributo `valorTotal`, na classe `Venda`, representado por `Venda::valorTotal`) e usando a expressão `init` para indicar que se trata de um valor inicial de atributo. A expressão OCL seria escrita assim:

```
Context Venda::valorTotal:Moeda init: 0,00
```

Pode-se omitir o tipo `Moeda`, pois a OCL não obriga a declaração de tipos nas suas expressões:

```
Context Venda::valorTotal init: 0,00
```

É possível também que um atributo tenha um valor inicial calculado de forma mais complexa. Mais adiante serão apresentados exemplos de expressões complexas com OCL que podem ser usadas para inicializar atributos com valores como somatórios, quantidade de elementos associados, maior elemento associado etc.

7.1.3. Atributos Derivados

Atributos derivados são valores alfanuméricos (novamente não se admitem objetos nem estruturas de dados como conjuntos e listas) que não são definidos senão através de um cálculo. Ao contrário dos valores iniciais, que são atribuídos na criação do objeto e depois podem ser mudados à vontade, os atributos derivados não admitem qualquer mudança diretamente neles. Em outras palavras, são atributos *read-only*.

Um atributo derivado deve ser definido por uma expressão. No diagrama, representa-se o atributo derivado com uma barra (/) antes do nome do

atributo seguida da expressão que o define. Na Figura 7.5, define-se que o lucro bruto de um produto é a diferença entre seu preço de venda e seu preço de compra.

Produto
<pre>+preçoCompra : Moeda + preçoVenda : Moeda + / lucro Bruto : Moeda = precoVenda-precoCompra</pre>

Figura 7.5: Um atributo derivado.

Em OCL, o mesmo atributo derivado poderia ser definido usando a expressão *derive*:

```
Context Produto::lucroBruto:Moeda
derive:
    precoVenda - precoCompra
```

Nessa classe, apenas os atributos *precoCompra* e *precoVenda* podem ser diretamente alterados por um *setter*. O atributo *lucroBruto* pode apenas ser consultado. Ele é o resultado do cálculo conforme definido.

Mecanismos de otimização de fase de implementação podem definir se atributos derivados como *lucroBruto* serão recalculados a cada vez que forem acessados ou se serão mantidos em algum armazenamento oculto e recalculados apenas quando um de seus componentes for mudado. Por exemplo, o *lucroBruto* poderia ser recalculado sempre que *precoCompra* ou *precoVenda* executarem a operação *set* que altera seus valores.

7.1.4. Enumerações

Enumerações são um meio-termo entre o conceito e o atributo. Elas são basicamente *strings* e se comportam como tal, mas há um conjunto predefinido de *strings* válidas que constitui a enumeração. Por exemplo, o dia da semana só pode assumir um valor dentre sete possíveis: “domingo”, “segunda-feira”, “terça-feira” etc. Assim, um atributo cujo valor pode ser um dia da semana poderá ser tipado com uma enumeração.

A enumeração pode aparecer nos diagramas UML como uma classe estereotipada. Sugere-se que não sejam colocadas no modelo conceitual, mas em um pacote específico para conter enumerações, como mostrado na Figura 7.6.

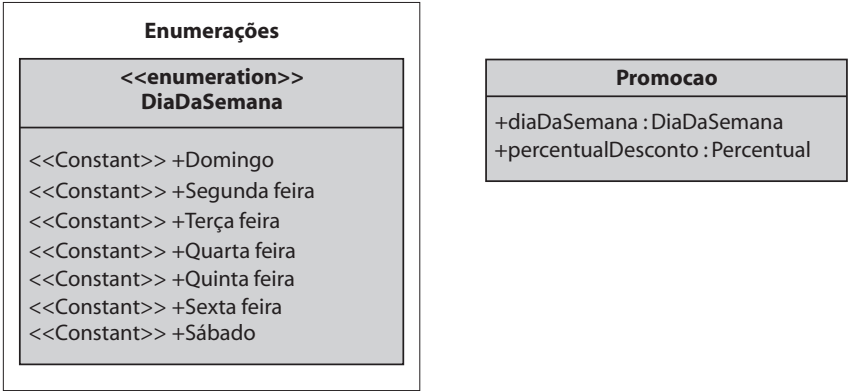


Figura 7.6: Um pacote com uma enumeração e seu uso em uma classe.

Na Figura 7.6, o atributo diaDaSemana, na classe Promocao pode assumir um dentre os sete valores da enumeração DiaDaSemana.

Em hipótese alguma enumerações podem ter associações com outros elementos ou atributos (os valores que aparecem dentro da declaração da enumeração são constantes e não atributos). Se isso acontecer, não se trata mais de uma enumeração, mas de um conceito complexo. Nesse caso, não se deve usar o estereótipo <<enumeration>> nem se pode usar o nome da enumeração como se fosse um tipo, como na Figura 7.6. Quando se trata de um conceito complexo, sua relação com outros conceitos tem de ser feita através de associações, como será mostrado mais adiante.

7.1.5. Tipos Primitivos

O analista pode e deve definir *tipos primitivos* sempre que se deparar com atributos que tenham regras de formação, como no caso do CPF. Tipos primitivos podem ser classes estereotipadas com <<primitive>>, como na Figura 7.50.

Alguns tipos primitivos como Date, Money etc. já são definidos em OCL e na maioria das linguagens de programação. Mas outros tipos podem ser criados e suas regras de formação podem ser definidas pelo analista. É o caso de CPF, CEP, NumeroPrimo e quaisquer outros tipos alfanuméricos cuja regra de formação possa ser analisada sintaticamente, ou seja, avaliando expressões em que não constem dados gerenciados pelo sistema. Por exemplo, CPF pode ser um tipo primitivo, mas CPFDeComprador não, pois para saber se o CPF

pertence a um comprador seria necessário avaliar os dados de compradores gerenciados pelo sistema.

7.2. Conceitos

É impossível falar de atributos sem falar nos conceitos, já que são fortemente ligados. Conceitos são usualmente agrupamentos coesos de atributos sobre uma mesma entidade de informação.

Conceitos são mais do que valores alfanuméricos. São também mais do que meramente um amontoado de atributos, pois eles trazem consigo um significado e podem estar associados uns com os outros.

7.2.1. Identificadores

Um *identificador* é um atributo que permite que uma instância de um conceito seja diferenciada de outras. Uma vez que um atributo tenha sido estereotipado como identificador (<<oid>> do inglês *Object Identifier*), não é possível que existam duas instâncias do mesmo conceito com o mesmo valor para esse atributo. Um exemplo de identificador é o número de CPF para os brasileiros (Figura 7.7) porque não existem duas pessoas com o mesmo CPF (pelo menos não oficialmente).

Pessoa
+nome : String <<oid>> + cpf : CPF +telefone : String +endereco : String

Figura 7.7: Um conceito com identificador.

Alguns dialetos usam o estereótipo <<PK>> para identificadores, derivado do inglês *Primary Key* (chave primária), um conceito de banco de dados que representa um atributo ou campo que não pode ser repetido em duas entidades.

Entretanto, a semelhança entre OID e PK não é perfeita porque um registro em um banco de dados relacional pode ter apenas uma chave primária (algumas vezes composta por mais de um atributo), enquanto um conceito

pode ter vários identificadores. Assim, o conceito de identificador assemelha-se mais ao conceito de *chave candidata* de banco de dados (Date, 1982).

7.2.2. Classe Controladora de Sistema

Usualmente, espera-se que um modelo conceitual corresponda a um grafo conexo, ou seja, que todos os conceitos se conectem de alguma forma. A não existência dessa situação pode indicar ao analista que algumas associações importantes ainda não foram descobertas.

Será muito útil, desde a fase de modelagem conceitual, adicionar ao diagrama uma classe que representa o sistema como um todo. Essa classe corresponderá à *controladora de sistema* ou *controladora-fachada* (*façade controller*), já mencionada no Capítulo 6.

Sendo uma classe de controle, admite-se que a controladora de sistema tenha apenas uma única instância (o sistema); ela pode ser uma classe estereotipada com <<control>> ou ainda desenhada de acordo com o padrão de Jacobson, como Livir na Figura 7.23.

7.2.3. Conceitos Dependentes e Independentes

Uma classificação interessante e útil para conceitos é verificar se são dependentes ou independentes.

Um conceito é *dependente* se precisa estar associado a outros conceitos para fazer sentido, ou seja, para representar uma informação minimamente compreensível. Não faria sentido ter um conceito de venda que não estivesse associado a um comprador e um conjunto de livros. Ele sozinho não faria sentido.

Um conceito é *independente* se pode ser compreendido sem estar associado a outros. Por exemplo, o conceito Pessoa pode ser compreendido a partir de seus atributos, sem necessidade de estar associado a outros conceitos. Uma pessoa não precisa ter comprado nada para ser uma pessoa. Essas associações com compras são opcionais para ela. Então, o conceito Pessoa é, nesse sentido, independente.

Um paralelo pode ser encontrado na linguística, onde os *verbos transitivos* precisam de complemento (correspondendo aos conceitos dependentes) e os *intransitivos* não precisam de complemento (correspondendo aos con-

ceitos independentes). Assim, pode-se dizer “João dormiu” sem acrescentar nenhum complemento ao verbo, e a frase faz sentido. Mas não se pode dizer “João fez” sem que haja um complemento, mesmo que por elipse. Para a frase ter sentido, João tem de ter feito *alguma coisa*.

Mas, para que serve essa distinção? É interessante notar que apenas os conceitos independentes se prestam a ser cadastros, ou seja, eles são operáveis através de um caso de uso *CRUD*. Pode-se ter, então, cadastros de pessoas, de livros, de fornecedores etc., mas, usualmente, os conceitos dependentes não se prestam a esse tipo de operação. Compras, vales-presente, pagamentos não podem simplesmente ser *cadastrados* na forma *CRUD*, mas são operados nos casos de uso que correspondem a processos de negócio, porque possivelmente comportam interações e exceções que fogem ao padrão.

Mais adiante, será visto que o acesso à informação no sistema (modelo dinâmico) inicia sempre na controladora-fachada. Ou seja, qualquer caminho de acesso à informação parte da controladora-fachada. Assim, conceitos diretamente ligados à controladora-fachada são aqueles cujas instâncias podem ser acessadas diretamente. Livros e compradores são cadastros (conceitos independentes) e, portanto, podem ser acessados diretamente.

Como consequência disso, apenas os conceitos independentes estarão inicialmente associados à classe controladora de sistema. Os conceitos dependentes não terão esse tipo de associação a não ser que ela represente algum tipo de informação adicional. Por exemplo, uma associação entre reservas (conceito dependente) poderia estar ligada à controladora-fachada através de uma associação ordenada para indicar a ordem de prioridade entre as reservas, conforme será visto mais adiante.

7.3. Como Encontrar Conceitos e Atributos

O processo de descoberta dos elementos do modelo conceitual pode variar. Porém, uma forma bastante útil é olhar para o texto dos casos de uso expandidos ou os diagramas de sequência de sistema. A partir desses artefatos, pode-se descobrir todos os elementos textuais que eventualmente referenciam informação a ser guardada e/ou processada.

Usualmente, esses elementos textuais são compostos por substantivos, como “pessoa”, “compra”, “pagamento” etc., ou por expressões que denotam substantivos (conhecidas em linguística como *sintagmas nominais*), como

“autorização de venda”. Além disso, no texto, algumas vezes alguns verbos podem indicar conceitos, pois o verbo pode exprimir um ato que corresponde a um substantivo, como por exemplo, “pagar”, que corresponde ao substantivo “pagamento”; “comprar”, que corresponde ao substantivo “compra” etc.

Via de regra, entretanto, o analista deve ter em mente os objetivos do sistema enquanto procura descobrir os elementos do modelo conceitual. Não é interessante representar, no modelo conceitual, a informação que é irrelevante para o sistema. Assim, nem todos os substantivos e verbos deverão ser considerados no modelo. O analista tem a responsabilidade de compreender quais as verdadeiras necessidades de informação e filtrar as irrelevâncias.

O processo de identificação dos conceitos e atributos, então, consiste em:

- a) identificar no texto dos casos de uso as palavras ou sintagmas que correspondem a conceitos sobre os quais se tem interesse em manter informação no sistema;
- b) agrupar as palavras ou expressões que são sinônimos, como, por exemplo, “compra” e “aquisição”, “comprador” e “cliente” etc.;
- c) identificar quais dos itens considerados correspondem a conceitos complexos e quais são meros atributos. Os atributos são aqueles elementos que podem ser considerados alfanuméricos, usualmente: nomes, números em geral, códigos, datas, valores em moeda, valores booleanos (verdadeiro ou falso) etc.

Aplicando essa técnica ao caso de uso da Figura 5.10, são encontrados os principais elementos de informação a serem trabalhados. Na Figura 7.8, os elementos identificados como conceitos ou atributos são grifados no texto.

Caso de Uso: Comprar livros

1. [IN] O comprador informa sua identificação.
2. [OUT] O sistema informa os livros disponíveis para venda (título, capa e preço) e o conteúdo atual do carrinho de compras.
3. [IN] O comprador seleciona os livros que deseja comprar.
4. O comprador decide se finaliza a compra ou se guarda o carrinho:

4.1 Variante: Finalizar a compra.

4.2 Variante: Guardar carrinho.

Variante 4.1: Finalizar a compra

4.1.1. [OUT] O sistema informa o valor total dos livros e apresenta as opções de endereço cadastradas.

4.1.2. [IN] O comprador seleciona um endereço para entrega.

4.1.3. [OUT] O sistema informa o valor do frete e total geral, bem como a lista de cartões de crédito já cadastrados para pagamento.

4.1.4. [IN] O comprador seleciona um cartão de crédito.

4.1.5. [OUT] O sistema envia os dados do cartão e valor da venda para a operadora.

4.1.6. [IN] A operadora informa o código de autorização.

4.1.7. [OUT] O sistema informa o prazo de entrega.

Variante 4.2: Guardar carrinho

4.2.1 [OUT] O sistema informa o prazo (dias) em que o carrinho será mantido.

Exceção 1a: Comprador não cadastrado

1a.1 [IN] O comprador informa seu CPF, nome, endereço e telefone.

Retorna ao passo 1.

Exceção 4.1.2a: Endereço consta como inválido

4.1.2a.1 [IN] O comprador atualiza o endereço.

Avança para o passo 4.1.2.

Exceção 4.1.6a: A operadora não autoriza a venda

4.1.6a.1 [OUT] O sistema apresenta outras opções de cartão ao comprador.

4.1.6a.2 [IN] O comprador seleciona outro cartão.

Retorna ao passo 4.1.5.

Figura 7.8: Um caso de uso com possíveis conceitos e atributos grifados.

O resultado dessa análise pode ser imediatamente transportado para uma diagrama de classes UML (Figura 7.9).

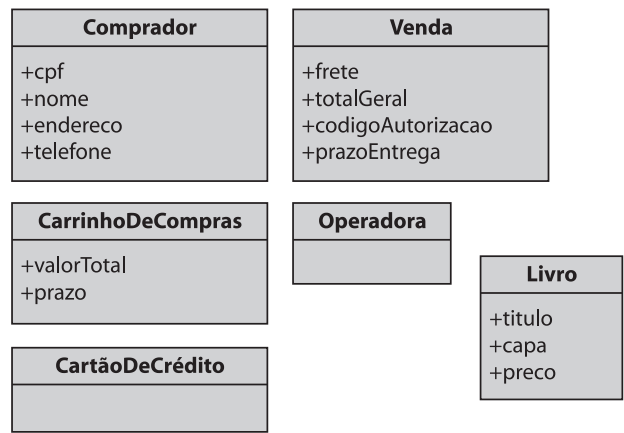


Figura 7.9: Modelo conceitual preliminar (ainda sem associações) criado a partir dos conceitos e atributos identificados no caso de uso da Figura 7.8.

Observa-se que, nos casos em que a informação passada no caso de uso foi detalhada (Comprador, Venda, Livro), os atributos foram devidamente identificados, mas, no caso em que a informação não foi devidamente detalhada no caso de uso (CartaoDeCredito Operadora), não foram identificados atributos. Isso demonstra a necessidade de fazer constar, no caso de uso, as informações que são efetivamente passadas dos atores ao sistema e vice-versa.

7.4. Associações

Quando dois ou mais conceitos complexos se relacionam entre si, diz-se que existe uma *associação* entre eles. Por exemplo, uma venda está associada aos livros que foram vendidos e também ao comprador a quem os livros foram vendidos. Um pagamento, quando existir, precisa estar obrigatoriamente associado a uma venda. Saindo um pouco do exemplo do sistema Livir, uma pessoa pode estar associada a um automóvel que seja de sua propriedade.

Os textos dos casos de uso mencionam associações com pouca frequência. Como os casos de uso descrevem ações de interação entre usuários e o sistema, então a sua descrição acaba mencionando principalmente as *operações*. Cabe aqui, portanto, definir claramente a diferença:

- a) *associação* é uma relação estática que pode existir entre dois conceitos complexos, complementando a informação que se tem sobre eles em um determinado instante ou referenciando informação associativa nova;

- b) *operação* é o ato de transformar a informação, fazendo-a passar de um estado para outro, mudando, por exemplo, a configuração das associações, destruindo e/ou criando novas associações ou objetos, ou modificando o valor dos atributos.

Assim, o texto dos casos de uso está frequentemente repleto de operações, mas não de associações. Quanto se têm, por exemplo, as classes Pessoa e Automovel, como na Figura 7.10, a associação estática que existe entre elas pode indicar a posse de uma pela outra.

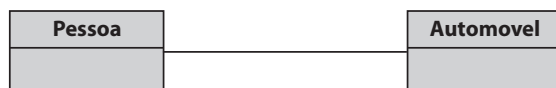


Figura 7.10: Representação de uma associação estática entre dois conceitos.

Porém, existem diferentes formas de associação entre pessoas e automóveis que não meramente a posse. Uma pessoa pode ser *passageira* de um automóvel ou *motorista* dele, ou ainda a associação pode simplesmente representar que uma determinada pessoa *gosta* de um determinado automóvel. Para eliminar tais ambiguidades, é conveniente, em muitos casos, utilizar um *nome de papel* para as associações, o qual pode ser colocado em um ou ambos os lados e deve ser lido como se fosse uma função. Na Figura 7.11, por exemplo, uma pessoa está no papel ou função de motorista de um automóvel.



Figura 7.11: Uma associação com nome de papel.

Diferentes papéis podem ser representados através de associações diferentes, como na Figura 7.12.



Figura 7.12: Múltiplas associações entre conceitos.

Na falta de um nome de papel explícito, o próprio nome da classe associada deve ser considerado como nome de papel. No caso da Figura 7.12, um automóvel explicitamente tem dois tipos de associação com pessoas: dono e motorista. Do ponto de vista inverso, porém, uma pessoa tem dois tipos de

associação com automóveis: aqueles dos quais é dona, correspondendo à sua frota, e aquele do qual é motorista, correspondendo simplesmente ao seu automóvel.

Voltando à discussão sobre as associações não serem operações, observa-se que uma pessoa pode comprar um automóvel. Isso é uma operação porque transforma as associações: uma associação de posse deixa de existir e outra é criada em seu lugar. Seria incorreto representar a operação como uma associação, como na Figura 7.13.



Figura 7.13: Uma operação incorretamente rotulando um papel de associação.

Segundo a definição da UML, associações também podem ter nomes, que são colocados no meio da linha que liga duas classes e não nas extremidades. Porém, tais nomes, além de serem difíceis de atribuir (analistas iniciantes preencherão seus diagramas com associações chamadas “possui” ou sinônimos), não têm qualquer resultado prático. Mais vale trabalhar com bons nomes de papel do que com nomes de associações. Então, é prático simplesmente ignorar que associações tenham nomes e viver feliz com os nomes de papel, muito mais úteis para definir possibilidades de navegação entre conceitos (Capítulos 8 e 9) e para nomear elementos de programação (Capítulo 12).

Algumas vezes, pode ser interessante guardar informações sobre operações ou transações que foram efetuadas. Assim, pode ser interessante para o sistema guardar as informações sobre a transação de compra, como data, valor pago etc. Nesse caso, a transação é tratada no modelo conceitual como um conceito complexo (Figura 7.14) e representa estaticamente a memória da operação que um dia foi efetuada.

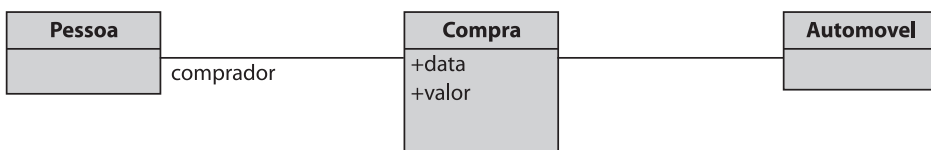


Figura 7.14: Transação representada como conceito.

Porém, como se pode notar, a transação é representada por um conceito, enquanto as associações continuam representando ligações estáticas entre conceitos e não operações ou transformações.

7.4.1. Como Encontrar Associações

Se a informação correspondente aos conceitos e atributos pode ser facilmente encontrada no texto dos casos de uso, o mesmo não ocorre com as associações.

Mas, então, como encontrar as associações entre os conceitos complexos? Há duas regras:

- conceitos dependentes (como Compra) precisam necessariamente estar ligados aos conceitos que os complementam (como Comprador e Item);
- informações associativas só podem ser representadas através de associações.

Informações associativas podem até aparecer nos casos de uso como conceitos. Mas quando se afirma que uma pessoa é dona ou motorista de um automóvel, essa informação só pode ser colocada na forma de uma associação. Analistas viciados em modelos de dados relacionais poderão fazer uma modelagem como a da Figura 7.15, mas está incorreta, pois atributos devem ser alfanuméricos e nunca conceitos complexos (para isso existem associações).



Figura 7.15: Um atributo representando *indevidamente* uma associação.

Também é incorreto utilizar chaves estrangeiras (Date, 1982), como na Figura 7.16. Quando dois conceitos complexos se relacionam, o elemento de modelagem é a associação, e fim de conversa!



Figura 7.16: Um atributo como chave estrangeira representando *indevidamente* uma associação.

Uma regra geral de *coesão* a ser usada é que um conceito só deve conter seus próprios atributos e nunca os atributos de outro conceito. Ora, o CPF do dono é um atributo de uma pessoa e não do automóvel. Por isso é inadequado representá-lo como na Figura 7.16. Outro motivo para que se tenham associações visíveis é prático: é muito mais fácil perceber quais objetos efetivamente têm referências para com outros. Mais adiante (Capítulo 9) será visto como essas linhas de *visibilidade* são importantes para que se possa projetar um código efetivamente orientado a objetos de qualidade.

7.4.2. Multiplicidade de Papéis

Na modelagem conceitual, é fundamental que se saiba a quantidade de elementos que uma associação permite em cada um de seus papéis. Por exemplo, na associação entre Pessoa e Automovel da Figura 7.11, quantos automóveis uma pessoa pode dirigir? Quantos motoristas um automóvel pode ter?

A resposta sempre dependerá de um estudo sobre a natureza do problema e sobre o real significado da associação, especialmente se ela representa o presente ou o histórico. Quer dizer, se a associação da Figura 7.11 representa o presente, pode-se admitir que um automóvel tenha um único motorista. Mas, se a associação representa o histórico, pode-se admitir que o automóvel tenha uma série de motoristas, que o dirigiram em momentos diferentes.

Assim, é fundamental que o analista decida claramente o que a associação significa antes de anotar a multiplicidade de seus papéis.

Há, basicamente, duas decisões a tomar sobre a multiplicidade de um papel de associação:

- a) se o papel é obrigatório ou não. Por exemplo, uma pessoa é obrigada a ter pelo menos um automóvel? Um automóvel deve obrigatoriamente ter um dono?;
- b) se a quantidade de instâncias que podem ser associadas através do papel tem um limite conceitual definido. Por exemplo, existe um número máximo ou mínimo de automóveis que uma pessoa pode possuir?

Deve-se tomar cuidado com algumas armadilhas conceituais. Em relação ao primeiro tópico, por exemplo, espera-se que a toda venda corresponda um pagamento. Mas isso não torna a associação obrigatória, pois a venda pode existir sem pagamento. Um dia ela possivelmente será paga, mas pode existir sem o pagamento por algum tempo. Então, esse papel *não* é obrigatório para a venda.

Outro caso refere-se ao limite máximo. Claro que o número máximo de automóveis que uma pessoa pode possuir é o número de automóveis que existe no planeta. Mas, à medida que outros automóveis venham a ser construídos, esse magnata poderá possuí-los também. Ou seja, embora exista um limite físico, não há um limite lógico para a posse. Então, o papel deve ser considerado virtualmente sem limite superior.

A multiplicidade de papel é representada por uma expressão numérica, onde:

- a) “*” representa o infinito;
- b) a vírgula (,) significa “e”;
- c) ponto-ponto (..) significa “até”.

A seguir são apresentados alguns exemplos usuais e seu significado:

- a) 1 exatamente um
- b) 0..1 zero ou um
- c) * de zero a infinito
- d) 1..* de um a infinito
- e) 2..5 de dois a cinco
- f) 2,5 dois ou cinco
- g) 2,5..8 dois ou de cinco a oito

Os valores de multiplicidade que incluem o zero são opcionais. Já os demais representam papéis obrigatórios.

A Figura 7.17 mostra que uma pessoa pode ter uma frota composta de um número qualquer de automóveis (opcional) e que pode ser motorista de um automóvel (também opcional). Por outro lado, mostra que um automóvel tem um único dono (obrigatório) e pode ter ou não um motorista (opcional).

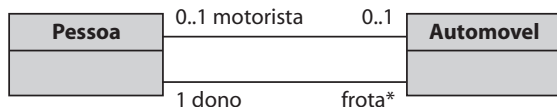


Figura 7.17: Associações com multiplicidade de papel.

7.4.3. Direção das Associações

Uma associação, no modelo conceitual, deve ser *não direcional*, isto é, a origem e o destino da associação não devem ser estabelecidos. Isso se deve ao fato de que, na atividade de análise, basta saber que dois conceitos estão associados.

Há pouca praticidade em definir prematuramente (antes da atividade de projeto) a direção de uma associação. Associações *unidirecionais* teriam como vantagem apenas o seguinte:

- a) não é necessário definir o nome de papel na origem de uma associação unidirecional;

- b) associações unidirecionais podem ser implementadas de forma mais eficiente que as bidirecionais.

Assim, essa decisão, que não afeta significativamente a modelagem conceitual, pode ser tomada com melhor conhecimento de causa (a direção das mensagens trocadas entre objetos) na atividade de projeto, como será visto no Capítulo 9.

7.4.4. Associação Derivada

Assim como algumas vezes pode ser interessante definir atributos derivados, que são calculados a partir de outros valores, pode ser também interessante ter associações derivadas, ou seja, associações que, em vez de serem representadas fisicamente, são calculadas a partir de outras informações que se tenha. Por exemplo, suponha que uma venda, em vez de se associar diretamente aos livros, se associe a um conjunto de itens, e estes, por sua vez, representem uma quantidade e um título de livro específico (Figura 7.18).

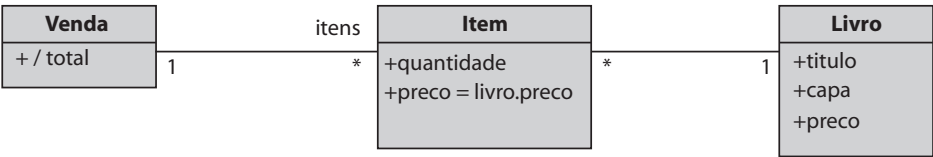


Figura 7.18: Uma venda e seus itens.

Esse tipo de modelagem é necessário quando os itens de venda não são representados individualmente, mas como quantidades de algum produto. Além disso, o livro enquanto produto pode ter seu preço atualizado, mas o item que foi vendido terá sempre o mesmo preço. Daí a necessidade de representar também o preço do item como um atributo com valor inicial igual ao preço do livro.

Porém, a partir da venda, não é mais possível acessar diretamente o conjunto de livros. Seria necessário tomar o conjunto de itens e, para cada item, verificar qual é o livro associado. Criar uma nova associação entre Venda e Livro não seria correto porque estaria representando informação que já existe no modelo (mesmo que de forma indireta). Além disso, uma nova associação entre Venda e Livro poderia associar *qualquer* venda com *qualquer* livro, não apenas aqueles que já estão presentes nos itens da venda, o que permitiria a representação de informações inconsistentes.

A solução de modelagem, nesse caso, quando for relevante ter acesso a uma associação que pode ser derivada de informações que já existem, é o uso de uma associação derivada, como representado na Figura 7.19.

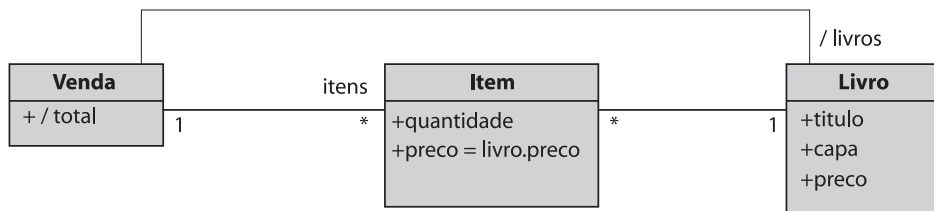


Figura 7.19: Uma associação derivada.

Uma associação derivada só tem papel e multiplicidade em uma direção (no caso, de Venda para Livro). Na outra direção ela é indefinida. Ao contrário de associações comuns que podem ser criadas e destruídas, as derivadas só podem ser consultadas (assim como os atributos derivados, elas são *read-only*). A forma de implementar uma associação derivada varia e otimizações podem ser feitas para que ela não precise ser recalculada a cada vez que for acessada.

Uma associação derivada pode ser definida em OCL. O exemplo da Figura 7.19 poderia ser escrito assim:

```
Context Venda::livros derive: self.itens.livro
```

Em relação a essa expressão OCL, pode-se observar que:

- o contexto é a própria associação derivada a partir da classe Venda conforme definido no diagrama;
- usa-se “derive” como no caso de atributos derivados. O que define se é um atributo ou associação derivada é o contexto e o tipo de informação, já que atributos são alfanuméricos e associações definem conjuntos de objetos;
- “self” denota uma instância do contexto da expressão OCL. No caso, qualquer instância de Venda;
- “.” é uma notação que permite referenciar uma propriedade de um objeto.

Propriedades que podem ser referenciadas pela notação “.” são:

- atributos;
- associações;
- métodos.

Na modelagem conceitual, usualmente faz-se referência apenas a atributos e associações. Assim, se o contexto é Venda, então `self.total` representa o atributo total de uma instância de Venda e `self.itens` representa um conjunto de instâncias de Item associadas à venda pelo papel itens.

Quando a notação “.” é usada sobre uma coleção, ela denota a coleção das propriedades de todos os elementos da coleção original. Assim, no contexto de Venda, a expressão “`self.itens.titulo`” referencia o conjunto de títulos (*strings*) dos itens de uma venda. Já a expressão `self.itens.livro`, que aparece na definição da associação derivada da Figura 7.19, representa o conjunto das instâncias de Livro associados às instâncias de Item associados a uma instância de Venda.

7.4.5. Coleções

Coleções de objetos não devem ser representadas como conceitos no modelo conceitual, mas como associações. De fato, uma associação com multiplicidade * representa um conjunto de objetos da classe referenciada. Assim, no exemplo da Figura 7.17, “frota” é um papel que associa um conjunto de automóveis a um dono. A modelagem da Figura 7.20 é, portanto, inadequada e desnecessária.



Figura 7.20: Uma coleção inadequadamente representada como conceito.

Assim, a forma correta de representar um conjunto ou coleção de objetos é através de um papel de associação, como na Figura 7.17 e não como um conceito (Figura 7.20).

Associações podem representar mais do que conjuntos; podem representar *tipos abstratos de dados*. Na verdade, podem também representar tipos concretos, mas estes não são importantes na atividade de análise, sendo utilizados apenas na atividade de projeto.

Para lembrar: tipos *abstratos* de dados são definidos apenas pelo seu comportamento. É o caso, por exemplo, do conjunto (*set*), onde elementos não se repetem e onde não há ordem definida; do multiconjunto (*bag*), onde

os elementos podem se repetir, mas não há ordem; da lista (*sequence*), onde elementos podem se repetir e há uma ordem entre eles; e também do conjunto ordenado (*ordered set*), fila (*queue*), pilha (*stack*) etc.

Tipos *concretos* de dados são as implementações físicas dos tipos abstratos. Por exemplo, uma lista pode ser implementada de diversas formas: como um *array*, como *lista encadeada*, como uma *árvore binária* etc. No caso dos tipos concretos, além do comportamento, há uma estrutura física que os define.

7.4.5.1. Conjuntos

Um papel de associação, na falta de mais detalhes, representa um *conjunto*, ou seja, elementos não se repetem e não há nenhuma ordem definida entre eles. Na Figura 7.17, frota é, dessa forma, um *conjunto* de automóveis de uma pessoa.

Se um mesmo automóvel for adicionado a essa associação para a mesma pessoa, o efeito é nulo, pois ele já pertence ao conjunto.

7.4.5.2. Conjunto Ordenado

Supondo que um livro que ainda não está em estoque tenha reservas de pessoas interessadas em comprá-lo assim que chegar, não se pode garantir que a quantidade de exemplares recebidos vá atender a todos os pedidos. Então, a forma mais justa de atender a esses pedidos seria atender prioritariamente as reservas mais antigas. Para isso elas devem estar ordenadas.

Por outro lado, uma mesma pessoa não pode reservar o mesmo livro mais de uma vez. Então, ainda se trata de um conjunto, sem repetições, mas os elementos se apresentam em ordem: primeiro, segundo, terceiro etc. Para definir esse tipo de estrutura de dados no modelo conceitual, basta adicionar a restrição {ordered} ao papel, como na Figura 7.21.



Figura 7.21: Um conjunto ordenado de reservas.

7.4.5.3. Multiconjunto

Há situações em que a ordem dos elementos não importa, mas um mesmo elemento pode aparecer mais de uma vez na coleção. Essa estrutura denomina-se *multiconjunto*, ou, mais simplesmente, em inglês, *bag*.

Por exemplo, pode-se querer saber que pessoas visualizaram os detalhes de um determinado livro e saber também quantas vezes cada pessoa visualizou esses detalhes. Pode não ser relevante saber quem visualizou primeiro, mas apenas quem visualizou mais vezes. Esse é um caso típico para utilização de um multiconjunto. Cada vez que uma pessoa visualiza um livro, uma nova associação é adicionada ao multiconjunto. A modelagem é mostrada na Figura 7.22.

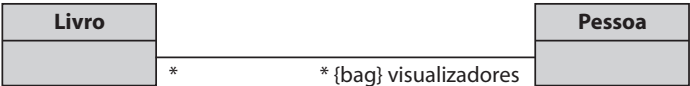


Figura 7.22: Um multiconjunto.

7.4.5.4. Lista

A lista (*sequence*) combina as propriedades de permitir a repetição de elementos e considerar a ordem entre eles. Um elemento pode aparecer mais de uma vez na lista.

Pode-se imaginar, por exemplo, que pessoas que compraram uma determinada quantidade de livros se habilitem para receber um brinde, mas apenas uma quantidade limitada de brindes é entregue por dia. Assim, uma lista de pessoas pode ser definida e os brindes vão sendo distribuídos para os primeiros da lista à medida que se tornam disponíveis. Se uma pessoa fizer duas ou mais compras na quantidade exigida para fazer jus ao brinde, pode entrar novamente na lista. Esse caso é mostrado na Figura 7.23.

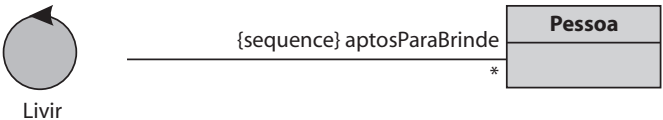


Figura 7.23: Uma lista.

A lista tem dois casos especiais importantes. Quando os primeiros elementos a serem retirados da lista são os mais antigos, como no caso da Figura 7.23, ela é uma *fila*. Nesse caso, pode-se usar a restrição {queue}.

Quando os primeiros elementos a serem retirados são os mais recentes, ela se comporta como uma *pilha*, que pode ser definida com a restrição {stack}.

Uma lista genérica pode ter inserções e retiradas em qualquer posição. Pilhas e filas só podem ter inserções e retiradas em um único local, embora qualquer de seus elementos possa ser visualizado.

7.4.5.5. Mapeamento

Quando um conceito tem um atributo identificador, pode-se criar um mapeamento do identificador para o conceito, de forma que seja muito mais fácil identificar e localizar instâncias específicas do conceito. Por exemplo, o ISBN de um livro é um identificador para o livro. Então, o cadastro de livros (associação a partir da controladora) pode ser qualificado pelo ISBN e, dessa forma, em vez de a controladora ter acesso a um mero conjunto de livros, ela terá acesso a um mapeamento que permite identificar um livro diretamente a partir de seu ISBN. A Figura 7.24 mostra a modelagem desse mapeamento.

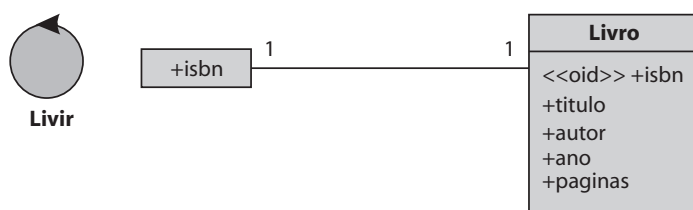


Figura 7.24: Um mapeamento.

O pequeno retângulo do lado esquerdo da associação representa um *qualificador* para o papel do lado oposto. Ou seja, a leitura que se deve fazer dessa associação é a seguinte: “para cada ISBN, há uma instância de livro”. O fato de o papel do lado direito estar marcado com multiplicidade 1 significa que a cada ISBN corresponde um e exatamente um livro, ou seja, não há dois livros com o mesmo ISBN. Isso tem de ser necessariamente verdade, pois o ISBN é um identificador do livro e, portanto, não admite repetições. Na prática, continua sendo uma associação de um para muitos, mas, do lado *muitos*, cada elemento é identificado unicamente por um qualificador.

O fato de o papel do lado esquerdo também estar marcado com multiplicidade 1 significa que toda e qualquer instância de livro tem um ISBN e participa desse mapeamento, ou seja, não pode estar de fora.

7.4.5.6. Partição

No caso da Figura 7.24, a multiplicidade 1 do lado direito define um mapeamento, ou seja, para cada valor do qualificador (isbn) corresponde exatamente uma instância da classe qualificada (Livro).

Mas se, em vez de 1, fosse uma multiplicidade maior como “*”? Nesse caso, a leitura seria: “para cada valor do qualificador corresponde um *conjunto* de instâncias da classe qualificada”.

Por exemplo, livros podem ser classificados por gênero. Como vários livros podem pertencer ao mesmo gênero, esse atributo não constitui um identificador (*oid*). Mas é possível definir uma *partição* do conjunto dos livros a partir do gênero, como na Figura 7.25.

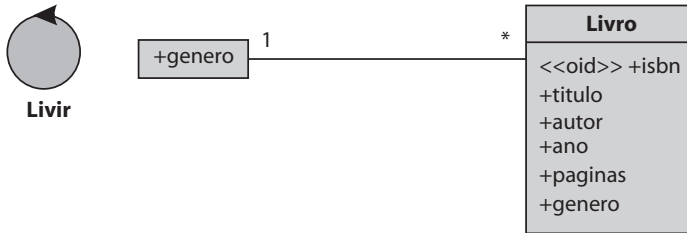


Figura 7.25: Uma partição.

A Figura 7.25 estabelece que, para cada valor possível de gênero (que poderia ser uma enumeração, por exemplo), corresponde um conjunto de livros (com zero ou mais elementos). O papel do lado esquerdo com multiplicidade 1 estabelece que todo livro participa da associação (é obrigatória) e, portanto, todo livro tem um gênero.

Essa restrição também faz admitir que o gênero poderia ser um atributo da classe Livro. Quando o qualificador é um atributo da classe, é chamado de *qualificador interno* (como na Figura 7.24); quando o qualificador não é atributo da classe qualificada, é chamado de *qualificador externo*, como nas Figuras 7.25 e 7.26.

7.4.5.7. Relação

Agora, se um livro puder ser classificado em mais de um gênero – por exemplo, o *Guia do Mochileiro das Galáxias* (Adams, 2004), que pode ser classificado no gênero “Humor”, mas também no gênero “Ficção Científica” – nesse caso, bastaria trocar a multiplicidade de papel do lado esquerdo da Figura 7.25 para * ou 1..*, resultando em uma relação como na Figura 7.26.

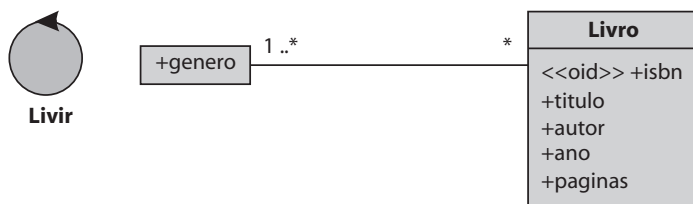


Figura 7.26: Uma relação.

A relação da Figura 7.26 diz que um livro tem um ou mais gêneros, e um gênero tem um conjunto de livros associado a ele. Nesse caso, o qualificador tem de ser, necessariamente, externo.

Possivelmente, a real necessidade das associações qualificadas nesses diagramas surgirá de forma mais enfática no capítulo seguinte, quando será necessário escrever expressões para referenciar objetos. Será mais fácil acessar um elemento indexado por uma associação qualificada do que obter todo o conjunto de instâncias referenciadas pela associação e procurar o elemento em questão ali dentro a partir de uma chave de busca.

7.4.6. Agregação e Composição

Algumas associações podem ser consideradas mais fortes do que outras no sentido de que elas definem um objeto que é composto por outros. Uma casa, por exemplo, é composta por seus cômodos.

Se existir exclusividade nessa associação, ou seja, se um item não pode ser parte de nenhum outro conceito, então a agregação é considerada forte e representada por um losango preto, como na Figura 7.27. Esse tipo de associação é também chamado de *composição*.

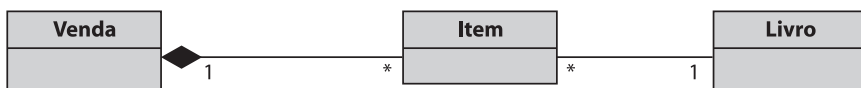


Figura 7.27: Composição.

A composição indica exclusividade na agregação. Quando essa exclusividade não é exigida, pode-se usar um losango branco, como na Figura 7.28, para indicar uma agregação compartilhada.

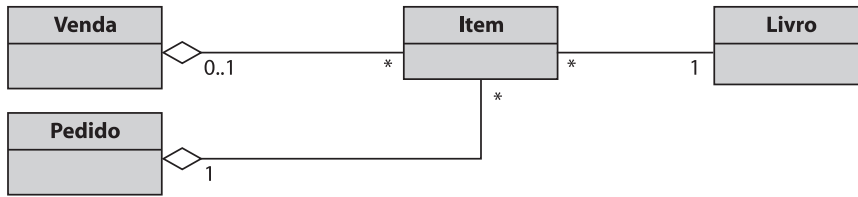


Figura 7.28: Agregação compartilhada.

O losango branco indica uma agregação compartilhada em que o componente pode ser agregado a vários conceitos ao mesmo tempo. Na Figura 7.28, um dado Item necessariamente faz parte de um Pedido, mas também poderá fazer parte de uma Venda.

Quanto à multiplicidade de papel do lado do agregador (lado onde está o losango), deve-se observar não ser possível uma multiplicidade diferente de 1 ou 0..1 quando a associação for de composição.

Agregação e composição são associações especiais e devem ser usadas com muita parcimônia no modelo, ou seja, só se deve usar quando se tem certeza.

Erros comuns são associar por agregação elementos que não são *parte-todo*. Por exemplo, um comprador não é parte de uma venda, visto que um comprador é um ente físico e uma venda é uma transação não física. Agregações e composições devem unir elementos de mesma natureza. Uma venda associa-se a um comprador, mas não é *feita de* comprador.

Existem poucas vantagens práticas na definição de agregações ou composições. Por isso, seu uso deve ser minimizado. Dentre as vantagens, pode-se citar basicamente o fato de que elementos agregados usualmente possuem atributos que se combinam nas partes e são derivados no todo. Por exemplo, o valor total de uma venda é a soma do valor dos seus itens. O peso total de uma encomenda é o peso de seus itens. Quando um automóvel é vendido, todos os seus componentes são vendidos. E assim por diante.

7.4.7. Associações n-árias

Pode-se dizer que a grande maioria das associações é binária. Porém pode haver situações em que devem ser criadas associações ente três ou mais classes. A representação gráfica dessas associações consiste em um polígono ligando por arestas todas as classes. Um exemplo de associação ternária é apresentado na Figura 7.29.

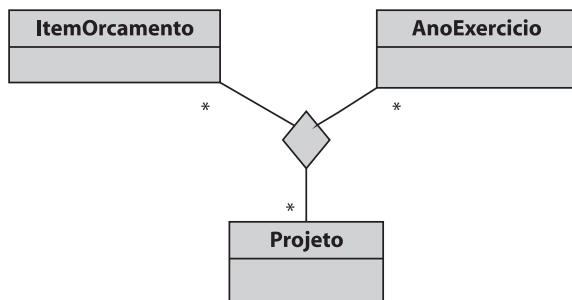


Figura 7.29: Exemplo de associação ternária.

Esse exemplo foi tirado de uma aplicação real de controle de orçamento. Cada projeto associa-se a um item de orçamento e um exercício. Para cada ano-exercício pode haver um variado número de itens de orçamento por projeto. Isso tem de ser representado por uma associação ternária.

Esses casos são *muito raros*, mas podem existir. Antes de decidir usar uma associação n-ária, que pode gerar inconvenientes no projeto e implementação, deve-se verificar se não é o caso de várias associações binárias. Por exemplo, o caso da Figura 7.30a não é uma associação ternária, mas duas associações binárias. A opção binária deve ser sempre preferida.

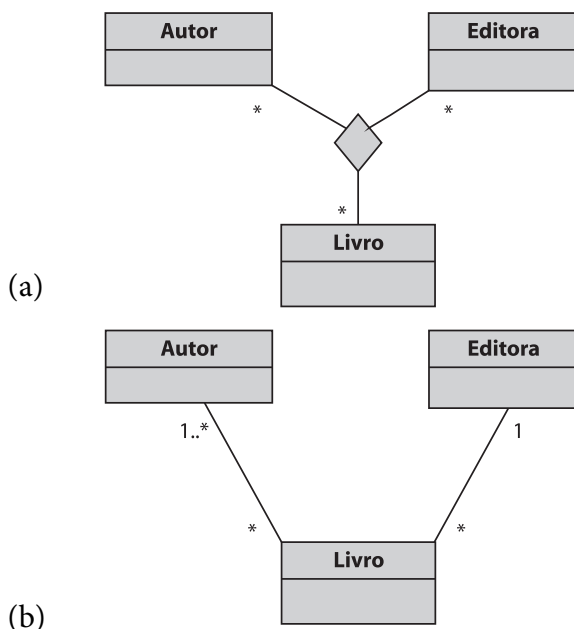


Figura 7.30: (a) Uma associação ternária indevida. (b) A solução correta com duas associações binárias.

No caso da Figura 7.30, a associação não deve ser ternária porque não existe uma relação de autor com editora independentemente do livro. Ou seja, o autor se relaciona ao livro e a editora se relaciona ao livro, mas o autor não se relaciona à editora.

7.5. Organização do Modelo Conceitual

A construção do modelo conceitual envolve mais do que simplesmente juntar conceitos, atributos e associações. Para que o modelo consista em uma representação fiel e organizada da informação gerenciada pelo sistema, é necessário que certas técnicas de modelagem sejam utilizadas.

As técnicas de organização de conceitos seguem três grupos distintos:

- a) *estruturais*: representando relações de generalização estrutural de conceitos, como, por exemplo, Pessoa, generalizando PessoaFisica e PessoaJuridica;
- b) *associativas*: representando relações de papéis associativos entre conceitos, como, por exemplo, Pessoa, podendo representar junto a uma empresa o papel de Comprador ou Funcionário;
- c) *temporais*: representando relações entre estados de um conceito como, por exemplo, um Livro e os estados da Figura 2.6: *encomendado, em estoque, vendido* etc.

Analistas principiantes e mesmo alguns experientes tendem a pensar que a única forma de fatorar informações é com herança. Mas deve-se saber distinguir quando usar cada uma das técnicas referidas. Só se usa herança quando um conceito efetivamente tiver dois ou mais subtipos. Uma instância nunca pode mudar de um subtipo para outro. Ela *nasce* no subtipo e *morre* no subtipo.

O caso, por exemplo, de Comprador e Funcionario não deve ser modelado com herança, ou seja, essas classes não devem ser subclasses de Pessoa, porque ninguém nasce comprador nem nasce funcionário. Além disso, uma mesma pessoa pode ser simultaneamente comprador e funcionário da mesma empresa ou até de diferentes empresas. Usar herança nesse caso vai causar problemas conceituais muito grandes no sistema (duplicações de cadastros, por exemplo).

As subseções seguintes detalham as três formas de organização de conceitos.

7.5.1. Generalização, Especialização e Herança

Durante anos, o uso de herança foi considerado como o grande mote da orientação a objetos. O mais importante na construção de um sistema era a definição de uma boa hierarquia de classes. Linguagens como *Smalltalk* (Goldberg & Robson, 1989) se estruturam totalmente sobre uma hierarquia de classes.

Com o passar do tempo, essa ênfase foi perdendo força, pois se percebeu que o uso da herança nem sempre era a melhor solução para problemas de modelagem, e hoje a herança é considerada apenas mais uma ferramenta de modelagem que ajuda a fatorar informações que, de outra forma, ficariam repetidas em diferentes conceitos.

A herança, em orientação a objetos é obtida quando duas classes se relacionam através de uma associação especial denominada *generalização* (no sentido da classe mais específica – *subclasse* –, para a mais genérica – *superclasse*) ou *especialização* (no sentido inverso).

A relação de generalização tem uma natureza muito diferente das associações do modelo conceitual. Ela só existe entre as classes, mas não entre as instâncias dessas classes. Se duas classes como Pessoa e Automovel são ligadas por uma associação simples, como na Figura 7.17, então as instâncias de Pessoa serão ligadas às instâncias de Automovel pelas realizações concretas dessa associação. Porém, se duas classes como Pessoa e PessoaFisica são ligadas pela relação de generalização, como na Figura 7.31, então as instâncias de PessoaFisica também *são* instâncias de Pessoa, ou seja, não existem instâncias de PessoaFisica ligadas a supostas instâncias de Pessoa: as instâncias de PessoaFisica são *simultaneamente* instâncias de Pessoa. Assim, toda instância de PessoaFisica tem atributos endereço e cpf. O inverso porém não ocorre, ou seja, nem toda instância de Pessoa é uma instância de PessoaFisica.

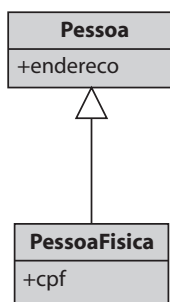


Figura 7.31: Generalização (herança).

Pode-se verificar então que, embora as associações simples entre classes do modelo conceitual se propaguem para as instâncias, a relação de herança não se propaga. Ela funciona como uma espécie de abreviação ou *macro* na definição das classes.

Assim, a única razão plausível para usar a relação de generalização existe quando é necessário fatorar as informações de duas ou mais classes (por “informações” das classes entende-se atributos e associações).

Se a superclasse puder ter suas próprias instâncias, ela é uma classe normal. Porém, se não for possível instanciar diretamente objetos da superclasse, mas apenas das subclasses, então a superclasse é *abstrata*. Classes abstratas são representadas em UML com seu nome escrito em itálico ou com a restrição {abstract}.

A generalização deve ser usada sempre que um conjunto de classes $X_1, \dots, X_n$, possuir diferenças específicas e semelhanças em comum, de forma que as semelhanças possam ser agrupadas em uma *superclasse* X (generalização das subclasses $X_1, \dots, X_n$), e as diferenças mantidas nas *subclasses* $X_1, \dots, X_n$. Essa situação está exemplificada na Figura 7.32.

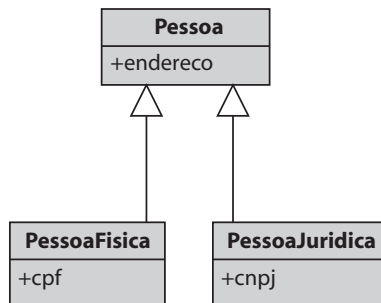


Figura 7.32: Representação esquemática de uma situação em que a herança pode ser usada.

Na Figura 7.32, a classe Pessoa é abstrata e não pode ter instâncias que não sejam instâncias de uma de suas subclasses. Instâncias de PessoaFisica têm cpf e endereco. Já instâncias de PessoaJuridica têm cnpj e endereco.

Não se deve usar a relação de generalização quando a superclasse não pode possuir nenhum atributo ou associação (Figura 7.33).

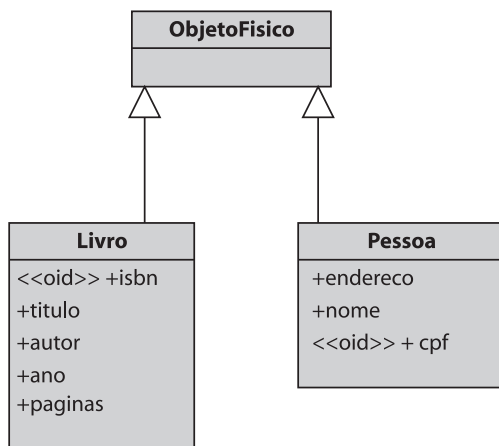


Figura 7.33: Situação em que a herança não deveria ser usada, pois não existem propriedades generalizadas.

Também não se deve usar herança quando as subclasses não possuem atributos ou associações que os diferenciem um do outro, como na Figura 7.34a. Nesses casos, usa-se apenas um atributo para diferenciar os tipos de pessoa, que estruturalmente são idênticos. Esse atributo diferenciador pode ser modelado como uma enumeração, como na Figura 7.34(b).

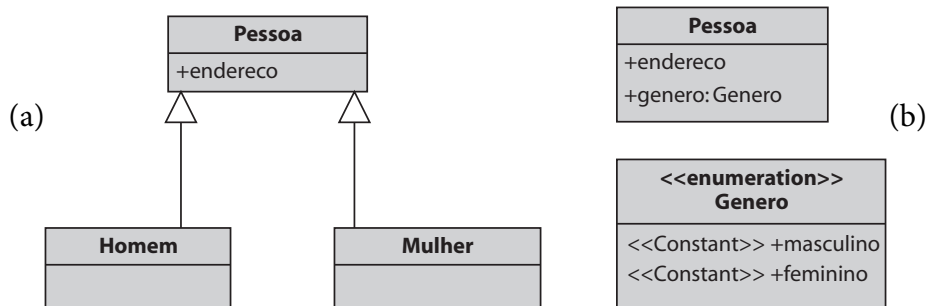


Figura 7.34: (a) Situação em que a herança não deve ser usada, pois não existem propriedades especializadas. (b) Modelagem alternativa mais adequada.

Além dessa regra, deve-se verificar, antes de decidir pelo uso da herança, se a generalização realmente representa uma classificação estrutural dos elementos, e não uma organização associativa ou temporária, como será visto nas seções seguintes.

7.5.2. Classes de Associação

Uma questão frequentemente mal modelada diz respeito a conceitos que no senso comum poderiam ser considerados subtipos, mas que na verdade são papéis. Por exemplo, em uma livraria, poderiam ser identificados conceitos como Comprador e Funcionario. Ao descobrir que ambos têm atributos em comum como nome, endereço etc., um analista menos avisado poderia criar uma superclasse Pessoa para generalizar esses dois conceitos, o que resultaria em um problema muito sério de modelagem, pois não se trata de tipos diferentes de pessoas, mas de papéis que pessoas têm em relação a uma empresa.

Para entender por que isso seria um problema, pode-se supor que um funcionário da livraria deseja comprar livros. Nesse caso, ele estará se comportando como comprador. Assim, a solução errada, mas frequente, acaba sendo criar um segundo cadastro do funcionário, agora como comprador. Como consequência, a mesma pessoa ficará com dois registros no sistema: um como funcionário e outro como comprador. Isso gera redundância nos dados e é uma fonte de inconsistências, visto que, por exemplo, se essa pessoa mudar de endereço, pode ser que seja registrado apenas que o Funcionario mudou de endereço, mantendo-se o endereço antigo para o Comprador.

A solução, nesse caso, é considerar que existe uma Pessoa que pode se relacionar com uma Empresa de pelo menos duas formas: como comprador ou como funcionário. As propriedades específicas de comprador (cartões de crédito, por exemplo) e de funcionário (salário, por exemplo), seriam propriedades da associação e não da pessoa. Para representar essas propriedades, define-se uma classe de associação para cada associação específica, como na Figura 7.35b.

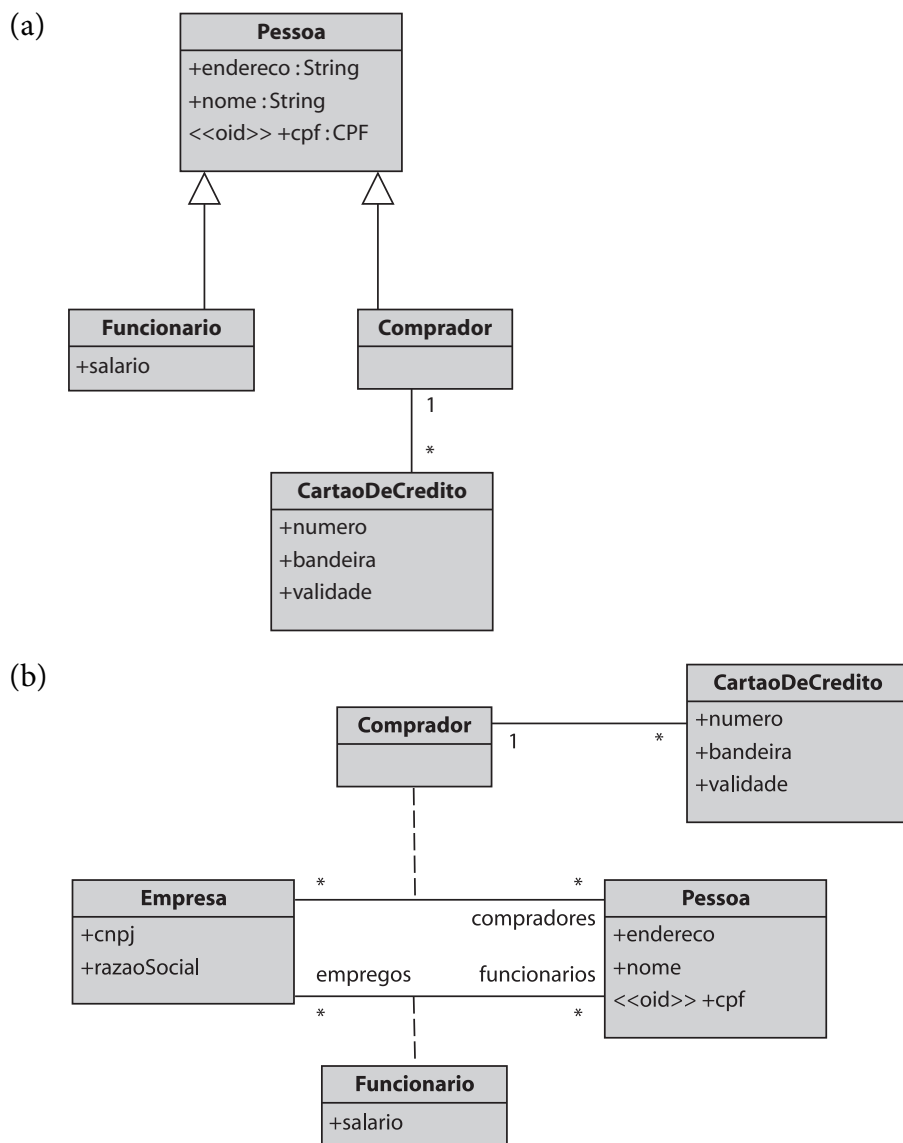


Figura 7.35: (a) Forma inadequada de representar papéis como herança. (b) Forma correta de representar papéis como classes de associação.

Portanto, quando uma mesma entidade pode representar diferentes papéis em relação a outras entidades, não se deve usar subclasses, mas classes de associação como solução de modelagem.

Para diferenciar a situação na qual se usa herança e a situação na qual se usa classe de associação, deve-se verificar se os subtipos do conceito considerado existem em função de um terceiro tipo ou não. Caso o subtipo só exista

de forma relativa, então se deve usar classe de associação. Por exemplo, ninguém pode ser *aluno*, simplesmente. Para alguém ser aluno deve haver uma instituição de ensino ou um professor. A pessoa tem de ser aluno *de* alguém ou alguma instituição.

Pode-se verificar, então, que é inadequado criar classes para representar tipos de pessoas que na verdade não são subclasses, mas papéis. Funcionários, professores, alunos, diretores, compradores etc. nunca poderiam ser subclasses de Pessoa. Esses conceitos só fazem sentido quando relacionados a outro conceito como empresa, escola, departamento etc. Deve-se ser professor *de* alguém, comprador *de* uma empresa, diretor *de* um departamento e assim por diante.

A diferença entre classe e associação e o uso de um conceito intermediário (transação) é bastante sutil. A Figura 7.36 mostra dois modelos alternativos parecidos, mas com uma pequena diferença.

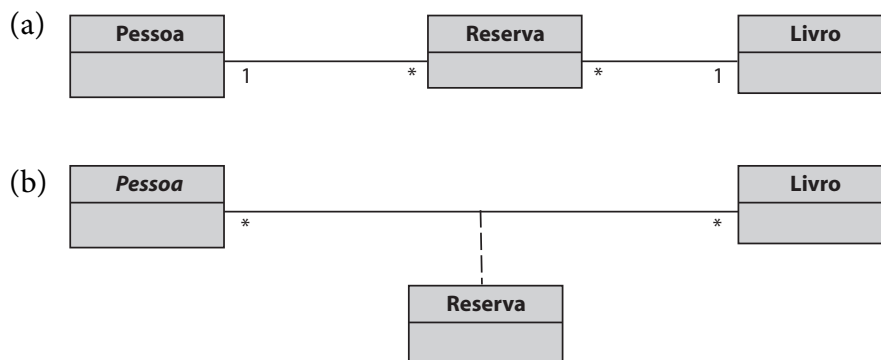


Figura 7.36: Modelagem de uma reserva (a) como um conceito e (b) como classe de associação.

No caso *a* da Figura 7.36, uma reserva associa uma pessoa a um livro. No caso *b* também. Mas, no caso *a*, uma pessoa pode ter várias reservas para o mesmo livro. No caso *b*, uma pessoa só pode ter, no máximo, uma única reserva para um mesmo livro.

7.5.3. Classes Modais

Classes modais ou *classes com estados* são usadas para modelar conceitos cujas instâncias podem mudar de um estado para outro ao longo de sua existência, alterando possivelmente sua estrutura, valores de atributos ou comportamento dos métodos. Embora algumas linguagens de programação, como *Smalltalk* (Goldberg & Robson, 1989), até permitem que instâncias de

objetos mudem de classe dinamicamente, isso não deve ser assumido como um princípio de modelagem, pois tais mudanças podem acarretar problemas estruturais complexos.

A rigor, uma instância, depois de criada, não poderá mudar de classe, visto que, mesmo que isso fosse possível, resta o problema de redefinir atributos e associações da nova instância. Assim, deve-se usar técnicas de modelagem que não exigem que objetos troquem dinamicamente de classe. Quando um objeto muda, é seu *estado* que muda, não sua classe.

São identificadas, aqui, três situações relacionadas à modelagem de estados:

- transição estável*: os diferentes estados de um objeto não afetam sua estrutura, mas apenas, possivelmente, valores de atributos;
- transição monotônica*: o objeto passa de um estado para outro e à medida que muda de estado vai ganhando novos atributos ou associações;
- transição não monotônica*: o objeto pode ganhar ou perder atributos ou associações à medida que muda de estado.

A modelagem da forma estável e da forma monotônica é simples. Já a modelagem da transição não monotônica exigirá a aplicação de um padrão de projeto denominado *Estado*, conforme será visto nas próximas subseções.

7.5.3.1. Transição Estável

Frequentemente, os diferentes estados de um objeto podem ser determinados através de um simples atributo. Por exemplo, o estado de uma Venda poderia ser modelado simplesmente com um atributo estado, tipado como uma enumeração de em andamento, concluída e paga.

Outro exemplo seria o atributo suspenso de um endereço, que indica que houve alguma devolução de mercadoria nesse endereço. O endereço fica nesse estado até que o comprador o atualize, pois possivelmente contém um erro.

Endereco
+cep
+rua
+numero
+bairro
+complemento
+suspenso : Booleano

Figura 7.37: Um conceito com transição de estado estável.

O atributo *suspenso* tem valor booleano e, se for verdadeiro, o endereço não pode ser usado para entregas.

A transição é considerada *estável* porque apenas o valor do atributo muda. A estrutura interna do objeto não é alterada, como nos dois subcasos seguintes.

7.5.3.2. Transição Monotônica

A situação é um pouco mais complexa quando, em função dos diferentes estados, o conceito pode adquirir diferentes atributos ou associações. Por exemplo, pagamentos no estado *pendente* têm apenas *vencimento* e *valorDevido*. Já os pagamentos no estado *liquidado* têm adicionalmente os atributos *dataDePagamento* e *valorPago* (Figura 7.38).

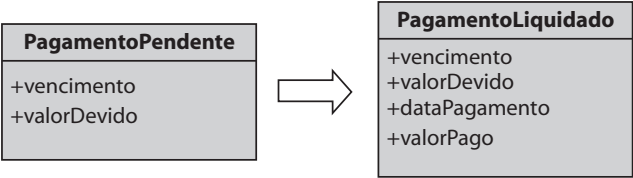


Figura 7.38: Um conceito com transição monotônica.

Diz-se que a transição de estado no caso da Figura 7.38 é monotônica porque novos atributos ou associações são acrescentados, mas nada é retirado. Isso implica que um pagamento liquidado não pode retroceder e se tornar novamente um pagamento pendente.

Seria incorreto modelar essa situação com herança de propriedades, como na Figura 7.39, pois, nesse caso, uma instância de *PagamentoPendente* só poderia se tornar uma instância de *PagamentoLiquidado* se fosse destruída e novamente criada, com todos os seus atributos (inclusive os comuns) novamente definidos. Essa forma não é muito prática e exige, quando implementada, mais processamento do que se poderia esperar, visto que, além dos atributos, várias associações poderão ter de ser refeitas.

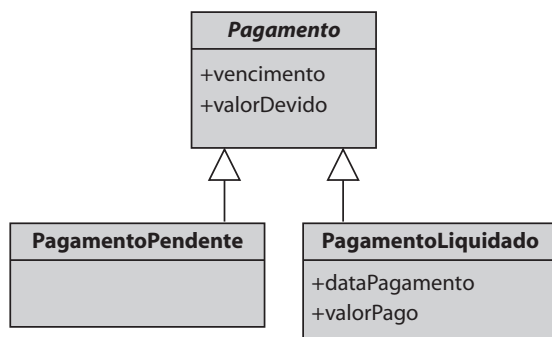


Figura 7.39: Forma inconveniente de modelar estados monotônicos com herança.

Outra solução não muito prática, mas ainda muito usada, consiste em criar uma única classe *Pagamento* e fazer com que certos atributos sejam nulos até que a classe mude de estado. Essa situação é indicada na Figura 7.40. Usualmente, a verificação da consistência da classe é feita nos métodos que a atualizam. Mas também poderia ser usada uma invariante (conforme explicado adiante) para garantir que nenhuma instância entre em um estado inválido, como, por exemplo, com *valorPago* definido e *dataDePagamento* indefinida.

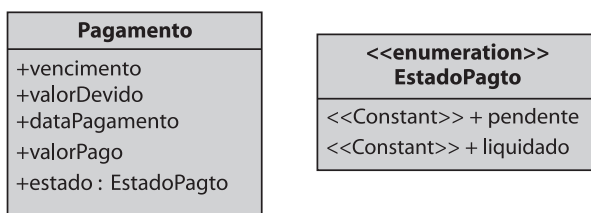


Figura 7.40: Modelagem inconveniente de estados monotônicos com uma única classe com atributos possivelmente nulos.

Essa forma de modelagem ainda não é boa, pois gera classes com baixa coesão e, portanto, com regras de consistência complexas que devem ser checadadas frequentemente. Essas classes com *baixa coesão* são altamente suscetíveis a erros de projeto ou programação.

Melhor seria modelar o conceito de pagamento de forma que o controle da consistência do objeto fosse feito através da própria estrutura do modelo. Como se trata de uma transição monotônica, é possível modelar essa situação simplesmente desdobrando o conceito original de pagamento em dois: um que representa o pagamento em aberto e outro que representa apenas os atributos ou associações adicionadas a um pagamento quando ele é liquidado.

Esses dois conceitos são, então, ligados por uma associação simples com multiplicidade de papel 1 no lado do conceito original e 0..1 no lado do conceito que complementa o original (Figura 7.41).

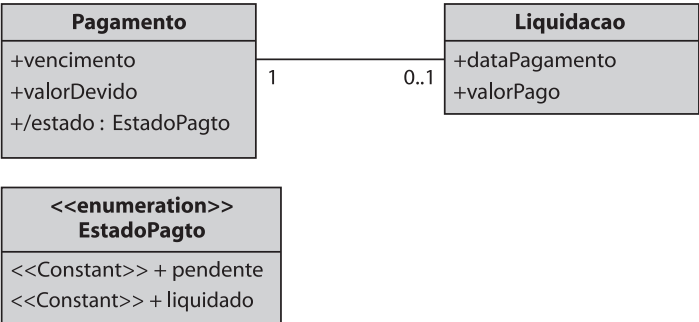


Figura 7.41: Forma eficaz de modelar classes modais com transição monotônica.

Com a modelagem indicada na Figura 7.41, percebe-se que é impossível que um pagamento não liquidado tenha dataDePagamento ou valorPago definidos. Já o estado do pagamento pode ser definido como um atributo derivado da seguinte forma: se existe uma associação com Liquidacao a partir da instância de Pagamento, então o estado é liquidado; caso contrário, o estado é pendente. Em OCL:

```
Context Pagamento::estado
derive:
    if self.liquidacao.isNull() then
        EstadoPagto::pendente
    else
        EstadoPagto::liquidado
    endIf
```

Em relação à notação, verifica-se que:

- a) a OCL possui estruturas de seleção na forma *if-then-else-endIf*. Se a expressão após o if for verdadeira, então a expressão toda é avaliada como a parte que vem entre o then e o else, caso contrário ela é avaliada como a parte que vem entre o else e o endIf;
- b) a função isNull() aplicada a uma propriedade retorna true se ela é indefinida (ou seja, null) e false caso contrário;
- c) a referência a uma constante de enumeração em OCL é feita usando-se o nome da enumeração seguido de “::” e o nome da constante, como EstadoPagto::pendente.

7.5.3.3. Transição Não Monotônica

Na transição monotônica, cada vez que um objeto muda de estado, ele pode adquirir novos atributos ou associações que não possuía antes. Contudo, se, além disso, o objeto puder ganhar e perder atributos ou associações, a transição é dita *não monotônica*.

Felizmente, é raro que em algum sistema se deseje *perder* alguma informação. Mas, às vezes, por questões práticas, isso é exatamente o que precisa acontecer.

Existem várias maneiras de se conceber e modelar um sistema de reservas em um hotel. Uma delas consiste em entender a hospedagem como uma entidade que evolui a partir de uma reserva da seguinte forma:

- inicialmente, um potencial hóspede faz uma reserva indicando os dias de chegada e saída, o tipo de quarto e o número de pessoas. O hotel lhe informa a tarifa;
- quando o hóspede faz o *checkin*, é registrado o dia de chegada (pode eventualmente ser diferente do dia previsto na reserva). O hotel lhe atribui um quarto, que eventualmente pode até ser diferente do tipo inicialmente reservado e, se for o caso, informa a nova tarifa. A data de saída prevista continua existindo, embora seu valor possa ser mudado no momento do *checkin*;
- quando o hóspede faz o *checkout*, deixa de existir a data prevista de saída para passar a existir a data de saída de fato; nesse momento, a conta precisa ser paga.

Esse conjunto de estados poderia ser modelado na fase de concepção por um diagrama de máquina de estados como o da Figura 7.42.

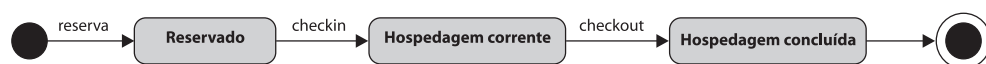


Figura 7.42: Uma máquina de estados para modelar uma hospedagem.

Se a hospedagem apenas adquirisse novos atributos e associações à medida que seus estados evoluem, ela poderia ser representada por uma sequência de conceitos ligados por associações de 1 para 0..1, como na Figura 7.43.

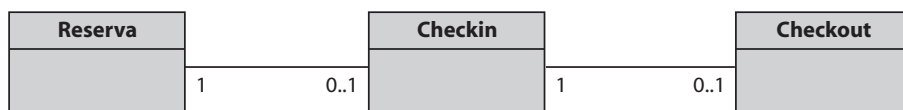


Figura 7.43: Possível modelagem de estados de uma reserva caso fosse monotônica.

Porém, observa-se que, com a transição de estados, alguns atributos e associações deixam de existir, como, por exemplo, as datas previstas, que são substituídas por datas reais, e a associação da reserva com um tipo de quarto, que é substituída pela associação da hospedagem com um quarto real.

Esse fato faz com que seja necessário usar um padrão de projeto (*design pattern*) conhecido como estado (*state*). De acordo com esse padrão, deve-se, primeiramente, separar o conceito de seu estado. Os atributos e associações que são comuns a todos os estados devem ser colocados no conceito original. Já os atributos e associações que são específicos dos estados devem ser modelados apenas nos estados onde devem ocorrer. Esses estados específicos são especializações de uma classe abstrata, como na Figura 7.44.

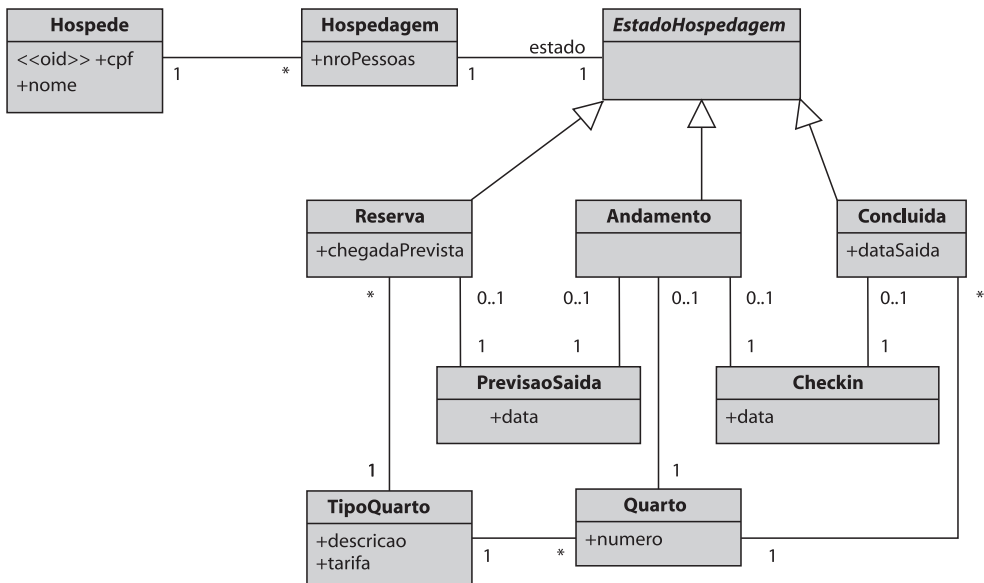


Figura 7.44: Modelagem de estados não monotônicos usando o padrão Estado.

Observa-se, na Figura 7.44, que o atributo `nroPessoas` é válido em qualquer estado de uma hospedagem, por isso é modelado no conceito `Hospedagem`. Há, depois, três estados possíveis:

- Reserva, pelo qual uma hospedagem, além do número de pessoas, tem chegada e saída previstas e o tipo de quarto;
- Andamento, pelo qual, além do número de pessoas, a hospedagem tem chegada efetiva, saída prevista e um quarto efetivo;
- Concluida, pelo qual, além do número de pessoas, a hospedagem tem chegada e saída efetivas e um quarto efetivo.

Quando um atributo é específico a um único estado, ele pode ser representado como atributo do estado, como, por exemplo, `chegadaPrevista` e `dataSaida` na Figura 7.44, mas atributos comuns a dois ou mais estados devem ser representados como conceitos separados, para que possam ser associados aos diferentes estados. No caso da Figura 7.44, a saída prevista é comum aos estados Reserva e Andamento. Por isso, ela foi transformada em conceito e ligada a ambos os estados por associação de 0..1 para 1. Já a data de chegada é comum aos estados Andamento e Concluída e, portanto, foi representada como um conceito à parte (Checkin) associado a ambos os estados por uma associação de 0..1 para 1.

Para complementar o modelo, ainda seria necessário estabelecer que uma instância de `PrevisaoSaida` deve se associar a apenas uma instância dentre Reserva e Andamento de cada vez. Igualmente, Checkin só pode se associar a uma instância de Andamento ou uma instância de Concluída de cada vez. Mais adiante, será explicado como fazer isso com invariantes.

Observa-se também que um quarto pode se associar a, no máximo, uma hospedagem em andamento (associação para 0..1), mas pode se associar a um número qualquer de hospedagens concluídas (associação para *). Assim, já fica modelada também a propriedade temporal dessa associação, que tem multiplicidade 1 no presente, mas * no histórico.

7.6. Padrões de Análise

Fazer um modelo conceitual é muito mais do que amontoar conceitos, atributos e associações em um diagrama. Frequentemente percebe-se que a modelagem simplesmente não funciona porque fica complicado demais continuar a enriquecê-la.

Existem técnicas, porém, que diminuem a complexidade desses diagramas e, ao mesmo tempo, aumentam sua expressividade, permitindo que sejam modeladas, de forma simples, situações que, abordadas de forma ingênua, poderiam gerar modelos altamente complexos.

Essas técnicas são chamadas por Fowler (2003) de *padrões de análise* e podem ser concebidas como um caso especial de padrões de projeto (Gamma *et al.*, 1999) aplicados ao modelo conceitual.

Padrões de análise ou projeto não são regras que obrigatoriamente devem ser aplicadas, mas sugestões baseadas em experiências prévias. Cabe ao analista decidir quando aplicar ou não determinado padrão em sua modelagem.

7.6.1. Coesão Alta

Um dos padrões mais fundamentais consiste na definição de conceitos de boa qualidade, ou seja, coesos. Um conceito coeso é mais estável e reusável do que um conceito não coeso, que pode se tornar rapidamente confuso e difícil de manter. A maioria dos sistemas poderia ter suas informações representadas em um único “tabelão” com baixíssima coesão, mas isso não seria nada prático.

Já foi mencionado que conceitos não devem ter atributos de outros conceitos (um automóvel não deve ter como atributo o CPF de seu dono). Atributos também não devem ser tipados com estruturas de dados (listas, conjuntos etc.), pois isso é uma evidência de baixa coesão (uma classe com atributos desse tipo estaria representando mais do que um único conceito). Por exemplo, uma Venda não deveria ter um atributo listaDeltens, pois os itens devem aparecer como um conceito separado ligado à Venda por uma associação de 1 para *.

Além disso, é importante que conceitos tenham atributos que sejam efetivamente compostos por uma estrutura simples e coesa. Quando alguns atributos podem ser nulos dependendo do valor de outros atributos, isso é sinal de baixa coesão. Restrições complexas poderão ser necessárias para manter o conceito consistente. Isso equivale a usar fita adesiva para tentar manter juntos os cacos de um vaso quebrado.

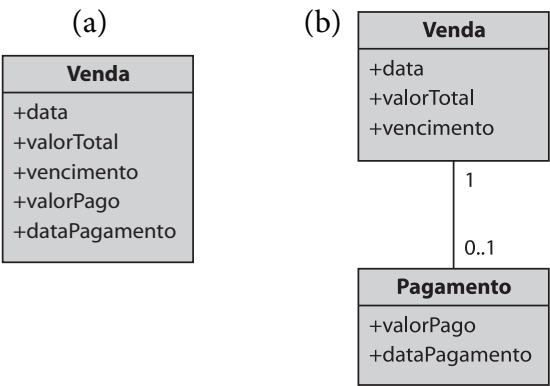


Figura 7.45: (a) Uma classe com baixa coesão por ter atributos dependentes de outros. (b) Uma solução de modelagem com classes mais coesas.

Na Figura 7.45a, os atributos valorPago e dataPagamento são mutuamente dependentes: ou ambos são nulos ou ambos são definidos. Uma restrição ou invariante de classe teria de estabelecer isso como regra para evitar que

instâncias inconsistentes surgissem. Mas uma forma melhor de modelar essa situação é mostrada na Figura 7.45b, na qual os conceitos Venda e Pagamento aparecem individualmente mais coesos. Nesse caso, não há mais atributos dependentes entre si.

Outro problema potencial é a existência de grupos de atributos fortemente correlacionados, como na Figura 7.46a, na qual se observa que grupos de atributos se relacionam mais fortemente entre si do que com outros, como rua, numero, cidade e estado, que compõe um endereço, ou ddd e telefone, que fazem parte de um telefone completo, ou ainda rg, orgaoExpedidor e ufOrgaoExpedidor, que são atributos do documento de identidade da pessoa.

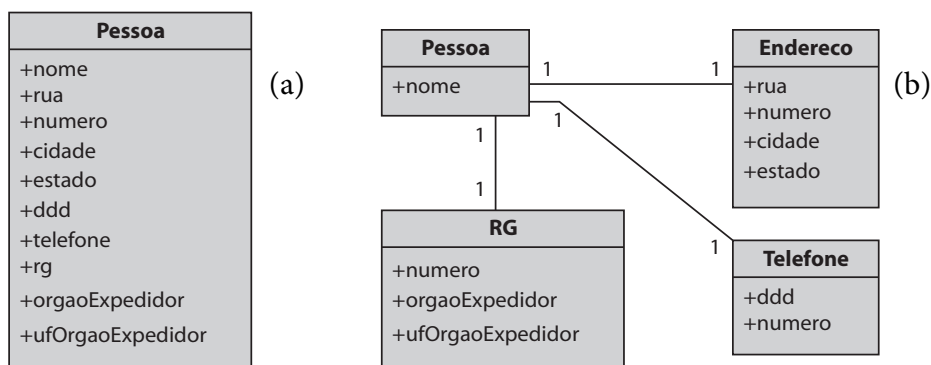


Figura 7.46: (a) Uma classe com baixa coesão por ter grupos de atributos fortemente correlacionados. (b) Uma solução com melhor coesão.

A solução para melhorar a coesão mostrada na Figura 7.46b também abre caminho para outras possibilidades de modelagem, como, por exemplo, permitir que uma pessoa tenha mais de um endereço ou mais de um telefone, caso as associações sejam trocadas por associações de um para muitos.

Outra situação ainda ocorre quando determinados atributos repetem sempre os mesmos valores em diferentes instâncias. A Figura 7.47a apresenta um exemplo.

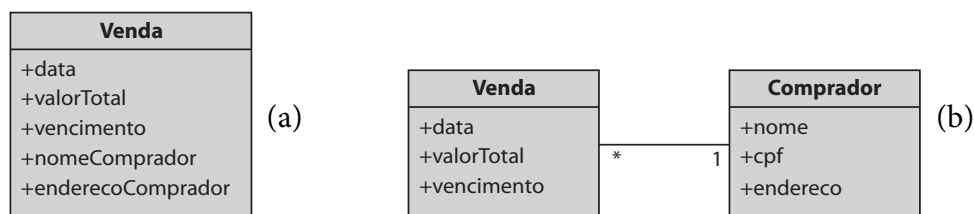


Figura 7.47: (a) Uma classe com baixa coesão por ter atributos que repetem valores nas instâncias. (b) Uma solução com melhor coesão.

Na Figura 7.47a, os atributos nomeComprador, cpfComprador e enderecoComprador vão se repetir em diferentes compras quando o comprador for o mesmo. Esse tipo de situação pode ser eliminado com a separação do conceito não coeso em dois conceitos associados, como na Figura 7.47b.

7.6.2. Classes de Especificação

Um caso especial de baixa coesão também ocorre quando se confunde um objeto com sua especificação. Até o momento, no sistema Livir, não foi discutida a diferença entre o conceito de obra literária e *cópia* de obra literária. Ambos os conceitos são tratados simplesmente como Livro, sem distinção. Mas ambos são entidades distintas e devem ser diferenciados. Por exemplo, título, ISBN, autor, preço de capa etc. aplicam-se à obra literária, pois se fossem aplicados a cada cópia iriam se repetir nas instâncias. Outra evidência de que se trata de conceitos diferentes é que, quando um livro é meramente reservado por não haver em estoque, reserva-se a obra literária, mas quando um livro é vendido, é vendida a cópia física.

Essa situação é muito frequente: muitas vezes produtos ou itens físicos compartilham uma especificação comum. Especificação e item físico devem ser modelados como dois conceitos separados, unidos por uma associação de um para muitos, como na Figura 7.48.

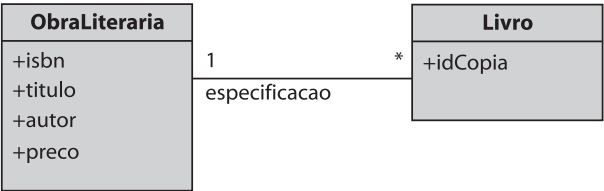


Figura 7.48: Uma classe com sua classe de especificação.

É possível que uma classe possua mais de uma especificação. Por exemplo, cópias de livros, além de serem especificadas pela obra literária, podem ser especificadas pelo seu estado (novo, usado, muito usado etc.). Em função do estado, um percentual de desconto pode ser aplicado ao livro. Assim, estado de uso seria uma classe com algumas instâncias que especificam cópias físicas de livros e definem seus percentuais de desconto, como na Figura 7.49.

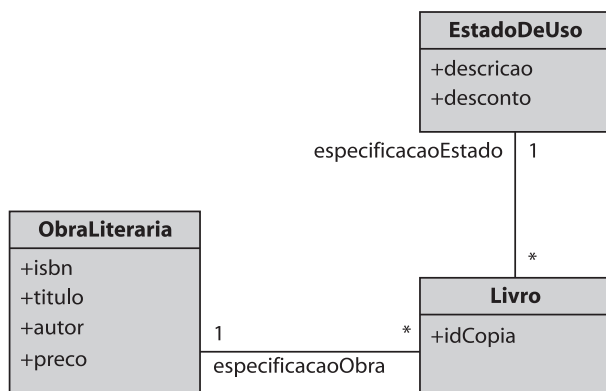


Figura 7.49: Uma classe com duas classes de especificação.

7.6.3. Quantidade

Frequentemente, o analista se depara com a necessidade de modelar quantidades que não são meramente números. O peso de um livro, por exemplo, poderia ser definido como 400. Mas 400 o quê? Gramas? Libras? Quilos? Uma solução é definir um tipo específico para o peso e então usá-lo sempre consistentemente. O atributo, então, seria declarado como `peso:Gramas`. Mas isso exige que o peso de todos os livros seja expresso em gramas. Se a informação vier em outra unidade, terá de ser convertida ou estará inconsistente.

Em alguns casos, espera-se que seja possível configurar o sistema informatizado para suportar diferentes medidas. Em alguns países se usam gramas, e em outros, libras. Se a classe for modelada com gramas, o sistema terá de ser refeito para aceitar libras.

Porém, o padrão “*Quantidade*” permite que diferentes sistemas de medição coexistam sem conflito e sejam facilmente intercambiáveis. O padrão consiste na criação de um novo tipo de dados primitivo *Quantidade*, com dois atributos, como mostrado na Figura 7.50.

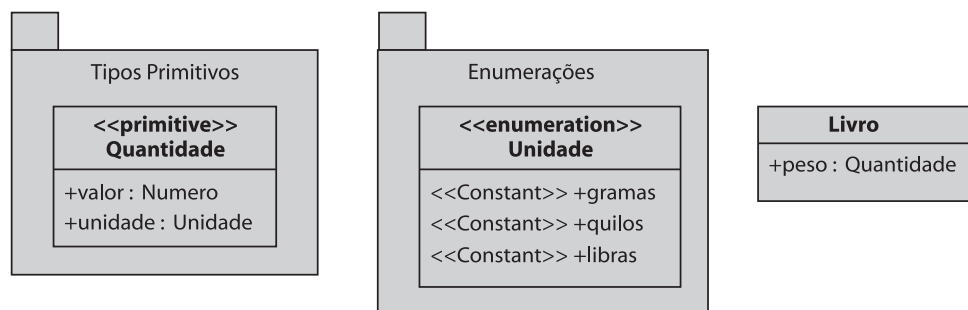


Figura 7.50: Definição e uso de *Quantidade*.

Dessa forma, o peso de cada livro será especificado como uma quantidade formada por um valor numérico e uma unidade, que corresponde a uma enumeração dos valores quilos, gramas e libras.

Caso se necessite estabelecer *razões de conversão* entre unidades, uma opção seria transformar a enumeração Unidade em uma classe normal e criar uma classe Razao associada a duas unidades: origem e destino, como na Figura 7.51. Quando uma quantidade da unidade origem tiver de ser convertida em uma quantidade da unidade destino, divide-se seu valor pelo valor da razão.

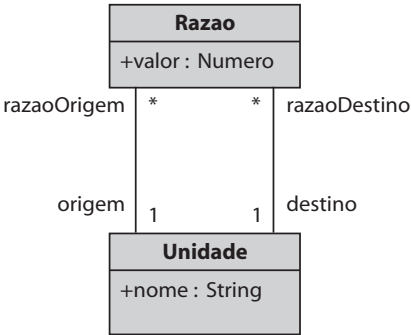


Figura 7.51: Unidades com razão de conversão.

Assim, por exemplo, a instância de Unidade, cujo nome é “gramas” pode estar ligada a uma instância de Razao, que por sua vez liga-se a uma instância de Unidade cujo nome é “quilos”. O valor dessa instância de Razao será então 1000 porque, para converter uma quantidade em gramas para uma quantidade em quilos, deve-se dividir por 1.000.

76.4. Medida

Uma evolução do padrão *Quantidade* é o padrão *Medida*, que deve ser usado quando for necessário realizar várias medidas diferentes, possivelmente em tempos diferentes a respeito de um mesmo objeto. Por exemplo, uma pessoa em observação em um hospital pode ter várias medidas corporais sendo feitas de tempos em tempos: temperatura, pressão, nível de glicose no sangue etc. Milhares de diferentes medidas poderiam ser tomadas, mas apenas umas poucas serão efetivamente tomadas para cada paciente. Então, para evitar a criação de um conceito com milhares de atributos dos quais a grande maioria permaneceria nulo, a opção é usar o padrão *Medida*, como na Figura 7.52.

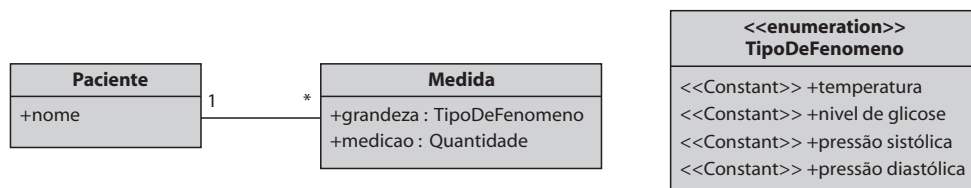


Figura 7.52: Definição e uso do padrão Medida.

Assim, um paciente terá uma série de medidas tomadas, cada uma avaliando um tipo de fenômeno e apresentando um valor que corresponde a uma quantidade (conforme padrão *Quantidade*).

Ainda é possível sofisticar mais uma medida adicionando atributos para indicar o instante do tempo em que a medida foi tomada e, também, o prazo de validade da medida. Por exemplo, o fato de que um paciente tinha febre há duas horas não continua necessariamente sendo verdadeiro no presente, ou seja, a medida já pode estar inválida.

7.6.5. Estratégia

Foi mencionado que um dos desafios dos requisitos é estar preparado para sua mudança. Especialmente os requisitos transitórios (aqueles que se prevê que vão mudar) devem ser acomodados no projeto do sistema de forma que sua mudança, quando ocorrer, minimize o impacto das alterações sobre o sistema e, conseqüentemente, seu custo.

Alguns casos são relativamente fáceis de tratar. Por exemplo, se houver uma previsão de que a moeda corrente do país poderá mudar (isso aconteceu muitas vezes entre 1980 e 1994), basta usar o padrão *Quantidade* ou, simplesmente, tratar o tipo de moeda como um parâmetro de sistema que pode ser alterado.

Mas há situações mais complexas. Por exemplo, a forma de calcular impostos pode variar muito. Há impostos que são calculados sobre o preço de venda dos produtos, outros são calculados sobre o lucro, outros são calculados sobre a folha de pagamento. As formas variam e, historicamente, uma quantidade significativa de novos impostos é criada ao longo de um ano. Os sistemas devem estar preparados para isso, mas as mudanças são completamente imprevisíveis.

Outra situação é a política de descontos da empresa. É possível que, no momento da contratação, a livraria aplique uma política de dar desconto de 10% para compras acima de 100 reais. Mas, com o passar do tempo, novas e imprevisíveis políticas podem ser criadas pelos departamentos comerciais, por exemplo:

- a) um livro grátis de até 50 reais para compras acima de 300 reais;
- b) 20% de desconto em até dois livros no dia do aniversário do comprador;
- c) 5% de desconto nos livros de suspense nas sextas-feiras 13.

Além disso, pode ser permitido combinar duas ou mais políticas, caso se apliquem, ou escolher apenas aquela que dá o maior desconto.

O padrão *Estratégia* sugere que, nesses casos, o procedimento deve ser separado dos dados aos quais ele se aplica. Ou seja, se é aplicado um desconto em uma venda, o desconto não deve ser meramente um método da venda a ser alterado quando houver mudanças. O desconto será representado por uma classe abstrata associada à venda. Essa classe abstrata terá subclasses concretas que representarão políticas concretas de desconto (Figura 7.53).

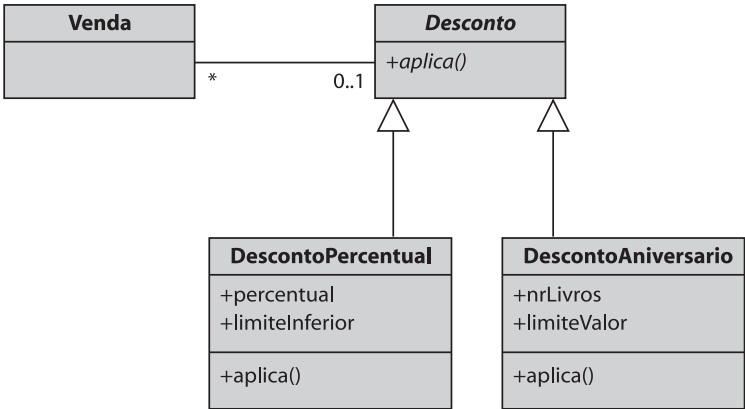


Figura 7.53: Padrão Estratégia.

Esse padrão não é puramente conceitual, pois sua realização envolve a existência de métodos (que pertencem ao domínio do projeto). Então, cada instância de *Venda* será associada a uma instância de uma das subclasses da estratégia de desconto. A classe abstrata *Desconto* implementa um método abstrato *aplica()*, que é aplicado de forma concreta nas subclasses. Caso alguma das estratégias concretas precise de dados específicos do comprador ou da

venda, eles podem ser acessados através das associações da classe de desconto concreta para as classes que contêm a informação necessária através de Venda.

Esse padrão minimiza dois problemas com a mudança desse tipo de requisito. Primeiro, ele mantém, para cada venda, o registro da estratégia de desconto aplicada na época em que a venda foi feita. Então, essa informação não se perde. Em segundo lugar, se novas estratégias de desconto forem criadas no futuro, basta implementar novas subclasses para Desconto. Isso não afeta o funcionamento das estratégias anteriores.

7.6.6. Hierarquia Organizacional

Outra situação comum consiste na necessidade de representar hierarquias organizacionais que nem sempre se comportam bem. É comum, por exemplo, representar a estrutura administrativa do Brasil com os níveis de estados e municípios, como na Figura 7.54.

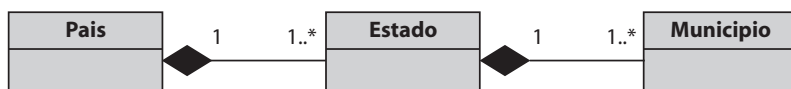


Figura 7.54: Representação direta de uma hierarquia organizacional usando classes e composição.

Porém, hierarquias normalmente não se comportam de forma tão simples. Pode-se observar, de início, que essa organização não se repete em muitos países, que adotam outras formas de divisão administrativa. Além disso, reformas políticas e administrativas podem mudar a hierarquia, criando subdivisões ou agrupando diferentes níveis hierárquicos. Aliás, diferentes estruturas organizacionais podem coexistir, por exemplo, com estados sendo divididos em municípios, do ponto de vista do executivo, mas em comarcas, do ponto de vista do judiciário.

Como, então, lidar com toda essa complexidade no modelo conceitual? Usando o padrão *Hierarquia Organizacional*.

A solução consiste em não considerar mais os diferentes tipos de organização como conceitos, mas como instâncias de um conceito único: a *estrutura organizacional*, como na Figura 7.55.

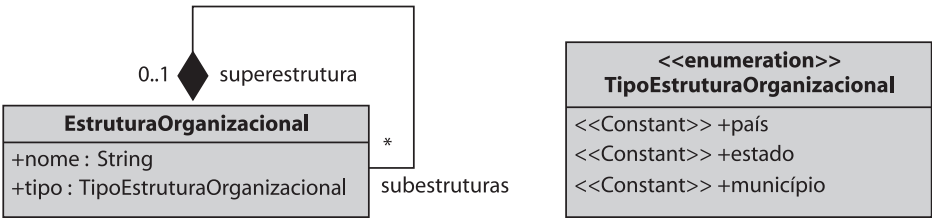


Figura 7.55: Aplicação do padrão Estrutura Organizacional.

Dessa forma, ganha-se flexibilidade em relação a poder lidar simultaneamente com estruturas organizacionais de diferentes países, bem como se pode mais facilmente lidar com suas mudanças, como a criação de novos níveis hierárquicos.

Esse padrão pode ter inúmeras variantes conforme se queira representar hierarquias concorrentes, estruturas sucessoras ou equivalentes e outras situações que podem surgir. Algumas serão comentadas nas seções seguintes.

7.6.7. Junção de Objetos

Um dos pressupostos de analistas que muitas vezes falha é que os usuários, ao operarem o sistema, farão tudo de acordo com o previsto. Isso nem sempre acontece. A falha humana ainda é frequente, apesar dos esforços no sentido de se construir interfaces cada vez mais à prova de falhas. Algum usuário poderia, por exemplo, cadastrar uma nova editora no sistema e, mais adiante, descobrir que ela, na verdade, já era cadastrada. Um código registrado erroneamente ou a impossibilidade de se ter um identificador único para um objeto com significado no mundo real podem causar essa situação.

A solução, quando esse erro de usuário ocorre, é fazer a junção dos objetos, usualmente copiando um sobre o outro. Essa operação pode ser executada diretamente como uma correção no banco de dados, mas em algumas situações pode ser necessário que o sistema esteja preparado para permitir ao próprio usuário fazer essa junção.

Além disso, nem sempre é um *erro* que provoca a necessidade de uma junção. Em algumas situações, a equivalência entre diferentes instâncias de um conceito pode não ser consenso, sendo necessário representar duas ou mais visões contraditórias simultaneamente. Em outros casos, como nas hierarquias organizacionais múltiplas, pode ser necessário indicar que duas es-

estruturas organizacionais em hierarquias diferentes são equivalentes ou, ainda, que uma é sucessora de outra.

As subseções seguintes vão apresentar as principais estratégias para lidar com este tipo de situação.

7.6.7.1. Copiar e Substituir

A primeira estratégia em que se pensa quando é necessário juntar dois objetos que na verdade são um só consiste em copiar os dados de um sobre o outro (*copy and replace* ou *copiar e substituir*). A operação de cópia deve ser definida por contrato (ver Capítulo 8) e o analista deve definir, para cada atributo e cada associação, o que deve acontecer durante a cópia. Regras serão definidas para dizer se um atributo será copiado sobre outro, se seus valores serão somados ou se o maior dentre eles deve permanecer etc. Quanto às associações, o analista deve decidir o que acontece: se uma associação sobrescreve outra, se elas se adicionam e assim por diante.

O registro da data da última inclusão ou alteração de um conceito pode ser uma ferramenta útil para que se tenha como decidir qual atributo manter no caso de conflito. Por exemplo, um comprador cadastrado duas vezes para o qual constam dois endereços diferentes possivelmente deverá manter apenas o registro do endereço mais recente. Por outro lado, todas as compras que esse comprador tenha efetuado devem ser agrupadas na instância resultante.

Depois de efetuar a cópia ou junção dos dados de uma instância sobre a outra, a instância que foi copiada deve ser destruída e quaisquer referências a ela devem ser redirecionadas para a instância que recebeu os dados da cópia.

7.6.7.2. Sucessor

Sucessor (*Superseding*) é uma técnica que pode ser usada quando se pretende manter o objeto original sem destruí-lo. Aplica-se *Sucessor*, por exemplo, no caso de estruturas organizacionais que se sucedem no tempo. Supondo que os departamentos de *venda* e *marketing* sejam unidos em um único departamento de *contato com clientes*, os departamentos originais devem ser mantidos mas marcados como não mais ativos, e o novo departamento deve ser marcado como sucessor deles. Implementa-se a estratégia *Sucessor* através de uma associação reflexiva, como na Figura 7.56.

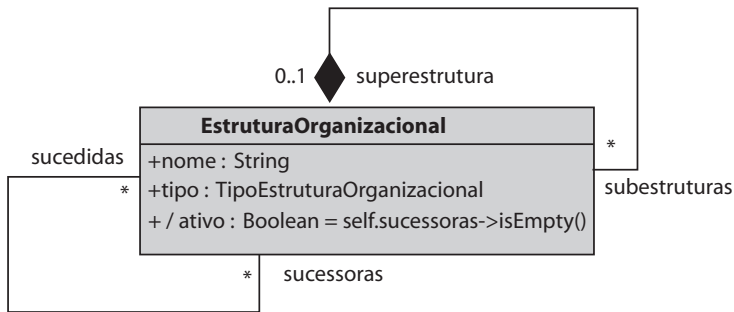


Figura 7.56: Um exemplo de aplicação da estratégia *Sucessor*.

O atributo derivado *ativo* é *true* se o conjunto representado pelo papel *sucessoras* é vazio e *false* caso contrário.

Manter a estrutura original, mesmo que ela não seja mais ativa, pode ser importante para fins de registro. Algum dia, alguém pode querer saber quanto se gastava por mês no antigo departamento de marketing, quanto se gastava no antigo departamento de vendas e quanto se gasta atualmente com o departamento de contato com clientes.

Pode ser útil adicionar uma classe de associação à associação *sucessoras/sucedidas* cujos atributos poderiam indicar, entre outras coisas, a data em que houve o evento de sucessão.

7.6.7.3. Essência/Aparência

Outra situação que ainda pode surgir com frequência é a existência de objetos equivalentes dos quais se queira manter a individualidade. Não se trata nesse caso de um objeto que sucede a outro, como em *Sucessor*, mas de objetos que são considerados equivalentes.

Pode-se ter, em alguns casos, diferentes manifestações de um mesmo objeto, mas uma única essência por trás. Quando algo muda na essência, muda também em todas as manifestações ou aparências.

Essa técnica pode ser modelada com a criação de um *objeto essência* para ser associado a um conjunto de objetos equivalentes. Diferentemente da técnica *Copiar e Substituir*, os objetos originais são mantidos, e diferentemente da técnica *Sucessor*, não há um objeto ativo e um objeto sucedido: todos os objetos são equivalentes. A Figura 7.57 mostra um exemplo de modelagem de uma classe que aceita que seus membros tenham objetos essência. Objetos são

considerados equivalentes se estão ligados ao mesmo objeto essência. Adicionalmente, a figura já introduz uma sofisticação que é a definição do conjunto de pessoas que aceitam a equivalência, que não é necessariamente unânime.

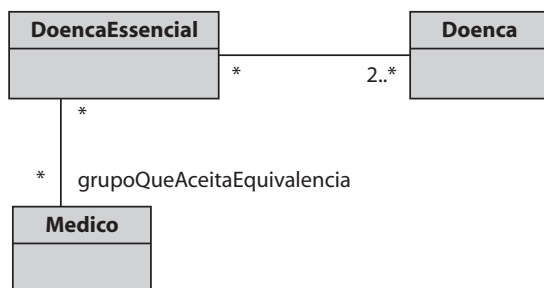


Figura 7.57: Exemplo da aplicação da técnica Essência/Aparência.

O exemplo aplica-se na área da saúde, na qual, em alguns casos, grupos de médicos aceitam que determinadas doenças são, na verdade, uma única doença, mas isso nem sempre é unanimidade.

O objeto essencial existe apenas para ligar objetos; ele não tem outras propriedades.

7.6.7.4. Desfazendo a Junção

Quando parece que o mundo real não pode ficar mais complexo, ele usualmente fica. Então, se existe a possibilidade de se descobrirem dados redundantes que necessitam de junção, também é possível que junções sejam feitas indevidamente e que tenham de ser desfeitas. Novamente, tais operações só são possíveis a partir de uma cirurgia de peito aberto no banco de dados ou a partir de operações bem planejadas disponíveis na interface do sistema. A segunda opção costuma ser menos invasiva.

Deve-se observar que, para que as junções feitas pela técnica *Copiar e Substituir* possam ser desfeitas, seria necessário guardar um *backup* dos objetos originais, pois a técnica destrói um dos objetos e descaracteriza o outro. No caso de *Sucessor*, a desvinculação ente os objetos é feita pela remoção da associação. No caso de *Essência/Aparência*, a junção é desfeita pela eliminação do objeto essencial. Porém, em todos os casos, deve-se decidir como tratar eventuais modificações que um objeto tenha sofrido quando estava ligado a outros.

Uma maneira de implementar a possibilidade de desfazer junções, bem como quaisquer outras operações, é manter um “log” de banco de dados, no qual cada modificação em um registro é anotada, sendo mantido, em uma tabela à parte, o valor anterior e o novo valor de cada campo de cada tabela alterada, bem como a hora exata e o usuário responsável pela modificação. Dessa forma, quaisquer operações podem ser desfeitas, mas ao custo de maior uso de armazenamento de dados.

76.8. Conta/Transação

Um padrão de cunho eminentemente comercial, mas de grande aplicabilidade, é o padrão *Conta/Transação*. Foi mencionado anteriormente que livros podem ser encomendados, recebidos, estocados, vendidos, entregues, devolvidos, reenviados e descartados. Tais movimentações, bem como as transações financeiras envolvidas, poderiam dar origem a uma série de conceitos como Pedido, Compra, Chegada, Estoque, Venda, Remessa, Devolução, ContasARceber, ContasAPagar etc., cada um com seus atributos e associações.

Porém, é possível modelar todos esses conceitos com apenas três classes simples e poderosas.

Uma *conta* é um local onde são guardadas quantidades de alguma coisa (itens de estoque ou dinheiro, por exemplo). Uma conta tem um *saldo* que, usualmente, consiste no somatório de todas as retiradas e depósitos.

Por outro lado, retiradas e depósitos, frequentemente, são apenas movimentações de bens ou dinheiro de uma conta para outra. Assim, uma *transação* consiste em duas movimentações, uma retirada de uma conta e um depósito de igual valor em outra. A Figura 7.58 ilustra essas classes.

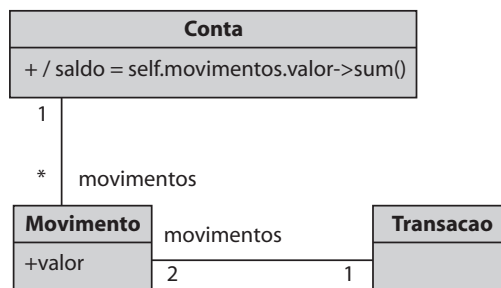


Figura 7.58: Classes do padrão *Conta/Transação*.

Para a classe *Transacao* ser consistente, é necessário que ela tenha exatamente dois movimentos de mesmo valor absoluto mas sinais opostos. Ou seja, se a transação tira cinco reais de uma conta, ela coloca cinco reais em outra conta. Então, a classe *Transacao* necessitaria de uma invariante (assunto da Seção 7.7) como a seguinte:

```
Context Transacao
inv:
    self.movimentos.valor→sum() = 0
```

Ou seja, para quaisquer instâncias de *Transacao*, a soma dos dois movimentos associados a ela tem de ser zero.

Por outro lado, o atributo derivado */saldo* da classe *Conta* é definido como o somatório de todos os movimentos daquela conta.

Então, as várias situações relacionadas a pedidos de livros podem ser modeladas a partir de um conjunto de instâncias da classe *Conta*. Por exemplo:

- a) para cada fornecedor (editora) corresponde uma instância de *Conta* da qual somente são retirados livros, ou seja, essa é uma *conta de entrada* e seu saldo vai ficando cada vez mais negativo à medida que mais e mais livros são encomendados;
- b) há uma conta para *saldo de pedidos*, que contém os livros pedidos mas ainda não entregues;
- c) há uma conta para *estoque* contendo os pedidos entregues e ainda não vendidos;
- d) há uma conta de *remessa* contendo os livros vendidos mas ainda não enviados;
- e) há uma conta de *envio*, contendo livros enviados mas cuja entrega ainda não foi confirmada;
- f) há uma conta de *venda confirmada* contendo os livros vendidos e cuja entrega foi confirmada pelo correio (possivelmente uma para cada comprador). Essa é uma conta de saída, cujo saldo vai ficando cada vez mais positivo à medida que transações são feitas. Seu saldo representa a totalidade de livros já vendidos.

Paralelamente, há contas para as transações em dinheiro feitas concomitantemente. Haverá contas a receber, contas a pagar, contas recebidas e pagas, investimentos, dívidas, valores separados para pagamento de impostos etc.

Assim, cada uma das possíveis transações de pedido, compra, venda, devolução e quebra de estoque pode ser modelada como instâncias de Transacao. Por exemplo:

- a) um *pedido* é uma transação que tira de uma conta de fornecedor e repassa à conta de saldo de pedidos;
- b) a *chegada da mercadoria* é uma transação que tira da conta de saldo de pedidos e coloca na conta de estoque;
- c) uma *venda* é uma transação que retira da conta de estoque e coloca na conta de remessa;
- d) um *registro de envio* é uma transação que retira da conta de remessa e coloca na conta de itens enviados;
- e) uma *devolução* é uma transação que retira da conta de itens enviados e coloca novamente na conta de estoque;
- f) uma *confirmação de entrega* é uma transação que retira da conta de itens enviados e coloca em uma conta de entrega definitiva.

Esse padrão comporta inúmeras variações e sofisticações, mas é muito interessante ver como uma simples ideia poderosa pode dar conta de tantas situações cuja modelagem ingênua poderia ser bastante complicada.

76.9. Associação Histórica

Frequentemente, o analista se defronta com a necessidade de que uma associação tenha memória. Por exemplo, pode-se representar a relação entre pessoas e seus empregos. Mas, quando uma pessoa muda de emprego, se a associação representa apenas a situação presente, a memória dos empregos anteriores se perde.

Caso se queira guardar informações históricas de uma associação, como empregos anteriores, pode-se usar o padrão *Associação Histórica*. Há um estereótipo associado a esse padrão, conforme mostrado na Figura 7.59.

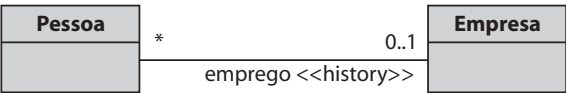


Figura 7.59: Uma associação histórica.

A associação da Figura 7.59 significa que uma pessoa pode ter no máximo um emprego em um dado instante de tempo, mas, se já teve outros empre-

gos antes, eles podem ser recuperados como em uma lista. Ou seja, é possível retornar o emprego anterior, o segundo anterior e assim por diante.

Na prática, tal padrão é implementado a partir de duas associações, a atual e a histórica, como na Figura 7.60. O estereótipo é, então, uma forma de abreviar essa estrutura mais complexa substituindo-a por uma forma mais simples.

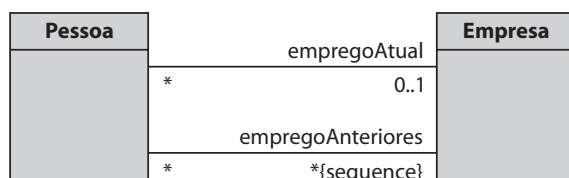


Figura 7.60: Desdobramento físico do estereótipo <<history>>.

Como será visto nos capítulos seguintes, a associação normal tem apenas um método para retornar seus elementos. No caso, a classe Pessoa terá um método `getEmprego()` que retorna o emprego da pessoa se ele existir ou o conjunto vazio, caso contrário. Se a associação for estereotipada com <<history>>, além desse método padrão, vai existir outro indexado `getEmprego(index)`, onde `index=1` representa o emprego atual, `index=2` representa o emprego anterior, e assim por diante. Se não houver um emprego para o `index` dado, a função retorna o conjunto vazio.

Esse tipo de associação, porém, não é capaz de indicar qual o emprego da pessoa em uma determinada data. Ela pode apenas dizer onde a pessoa trabalhava antes, mas não quando saiu de lá. Pode-se optar, então, se necessário, por um padrão em que, além da memória da sequência, a associação tenha memória de tempo, como na Figura 7.61.

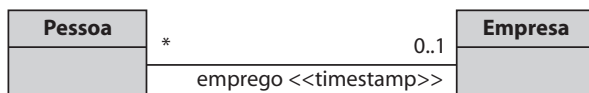


Figura 7.61: Uma associação histórica com registro de tempo.

A associação histórica com registro de tempo, além dos métodos `get()` e `get(index)`, já mencionados, ainda permite o acesso ao elemento em dado período de tempo pelo método `get(time)`. Seria possível consultar a associação da Figura 7.61 de três formas:

- a) `getEmprego()`, que retorna o emprego atual ou o conjunto vazio, se ele não existir;
- b) `getEmprego(index)`, que retorna um emprego anterior, cujo número de ordem é dado pelo valor inteiro passado como parâmetro, ou o conjunto vazio se tal emprego não existir;
- c) `getEmprego(time)`, em que `time` é um valor que corresponde a uma data/hora válida. Se a pessoa em questão tinha um emprego naquela data/hora, ele será retornado, caso contrário é retornado o conjunto vazio.

A Figura 7.62 apresenta uma possível implementação do estereótipo `<<timestamp>>`.

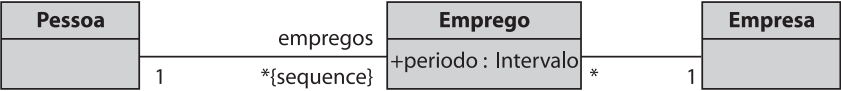


Figura 7.62: Uma possível implementação para o estereótipo `<<timestamp>>`.

Observa-se que a classe `Emprego`, em vez de ter atributos para data inicial e data final, possui um único atributo `período:Intervalo` representando um período contínuo de tempo. Isso leva ao padrão seguinte: *Intervalo*.

7.6.10. Intervalo

Sempre que um objeto qualquer tem pares de atributos representando um início e um fim, como data inicial e final, em vez de representar essa situação como dois atributos, é preferível definir e utilizar um tipo primitivo chamado `Intervalo`, como na Figura 7.63.



Figura 7.63: Tipo primitivo `Intervalo`.

Há dois motivos para isso: o primeiro é que a existência dos dois atributos fortemente correlacionados (início e fim) em uma classe fere o princípio de coesão, já explicado. O segundo é que, possivelmente, várias operações específicas sobre intervalos serão necessárias, como, por exemplo, verificar se

uma determinada data está dentro de um intervalo ou verificar se dois intervalos se sobrepõem.

É muito mais razoável implementar essas operações uma única vez em uma classe primitiva do que implementá-las inúmeras vezes nas classes conceituais cada vez que houver necessidade delas.

7.7. Invariantes

Existem situações em que a expressividade gráfica do diagrama de classes é insuficiente para representar determinadas regras do modelo conceitual. Nesses casos, necessita-se fazer uso de invariantes.

Invariantes são restrições sobre as instâncias e classes do modelo. Certas restrições podem ser representadas no diagrama: por exemplo, a restrição de que uma venda não pode ter mais do que cinco livros poderia ser representada como na Figura 7.64.

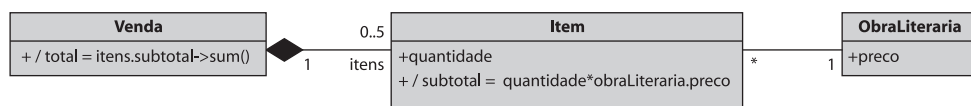


Figura 7.64: Uma restrição que pode ser representada no diagrama.

Mas nem todas as restrições podem ser representadas tão facilmente. Se houvesse uma restrição que estabelecesse que nenhuma venda pode ter valor superior a mil reais, isso não seria passível de representação nas associações nem nos atributos do diagrama da Figura 7.64. Mas seria possível estabelecer tal restrição usando invariantes de classe como a seguir:

```

Context Venda
inv:
    self.total <= 1000,00
  
```

Talvez a maioria dos desenvolvedores de software, quando se depara com regras desse tipo acaba incorporando-as nos métodos que fazem algum tipo de atualização nas instâncias da classe. Por exemplo, no caso anterior, poderia ser colocado um teste no método que faz a adição de um novo item em uma venda para verificar se o valor total passaria de 1.000 e, se for o caso, impedir a adição.

O problema com essa abordagem é que apenas *naquele* método seria feita a verificação, mas não fica uma regra geral para ser observada em outros métodos. Até é possível que o analista hoje saiba que a regra deve ser seguida, mas, e se outro analista fizer a manutenção do sistema dentro de cinco ou 10 anos? Ele não saberá necessariamente que essa regra existe, provavelmente não vai consultar o documento de requisitos, já desatualizado, e poderá introduzir erro no sistema se permitir a implementação de métodos que não obedeçam à regra.

Então, todas as regras gerais para o modelo conceitual devem ser explicitadas no modelo para que instâncias inconsistentes não sejam permitidas. Se for possível, as restrições devem ser explicitadas graficamente; caso contrário, através de invariantes.

Outro exemplo, que ocorre com certa frequência, é a necessidade de restringir duas associações que a princípio são independentes. Na Figura 7.65, considera-se que cursos têm alunos, cursos são formados por disciplinas e alunos matriculam-se em disciplinas, mas o modelo mostrado na figura não estabelece que alunos só podem se matricular nas disciplinas de seu próprio curso.

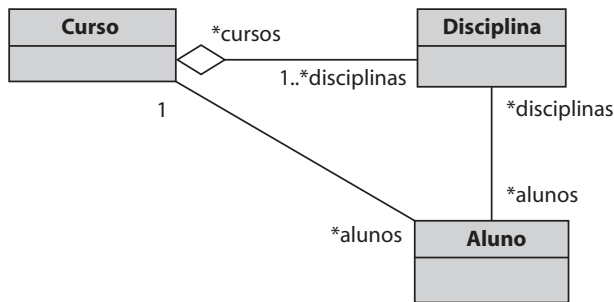


Figura 7.65: Uma situação que necessita de uma invariante para que a consistência entre associações se mantenha.

Para que um aluno só possa se matricular em disciplinas que pertencem ao curso ao qual está associado, é necessário estabelecer uma invariante como:

Context Aluno

inv:

```
self.disciplinas->forall (d|d.cursos->includes(self.curso))
```

A invariante diz que, para todas as disciplinas (d) cursadas por um aluno (self), o conjunto de cursos nos quais a disciplina é oferecida contém o curso no qual o aluno está matriculado.

A mensagem `forAll` afirma que uma expressão lógica é verdadeira para todos os elementos de um conjunto; no caso, o conjunto dado por `self.disciplina`.

A variável “`d`” entre parênteses equivale a um *iterador*, ou seja, “`d`” substitui na expressão lógica cada um dos elementos do conjunto. A mensagem `includes` corresponde ao símbolo matemático de pertença invertida ($\ni$), ou seja, afirma que um determinado conjunto contém um determinado elemento.

É possível, ainda, simplificar a expressão eliminando a variável `self` e o iterador `d`, visto que podem ser inferidos pelo contexto em que se encontram. A expressão anterior poderia, então, ser escrita assim:

```
Context Aluno
```

```
inv:
```

```
disciplinas→forAll(cursos→includes(curso))
```

7.8. Discussão

Um bom modelo conceitual produz um banco de dados organizado e normalizado. Um bom modelo conceitual incorpora regras estruturais que impedem que a informação seja representada de forma inconsistente. Um bom modelo conceitual vai simplificar o código gerado porque não será necessário fazer várias verificações de consistência que a própria estrutura do modelo já garante.

O uso de padrões corretos nos casos necessários simplifica o modelo conceitual e torna o sistema mais flexível e, portanto, lhe dá maior qualidade. É, dessa maneira, uma ferramenta poderosa. Muitos outros padrões existem e os analistas podem descobrir e criar seus próprios padrões. Apenas é necessário sempre ter em mente que só vale a pena criar um padrão quando os seus benefícios compensam o esforço de registrar sua existência.

Contratos

Até o início da modelagem funcional, o processo de análise deve ter produzido dois artefatos importantes:

- a) o *modelo conceitual*, que representa estaticamente a informação a ser gerenciada pelo sistema;
- b) os *diagramas de sequência de sistema*, que mostram como possíveis usuários trocam informações com o sistema, sem mostrar, porém, como a informação é processada internamente.

Na fase de construção dos diagramas de sequência de sistema, foram identificadas as operações e consultas de sistema. Cada operação ou consulta desse tipo implica a existência de uma intenção por parte do usuário. Essa intenção é capturada pelos contratos de operações de sistema e pelos contratos de consulta de sistema, que correspondem à *modelagem funcional* do sistema.

Um contrato de operação de sistema pode ter três seções:

- a) condições (opcional);
- b) pós-condições (obrigatória);
- c) exceções (opcional).

Já um contrato para uma consulta de sistema pode ter duas seções:

- a) condições (opcional);
- b) resultados (obrigatória).

As *precondições* existem nos dois tipos de contratos e devem ser cuidadosamente estabelecidas. Elas complementam o modelo conceitual no sentido de definir o que será verdadeiro na estrutura da informação do sistema quando a operação ou consulta for executada. Isso significa que elas não serão testadas durante a execução, mas algum mecanismo externo deverá garantir sua validade antes de habilitar a execução da operação ou consulta de sistema correspondente.

As *pós-condições* também devem ser muito precisas. Elas estabelecem o que uma operação de sistema muda na informação.

Deve-se tomar cuidado para não confundir as pós-condições com os *resultados* das consultas. As pós-condições só existem nas operações de sistema porque elas especificam alguma *alteração nos dados armazenados*. Assim, pelo princípio de separação entre operação e consulta, não é apropriado que uma operação de sistema retorne algum resultado (exceto no caso mencionado na Seção 8.8.1). Já as consultas, por definição, devem retornar algum resultado, mas não podem alterar os dados armazenados. Daí os contratos das consultas de sistema terem resultados mas não pós-condições.

Ao contrário das precondições, que devem ser garantidamente verdadeiras durante a execução de uma operação, as *exceções* são situações que usualmente não podem ser garantidas *a priori*, mas serão testadas *durante* a execução da operação. Exceções são eventos que, se ocorrerem, impedem o prosseguimento correto da operação.

Esse tipo de exceção ocorre apenas nas operações de sistema quando se tenta alterar alguma informação com dados que não satisfazem alguma regra de negócio (por exemplo, tentar cadastrar um comprador que já tem cadastro).

Apenas elementos conceituais (conceitos, atributos e associações) podem constar nos contratos de análise. Esses elementos terão necessariamente relação com as regras de negócio do sistema sendo analisado. As exceções aqui elencadas, portanto, são referentes às regras de negócio e não exceções referentes a problemas de hardware ou de comunicação. As exceções que podem ocorrer nos níveis de armazenamento, comunicação ou acesso a dispositivos externos são tratadas por mecanismos específicos nas camadas arquitetônicas correspondentes na atividade de projeto, e o usuário normalmente nem toma conhecimento delas.

Como as consultas não alteram a informação armazenada, elas não geram exceções referentes às regras de negócio.

8.1. Precondições

As precondições estabelecem o que é verdadeiro quando uma operação ou consulta de sistema for executada. Por exemplo, considerando o modelo conceitual de referência da Figura 8.1, se um usuário estiver comprando um livro, poderá ser assumido como precondição que o seu CPF, passado como parâmetro para a operação, corresponde a um comprador válido, ou seja, existe uma instância de Pessoa no papel de comprador cujo atributo cpf é igual ao CPF passado como parâmetro. Essa expressão pode ser assim representada em OCL:

```
Context Livir::identificaComprador(umCpf)
pre:
    compradores → select (cpf=umCpf) → notEmpty()
```



Figura 8.1: Modelo conceitual de referência.

A expressão então afirma que, no contexto do método `identificaComprador`, que é uma operação de sistema implementada na controladora `Livir`, há uma precondição que estabelece que o conjunto de compradores filtrado (`select`) pela condição de que o atributo `cpf` do comprador seja igual ao parâmetro `umCpf` é não vazio, ou seja, há pelo menos um comprador cujo CPF é igual ao parâmetro passado.

Se a associação `compradores` da Figura 8.1 for qualificada como na Figura 8.2, a precondição de garantia de parâmetro mencionada anteriormente poderá ser escrita de forma mais direta:

```
Context Livir::identificaComprador(umCpf)
pre:
    compradores[umCpf] → notEmpty()
```

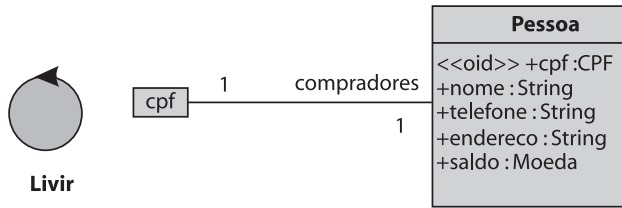


Figura 8.2: Modelo conceitual de referência com associação qualificada.

Quando a associação é qualificada, como na Figura 8.2, pode-se usar um valor para indexar diretamente o mapeamento representado pela associação. Assim, como a operação `identificaComprador` tem um parâmetro `umCpf`, a expressão `compradores[umCpf]` produz o mesmo resultado que a expressão `compradores→select(cpf=umCpf)`.

Para serem úteis ao processo de desenvolvimento de software, as precondições não podem ser expressas de maneira descuidada. Elas devem refletir fatos que possam ser identificados diretamente no modelo conceitual já desenvolvido para o sistema. Isso justifica a utilização de linguagens formais como OCL (Warmer & Kleppe, 1998) para escrever contratos.

Pode-se identificar duas grandes famílias de precondições:

- a) *garantia de parâmetros*: precondições que garantem que os *parâmetros* da operação ou consulta correspondem a elementos válidos do sistema de informação, como, por exemplo, que existe cadastro para o comprador cujo CPF corresponde ao parâmetro da operação ou consulta;
- b) *restrição complementar*: precondições que restringem ainda mais o modelo conceitual para a execução da operação ou consulta, de forma a garantir que a informação se encontra em uma determinada situação desejada, por exemplo, que o endereço para entrega informado pelo comprador não esteja no estado *inválido*.

Sobre o segundo tipo de precondição, pode-se entender que ela pode estabelecer restrições mais fortes sobre o modelo conceitual. Assim, se o modelo conceitual especifica que uma associação tem multiplicidade de papel 0..1, uma precondição complementar poderá especificar que, durante a execução de determinada operação de sistema, esse papel está preenchido (1) ou não (0). Por exemplo, uma Venda pode ter ou não um Pagamento (0..1), mas a operação de efetuar o pagamento de uma venda exige uma venda que não esteja paga ainda (0).

É necessário lembrar que uma precondição nunca poderá *contradizer* as especificações do modelo conceitual, mas apenas restringi-las ainda mais. Se o modelo conceitual exige 0 ou 1, nenhuma precondição poderá garantir 2.

8.1.1. Garantia de Parâmetros

Em relação às precondições de *garantia de parâmetros*, deve-se tomar cuidado para não confundir as precondições que testam os parâmetros *semanticamente* com as simples verificações sintáticas. Para garantir que um parâmetro seja, por exemplo, um número maior do que zero, basta usar *tipagem* (por exemplo, “x:InteiroPositivo”), não sendo necessário escrever isso como precondição.

A *tipagem* deve ser definida na assinatura da operação. Por exemplo, a tipagem é que vai definir que um determinado parâmetro deve ser um número inteiro, um número maior do que 100, ou mesmo um número primo. Se o tipo não existir, deve-se definir uma classe com o estereótipo <<primitive>> para o novo tipo.

Será considerada precondição semântica apenas uma asserção para a qual a determinação do valor verdade implica verificar os dados gerenciados pelo sistema. Assim, determinar se um número de CPF está bem formado pode ser feito sintaticamente (aplicando-se uma fórmula para calcular os dígitos verificadores), mas verificar se existe um comprador cadastrado com um dado número de CPF é uma verificação semântica, pois exige a consulta aos dados de compradores. Assim, a primeira verificação deve ser feita por tipagem e a segunda por precondição.

8.1.2. Restrição Complementar

Uma *restrição complementar* consiste na garantia de que certas restrições mais fortes do que aquelas estabelecidas pelo modelo conceitual são obtidas. É possível identificar vários tipos de restrições, como, por exemplo:

- a) fazer uma *afirmação específica* sobre uma instância ou um conjunto de instâncias;
- b) fazer uma *afirmação existencial* sobre um conjunto de instâncias;
- c) fazer uma *afirmação universal* sobre um conjunto de instâncias.

Um exemplo de *afirmação específica* sobre uma instância, considerando o modelo da Figura 8.2, poderia ser afirmar que o comprador com o CPF 12345678910 tem saldo igual a zero:

```
Context Livir::operacaoQualquer()
pre:
    compradores[12345678910].saldo = 0
```

Um exemplo de *afirmação existencial* seria dizer que existe pelo menos um comprador com saldo igual a zero (embora não se saiba necessariamente qual):

```
Context Livir::operacaoQualquer()
pre:
    compradores → exists(c | c.saldo = 0)
```

Um exemplo de *afirmação universal* seria dizer que todos os compradores têm saldo igual a zero.

```
Context Livir::operacaoQualquer()
pre:
    compradores → forAll(c | c.saldo = 0)
```

Tanto a expressão *exists* quanto *forAll* usadas poderiam ser simplificadas para *exists(saldo=0)* ou *forAll(saldo=0)*, mantendo o mesmo significado.

8.1.3. Garantia das Precondições

Como as precondições não são testadas pela operação, admite-se que algum mecanismo externo as garanta. Pode-se, por exemplo, antes de chamar a operação, executar uma consulta que testa a precondição e só chama a operação se o resultado for positivo. Pode-se, ainda, criar mecanismos restritivos de interface que garantam que a operação só é executada se a precondição for observada.

Por exemplo, em vez de digitar um CPF qualquer, o usuário terá de selecioná-lo de uma lista de CPFs válidos. Dessa forma, o parâmetro já estará garantidamente validado antes de executar a operação.

8.1.4. Precondição *versus* Invariante

Usam-se invariantes no modelo conceitual para regras que valem sempre, independentemente de qualquer operação. Usam-se precondições para

regras que valem apenas quando determinada operação ou consulta está sendo executada.

Quando já existir uma invariante para determinada situação, não é necessário escrever precondições para a mesma situação. Por exemplo, se já existir uma invariante na classe *Aluno* afirmando que ele só pode se matricular em disciplinas do seu curso, não é necessário escrever precondições nas operações de matrícula para verificar isso. Assume-se que o projeto deva ser efetuado de forma que tanto a invariante quanto as eventuais precondições nunca sejam desrespeitadas.

Mecanismos de teste de projeto poderão verificar as invariantes e precondições durante a fase de teste do sistema. Caso, em algum momento, as condições sejam falsas, devem ser sinalizadas exceções. Porém, nesses casos, o projetista deve imediatamente corrigir o sistema para que tais exceções não venham mais a acontecer. Quando o sistema for entregue ao usuário final, deve-se ter garantias de que as precondições e invariantes nunca sejam falsas.

8.2. Associações Temporárias

Quando se utiliza a estratégia *statefull*, mencionada no Capítulo 6, é necessário que a controladora guarde “em memória” certas informações que não são persistentes, mas que devem ser mantidas durante a execução de um conjunto de operações.

Pode-se, então, definir certas associações ou atributos temporários (ambos estereotipados com `<<temp>>`) para indicar informações que só são mantidas durante a execução de um determinado caso de uso e descartadas depois.

Por exemplo, para que a controladora guarde a informação sobre quem é o comprador correntemente sendo atendido, pode-se utilizar uma associação temporária para indicar isso no modelo conceitual refinado, como na Figura 8.3.

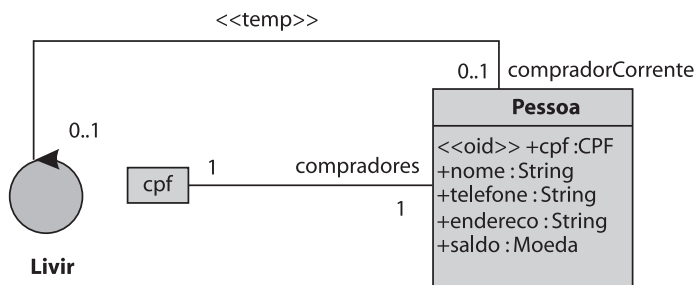


Figura 8.3: Uma associação temporária.

Assim, uma précondição de uma operação, por exemplo, poderia afirmar que já existe um comprador corrente identificado da seguinte forma:

```
Context Livir::operacaoQualquer()
pre:
    compradorCorrente → notEmpty()
```

8.3. Retorno de Consulta

Conforme mencionado, operações de sistema provocam alterações nos dados, enquanto consultas apenas retornam dados. Os contratos de consultas devem ter obrigatoriamente uma cláusula de retorno, representada em OCL pela expressão *body*.

As expressões que representam précondições vistas até aqui são todas booleanas, mas expressões utilizadas na cláusula *body* podem ser de qualquer tipo. Podem retornar *strings*, números, listas, tuplas etc. Os exemplos seguintes são baseados no modelo conceitual da Figura 8.3.

Inicialmente, define-se uma consulta de sistema que retorna o *saldo* do *comprador corrente*:

```
Context Livir::saldoCompradorCorrente():Moeda
body:
    compradorCorrente.saldo
```

As consultas de sistema sempre têm por contexto a controladora. Portanto, nessa expressão, *compradorCorrente* é uma propriedade da controladora; no caso, um papel de associação.

A consulta a seguir retorna nome e telefone do comprador cujo CPF é dado:

```
Context Livir::nomeTelefoneComprador(umCpf):Tuple
body:
    Tuple{
        nome = compradores[umCpf].nome,
        telefone = compradores[umCpf].telefone
    }
```

O construtor *Tuple* é uma das formas de representar *DTOs* em OCL; a tupla funciona como um registro, no caso com dois campos: *nome* e *telefone*. Os valores dos campos são dados pelas expressões após o sinal “=”.

Para não ter de repetir a expressão `compradores[umCpf]` ou possivelmente expressões até mais complexas do que essa em contratos OCL, pode-se usar a cláusula `def` para definir um identificador para a expressão que pode ser reutilizado. Usando a cláusula `def`, o contrato ficaria assim:

```
Context Livir::nomeTelefoneComprador(cpfComprador):Tuple
def:
    comprador = compradores[cpfComprador]
body:
    Tuple{
        nome = comprador.nome,
        telefone = comprador.telefone
    }
```

A expressão a seguir faz uma *projeção*, retornando um conjunto com todos os nomes de compradores:

```
Context Livir::listaNomeCompradores():Set
body:
    compradores.nome
```

A próxima expressão aplica um *filtro* e uma projeção, retornando os nomes de todos os compradores que têm saldo igual a zero:

```
Context Livir::listaNomeCompradoresSaldoZero():Set body:
    compradores→select(saldo=0).nome
```

Como último exemplo, a expressão a seguir retorna CPF, nome e telefone de todos os compradores que têm saldo igual a zero:

```
Context Livir::listaCpfNomeTelefoneCompradoresSaldoZero():Set
body:
    compradores→select(saldo=0)→collect(c|
        Tuple {
            cpf = c.cpf,
            nome = c.nome,
            telefone = c.telefone
        }
```

A expressão `collect` é uma forma de obter um conjunto cujos elementos são propriedades ou transformações de outro conjunto. A própria notação “.”,

aplicada sobre conjuntos, é uma forma abreviada de `collect`. Por exemplo, `compradores.nome` é equivalente a `compradores→collect(nome)`.

Quando for possível, usa-se a notação “.”, por ser mais simples. Mas, no exemplo anterior, a necessidade de criar uma tupla em vez de acessar uma propriedade dos elementos do conjunto impede o uso da notação “.”. Assim, a expressão `collect` tem de ser explicitamente usada nesse caso. Novamente, pode-se omitir o indexador, e a expressão poderá ainda ser simplificada para:

```
Context Livir::listaCpfNomeTelefoneCompradoresSaldoZero():Set
  body:
    compradores→select(saldo=0)→collect(
      Tuple {
        cpf = cpf,
        nome = nome,
        telephone = telefone
      }
    )
```

Nesse caso, as coincidências de nomes de identificadores de campo e atributos podem deixar a expressão estranha, mas os significados desses identificadores é não ambíguo pelo contexto.

8.4. Pós-condições

As pós-condições estabelecem o que muda nas informações armazenadas no sistema após a execução de uma operação de sistema. As pós-condições também devem ser claramente especificadas em termos que possam ter correspondência nas definições do modelo conceitual. Assim, uma equivalência com as expressões usadas como pós-condição e expressões passíveis de escrita em OCL é altamente desejável para evitar que os contratos sejam ambíguos ou incompreensíveis.

Uma pós-condição em OCL é escrita no contexto de uma operação (de sistema) com o uso da cláusula `post`, conforme exemplo a seguir:

```
Context Livir::operacaoX()
  post:
    <expressão OCL>
```

Havendo mais de uma pós-condição que deve ser verdadeira após a execução da operação de sistema, faz-se a combinação das expressões com o operador AND:

```
Context Livir::operacaoX()  
    post:  
        <expressão 1> AND  
        <expressão 2> AND  
        ...  
        <expressão n>
```

Para se proceder a uma classificação dos tipos de pós-condições possíveis e úteis em contratos de operação de sistema, deve-se considerar que o modelo conceitual possui apenas três elementos básicos, que são os *conceitos* (representados pelas classes), as *associações* e os *atributos*. Assim, considerando que instâncias de classes e associações podem ser criadas ou destruídas e que atributos apenas podem ter seus valores alterados, chega-se a uma classificação com cinco tipos de pós-condições:

- a) modificação de valor de atributo;
- b) criação de instância;
- c) criação de associação;
- d) destruição de instância;
- e) destruição de associação.

Para que essas pós-condições possam ser definidas de forma não ambígua, é necessário que inicialmente se proceda a uma definição de certas *operações básicas* sobre essas estruturas conceituais e seu comportamento esperado.

As operações consideradas básicas são aquelas que em orientação a objetos operam diretamente sobre os elementos básicos do modelo conceitual. Seu significado e comportamento são definidos por padrão.

Infelizmente, as linguagens de programação não oferecem ainda um tratamento padronizado para as operações básicas. Sua programação muitas vezes é fonte de trabalho braçal para programadores.

As operações conforme definidas nas subseções seguintes são efetivamente básicas, no sentido de que não fazem certas verificações de consistência. Por exemplo, uma operação que cria uma associação não vai verificar se o limite máximo de associações possíveis já foi atingido. Essas verificações de

consistência devem ser feitas em relação ao conjunto das pós-condições, ou seja, após avaliar todas as pós-condições é que se vai verificar se os objetos ficaram ou não em um estado consistente.

Por exemplo, suponha que um objeto A tenha uma associação obrigatória com um objeto B1, e que uma operação de sistema vai trocar essa associação por outra entre A e B2. É necessário destruir a associação original e criar uma nova. No intervalo de tempo entre essas duas operações, o objeto A estaria inconsistente (sem a associação obrigatória), mas considerando o conjunto das pós-condições observa-se que o resultado final é consistente, pois uma foi destruída e outra criada em seu lugar.

A discussão sobre a manutenção de consistência do conjunto de pós-condições de uma operação de sistema será feita na Seção 8.4.6.

8.4.1. Modificação de Valor de Atributo

O tipo mais simples de pós-condição é aquele que indica que o valor de um atributo foi alterado. Pode-se indicar tal condição com uma operação básica denotada pela expressão “set” seguida do nome do atributo, com o novo valor entre parênteses.

A mensagem referente a essa operação é enviada a uma instância da classe que contém o atributo. Por exemplo, se o objeto pessoa, instância da classe Pessoa, tem um atributo dataNascimento e uma determinada operação de sistema vai alterar essa data de nascimento para um valor dado por novaData, então a pós-condição da expressão deverá conter:

```
pessoa^setDataNascimento(novaData)
```

A notação “^” usada aqui difere da notação “.” no seguinte aspecto: o *ponto* forma uma expressão cujo valor é o retorno da avaliação da expressão, ou *null*, se não houver retorno; já o *circunflexo* apenas indica que a mensagem *foi enviada ao objeto*. O valor de uma expressão com circunflexo, portanto, só pode ser booleano.

Outra coisa: a expressão só diz que a instância de pessoa *recebeu* a mensagem, mas não diz quem *enviou*. A decisão sobre qual objeto vai enviar a mensagem é tomada na atividade de projeto, durante a modelagem dinâmica.

Quando usadas como pós-condições, tais expressões são asserções, ou seja, afirmações. Assim, a leitura da expressão OCL acima seria: “O objeto pessoa recebeu a mensagem setDataNascimento com o parâmetro novaData.”

8.4.2. Criação de Instância

A operação de criação de instância deve simplesmente criar uma nova instância de uma classe. Embora a OCL não seja uma linguagem imperativa, ela possui um construtor para referenciar uma nova instância de uma classe dada. Uma nova instância de Livro, conforme a Figura 8.4, poderia ser referenciada assim:

```
Livro::newInstance()
```

Livro
+isbn
+titulo
+preco = 0,00
+autor
+ / status

Figura 8.4: Uma classe a ser instanciada.

Assume-se que atributos com valores iniciais (cláusula init) já sejam definidos automaticamente pela operação de criação (sem necessidade de especificar novamente pós-condições para dar valor a eles). Além disso, atributos derivados são calculados e não podem ser modificados diretamente. Mas, o que acontece com os demais atributos e associações no momento da criação?

Há dois padrões a seguir aqui:

- a) a operação básica de criação de instância simplesmente produz a instância, sem inicializar seus atributos e associações obrigatórias. Nesse caso, a validação é feita depois e a checagem de consistência é feita no nível da operação de sistema, como mencionado antes, e não no nível da operação básica;
- b) a operação básica de criação de instância inicializa atributos e associações obrigatórias de forma que a instância não fique inconsistente em relação ao modelo conceitual. Nesse caso, a operação básica já produz uma instância consistente.

A segunda forma exigirá operações de criação mais complexas. As instâncias da classe referenciada na Figura 8.4, por exemplo, teriam de ser criadas já com todos os seus parâmetros:

```
Livro::newInstance(umISBN, umTitulo, umAutor)
```

Então, a operação básica não seria mais tão básica, pois faria-se necessário descrever de que forma esses parâmetros são usados para inicializar os atributos da instância. Assim, a operação de criação de instância teria chamadas de operações básicas dentro dela.

O primeiro padrão é mais simples: a operação básica de criação simplesmente cria a instância, e outras operações básicas tratam da inicialização de atributos e associações. A consistência dos objetos em relação ao modelo conceitual é checada após a execução da operação de sistema e não após cada operação básica (o que seria o caso, se fosse aplicado o segundo padrão). Assim, aplicando o primeiro padrão, a classe da Figura 8.4 poderia ser criada e inicializada como no exemplo a seguir (onde *criaLivro* é uma operação de sistema):

```
Context Livir::criaLivro(umIsbn, umTitulo, umAutor)
def:
    novoLivro = Livro::newInstance()
post:
    ...
    novoLivro^setIsbn(umIsbn) AND
    novoLivro^setTitulo(umTitulo) AND
    novoLivro^setAutor(umAutor)
```

Uma pós-condição ficou implícita na cláusula *def*: a criação da instância de *Livro*. Nota-se que o atributo *preco*, que tem valor predefinido, não precisa ser inicializado, bem como o atributo derivado *status*, que é calculado (por uma cláusula “*derive*” na definição da classe *Livro*).

Há mais um “porém” aqui: em pós-condições de contratos, de nada adianta mencionar a criação de uma instância se ela não for também imediatamente associada a alguma outra instância, porque a informação inacessível em um sistema simplesmente não é informação. Então, a criação de instância vai ocorrer sempre em conjunto com uma criação de associação, conforme será visto na seção seguinte.

8.4.3. Criação de Associação

Como visto, outro tipo de operação básica é aquela que indica que uma *associação* foi criada entre duas instâncias. A criação de associações pode ser

limitada superiormente e inferiormente, dependendo da multiplicidade de papel. Por exemplo, uma associação 0..5 que já tenha cinco objetos não poderá aceitar um sexto objeto. Uma associação *para um* não pode aceitar um segundo elemento, nem o primeiro pode ser removido. Essa verificação, porém, conforme foi dito, será feita para a operação de sistema como um todo e não individualmente para cada operação básica.

Existem vários dialetos para nomear operações que modificam e acessam associações. Aqui será usado o prefixo “add” seguido do nome de papel para nomear essa operação (outra opção seria usar *set*, como no caso de atributos).

Assim, considerando a associação entre as classes *Automovel* e *Pessoa*, conforme a Figura 8.5, e considerando duas instâncias, respectivamente, *jipe* e *joao*, pode-se admitir que a associação possa ser criada do ponto de vista do automóvel por:

```
jipe^addPassageiro(joao)
```

ou, do ponto de vista da pessoa, por:

```
joao^addAutomovel(jipe)
```

As duas expressões são simétricas e produzem exatamente o mesmo resultado (a criação da associação).

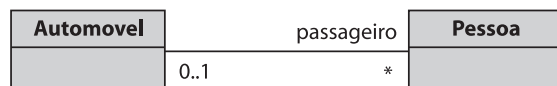


Figura 8.5: Um modelo de referência para operações de criação de associação.

Associações com papel obrigatório, como na Figura 8.6, normalmente são criadas juntamente com um dos objetos (o do lado não obrigatório). Assim, usualmente, esse tipo de pós-condição combinada de criação de instância e sua associação obrigatória pode ser feita como indicado a seguir:

```
venda^addPagamento(Pagamento::newInstance())
```



Figura 8.6: Um modelo de referência para operações de criação de associação com papel obrigatório.

A situação se complica mais quando o limite inferior for maior do que 1, o que implica que um objeto já teria de ser criado com vários outros objetos associados. Nesse caso, o padrão utilizado neste livro, que considera a consis-

tência do contrato como um todo e não de cada operação básica, é novamente mais simples: basta criar o objeto e adicionar associações até chegar ao limite exigido. A consistência será verificada ao final do conjunto de operações.

Complementando, então, o exemplo da seção anterior, a expressão a seguir mostra a criação de um novo livro e sua inicialização, inclusive com a criação de uma associação entre a controladora e o novo livro:

```
Context Livir::criaLivro(umIsbn, umTitulo, umAutor)
def:
    novoLivro = Livro::newInstance()
post:
    self^addLivro(novoLivro) AND
    novoLivro^setIsbn(umIsbn) AND
    novoLivro^setTitulo(umTitulo) AND
    novoLivro^setAutor(umAutor)
```

8.4.4. Destruição de Instância

A destruição de objetos deve também ser entendida do ponto de vista declarativo da OCL. Há duas abordagens para indicar que uma instância foi destruída:

- a) *explícita*: declara-se que um objeto foi destruído através do envio de uma mensagem explícita de destruição;
- b) *implícita*: removem-se todas as associações para o objeto de forma que ele passe a não ser mais acessível. Em linguagens de programação, é possível implementar coletores de lixo (*garbage collection*) para remover da memória objetos que não são mais acessíveis.

Neste livro, será assumida a abordagem explícita, visto que ela deixa mais claro qual a real intenção do analista. Um objeto que foi destruído, então, terá recebido uma mensagem como:

```
objeto^destroy()
```

O significado dessa expressão, em uma pós-condição de operação de sistema, é de que o objeto referenciado foi destruído durante a execução da operação.

Assume-se que todas as associações desse objeto também são destruídas com ele.

8.4.5. Destruição de Associação

A destruição de uma associação é referenciada pela operação básica com prefixo `remove` seguida do nome de papel e tendo como parâmetro o objeto a ser removido (em alguns dialetos poderia ser `unset`). Por exemplo, considerando a Figura 8.5, para remover um pagamento `p1` associado à venda `v`, pode-se escrever:

```
v^removePagamento(p1)
```

Deve-se assumir, nesse caso, que, como a multiplicidade de papel de `Pagamento` para `Venda` é obrigatória (igual a 1), a remoção da associação implicará necessariamente a destruição do pagamento ou a criação posterior de uma nova associação com outra venda.

Se a multiplicidade do papel a ser removido fosse 1 ou 0..1, seria opcional informar o parâmetro, pois haveria uma única possível associação a ser removida. Observando novamente a Figura 8.5, se a remoção da associação fosse feita a partir do pagamento, a operação poderia ser chamada sem o parâmetro, pois só há uma venda possível a ser removida:

```
p1^removeVenda()
```

Novamente, deve-se ter em mente que a remoção dessa associação obriga à criação de uma nova associação para o pagamento `p1` ou sua destruição.

Tentar remover uma associação inexistente é um erro de projeto e não pode ser tentado em pós-condições bem formadas.

8.4.6. Pós-condições bem Formadas

Considerando-se então que as operações básicas que denotam as pós-condições mais elementares não comportam checagem de consistência nos objetos em relação ao modelo conceitual, o conjunto de pós-condições é que precisa ser verificado para se saber se, ao final da execução das operações, os objetos estão em um estado consistente com as definições do modelo.

Pode-se resumir assim as checagens a serem efetuadas:

- a) uma instância recém-criada deve ter pós-condições indicando que todos os seus atributos foram inicializados, exceto: (1) atributos derivados (que são calculados), (2) atributos com valor inicial (que já são definidos por uma cláusula `init` no contexto da classe e não precisam ser novamente definidos para cada operação de sistema) e (3) atributos que possam ser nulos (nesse caso, a inicialização é opcional);

- b) uma instância recém-criada deve ter sido associada a alguma outra que, por sua vez, possua um caminho de associações que permita chegar à controladora de sistema. Caso contrário, ela é inacessível, e não faz sentido criar um objeto que não possa ser acessado por outros;
- c) todas as associações afetadas por criação ou destruição de instância ou associação devem estar com seus papéis dentro dos limites inferior e superior;
- d) todas as invariantes afetadas por alterações em atributos, associações ou instâncias devem continuar sendo verdadeiros.

Foge ao escopo deste livro a definição e um sistema de verificação de restrições, o que seria necessário para implementar automaticamente a checagem de invariantes e limites máximo e mínimo em associações. O analista, ao preparar os contratos, deve estar ciente de que os objetos devem ser deixados em um estado consistente após cada operação de sistema. Havendo a possibilidade de implementar um sistema de checagem automática dessas condições, seria uma grande ajuda à produtividade do analista. Porém, salvo melhor juízo, tal sistema ainda não está disponível nas ferramentas CASE comerciais.

8.4.7. Combinações de Pós-condições

Cada operação de sistema terá um contrato no qual as pós-condições vão estabelecer tudo o que essa operação muda nos objetos, associações e atributos existentes. Usualmente, uma operação de sistema terá várias pós-condições, que podem ser unidas por operadores AND, como mencionado anteriormente. Mas também é possível usar operadores OR, que indicam que pelo menos uma das pós-condições ocorreu, mas não necessariamente todas:

```
post:
    <pos-condição 1> OR
    <pos-condição 2>
```

Além disso, é possível utilizar o operador IMPLIES, com o mesmo significado da implicação lógica. Mas esse operador também pode ser substituído pela forma if-then-endif. Assim, a expressão:

```
post:
    <condição> IMPLIES <pos-condição>
```

pode ser escrita como:

```
post:
```

```
if <condição> then
    <pos-condição>
endIf
```

Muitas vezes, a condição é construída com valores que os atributos tinham antes de a operação ser executada. Esses valores anteriores devem ser anotados com a expressão @pre. Por exemplo, uma pós-condição que estabeleça que *se o saldo da venda corrente era zero então o saldo passou a ser 1* poderia ser escrita assim:

```
post:
    if self.vendaCorrente.saldo@pre = 0 then
        self.vendaCorrente^setSaldo(1)
    endIf
```

ou:

```
post:
    vendaCorrente.saldo@pre=0 IMPLIESvendaCorrente^setSaldo(1)
```

8.4.8. Pós-condições sobre Coleções de Objetos

É possível, com uma única expressão OCL, afirmar que todo um conjunto de objetos foi alterado. Por exemplo, para afirmar que o preço de todos os livros foi aumentado em x%, pode-se usar a expressão forAll para indicar que todas as instâncias foram alteradas:

```
Context Livir::reduzPrecoLivros(x)
post: self.livros→forAll(livro|
    livro^setPreco(livro.preco@pre * (1-x/100))
)
```

8.5. Exceções

As exceções em contratos são estabelecidas como situações de falha que não podem ser evitadas ou verificadas antes de iniciar a execução da operação propriamente dita.

Já foi visto anteriormente que, muitas vezes, situações identificadas como exceções são na verdade pré-condições sobre as quais o analista não pensou muito. Sempre que for possível transformar uma exceção em pré-condição, é conveniente fazê-lo, pois é preferível limitar a possibilidade de o

usuário gerar um erro do que ficar indicando erros em operações que ele já tentou executar e falhou.

Nos contratos de operação de sistema, basta indicar a exceção dizendo qual condição que a gera. O tratamento da exceção será feito necessariamente na atividade de projeto da camada de interface do sistema. O exemplo a seguir mostra um contrato com uma exceção indicada:

```
Context Livir::identificaComprador(umCpf)
def:
    comprador = compradores→select(cpf = umCpf)
post:
    self^addCompradorCorrente(comprador)
exception:
    comprador→size() = 0 IMPLIES self^throw("cpf inválido")
```

Considera-se como exceção de contrato apenas a situação que não possa ser tratada dentro da própria operação, exigindo possivelmente o retorno do controle da aplicação ao usuário para tentar outras operações.

Assume-se também que, quando uma exceção ocorre em uma operação de sistema, a operação não é concluída e nenhuma de suas pós-condições é obtida. O sistema de informação deve ser mantido em um estado consistente, mesmo quando ocorrer uma exceção.

Como mencionado, algumas exceções podem ser convertidas em pre-condições desde que o analista vislumbre algum meio de verificar a condição antes de tentar executar a operação. Assim, o contrato com uma exceção poderia ser transformado em:

```
Context Livir::identificaComprador(umCpf)
def:
    comprador = compradores→select(cpf = umCpf)
pre:
    comprador→size() = 1
post:
    self.addCompradorCorrente(comprador)
```

Nesse caso, não há verificação da condição. Quem chamar a operação `identificaComprador` deve garantir que o CPF passado como parâmetro seja válido.

8.6. Contratos Padrão CRUD

O processo de criação de contratos está intimamente ligado ao caso de uso expandido e ao modelo conceitual. Deve-se usar o modelo conceitual como referência em todos os momentos porque ele é a fonte de informação sobre quais asserções podem ser feitas.

Os contratos devem ser sempre escritos como expressões interpretáveis em termos dos elementos do modelo conceitual. Assim, asserções como “foi impresso um recibo” dificilmente serão pós-condições de um contrato, visto que não representam informação do modelo conceitual. Tais expressões não podem sequer ser representadas em OCL.

Mesmo que o analista quisesse, por algum motivo, armazenar a informação de que um recibo foi impresso após a execução de alguma operação, a pós-condição deveria ser escrita de forma a poder ser interpretada como alteração de alguma informação no modelo conceitual, como, por exemplo, “o atributo `reciboImpresso` do `emprestimoAberto` foi alterado para `true`” ou, em OCL:

post:

```
emprestimoAberto^setReciboImpresso(true)
```

As subseções seguintes apresentam modelos de contratos para as operações típicas CRUD. São três contratos de operação e sistema, e um contrato de consulta de sistema. As operações são executadas sobre a classe `Livro`, definida segundo o modelo conceitual da Figura 8.7.

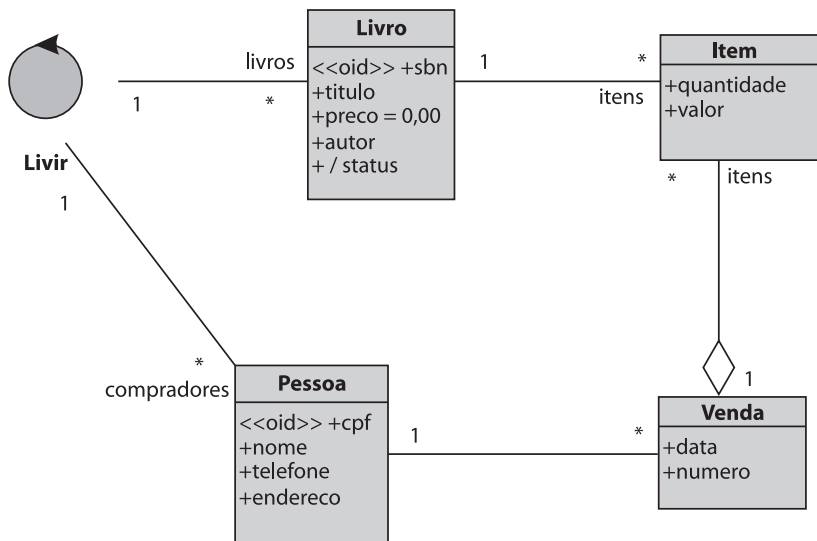


Figura 8.7: Modelo conceitual de referência para contratos de operações CRUD.

8.6.1. Operações de Criação (Create)

Para as operações e consultas ligadas à manutenção de informações (cadastros), os contratos serão mais ou menos padronizados. Inserção (*create*) de informação sempre vai incluir a criação de uma instância, alteração de seus atributos e a criação de uma associação com a controladora do sistema ou com alguma outra classe:

```
Context Livir::criaLivro(umIsbn, umTitulo, umAutor)
def:
    novoLivro = Livro::newInstance()
post:
    self^addLivros(novoLivro) AND
    novoLivro^setIsbn(umIsbn) AND
    novoLivro^setTitulo(umTitulo) AND
    novoLivro^setAutor(umAutor)
```

Como o atributo isbn já está estereotipado com <<oid>>, não é necessário estabelecer como exceção a tentativa de inserir um livro cujo isbn já exista, pois essa exceção já é prevista pelo próprio estereótipo. Mas, se em vez de exceção, esse fato for assumido como precondição, ela deve ficar explícita:

```
Context Livir criaLivro(umIsbn, umTitulo, umAutor)
def:
    novoLivro = Livro::newInstance()
pre:
    livros→select(isbn=umIsbn)→isEmpty()
post:
    self^addLivros(novoLivro) AND
    novoLivro^setIsbn(umIsbn) AND
    novoLivro^setTitulo(umTitulo) AND
    novoLivro^setAutor(umAutor)
```

8.6.2. Operações de Alteração (Update)

As alterações de informação vão envolver apenas a alteração de atributos ou, possivelmente, a criação e/ou destruição de associações:

```
Context Livir alteraLivro(umIsbn, novoIsbn, umTitulo, umAutor)
def:
    livro = livros→select(isbn=umIsbn)
pre:
    livro→size() = 1
post:
    livro^setIsbn(novoIsbn) AND
    livro^setTitulo(umTitulo) AND
    livro^setAutor(umAutor)
```

Há dois padrões aqui envolvendo a alteração de atributos marcados com <<oid>>:

- não se permite que sejam alterados*. O objeto deve ser destruído e um novo objeto criado;
- permite-se a alteração*. Nesse caso, a operação passa dois argumentos: o ISBN anterior e o novo, como foi feito no exemplo anterior. Se o novo ISBN corresponder a um livro já existente, haverá uma exceção porque esse atributo foi marcado como oid.

Também há duas opções em relação a verificar se o livro com identificador umISBN existe ou não:

- entende-se como *exceção* o fato de não existir um livro com o ISBN dado;
- define-se uma *precondição* que garante que o livro com um ISBN existe, como foi feito no exemplo.

8.6.3. Operações de Exclusão (Delete)

As operações de sistema que excluem objetos terão de considerar as regras de restrição estrutural do modelo conceitual antes de decidir se um objeto pode ou não ser destruído e, caso possa, verificar se outros objetos também devem ser destruídos junto com ele.

No caso da Figura 8.7, por exemplo, uma instância de Livro não pode ser simplesmente destruída sem que se verifique o que acontece com possíveis instâncias de Item ligadas ao livro, já que do ponto de vista dos itens a associação com um livro é obrigatória.

Para que a exclusão seja feita sem ferir esse tipo de restrição estrutural, pode-se escolher uma dentre três abordagens:

- a) garantir por *precondição* que o livro sendo excluído não possui nenhum item ligado a ele. A associação, do ponto de vista do livro, é opcional. Por essa abordagem, apenas livros que não têm itens associados podem ser selecionados para exclusão;
- b) garantir, por *pós-condição*, que todos os itens ligados ao livro também serão excluídos. Usa-se essa abordagem quando se quer que a operação de exclusão se propague para todos os objetos (no caso, itens) que têm associações obrigatórias com o livro. Essa propagação continua atingindo outros objetos em cadeia até que não haja mais ligações baseadas em associações obrigatórias;
- c) utilizar uma *exceção* para abortar a operação caso seja tentada sobre um livro que tenha itens associados a ele.

Um possível contrato usando a abordagem de *precondição* teria esse formato:

```
Context Livir::excluiLivro(umIsbn)
def:
    livro = livros→select(isbn=umIsbn)
pre:
    livro→size() = 1 AND
    livro.itens→size() = 0
post:
    livro^destroy()
```

Conforme indicado, a mensagem `destroy` elimina a instância de `Livro`, bem como todas as suas associações que, portanto, não precisam ser removidas uma a uma.

Um possível contrato usando a abordagem de *pós-condição*, que propaga a exclusão a todos os objetos ligados ao livro por associações de multiplicidade de papel obrigatória, teria o seguinte formato:

```
Context Livir::excluiLivro(umIsbn)
def:
    livro = livros→select(isbn=umIsbn)
pre:
    livro→size() = 1
post:
    livro.itens→forAll(item|item^destroy()) AND
    livro^destroy()
```

A pós-condição afirma então que, além do livro, todos os itens ligados a ele foram destruídos. Como a classe `Item` não possui associações obrigatórias vindas de outras classes, a destruição pode parar por aí, mas, caso contrário, seria necessário destruir outros objetos.

Um possível contrato usando a abordagem de *exceção* teria este formato:

```
Context Livir::excluiLivro(umIsbn)

def:
    livro = livros→select(isbn=umIsbn)
pre:
    livro→size() = 1
post:
    livro^destroy()
exception:
    livro.itens→notEmpty() IMPLIES
    self^throw("não é possível excluir este livro")
```

Escolhe-se a abordagem de pós-condição quando se quer propagar a exclusão a todos os elementos dependentes do objeto destruído. Por exemplo, se um comprador for destruído, quaisquer reservas que ele tenha no sistema podem ser destruídas junto.

Mas há situações em que não se quer essa propagação. Por exemplo, a remoção de um livro do catálogo não deveria ser possível se cópias dele já foram vendidas. Nesse caso, deve-se optar pelas abordagens de pré-condição ou exceção. A primeira vai exigir que se impeça que um livro com itens associados possa sofrer a operação de exclusão. Por exemplo, a lista de livros disponível para a operação de exclusão poderia conter apenas aqueles que não possuem itens associados. Caso não se queira ou não se possa dar essa garantia, resta a abordagem de exceção. Deixa-se o usuário tentar a exclusão, mas, se ela não for possível, uma exceção é criada.

Usualmente, informações cadastrais como essas nunca são removidas de sistemas de informação, mas marcadas com algum atributo booleano que indica se são instâncias ativas ou não. Afinal, não se poderia ter registros históricos de vendas de livros se os livros que saem de circulação fossem simplesmente removidos do sistema de informação.

8.6.4. Consultas (Retrieve)

A consulta simples do padrão CRUD implica simplesmente a apresentação de dados disponíveis sobre uma instância de um determinado conceito a partir de um identificador dessa instância. Nessas consultas, não se fazem totalizações ou filtragens, ficando essas operações restritas aos relatórios (estereótipo <<rep>>).

Então, uma consulta simples para a classe Livro da Figura 8.7 seria:

```
Context Livir::consultaLivro(umIsbn):Tuple
def:
    livro = livros→select(isbn=umIsbn)
body:
    Tuple{
        isbn = livro.isbn,
        titulo = livro.titulo,
        preco = livro.preco,
        autor = livro.autor,
        status = livro.status
    }
```

A consulta do tipo CRUD retorna sempre uma tupla com os dados constantes nos atributos do objeto selecionado por seu identificador.

8.7. Contrato Padrão Listagem

Frequentemente, é necessário utilizar operações de listagem de um ou mais atributos de um conjunto de instâncias de uma determinada classe para preencher listas ou menus em interfaces.

Um primeiro contrato de listagem (simples) vai apenas listar os ISBN dos livros catalogados na livraria:

```
Context Livir::listaIsbn():Set
body:
    self.livros.isbn
```

Caso se deseje uma lista de múltiplas colunas como, por exemplo, ISBN e título, é necessário retornar uma coleção de tuplas, como:

```
Context Livir::listaIsbnTitulo():Set
```

```
body:
  self.livros→collect(livro|
    Tuple{
      isbn = livro.isbn,
      titulo = livro.titulo
    }
  )
```

Por vezes, será necessário aplicar um filtro à lista, como, por exemplo, retornando apenas o ISBN e título de livros que não têm nenhum item associado (nunca foram vendidos). Nesse caso, aplica-se um `select` ao conjunto antes de formar as tuplas:

```
Context Livir::listaIsbnTituloNaoVendidos():Set
body:
  self.livros→select(livro|
    livro.itens→isEmpty()
  )→ collect(livro|
    Tuple{
      isbn = livro.isbn,
      titulo = livro.titulo
    }
  )
```

A maioria das consultas de simples listagem terá apenas estes dois construtores: um `select` (filtro) seguido de um `collect` (projeção dos atributos que serão retornados). Mas algumas poderão ser mais complexas. Nesses casos, elas já não se encaixam no padrão Listagem, mas no padrão Relatório (<<rep>>).

8.8. Contratos Relacionados com Casos de Uso

Para as operações associadas com casos de uso, frequentemente haverá uma cadeia de execução ao longo de um dos fluxos, em que duas ou mais operações de sistema serão executadas. Possivelmente, cada operação poderá deixar informações para as demais na forma de associações temporárias. Para melhor construir os contratos dessas operações, uma abordagem possível é seguir a sequência das operações de sistema do caso de uso expandido. Nesse processo, o analista deve se perguntar:

- a) qual é o objetivo de cada operação?
- b) o que cada uma delas produz?
- c) o que cada uma delas espera que tenha sido produzido pelas anteriores?
- d) que exceções poderiam ocorrer durante a execução?
- e) a operação segue algum padrão como CRUD, Listagem ou REP?

Ao responder a essas perguntas, o analista estará construindo contratos que permitirão que as operações sejam executadas de forma consistente no contexto de uma transação. Se for necessário adicionar novas consultas no diagrama de sequência para garantir certas precondições, isso deve ser feito nesse momento.

As subseções seguintes vão apresentar todos os contratos para as operações e consultas de sistema relacionadas ao caso de uso apresentado na forma de diagrama de sequência nas Figuras 6.5 (*stateless*) e 6.6 (*statefull*).

8.8.1. Contratos para Estratégia Stateless

Em função de a estratégia ser *stateless*, as operações e consultas da Figura 6.5 vão enviar as informações ao sistema cada vez que elas forem necessárias. Informações temporárias não serão guardadas. Isso afeta os objetivos das operações e consultas. A Tabela 8.1 apresenta a lista das operações e consultas de sistema identificadas na Figura 6.5.

Tabela 8.1

```

criaCompra(idComprador):LongInt
listaLivrosDisponiveis():Set
adicionaCompra(idCompra, idLivro, quantidade)
consultaTotal(idCompra):Money
listaEnderecos(idComprador):Set
defineEnderecoEntrega(idCompra, idEndereco)
consultaFrete(idCompra):Money
consultaTotalGeral(idCompra):Money
listaCartoes(idComprador):Set
defineCartao(idCompra,idCartao)
solicitacaoPagto(idCompra):Tuple
registraPagto(idCompra, codigoAutorizacao)
consultaPrazoEntrega(idCompra):Date

```

A Figura 8.8 apresenta o modelo conceitual de referência para essas operações.

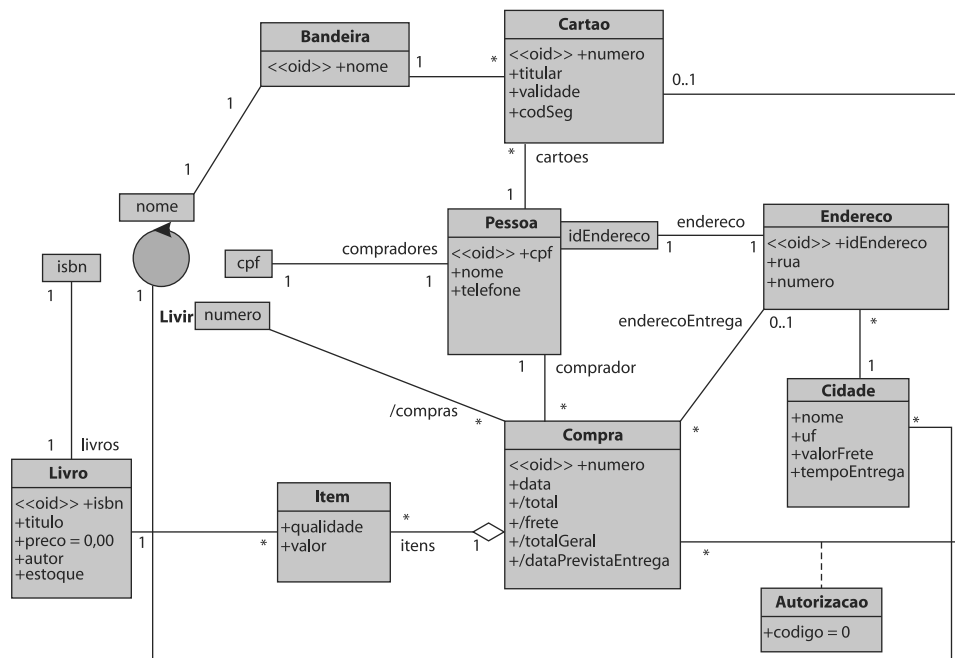


Figura 8.8: Modelo conceitual de referência para as operações da Tabela 8.1.

O primeiro contrato refere-se a uma operação que cria uma nova compra para um comprador identificado pelo seu `idComprador`. A operação deve retornar um `idCompra` (criado automaticamente pelo sistema) para ser usado como parâmetro para identificar essa nova compra nas demais operações. Trata-se de um contrato cuja operação encaixa-se na situação já mencionada, que permite que um retorno seja enviado contendo o identificador de um objeto criado pela operação de sistema. Excepcionalmente, essa operação terá dentro da cláusula `post` um comando `return` para indicar que é uma operação que retorna um valor:

```
Context Livir::criaCompra(idComprador):LongInt
def:
    novaCompra = Compra::newInstance()
def:
    comprador = compradores[idComprador]
post:
    novaCompra^setNumero(novoNumeroAutomatico()) AND
```

```

novaCompra^setData(dataAtual()) AND
novaCompra^addComprador(comprador) AND
return: novaCompra.numero
exception:
  comprador→size() = 0 IMPLIES
    self^throw("Comprador não cadastrado")

```

As pós-condições referenciam duas funções que a princípio seriam definidas em bibliotecas específicas, que são `novoNumeroAutomatico`, para gerar um novo identificador para uma venda, e `dataAtual`, que retorna a data do sistema.

A consulta de sistema `listaLivrosDisponíveis` segue o padrão listagem (com filtro) e deve retornar as informações sobre ISBN, título, autor e preço de todos os livros que tenham pelo menos um exemplar em estoque. É necessário aplicar um filtro ao conjunto de livros antes de obter seus dados:

```

Context Livir::listaLivrosDisponiveis():Set
body:
  livros→select(
    estoque>0
  )→collect(livro|
    Tuple{
      isbn = livro.isbn,
      titulo = livro.titulo,
      preco = livro.preco,
      autor = livro.autor
    }
  )

```

A operação `adicionaCompra` deve adicionar uma quantidade indicada de exemplares do livro à compra e reduzir do estoque a mesma quantidade. Caso a quantidade solicitada seja superior à quantidade em estoque, deve ocorrer uma exceção:

```

Context Livir::adicionaCompra(idCompra, idLivro, quantidade)
def:
  compra = compras[idCompra]

```

```

def:
    livro = livros[idLivro]
def:
    item = Item::newInstance()
pre:
    compra→size() = 1 AND
    livro→size() = 1
post:
    item^setQuantidade(quantidade) AND
    item^setValor(livro.preco) AND
    item^addCompra(compra) AND
    item^addLivro(livro) AND
    livro^setEstoque(livro.estoque@pre - quantidade)
exception:
    quantidade > livro.estoque IMPLIES
    self^throw("quantidade insuficiente em estoque")

```

Seria possível perguntar por que a exceção referencia `livro.estoque` e não `livro.estoque@pre`. Isso se deve ao fato de que a exceção, assim como as pre-condições e definições, referem-se a valores existentes *antes* de a operação ser executada. Apenas as *pós-condições* referenciam valores posteriores e, por isso, em alguns casos exigem o uso de `@pre`.

Seguem os contratos das demais operações e consultas da Tabela 8.1:

```

Context Livir::consultaTotal(idCompra):Money
def:
    compra = compras[idCompra]
pre:
    compra→size() = 1
body:
    compra.total
Context Livir::listaEnderecos(idComprador):Set
def:
    comprador = compradores[idComprador]
pre:

```

```

    comprador→size() = 1
body:
    comprador.enderecos→collect(endereco|
        Tuple {
            id = endereco.idEndereco,
            rua = endereco.rua,
            numero = endereco.numero,
            cidade = endereco.cidade.nome,
            uf = endereco.cidade.uf
        }
    )

```

```

Context Livir::defineEnderecoEntrega(idCompra, idEndereco)
def:
    compra = compras[idCompra]
def:
    endereco = compra.comprador.enderecos[idEndereco]1
pre:
    compra→size() = 1 AND
    endereco→size() = 1
post:
    compra^addEnderecoEntrega(endereco)

```

```

Context Livir::consultaTotalGeral(idCompra):Money
def:
    compra = compras[idCompra]
pre:
    compra→size() = 1
body:
    compra.totalGeral

```

```
Context Livir::listaCartoes(idComprador):Set
```

¹ Aqui não é necessário verificar por precondição que o comprador existe e é único porque isso já é uma condição estrutural do modelo conceitual, já que a associação de Compra para Pessoa tem multiplicidade 1.

```
def:
    comprador = compradores[idComprador]
pre:
    comprador→size() = 1
body:
    comprador.cartoes→collect(cartao|
        Tuple {
            bandeira = cartao.bandeira.nome,
            numero = cartao.numero
        }
    )
```

Context Livir::defineCartao(idCompra,idCartao)

```
def:
    compra = compras[idCompra]
def:
    cartao=compra.comprador.cartoes→select(numero=idCartao)
pre:
    compra→size() = 1 AND
    cartao→size() = 1
post:
    compra^addCartao(cartao)
```

Context Livir::solicitacaoPagto(idCompra):Tuple

```
def:
    compra = compras[idCompra]
pre:
    compra→size() = 1
body:
    Tuple {
        numero = compra.cartao.numero,
        titular = compra.cartao.titular,
        validade = compra.cartao.validade,
```



```

        codSeg = compra.cartao.codSeg,
        valor = compra.totalGeral
    }

```

```

Context Livir::registraPagto(codigoAutorizacao, idCompra)2
def:
    compra = compras[idCompra]
pre:
    compra → size() = 1
post:
    compra.autorizacao ^ setCodigo(codigoAutorizacao)3

```

```

Context Livir::consultaPrazoEntrega(idCompra):Date
def:
    compra = compras[idCompra]
pre:
    compra → size() = 1
body:
    compra.enderecoEntrega.cidade.tempoEntrega

```

A situação aqui representada é simplificada com o objetivo de mostrar como possíveis contratos poderiam ser feitos. Não se pretende demonstrar uma situação real de compra, que seria bem mais complexa e, portanto, fugiria dos objetivos do livro.

8.8.2. Contratos para a Estratégia Statefull

A estratégia *statefull* pressupõe que o sistema seja capaz de “lembrar” determinadas informações temporárias, o que não é possível com a estratégia *stateless*. Por isso, não é necessário tanta passagem de parâmetros quando se

² Aqui não se considerou a possível exceção de a compra eventualmente não ser autorizada.

³ Como **Autorização** é uma classe de associação, esta instância foi criada no momento em que o cartão foi associado com a compra na operação **defineCartao**. Porém, naquele momento o código de autorização era zero.

usa a estratégia *statefull*, mas é necessário estabelecer *como* essas informações serão armazenadas.

Uma possibilidade seria armazenar essas informações em variáveis globais ou da classe controladora. Mas tais soluções são pouco elegantes por fugirem da estrutura usual do modelo conceitual. Melhor seria representar essas informações temporárias como associações temporárias adicionadas ao modelo conceitual, como na Figura 8.9.

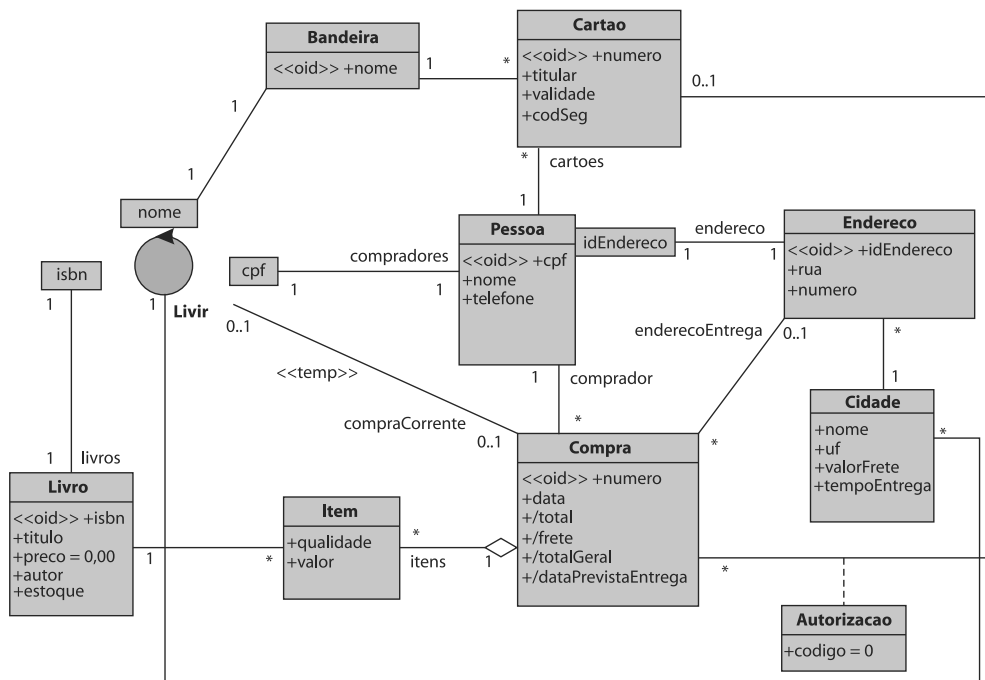


Figura 8.9: Modelo conceitual de referência para estratégia *statefull*.

Nesse caso, basta guardar a informação da compra corrente, pois comprador, cartão e endereço já podem ser inferidos pelas associações persistentes existentes.

Por outro lado, não é mais necessária a associação derivada para encontrar a compra corrente a partir de seu número, visto que a associação temporária permite acesso direto a essa instância.

A Tabela 8.2 apresenta as operações e consultas de sistema para a estratégia *statefull*, conforme o diagrama da Figura 6.6.

Tabela 8.2: Operações e Consultas de Sistema da Figura 6.6

```

criaCompra(idComprador)
listaLivrosDisponiveis():Set
adicionaCompra(idLivro, quantidade)
consultaTotal():Money
listaEnderecos():Set
defineEnderecoEntrega(idEndereco)
consultaFrete():Money
consultaTotalGeral():Money
listaCartoes():Set
defineCartao(idCartao)
solicitacaoPagto():Tuple
registraPagto(codigoAutorizacao)
consultaPrazoEntrega():Date

```

A primeira operação, `criaCompra(idComprador)`, não precisa mais retornar o `idCompra`, pois a compra corrente ficará registrada na associação temporária `compraCorrente`. Seu contrato fica, portanto, assim:

```

Context Livir::criaCompra(idComprador)
def:
    novaCompra = Compra::newInstance()
def:
    comprador = compradores[idComprador]
post:
    novaCompra^setNumero(novoNumeroAutomatico()) AND
    novaCompra^setData(dataAtual()) AND
    novaCompra^addComprador(comprador) AND
    self^addCompraCorrente(novaCompra)
exception:
    comprador → size() = 0 IMPLIES
    self^throw("Comprador não cadastrado")

```

Os contratos da consulta `listaLivrosDisponiveis` são idênticos nos dois casos. Seguem os contratos das demais consultas e operações de sistema:

```
Context Livir::adicionaCompra(idLivro, quantidade)
def:
    livro = livros[idLivro]
def:
    item = Item::newInstance()
pre:
    livro→size() = 1 AND
    compraCorrente→size() = 1
post:
    item^setQuantidade(quantidade) AND
    item^setValor(valor) AND
    item^addCompra(compraCorrente) AND
    item^addLivro(livro) AND
    livro^setEstoque(livro.estoque@pre - quantidade)
exception:
    quantidade>livro.estoque IMPLIES
        self^throw("quantidade insuficiente em estoque")
```

```
Context Livir::consultaTotal():Money
pre:
    compraCorrente→size() = 1
body:
    compraCorrente.total
```

```
Context Livir::listaEnderecos():Set
pre:
    compraCorrente→size() = 1
body:
    compraCorrente.comprador.enderecos→collect(endereco|
        Tuple {
            id = endereco.idEndereco,
            rua = endereco.rua,
            numero = endereco.numero,
```

```

        cidade = endereco.cidade.nome,
        uf = endereco.cidade.uf
    }
)

```

```

Context Livir::defineEnderecoEntrega(idEndereco)
def:
    endereco=compraCorrente.comprador.enderecos[idEndereco]
pre:
    endereco→size() = 1 AND
    compraCorrente→size() = 1
post:
    compraCorrente^addEnderecoEntrega(endereco)

```

```

Context Livir::consultaFrete():Money
pre:
    compraCorrente→size() = 1
body:
    compraCorrente.frete

```

```

Context Livir::consultaTotalGeral():Money
pre:
    compraCorrente→size() = 1
body:
    compraCorrente.totalGeral

```

```

Context Livir::listaCartoes():Set
pre:
    compraCorrente→size() = 1
body:
    compraCorrente.comprador.cartoes→collect(cartao|
        Tuple {
            bandeira = cartao.bandeira.nome,

```

```
        numero = cartao.numero
    }
)
```

```
Context Livir::definecartao(idCartao)
def:
    cartao = compraCorrente.comprador.cartoes
    →select(numero=idCartao)
pre:
    cartao→size() = 1 AND
    compraCorrente→size() = 1
post:
    compraCorrente^addCartao(cartao)
```

```
Context Livir::solicitacaoPagto():Tuple
pre:
    compraCorrente→size() = 1 AND
    compraCorrente.cartao→size() = 1
body:
    Tuple {
        numero = compraCorrente.cartao.numero,
        titular = compraCorrente.cartao.titular,
        validade = compraCorrente.cartao.validade,
        codSeg = compraCorrente.cartao.codSeg
    }
```

```
Context Livir::registraPagto(codigoAutorizacao)
pre:
    compraCorrente→size() = 1 AND
    compraCorrente.cartao→size = 1
post:
    compraCorrente.autorizacao^setCodigo(codigoAutorizacao)
```

```
Context Livir::consultaPrazoEntrega():Date
pre:
    compraCorrente→size() = 1
body:
    compraCorrente.enderecoEntrega.cidade.tempoEntrega
```

Observa-se que, na maioria das operações e consultas, a principal alteração entre as estratégias *stateless* e *statefull* foi a troca da expressão, que era definida como `compras[idCompra]` na estratégia *stateless* e por `self.compraCorrente` na estratégia *statefull*.

Projeto da Camada de Domínio

Ainda durante a fase de *elaboração*, após as atividades de análise, vêm as atividades de projeto. Mas o que é *projeto de software orientado a objetos*? Pode-se dividir as atividades de projeto em dois grandes grupos:

- a) o *projeto lógico*, que inclui os diagramas de classe que evoluem a partir do modelo conceitual e os diagramas de modelagem dinâmica que representam a maneira como os objetos interagem para executar as operações e consultas de sistema;
- b) o *projeto tecnológico*, que inclui todos os aspectos do problema que são inerentes à tecnologia empregada: interface, armazenamento de dados, segurança, comunicação, tolerância a falhas etc.

O *projeto* do software visa produzir uma *solução* para o problema que, nesse ponto, já deve estar suficientemente esclarecido pela análise. O *problema* está especificado no modelo conceitual, nos contratos e nos diagramas de sequência de sistema ou casos de uso expandidos. Resta agora projetar uma arquitetura de software (e possivelmente hardware) para realizar lógica e tecnologicamente o sistema, ou seja, para apresentar uma solução ao problema enunciado.

O projeto lógico também é conhecido como projeto da *camada de domínio*. Essa camada da arquitetura do software corresponde ao conjunto de

classes que vai realizar toda a lógica de acesso e transformação dos dados do sistema de informação. As demais camadas (persistência, interface, segurança etc.) são derivadas ou dependentes da camada de domínio, servindo para conectar essa lógica pura com os aspectos físicos da computação (redes, interfaces, dispositivos de armazenamento etc.).

O projeto da camada de domínio consiste basicamente em duas atividades que podem ser executadas concomitantemente:

- a) *modelagem dinâmica*, que consiste em construir modelos de execução para os contratos de operação e consulta de sistema. Em orientação a objetos, tais modelos devem ser modelos de interação entre objetos, que usualmente são representados através de diagramas de comunicação ou diagramas de sequência da UML ou, ainda, diretamente na forma algorítmica;
- b) *elaboração do diagrama de classes de projeto (DCP)*, que consiste basicamente em adicionar ao modelo conceitual algumas informações que não era possível ou desejável obter na atividade de análise, como, por exemplo, a direção das associações e os métodos a serem implementados nas classes. Esses aspectos só podem ser efetivamente inseridos no diagrama durante a modelagem dinâmica.

As demais fases do projeto que são abordadas neste livro em capítulos subsequentes são:

- a) *projeto da camada de interface* (Capítulo 10), em que são vistas técnicas para manter a independência entre a camada de domínio e a interface do software;
- b) *projeto da camada de persistência* (Capítulo 11), em que é visto como implementar um sistema de persistência que automatiza o salvamento e a recuperação de dados em memória secundária, retirando do projetista a necessidade de se preocupar com esses aspectos.

O projeto lógico do software pode ser realizado de forma bastante sistemática, desde que o projetista possua dois artefatos da atividade de análise corretamente construídos:

- a) o modelo conceitual;
- b) os contratos de operações e consultas de sistema (modelo funcional).

O trabalho do projetista consistirá em:

- a) construir um diagrama de comunicação (ou de sequência) para cada operação e consulta de sistema, levando em conta o modelo conceitual e o respectivo contrato;
- b) construir e aprimorar o DCP a partir do modelo conceitual e dos diagramas de comunicação desenvolvidos.

A modelagem dinâmica, como foi explicado, pode se valer de diagramas de comunicação, diagramas de sequência ou ainda de algoritmos. Cada uma das formas tem vantagens e desvantagens:

- a) *algoritmos* são os mais fáceis de fazer, mas é difícil perceber claramente as conexões entre os objetos em simples textos. Assim, algoritmos podem fazer com que o acoplamento entre as classes seja aumentado, prejudicando a qualidade do projeto;
- b) *diagramas de comunicação* são melhores do que os algoritmos para visualizar e distribuir responsabilidades e melhores do que os diagramas de sequência para visualizar as dependências de visibilidade entre os objetos. Mas pode ser difícil organizá-los graficamente, no caso de colaborações mais complexas, e algumas vezes são mais difíceis de ler do que os diagramas de sequência;
- c) *diagramas de sequência* são mais fáceis de entender, mas não explicitam as ligações de visibilidade entre os objetos, podendo permitir, caso o projetista não esteja atento, comunicações inválidas ou impossíveis.

Inicialmente, este capítulo usará diagramas de comunicação para mostrar a importância de perceber que os objetos se comunicam através de linhas de visibilidade. Posteriormente, serão utilizados diagramas de sequência nos exemplos, por serem aparentemente mais populares (e, talvez por isso, inúmeras vezes mal elaborados).

A distribuição de responsabilidades entre os objetos tem a ver com a questão: *quais métodos devem ficar em cada classe?*

Muitos projetistas têm dificuldade para construir uma solução elegante para esse problema quando tentam simplesmente acrescentar métodos em um diagrama de classes. O uso de diagramas de comunicação e padrões de projeto pode, entretanto, permitir uma forma muito mais eficiente de *descobrir* o local adequado para implementar cada método.

A definição dos métodos é feita na atividade de projeto do sistema, quando se constrói o DCP. Para construir esse diagrama, deve-se observar

inicialmente o modelo conceitual (conceitos e associações que representam a estrutura da informação descoberta na atividade de análise de domínio).

Depois, é necessário observar os diagramas de comunicação ou sequência. Esses diagramas apresentam objetos trocando mensagens para realizar contratos. Existe todo um processo para se definir corretamente esses diagramas. A construção deles para representar os aspectos dinâmicos internos do sistema é, especialmente no caso de projetistas menos experientes, superior ao processo de escrever algoritmos porque, quando um projetista faz um algoritmo, ele pode tender a concentrar todas as responsabilidades em uma única classe, enquanto o uso dos diagramas (especialmente os de comunicação) permite melhor visualização da distribuição espacial dos objetos e suas formas de visibilidade, o que possibilita melhor aplicação dos padrões de projeto para distribuição de responsabilidades.

Quando se trabalha com orientação a objetos sem um método adequado, ou seja, simplesmente fazendo um diagrama de classes e adicionando métodos nas classes, sem uma técnica sistemática e padronizada que dirija essa atividade, as responsabilidades das classes acabam sendo mal distribuídas e o resultado final acaba sendo tão desestruturado quanto os chamados programas *spaghetti*.

Assim, para um sistema ser elegante, as responsabilidades têm de estar bem distribuídas. Se não se usa um método sistemático, pode acontecer que as responsabilidades acabem ficando concentradas na classe que representa o sistema como um todo (como *Livir*) ou naquelas classes que representam seres humanos, como *Comprador* ou *Funcionário*. Então, acabaria acontecendo que classes como *Livro*, *Venda*, *Pagamento* etc. não teriam nenhum método relevante.

Quando uma ou duas classes fazem tudo e as outras são meras pacientes desse processo, não existe propriamente orientação a objetos, mas uma estrutura concentradora. Seria preferível fazer um projeto estruturado benfeito do que um projeto orientado a objetos dessa forma.

Projetistas podem cometer o erro de acreditar que um sistema orientado a objetos é uma simulação do mundo real. Mas isso não é normalmente verdade. O sistema representa as *informações* do mundo real e não as *coisas* propriamente ditas. Há aqui uma diferença sutil, mas importante. Os métodos não correspondem a ações do mundo real, mas à realização interna de contratos de operações e consultas de sistema. Por esse motivo é que os métodos

internos são citados apenas na atividade de projeto e sequer aparecem na atividade de análise.

Sendo assim, projetar software orientado a objetos deve ser compreendido como um processo muito preciso e guiado por padrões já aprendidos, e não simplesmente como o ato de criar classes e associar métodos a elas de forma *ad-hoc*.

9.1. Responsabilidades e Operações Básicas

Para que haja uma boa distribuição de responsabilidades entre os diferentes tipos de objetos, inicialmente será definida uma classificação. Basicamente, há dois grandes grupos de responsabilidades:

- a) responsabilidades de *conhecer*, que correspondem às consultas;
- b) responsabilidades de *fazer* ou *alterar*, que correspondem às operações.

Ambos os grupos ainda se subdividem em três subgrupos:

- a) coisas que objeto conhece ou faz sobre *si mesmo*;
- b) coisas que o objeto conhece ou faz a respeito das suas *vizinhanças*;
- c) outras coisas que o objeto conhece ou faz não classificadas nos subgrupos anteriores; normalmente *conhecimentos derivados* e *ações coordenadas*.

No caso das responsabilidades de conhecer, os três subgrupos poderiam ser assim caracterizados:

- a) *coisas que o objeto conhece sobre si mesmo*: equivale a poder acessar o valor dos atributos do objeto. Tais responsabilidades são incorporadas às classes através de operações básicas de consulta, que são nomeadas com o prefixo *get* seguido do nome do atributo. Por exemplo, se a classe *Pessoa* tem um atributo *dataNascimento*, então o método *getDataNascimento()* realiza a responsabilidade de conhecer o valor desse atributo;
- b) *coisas que o objeto conhece sobre suas vizinhanças*: equivale a poder acessar outros objetos que estão associados diretamente a ele. Essa responsabilidade é então realizada por métodos que acessam o conjunto de objetos associados através de cada uma das associações de um objeto. Tais métodos também são usualmente representados por um prefixo

get seguido do nome de papel da associação. Por exemplo, se a classe Pessoa tem uma associação com Reserva com nome de papel reservas, então o método `getReservas()` retorna o conjunto de reservas de uma pessoa;

- c) *coisas que o objeto conhece de forma derivada*: equivale a conhecimentos que são combinações de outros, por exemplo, uma venda pode saber seu valor total a partir da soma dos preços dos produtos que estão associados a ela. Essa responsabilidade é também identificada pelo prefixo `get`, seguido do nome da informação (por exemplo, `getTotal()`), e frequentemente corresponde ao método de acesso de um atributo derivado, ou seja, `getTotal()` será o método de acesso ao atributo derivado total.

No caso das responsabilidades de fazer ou alterar informações, os três subgrupos poderiam ser assim caracterizados:

- a) *coisas que o objeto faz sobre si mesmo*: corresponde às operações básicas de alteração de atributos, identificadas pelo prefixo `set` seguido do nome do atributo. Então, se a classe Pessoa tem o atributo `dataNascimento`, o método `setDataNascimento(umaData)` realiza essa responsabilidade;
- b) *coisas que o objeto faz sobre suas vizinhanças*: corresponde às operações básicas de adição e remoção de associações, identificadas, respectivamente, pelos prefixos `add` e `remove`, seguidos do nome de papel da associação. Então, se a classe Pessoa possui associação com Reserva e nome de papel reservas, os métodos `addReservas(umaReserva)` e `removeReservas(umaReserva)` realizam essa responsabilidade do ponto de vista da classe Pessoa;
- c) *coisas que o objeto faz de forma coordenada*: corresponde a operações múltiplas que alteram objetos e que são coordenados a partir de um objeto que detenha a melhor visibilidade possível para os objetos a serem alterados. Tais operações são conhecidas como *métodos delegados* e são bastante importantes na atividade de projeto porque os demais métodos (básicos) são definidos por padrão, mas os delegados precisam ser elaborados. Por exemplo, se todos os livros de uma determinada venda devem ser marcados como entregues, quem deve coordenar essas atividades de marcação é a própria venda, que possui acesso (ou visibilidade) mais direta para os livros que devem sofrer a operação.

A Tabela 9.1 resume os seis tipos de responsabilidades e os métodos tipicamente associados a cada tipo.

Tabela 9.1: Tipos de Responsabilidades

	Conhecer	Fazer
Sobre si mesmo	Consultar atributos getAtributo()	Modificar atributos setAtributo(valor)
Sobre suas vizinhanças	Consultar associações getPapel()	Modificar associações addPapel(umObjeto) removePapel(umObjeto)
Outros	Consultas gerais, Associações e atributos derivados consultaInformação() getAtributoDerivado() getAssociaçãoDerivada	Métodos delegados -- nomes variam

Não há um prefixo padrão para métodos derivados, que serão nomeados conforme a operação que executem. Usualmente, esses nomes não devem incluir o nome da classe onde estão implementados. Por exemplo, *Livir* poderá ter uma operação de sistema `encerrarVenda()`. Se essa operação for delegada à classe *Venda*, o nome do método delegado deverá ser simplesmente `encerrar()`, pois possivelmente será invocado assim: `venda.encerrar()`.

9.2. Visibilidade

Para que dois objetos possam trocar mensagens para realizar responsabilidades derivadas e coordenadas, é necessário que exista *visibilidade* entre eles. Existem quatro formas básicas de visibilidade:

- por associação*: quando existe uma associação entre os dois objetos de acordo com as definições de suas classes no modelo conceitual;
- por parâmetro*: quando um objeto, ao executar um método, recebe outro como parâmetro;
- localmente declarada*: quando um objeto, ao executar um método, recebe o outro como retorno de uma consulta;
- global*: quando um objeto é declarado globalmente.

Nenhuma das formas de visibilidade é necessariamente simétrica. Ou seja, se *x* tem visibilidade para *y*, isso não significa que *y* tenha necessariamente visibilidade para *x*.

9.2.1. Visibilidade por Associação

Somente pode existir *visibilidade por associação* entre dois objetos quando existir uma associação entre as classes correspondentes no modelo conceitual ou DCP.

O tipo de visibilidade que se tem varia conforme a multiplicidade de papel e de outras características, como o fato de a associação ser qualificada ou possuir classe de associação.

Um papel de associação *sempre* pode ser tratado como um conjunto de instâncias. Mas, no caso da multiplicidade para um, ainda é possível interpretar o papel como sendo um objeto individual. Em relação à multiplicidade, pode-se identificar os seguintes tipos:

- a) se a multiplicidade de papel for para um, tem-se visibilidade diretamente para uma instância;
- b) qualquer outra multiplicidade de papel dá visibilidade a um conjunto de instâncias.

Essa multiplicidade, como será visto, não é restrita apenas ao modelo conceitual. Caso o contrato da operação ou consulta em questão possua uma pre-condição que estabeleça uma multiplicidade mais restrita do que a do modelo conceitual, é a multiplicidade do contrato a que vale.

9.2.1.1. Visibilidade para Um

Na Figura 9.1a, a classe Pagamento tem uma associação *para um* com Venda. Nesse caso, qualquer instância de Pagamento terá visibilidade direta para uma instância de Venda. Dessa forma, quando tais instâncias forem representadas em um diagrama de comunicação, como na Figura 9.1b, a instância de Pagamento poderá enviar mensagens diretamente à instância de Venda.

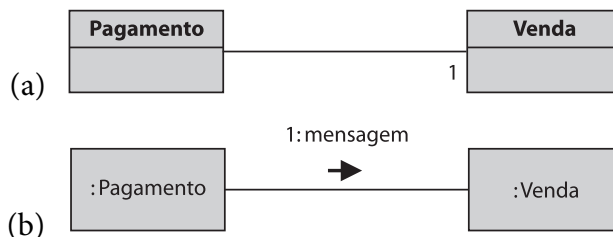


Figura 9.1: (a) Um diagrama de classe com associação para um. (b) Um diagrama de comunicação onde um objeto :Pagamento tem visibilidade por associação para um objeto :Venda.

9.2.1.2. Visibilidade para Muitos

Outras formas de multiplicidade diferentes de *para um*, como “*”, “1..*”, “0..1”, “5” etc. habilitam a visibilidade não para uma, mas para uma coleção de instâncias. O caso 0..1 ainda poderia ser tratado como uma visibilidade *para um* que aceita o objeto *null*, mas por questão de padronização convém tratar esse caso como um conjunto que pode ser unitário ou vazio. A partir de agora, todos esses casos serão tratados genericamente como “*”.

Na Figura 9.2a a classe *Venda* tem uma associação para “*” para a classe *Item*. Verifica-se que, nesse caso, uma instância de *Venda* pode enviar mensagens a conjuntos de instâncias de itens. É possível enviar mensagens ao conjunto propriamente dito, como na Figura 9.2b (por exemplo, consultando o tamanho do conjunto ou adicionando um elemento a ele) ou, ainda, enviar mensagens a cada um dos elementos do conjunto, como na Figura 9.2c (por exemplo, solicitando o valor do subtotal de cada item). Ou seja, a mensagem pode ser endereçada a estrutura de dados que contém os elementos ou a cada um dos elementos iterativamente.

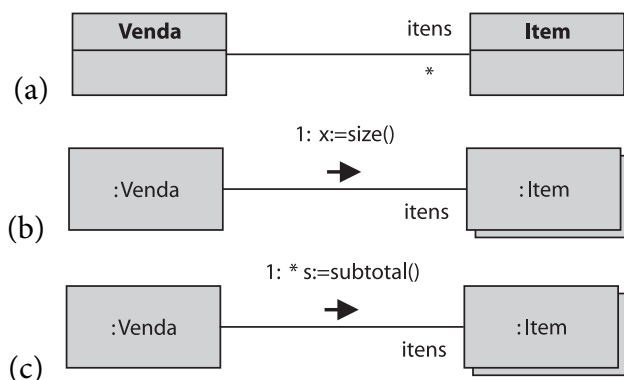


Figura 9.2: (a) Diagrama de classes com associação *para muitos*. (b) Diagrama de comunicação com mensagem para um conjunto. (c) Diagrama de comunicação com mensagem para cada um dos elementos de um conjunto.

No caso da Figura 9.2c, o asterisco antes da mensagem propriamente dita identifica que se trata de uma mensagem enviada a cada um dos elementos do conjunto e não ao conjunto em si, como na Figura 9.2b.

9.2.1.3. Associações Ordenadas

Se a associação for ordenada (*OrderedSet* e *Sequence*), além da visibilidade ao conjunto todo, pode-se ter visibilidade a um elemento específico da

associação com base na sua posição: primeiro, ultimo e *n*-ésimo. A Figura 9.3 exemplifica esse caso.

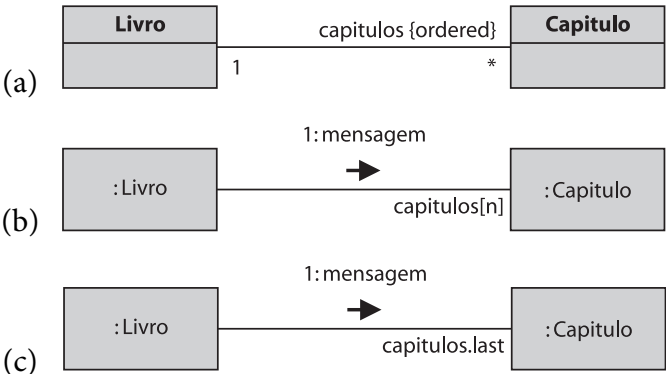


Figura 9.3: Diagrama de classes com associação ordenada. (b) Visibilidade ao *n*-ésimo elemento. (c) Visibilidade ao último elemento.

9.2.1.4. Associações Qualificadas

Se a associação para muitos for qualificada como mapeamento (com multiplicidade 1 no lado oposto ao qualificador), haverá duas formas de visibilidade direta:

- a) visibilidade para o conjunto como um todo exatamente como se fosse uma associação para “*”;
- b) se o objeto do lado do qualificador possuir uma chave para acessar a associação, ele terá visibilidade direta para o elemento qualificado por essa chave.

A Figura 9.4a apresenta um diagrama de classes com uma associação qualificada entre Pessoa e Cartao. A Figura 9.4b demonstra que, nesse caso, uma instância de Pessoa tem visibilidade para o conjunto de seus cartões. A Figura 9.4c mostra que, caso a pessoa em questão possua o valor do qualificador (número do cartão), ela terá visibilidade direta para o elemento qualificado por esse número. Além disso, ela ainda pode acessar todos os elementos da associação, como no caso da Figura 9.2c.



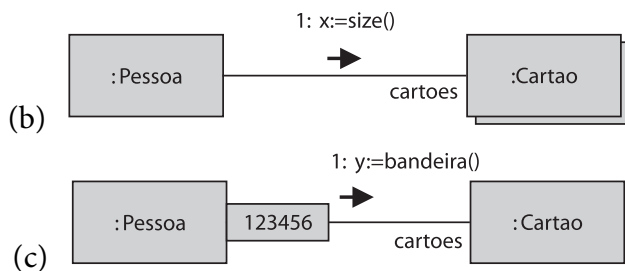


Figura 9.4: (a) Diagrama de classe com associação qualificada. (b) Diagrama de comunicação representando visibilidade para o conjunto. (c) Diagrama de comunicação representando visibilidade para um elemento qualificado.

Por outro lado, se a multiplicidade do lado oposto ao qualificador for diferente de *um*, o que se tem é uma partição, ou seja, uma divisão de um conjunto maior em subconjuntos (Figura 9.5a). Nesse caso, a visibilidade sem o qualificador (Figura 9.5b) representa o conjunto completo (no exemplo, todos os livros) e a visibilidade com o qualificador (Figura 9.5c) representa um subconjunto (no caso, os livros infantis).

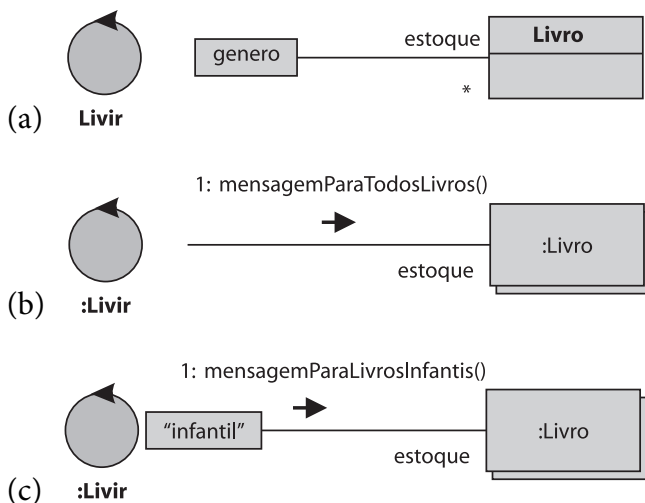


Figura 9.5: (a) Diagrama de classes com qualificador definindo partição. (b) Mensagem para o conjunto como um todo. (c) Mensagem para um subconjunto.

Tanto no caso da Figura 9.5b quanto na 9.5c, trata-se de mensagens enviadas a conjuntos. Para enviar mensagens a cada um dos elementos desses conjuntos, seria necessário prefixar a mensagem com `*`.

9.2.1.5. Classes de Associação

Conforme já explicado no Capítulo 7, uma classe de associação tem instâncias que são criadas e destruídas conforme associações definidas entre duas outras classes sejam criadas e destruídas. Assim, se A tem uma associação com B, e C é a classe de associação correspondente, pode-se inferir que instâncias de A terão visibilidade para a mesma quantidade de instâncias de B e de C. Simetricamente, instâncias de B terão visibilidade para a mesma quantidade de instâncias de A e C. Por outro lado, uma instância C só terá visibilidade para uma única instância de A e uma única instância de B.

A Figura 9.6a apresenta um diagrama de classe típico com classe de associação. A Figura 9.6b representa a visibilidade que :Pessoa tem para :Empresa (corresponde à multiplicidade do papel emprego, ou seja, uma visibilidade para um conjunto). Já a Figura 9.6c representa a visibilidade que :Pessoa tem para :Emprego. Como uma pessoa tem um emprego para cada empresa, essa visibilidade também corresponde à multiplicidade do papel emprego. Há, então, uma dualidade no papel emprego nesse caso, visto que ele permite que uma :Pessoa acesse tanto o conjunto de empresas quanto o conjunto de empregos.

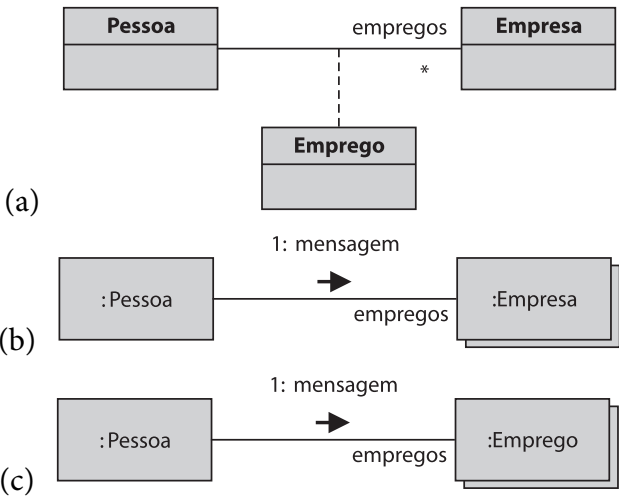


Figura 9.6: (a) Modelo conceitual com classe de associação. (b) Visibilidade usual para um conjunto dado pelo papel. (c) Visibilidade para um conjunto obtido a partir da classe de associação.

A classe de associação também funciona como mapeamento similarmente à associação qualificada. No caso da Figura 9.6a, esse mapeamento equivale a mapear um emprego para cada empresa onde a pessoa trabalha.

Assim, havendo uma empresa dada, é possível determinar de forma única um emprego de uma dada pessoa (Figura 9.7).

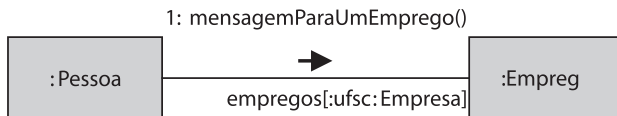


Figura 9.7: Uma representação de visibilidade onde a associação qualificada funciona como um mapeamento.

Ainda na Figura 9.7, observa-se que `:ufsc` deve ser uma instância da classe `Empresa` para que o acesso direto ao emprego de `:Pessoa` seja possível.

Finalmente, a Figura 9.8 mostra a visibilidade que instâncias da classe de associação têm das demais classes participantes. No caso, uma visibilidade necessariamente *para um*.

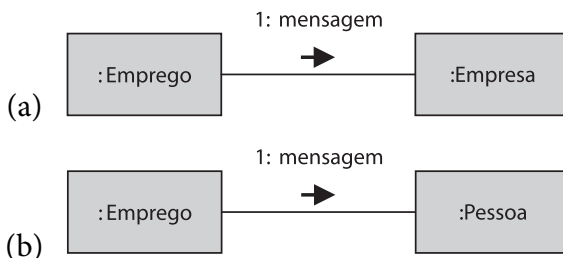


Figura 9.8: Visibilidade da classe de associação para as classes participantes: (a) Visibilidade de `:Emprego` para `:Empresa`. (b) Visibilidade de `:Emprego` para `:Pessoa`.

9.2.1.6. Influência das Precondições na Visibilidade por Associação

Quando uma precondição de operação ou consulta de sistema restringe ainda mais uma multiplicidade de papel, será sempre a precondição que vai valer como determinante da visibilidade no contexto daquela operação.

Por exemplo, se um diagrama de classe como o da Figura 9.9a define que uma venda pode ter ou não um pagamento, mas a operação sendo executada tem como precondição algo como:

```

pre:
    venda.pagamento->size() = 1
  
```

então, *no contexto daquela operação*, passa a valer a visibilidade *para um* e não mais *para zero ou um*. É como se, durante a execução dessa operação, o modelo conceitual fosse temporariamente alterado para ficar como na Figura 9.9b.

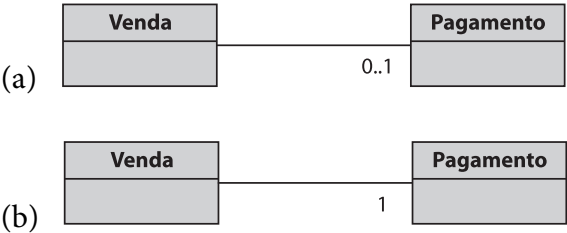
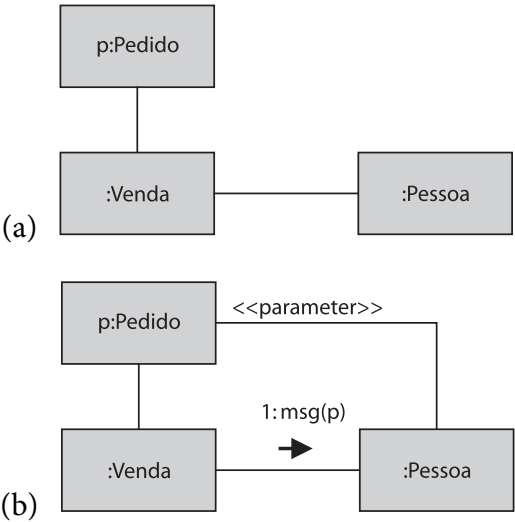


Figura 9.9: (a) Visibilidade opcional de Venda para Pagamento. (b) Como fica a visibilidade temporariamente durante uma operação que tenha como pré-condição a necessidade da existência do pagamento para uma determinada instância de Venda.

9.2.2. Visibilidade por Parâmetro

A visibilidade por parâmetro é obtida quando um objeto, ao executar um método, recebe outro objeto como parâmetro. Não precisa haver, nesse caso, associação entre as classes para que o primeiro objeto possa se comunicar com o segundo.

Por exemplo, uma instância de Pessoa, ao executar o método msg, recebe como parâmetro uma instância de Pedido identificada pelo argumento p. Nesse caso, independentemente de existir associações entre Pessoa e Pedido, a instância de Pessoa que está executando o método passa a ter visibilidade *por parâmetro* para o pedido p (Figura 9.10).



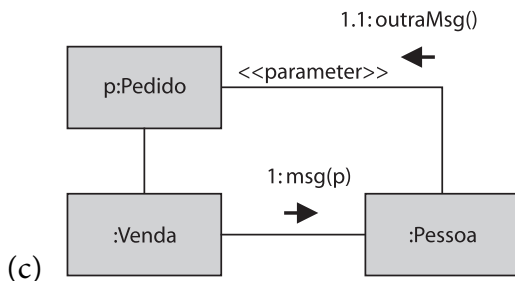


Figura 9.10: (a) Situação inicial, onde :Venda tem visibilidade para p:Pedido e :Pessoa. (b) Após :Venda enviar mensagem msg para :Pessoa passando p:Pedido como parâmetro, :Pessoa adquire visibilidade por parâmetro para p:Pedido. (c) A partir desse ponto, :Pessoa pode se comunicar diretamente com p:Pedido.

No caso da visibilidade por parâmetro, deve-se admitir que o objeto que enviou a mensagem msg para a instância de Venda detinha algum tipo de visibilidade para a instância de Pessoa que enviou como parâmetro.

9.2.3. Visibilidade Localmente Declarada

Outra forma de visibilidade possível ocorre quando um objeto, ao enviar uma consulta a outro, recebe *como retorno* um terceiro objeto, como demonstrado na Figura 9.11.

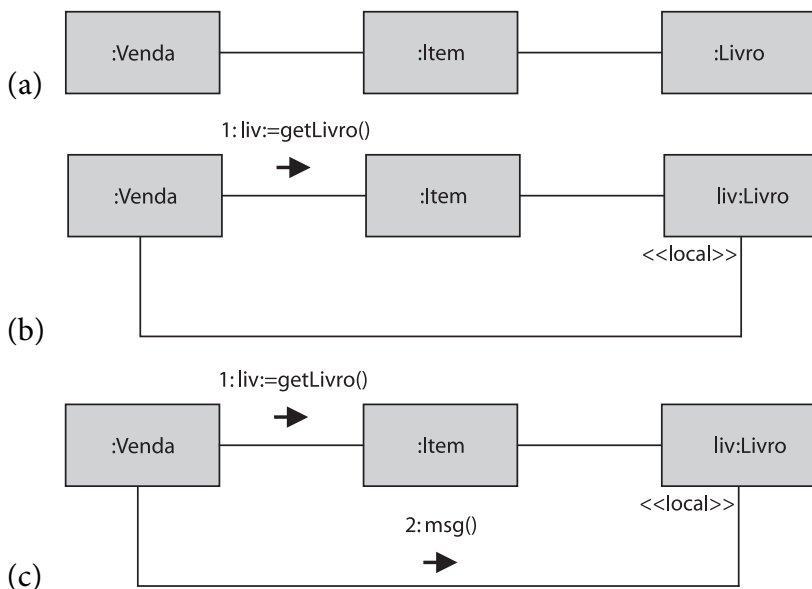


Figura 9.11: (a) Situação inicial. (b) :Venda envia uma mensagem a :Item e recebe liv:Livro como retorno. A partir desse ponto, :Venda adquire visibilidade local para liv:Livro. (c) Agora :Venda pode se comunicar diretamente com liv:Livro.

Tanto a visibilidade local quanto a visibilidade por parâmetro somente são válidas durante a execução do método que deu origem a elas, ou seja, assim como variáveis locais e parâmetros de procedimentos, elas só têm validade dentro da operação onde foram declaradas, desaparecendo após o término desta. Já a visibilidade por associação é permanente, persistindo até que seja explicitamente removida por uma operação básica de destruição de associação.

Deve-se evitar ao máximo usar a visibilidade por parâmetro e a localmente declarada para enviar mensagens a objetos.

Por princípio, deve-se optar por enviar mensagens apenas através de ligações de visibilidade por associação. Objetos recebidos como parâmetro ou retorno devem apenas ser repassados como parâmetro, se possível. Já a visibilidade local deveria, por princípio, ser usada *apenas* quando o método que retorna o objeto efetivamente *cria* o objeto. Esse princípio é conhecido em padrões de projeto como “*Não Fale com Estranhos*” ou “*Law of Demeter*” (Lieberherr & Holland, 1989).

9.2.4. Visibilidade Global

Existe visibilidade global para um objeto quando ele é declarado globalmente. O padrão de projeto *Singleton* (Gamma *et al.*, 1995) admite uma instância globalmente visível apenas quando ela é a *única instância possível da sua classe*. Isso faz sentido porque, se essa classe possui uma única instância, não é necessário que outros objetos possuam associação para ela. Também não é necessário passá-la como parâmetro ou como retorno de métodos.

Um exemplo seriam classes de projeto que não representam conceitos do modelo conceitual, mas serviços, como: *ConversorDeMoedas*, *ContadorDeTempo* etc. Haverá uma única instância dessas classes de serviço que podem ser acessadas por qualquer objeto a qualquer tempo (Figura 9.12).



Figura 9.12: Visibilidade global.

9.3. Realização Dinâmica das Pós-condições

Já foi visto que os contratos de operação de sistema apresentam um conjunto de pós-condições que correspondem a certas operações básicas de criação e destruição de instâncias, criação e destruição de associações e modificação de valor de atributos. Os contratos apenas indicam *o que* deve acontecer, sem mostrar como mensagens reais são trocadas entre objetos para realizar tais ações. Os diagramas de comunicação podem ser usados exatamente para mostrar como essas trocas são feitas. Os seguintes princípios devem ser observados:

- a) a visibilidade entre instâncias de objetos é regida pela multiplicidade de papel estabelecida no modelo conceitual e eventuais precondições que restringem mais ainda tal multiplicidade;
- b) cada uma das operações básicas estabelecidas em pós-condições deve ser realizada no diagrama de comunicação através do envio de uma mensagem básica ao objeto que detém a *responsabilidade* de forma mais imediata;
- c) o fluxo de execução em um diagrama de comunicação ou sequência que representa uma operação de sistema *sempre* inicia na instância da *controladora de sistema* recebendo uma mensagem da *interface*;
- d) quando o objeto que detém o controle de execução não tiver visibilidade para o objeto que deve executar a operação básica, ele deve delegar a responsabilidade (e o controle) a outro objeto que possa fazê-lo ou que esteja mais próximo deste último.

Aplicam-se, ainda, padrões de projeto que vão auxiliar o projetista a construir métodos que sejam efetivamente reusáveis e com baixo acoplamento. Santos (2007) apresenta uma sistematização desses princípios em um sistema de regras de produção capaz de gerar bons diagramas de comunicação a partir de uma ampla gama de contratos.

9.3.1. Criação de Instância

Quando um contrato estabelece que um instância foi criada, algum objeto deve enviar uma mensagem de criação no respectivo diagrama de comunicação. O padrão “Criador” (Gamma, 1995) estabelece que deva ser prioritariamente:

- um objeto de uma classe que tenha relação de *composição* ou *agregação* para com o objeto a ser criado;
- um objeto que tenha relação de *um para muitos* com o objeto a ser criado;
- um objeto que tenha os *dados de inicialização* do objeto a ser criado e cuja classe esteja associada à classe dele.

Cada regra só se aplica se a anterior não for possível ou quando houver empate.

Considerando o diagrama da Figura 9.13, a criação de uma instância da classe *Item* deverá ser feita, de acordo com o padrão *Criador*, por uma instância da classe *Venda*, visto que existe entre elas uma relação de agregação. Se não houvesse essa agregação, a criação poderia ser feita pela classe *Livro*, mas nunca pelas classes *Pessoa* ou *Livir*, que não têm associação com *Item*.

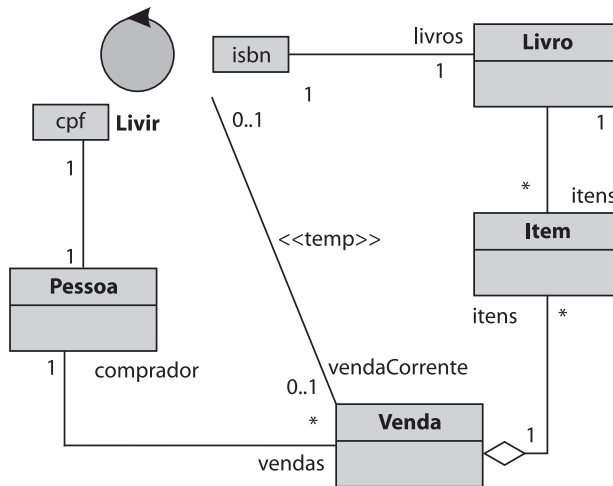


Figura 9.13: Modelo conceitual de referência.

Um fragmento de uma possível operação de sistema com a criação de uma instância de *Item* é mostrado a seguir:

```

Context Livir::adicionaItem(...)
def:
    item = Item::newInstance()
pre:
    vendaCorrente->size() = 1
post:
    ...
  
```

Aplicando-se tais princípios a esse contrato no diagrama de comunicação da Figura 9.14, deve-se iniciar o fluxo pela controladora: a controladora recebe a mensagem referente à operação de sistema diretamente da interface. Essa mensagem não deve ser numerada. Como *Livir* não tem visibilidade para *Item*, ela não pode criar essa instância, devendo delegar (mensagem 1) a uma instância de *Venda*, a venda corrente, que, por sua vez, fará a criação propriamente dita (mensagem 1.1).

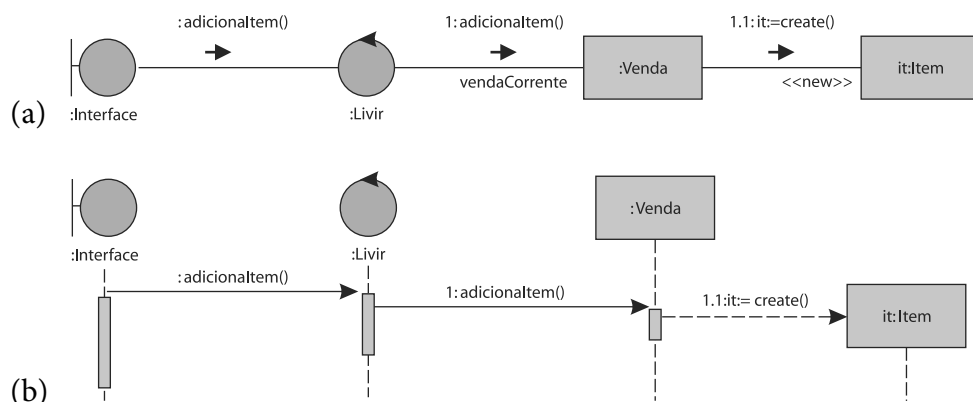


Figura 9.14: Diagrama de comunicação (a) e de sequência (b) com uma mensagem de criação de instância.

É importante observar que os diagramas da Figura 9.14 possuem três tipos de mensagens:

- mensagem que ativa uma *operação de sistema* entre a interface e a controladora *:Livir*;
- mensagem *delegada* entre a controladora e a instância de *Venda*;
- mensagem referente a uma *operação básica* entre *:Venda* e *it:Item*.

O primeiro e o terceiro tipo de mensagens *sempre* aparecerão nesses diagramas: a operação de sistema uma única vez, consistindo na raiz da árvore de mensagens enviadas. As mensagens básicas aparecerão na mesma quantidade das pós-condições dos contratos (incluindo criação de instância, que usualmente aparece na cláusula *def*). As mensagens básicas consistem nas folhas da árvore de mensagens. Elas não devem invocar novas mensagens.

Já o segundo tipo, operações delegadas, é opcional, aparecendo sempre que a controladora não for capaz de realizar as pós-condições sem delegar a outros objetos alguma responsabilidade. As operações delegadas são os ramos intermediários da árvore de mensagens e *sempre* precisam ter alguma continuação. O fluxo de mensagens só termina nas mensagens básicas (folhas).

9.3.2. Criação de Associação

Já foi comentado que, quando uma instância é criada, ela deve ser imediatamente associada a alguma outra instância para que seja acessível. Então, o contrato da seção anterior deve ser complementado para realizar também essa pós-condição adicionando o item à venda corrente:

```
Context Livir::adicionaItem(...)
def:
    item = Item::newInstance()
pre:
    vendaCorrente → size() = 1
post:
    vendaCorrente ^ addItens(item)
```

Conforme já comentado, a pós-condição que cria uma associação tem duas formas simétricas: no caso anterior, `vendaCorrente ^ addItens(item)` também poderia ter sido escrita como `item ^ addVenda(vendaCorrente)`. Embora ambas produzam o mesmo efeito final, usualmente será mais prático que o objeto que recebe a mensagem seja aquele mais próximo da controladora. No caso, a escolha seria por enviar a mensagem de `:Venda` para `:Item`, já que `:Venda` tem uma ligação direta com a controladora enquanto `:Item` só tem ligações indiretas (Figura 9.15).

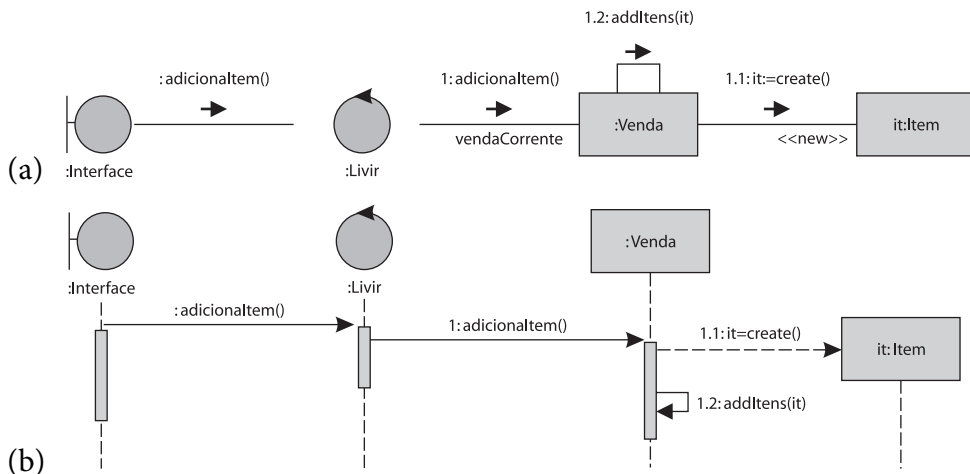


Figura 9.15: Um diagrama de comunicação (a) e de sequência (b) com uma mensagem de criação de associação.

Para que esse contrato fosse ainda mais completo, deveria haver a associação do item com um livro existente, cujo código fosse passado como parâmetro. Para acessar uma instância de Livro a partir de seu código, pode-se usar a mensagem básica de consulta, ou *get*. A mensagem *get* básica retorna todas as instâncias associadas ao papel, ou seja, um conjunto, mas quando a associação é qualificada, pode-se usar uma mensagem *get* com parâmetro correspondendo ao qualificador do objeto, como na Figura 9.16. O contrato completo é apresentado a seguir:

```
Context Livir::adicionaItem(idLivro)
def:
    item = Item::newInstance()
def:
    livro = livros[idLivro]
pre:
    vendaCorrente->size() = 1
post:
    vendaCorrente^addItens(item) AND
    item^addLivro(livro)
```

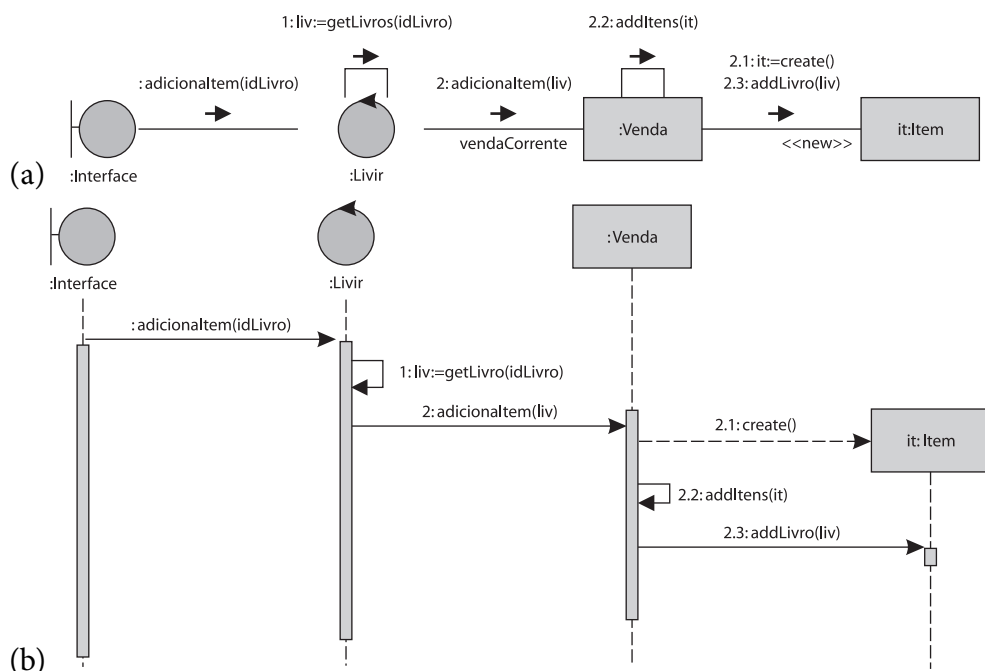


Figura 9.16: Um diagrama de comunicação (a) e de sequência (b) com uma criação de instância e duas operações de criação de associação.

Observa-se, na Figura 9.16, que a operação de sistema recebe como parâmetro um identificador alfanumérico. A controladora que tem acesso imediato ao conjunto dos livros faz o acesso através da consulta básica `getLivros`, passando o `idLivro` e recebendo `liv` como retorno, para o qual passa a ter visibilidade local. Em seguida, a controladora delega à venda a operação de adição, mas passando a instância `liv` em vez do simples identificador. A venda, por sua vez, tem capacidade para executar diretamente as três operações básicas exigidas no contrato: a criação do item, a associação da venda com o item e a associação do item com o livro, já que sua classe tem associação direta com a classe `Item`.

9.3.3. Modificação de Valor de Atributo

Outra operação básica é a operação de modificação de valor de atributo, que pode ser enviada pelo objeto que contém o atributo ou por um de seus vizinhos diretos. Continuando o exemplo anterior, adiciona-se ao contrato a necessidade de complementar o item com um valor para o atributo quantidade (que não aparece na Figura 9.13, mas está implícito). Então, além das operações já executadas, a venda ainda poderá alterar o valor desse atributo (recebido como parâmetro) através de uma mensagem básica do tipo `set`, como na Figura 9.17. Já o contrato completo aparece a seguir:

```
Context Livir::adicionaItem(idLivro, quantidade)
def:
    item = Item::newInstance()
def:
    livro = livros[idLivro]
pre:
    vendaCorrente->size() = 1
post:
    vendaCorrente^addItens(item) AND
    item^addLivro(livro) AND
    item^setQuantidade(quantidade)
```

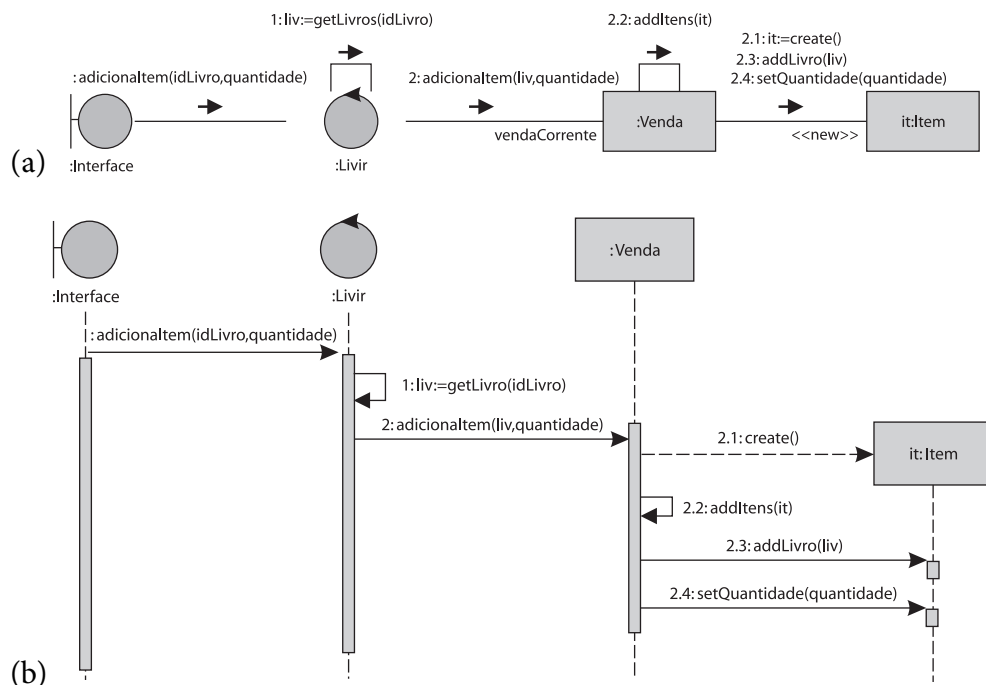


Figura 9.17: Um diagrama de comunicação com operação básica de alteração de valor de atributo.

Como se pode observar até aqui, os diagramas de comunicação podem ser substituídos por diagramas de sequência, caso se prefira trabalhar com eles. A principal inconveniência é que os diagramas de sequência não apresentam as linhas de visibilidade explicitamente.

Uma dúvida que poderia ser suscitada nesse ponto é por que a consulta `getLivro` é enviada da controladora para si própria e não para o conjunto de livros. Isso se deve ao fato de que aqui já se trata de uma operação de consulta padronizada sobre associações. A classe que possui uma associação tem visibilidade para um conjunto de instâncias de outra classe. A forma básica de acessar esse conjunto é através do envio da mensagem `get`, seguido do nome de papel à própria instância associada. No caso anterior, `Livir` tem associação com `Livro` (papel `livros`), e a forma correta de obter o conjunto de livros é enviando a mensagem `getLivros` à instância de `Livir`.

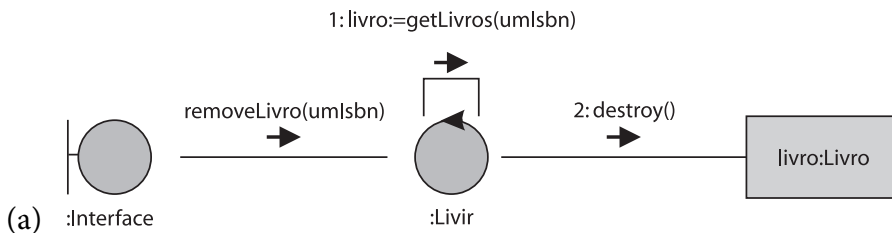
9.3.4. Destruição de Instância

A destruição de uma instância é representada pelo envio da mensagem `destroy()`. Após isso, mais nenhuma mensagem pode ser enviada a essa instância. Aplicam-se aqui também os mesmos princípios do padrão *Criador*: o objeto que destrói uma instância deve ter agregação ou composição com o objeto a ser destruído ou uma associação de um para muitos, ou, pelo menos, ser associado ao objeto.

Deve-se tomar todos os cuidados estruturais para evitar que uma instância destruída mantenha associações com objetos que precisariam dela. Se os contratos de operação de sistema estiverem bem formados, a atividade de projeto dos diagramas de comunicação só precisa representar as pós-condições já mencionadas, sem necessidade de novas checagens de consistência que já terão sido consideradas.

Na Figura 9.18, apresenta-se um modelo em que uma instância de *Livro* é removida. A operação considera que o livro em questão não está associado a nenhum item. Segue o respectivo contrato:

```
Context Livir::removeLivro(umIsbn)
def:
    livro = livros → select(livro |
        livro.isbn = umIsbn
    )
pre:
    livro.itens → size() = 0
post:
    livro ^ destroy()
```



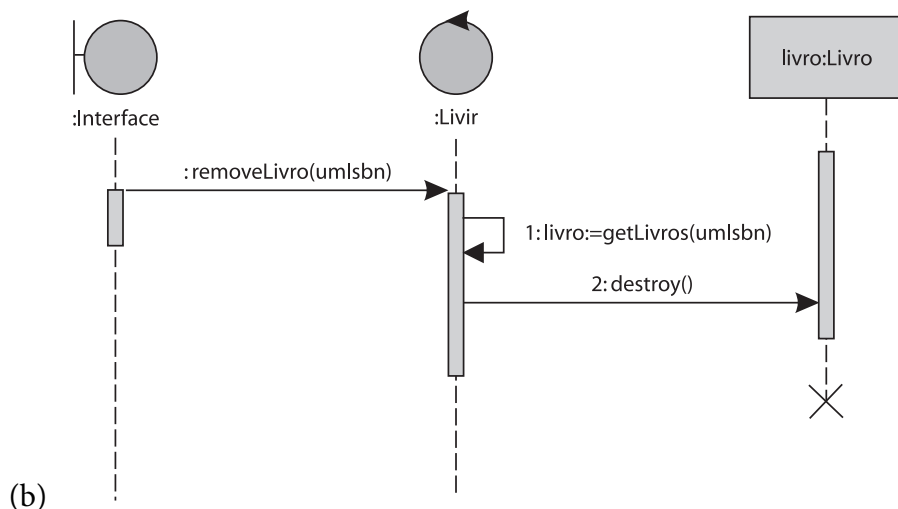


Figura 9.18: Diagrama de comunicação (a) e sequência (b) com operação básica de destruição de instância.

9.3.5. Destruição de Associação

A destruição de uma associação é realizada pela operação básica prefixada por *remove*, seguida do nome de papel. A Figura 9.19 apresenta um exemplo de remoção de associação para a operação de sistema que faz um automóvel trocar de dono, conforme o contrato a seguir:

```

Context  Control::trocaDono(idAntigoDono, idNovoDono,
idAutomovel)

def:
    antigoDono = pessoas[idAntigoDono]
def:
    novoDono = pessoas[idNovoDono]
def:
    automovel = automoveis[idAutomovel]
pre:
    antigoDono->size() = 1 AND
    novoDono->size() = 1 AND
    automovel->size() = 1
post:
    automovel^removeDono(antigoDono) AND
    automovel^addDono(novoDono)
    
```

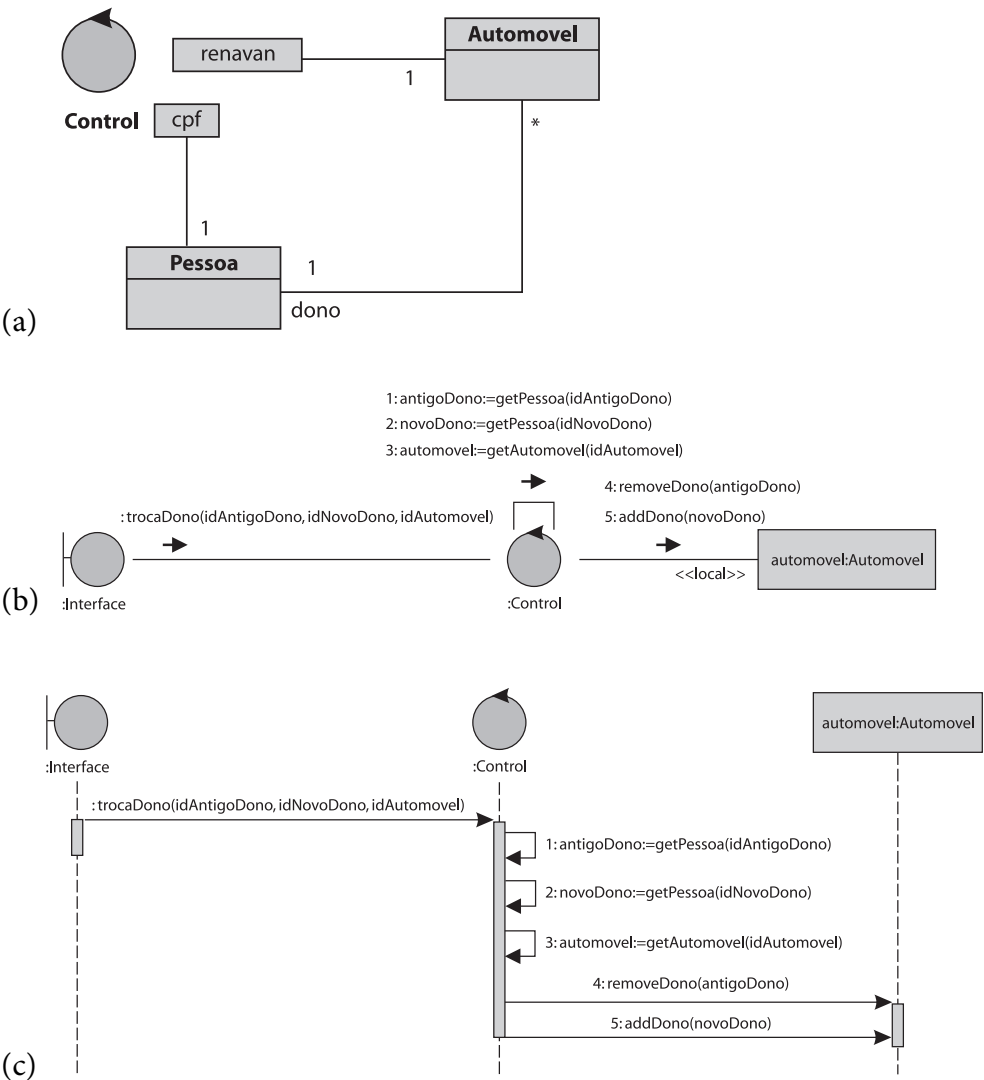



Figura 9.19: (a) Modelo conceitual de referência. Diagrama de comunicação (b) e de sequência (c) com operação básica de remoção de associação.

9.3.6. Pós-condições Condicionais

Conforme já explicado, por vezes pode-se afirmar que uma pós-condição só é obtida quando determinadas condições iniciais são satisfeitas. Nesses casos, é possível usar condicionais nas mensagens do diagrama de sequência de forma semelhante à condição de guarda que existe nos diagramas de atividade e de máquina de estados. Por exemplo, uma operação de sistema sobre

uma venda poderá aplicar um desconto de 10% caso o valor total da venda seja superior a 1.000 reais. O contrato seria algo como:

```
Context Livir::aplicaDesconto()
pre:
    vendaCorrente→size() = 1
post:
    vendaCorrente.valorTotal > 1000 IMPLIES
    vendaCorrente^setValorTotal(vendaCorrente.valorTotal@pre/
        1.1)
```

O modelo dinâmico deveria representar a cláusula condicional como na Figura 9.20.

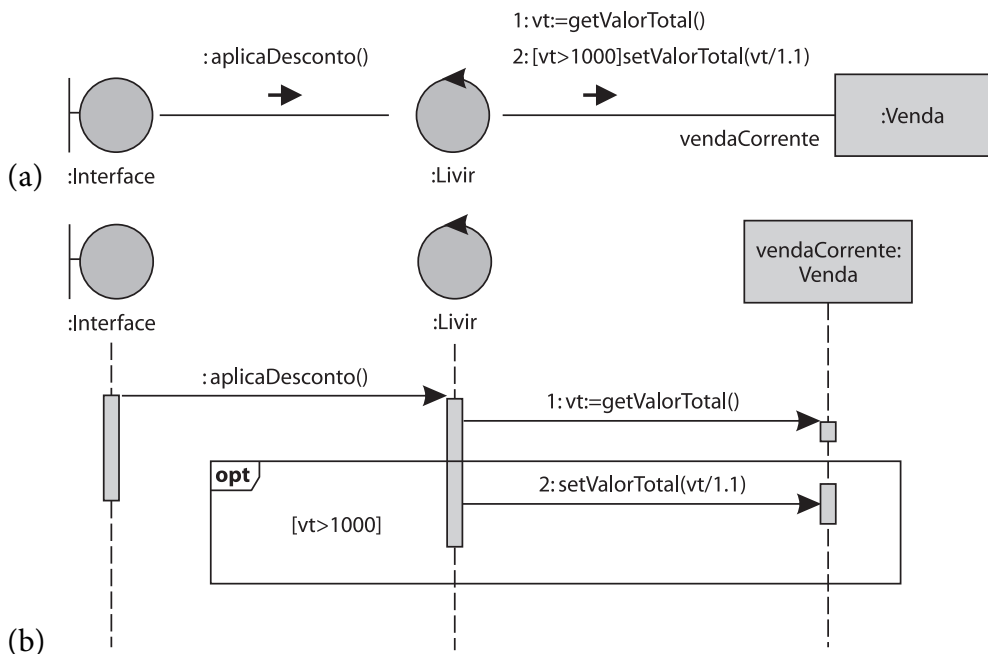


Figura 9.20: Diagrama de comunicação (a) e sequência (b) com mensagem condicional.

Observa-se que, se o valor `vt` for menor ou igual a 1.000, a operação não produz nenhum resultado.

Caso a condicional envolvesse uma estrutura do tipo *if-then-else-endif*, deveria haver uma condicional para a cláusula *then* e outra condicional negando a primeira para a cláusula *else*.

No caso de estruturas *CASE*, deveria haver uma mensagem condicional para cada um dos casos definidos.

9.3.7. Pós-condições sobre Coleções

Quando operações de sistema têm contratos com pós-condições que especificam alterações em coleções de objetos, pode-se usar a estrutura de repetição “*” para indicar que uma mensagem é enviada a todos os elementos de uma coleção. Por exemplo, a seguinte operação de sistema aumenta o preço de todos os livros em 10%:

```
Context Livir::aumentaPrecos()
post:
  livros → forall (livro |
    livro ^ setPreco (livro.preco@pre * 1.1)
  )
```

Os diagramas da Figura 9.21 foram elaborados para esse contrato.

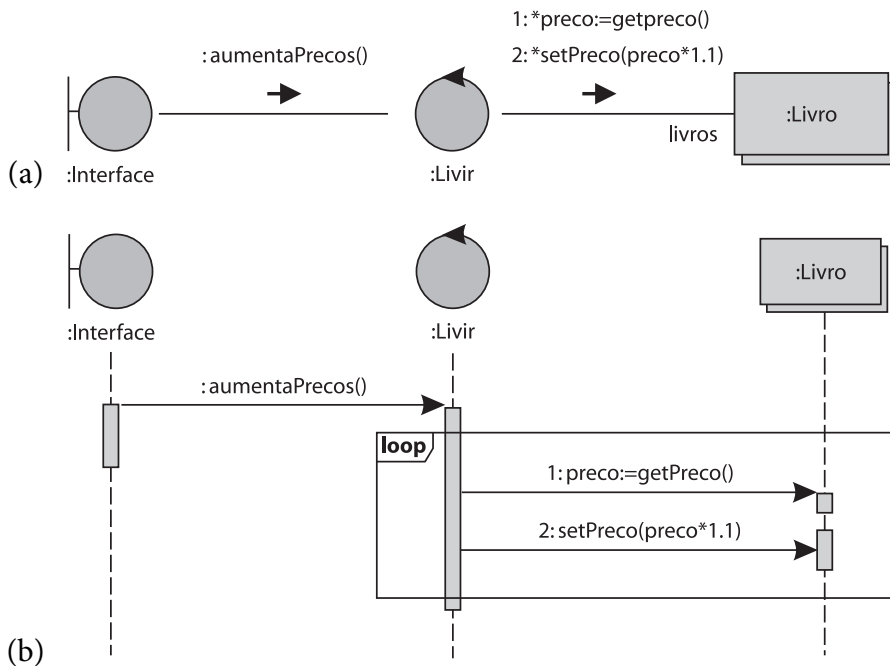


Figura 9.21: Diagrama de comunicação (a) e sequência (b) com iteratividade.

Outra situação comum ocorre quando a iteração não deve ocorrer sobre todos os elementos de uma coleção, mas apenas sobre aqueles que satisfazem um determinado critério. O contrato a seguir apresenta uma operação que majora em 10% apenas os livros cujo preço é inferior a 100 reais:

```
Context Livir::aumentaLivrosBaratos()
post:
  livros → select (preco < 100) → forAll (livro |
    livro ^ setPreco (livro.preco @ pre * 1.1)
  )
```

Os diagramas de comunicação e sequência correspondentes são mostrados na Figura 9.22.

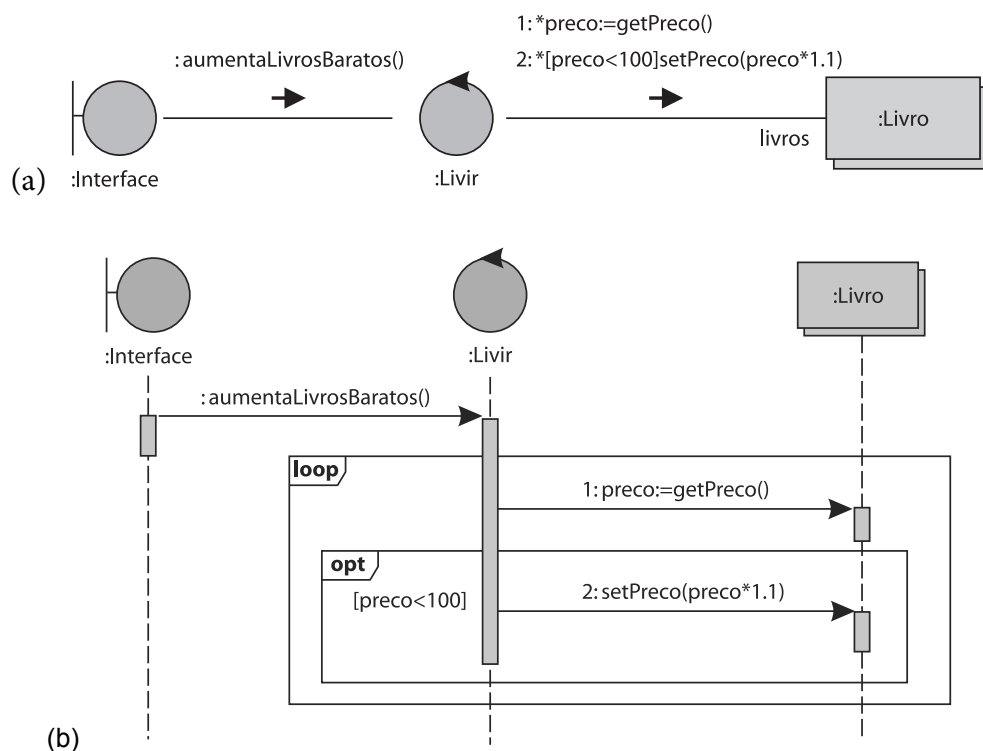


Figura 9.22: Diagrama de comunicação (a) e sequência (b) com iteração e filtro.

9.4. Consultas de Sistema

Em relação às consultas de sistema, pode-se observar no diagrama de comunicação também a presença de três tipos de mensagens:

- a) a *consulta de sistema*, que é única em cada diagrama, vem do diagrama de sequência de sistema e inicia as chamadas, sendo a raiz da árvore de mensagens. Ela também é a responsável por retornar o resultado, que deve ser uma estrutura *DTO* ou um alfanumérico;
- b) as *consultas básicas*, que são as folhas do diagrama e correspondem às operações *get* básicas sobre atributos ou papéis de associação. Elas correspondem às folhas da árvore de mensagens e não precisam ser mais detalhadas no diagrama;
- c) as *consultas intermediárias*, que correspondem aos atributos derivados quando seu retorno é um valor alfanumérico ou a consultas intermediárias propriamente ditas quando o retorno é uma estrutura de dados como uma lista, conjunto ou tupla.

A Figura 9.23 apresenta um diagrama para o contrato de consulta de sistema CRUD para consultar Livro, conforme apresentado no capítulo anterior.

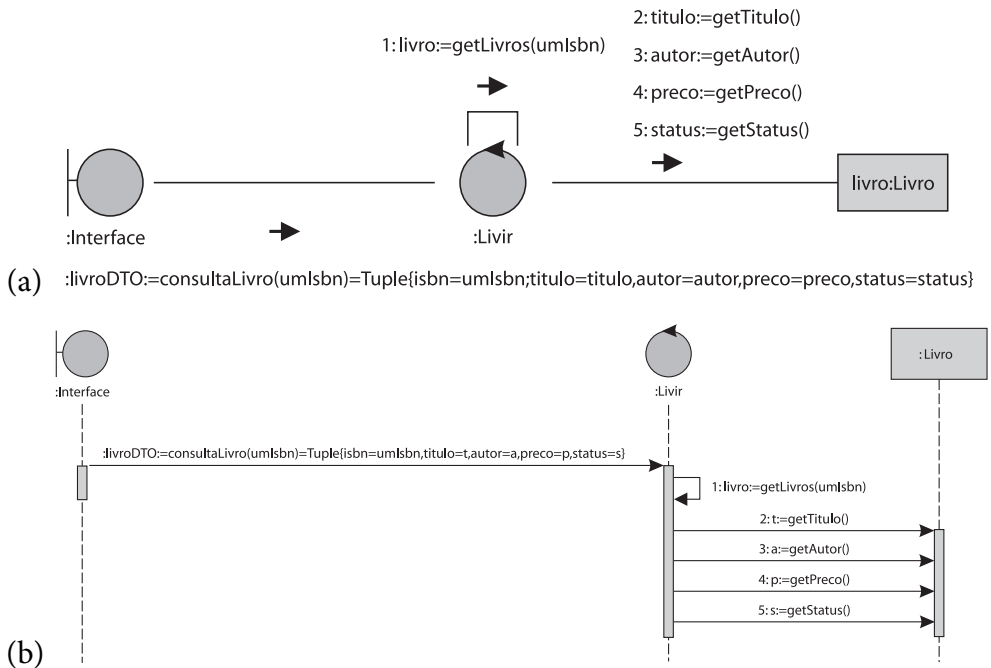


Figura 9.23: Diagrama de comunicação (a) e sequência (b) para consulta de sistema *CRUD-retrieve*.

Porém, nem o diagrama de sequência, nem o diagrama de comunicação possuem estruturas adequadas para representar combinações complexas de valores como funções matemática, filtros etc., o que faz com que não sejam

boas ferramentas para representar tais contratos. No exemplo anterior, o acesso a cada um dos atributos do livro é indicado por uma mensagem individual, e elas são combinadas na chamada da consulta de sistema após o sinal de “=”, onde se especifica como a tupla que retorna da consulta de sistema é formada.

Em outras palavras, o trabalho que se tem para criar diagramas de consulta de sistema usualmente não compensa o que se possa aprender de novo com eles. Nesses casos, recomenda-se o uso de algoritmos ou programação direta a partir da especificação do contrato da consulta, tendo sempre em vista o seguinte:

- a) sempre que uma consulta intermediária retornar um valor alfanumérico simples, verificar se não faria sentido transformar esse valor em um atributo derivado da classe com grande potencial de reusabilidade;
- b) sempre que possível, criar novos padrões de consulta que permitam agrupar várias consultas básicas em estruturas de mais alto nível, como, por exemplo, uma consulta que já aplique um filtro nos resultados ou que transforme os atributos de uma classe diretamente em uma tupla.

9.4.1. Padrões de Consulta com Filtro

Nem sempre o que se deseja é uma simples consulta que retorne todas as instâncias de uma classe ou todas as instâncias associadas a um determinado objeto. Muitas vezes, aplicam-se filtros nas consultas, desejando, por exemplo, apenas as venda do mês de janeiro ou apenas os clientes que moram em apartamento etc.

Existem pelo menos três padrões para tratar essa diversidade de consultas com filtros:

- a) implementar na classe que detém a responsabilidade uma única consulta que retorne todos os elementos da associação e deixar que o objeto que solicitou tais elementos faça a filtragem;
- b) implementar uma consulta específica para cada filtro possível;
- c) implementar uma consulta genérica que tenha como parâmetro um objeto filtro.

Nos casos *a* e *c*, implementa-se uma única consulta, o que deixa a classe que detém a responsabilidade mais simples, mas, por outro lado, gera mais código nas classes que solicitam a informação.

No caso *b*, implementam-se mais métodos na classe que detém a responsabilidade, mas as classes que solicitam a informação farão chamadas mais simples, recebendo a informação pronta.

A escolha entre um padrão ou outro deve ser tomada pelo projetista em função da quantidade de possíveis filtros e da quantidade de possíveis chamadas. Se existem poucas possibilidades de filtros e muitas chamadas, a melhor escolha é o padrão *b*. Se a quantidade de filtros é grande e há poucas chamadas para cada um individualmente, deve-se escolher entre os padrões *a* e *c*. O padrão *a* é mais direto, mas gera muito trabalho na fase de programação: cada vez que um mesmo filtro for usado, o código deve ser repetido na classe que chama a consulta. Já o padrão *c* requer que se conheça o funcionamento da classe Filtro, mas depois de dominado tende a ser mais simples do que o padrão *a*.

Um *objeto filtro* é um parâmetro que é passado ao método de consulta. Esse objeto deve conter atributos e associações para outros objetos, de forma que o método de consulta faça uma verificação e só retorne as instâncias que correspondem à definição dada pelo objeto filtro.

Por exemplo, no modelo da Figura 9.24, deseja-se fazer uma consulta para retornar os livros cadastrados no sistema Livir.

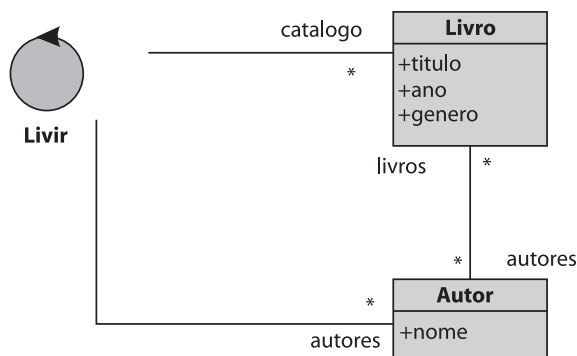


Figura 9.24: Modelo de referência para exemplos de consultas.

Os filtros possíveis são:

- pelo autor;
- pelo ano de publicação (inicial e final);
- pelo gênero.

Aplicando o padrão *a*, implementa-se na classe Livir (que detém a responsabilidade porque tem visibilidade para todo o conjunto de livros) uma

única consulta básica `getCatalogo():SET[Livro]` , que retorna o conjunto de todos os livros da livraria. Para implementar consultas pelos filtros, deve-se chamar `getCatalogo()` e aplicar o filtro ao resultado.

Aplicando o padrão *b*, deve-se implementar, na classe `Livir`, três consultas que podem ser consideradas variantes da consulta básica:

- a) `getCatalogoPorAutor(umAutor: Autor): SET[Livro]`
- b) `getCatalogoPorAno(umIntervalo: Intervalo): SET[Livro]`
- c) `getCatalogoPorGenero(umGenero: String): SET[Livro]`

Cada uma das consultas retorna apenas os elementos que satisfazem os parâmetros passados.

O padrão *c* exige a definição de uma classe para representar o objeto filtro. No caso das consultas derivadas de `getCatalogo`, será necessário definir uma classe `FiltroLivros` segundo a definição da Figura 9.24.

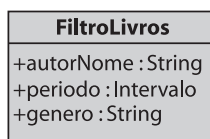


Figura 9.25: Uma classe `FiltroLivros` para consultas de catálogo.

Ainda seria possível implementar essa classe com uma associação à classe `Autor` para evitar o uso do atributo `autorNome`. Mas, nesse caso especificamente, pode ser interessante manter a classe `FiltroLivro` (que não é conceitual) independente das classes conceituais.

O método de consulta `getCatalogo(umFiltro:FiltroLivro):SET[Livro]` pode então ser implementado assim:

CLASSE `Livir`

VAR PRIVADA `catalogo : SET[Livro]`

MÉTODO `getCatalogo(umFiltro:FiltroLivro):SET[Livro]`

VAR `resultado:SET[Livro]`

PARA CADA `livro` EM `catalogo` FAÇA

SE `umFiltro.getNomeAutor = livro.getAutor().getNome`

OU `umFiltro.getNomeAutor().isNull()` ENTÃO

SE `umFiltro.getIntervalo().contains(livro.getAno())`

OU `umFiltro.getIntervalo().isNull()` ENTÃO


```
SE umFiltro.getGenero() = livro.getGenero()
OU umFiltro.getGenero().isNull() ENTÃO
    resultado.add(livro)
FIM SE
FIM SE
FIM SE
FIM MÉTODO
FIM CLASSE
```

Então, uma consulta, para retornar os livros entre 2004 e 2006 do autor “raul”, deveria ser invocada com a passagem de um objeto filtro devidamente instanciado. Como não se especifica a editora, serão retornados resultados de todas as editoras:

```
...
meuFiltro := FiltroLivro.newInstance()
meuFiltro.setAutorNome("raul")
meuFiltro.setPeriodo(Intervalo[2004..2006])
livros := self.getCatalogo(meuFiltro)
...
```

9.5. Delegação e Acoplamento Baixo

Até aqui, foram vistas algumas técnicas para construção de diagramas de comunicação para realizar contratos. Mas, em várias situações, há mais de uma opção de projeto e mais de um meio de delegar mensagens. A partir de agora, será aprofundada a discussão sobre os padrões de projeto que devem ser usados para produzir um projeto elegante.

Como visto anteriormente, muitas vezes acontece que o objeto que detém o fluxo de controle de execução não tem visibilidade para o objeto com uma responsabilidade a executar (por exemplo, alterar o valor de um atributo). Nesse caso, pode-se identificar duas abordagens diametralmente opostas para o projeto:

- a) o objeto que detém o fluxo de informação procura obter uma referência (local) para o objeto que poderia executar a ação e comunica-se diretamente com ele;
- b) o objeto delega o envio da mensagem a outro que tenha contato mais direto com o objeto que poderia executar a ação.

Pode-se parodiar essas abordagens da seguinte forma: imagine que João seja chefe de Pedro. João quer contratar um *buffet* para a festa do escritório, mas não conhece nenhuma empresa que faça tal serviço. De alguma maneira, João fica sabendo que Pedro tem contato com uma empresa de *buffet*. Então:

- a) na primeira abordagem, João solicita a Pedro que lhe dê o telefone da empresa e, depois, João faz a encomenda direto à empresa. A única coisa que Pedro faz é passar o contato a João;
- b) na segunda abordagem, João passa os parâmetros a Pedro (quantas pessoas, que tipo de comida, até quanto pode gastar etc.) e pede a Pedro que contrate o *buffet*. A festa acontece e João nunca fica sabendo qual é o telefone do *buffet*. Ele delegou tarefa a Pedro.

Embora na vida real possa ser interessante que as pessoas tenham muitos contatos e relacionamentos, tal não ocorre em sistemas orientados a objetos. Quanto mais conectados os objetos estiverem, mais complexos serão os sistemas. Então, é necessário evitar ao máximo criar conexões entre objetos que não estejam já conectados pelo modelo conceitual.

A primeira abordagem chama-se *concentração* e faz com que o objeto que detém o fluxo de informação procure obter acesso a todos os objetos necessários e comande ele mesmo as atividades. A segunda abordagem chama-se *delegação*, e faz com que o objeto procure delegar as responsabilidades distribuindo-as entre vários outros objetos, quando necessário. A segunda abordagem normalmente é preferível, pois provê maior potencial de reuso, criando métodos intermediários ou delegados nas classes.

Um exemplo concreto disso é mostrado a partir do modelo conceitual da Figura 9.26 (onde a quantidade de atributos foi reduzida ao mínimo necessário para o exemplo). O contrato de consulta de sistema a seguir tem como objetivo obter o valor total da venda corrente. Esse valor total deve ser calculado a partir do valor e quantidade de cada um dos itens da venda:

```
Context Livir::getTotalVendaCorrente():Moeda
pre:
```

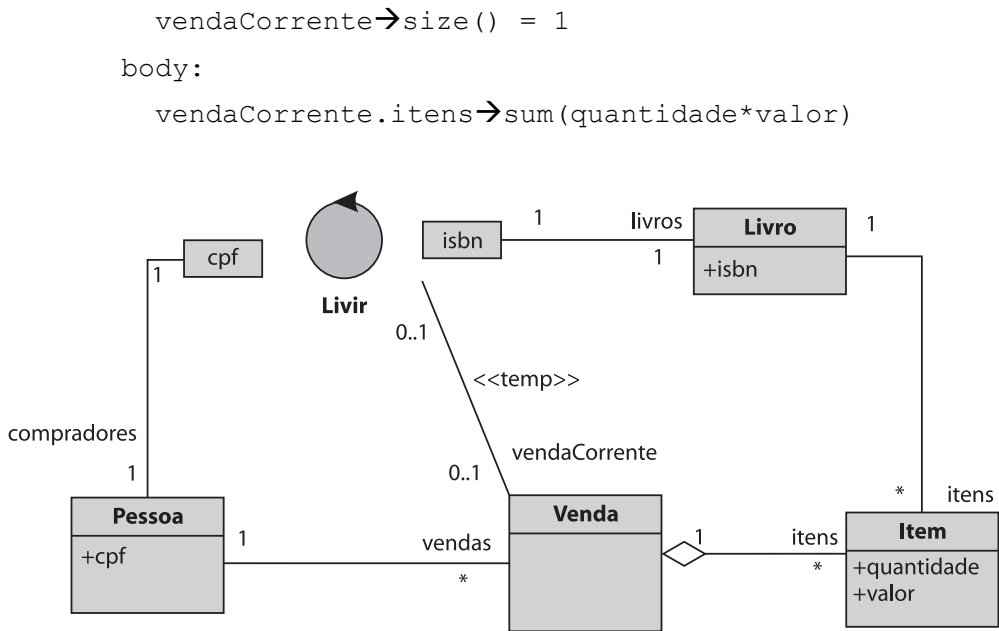


Figura 9.26: Modelo conceitual de referência.

Com a abordagem de concentração, o algoritmo será implementado na classe *Livir* (aliás, seguindo essa abordagem, *todos* os algoritmos serão implementados nessa classe). *Livir* tentará obter todos os dados necessários para realizar o cálculo e retornará o resultado. O pseudocódigo poderá ser descrito assim:

```

CLASSE Livir
VAR compradores : MAP[String->Pessoa]
VAR livros : MAP[String->Livro]
VAR vendaCorrente : Venda
MÉTODO getTotalVendaCorrente() : Moeda
    VAR itens : SET[Item]
    VAR total : Moeda
    itens := vendaCorrente.getItems()
    total := 0
    PARA CADA item EM itens FAÇA
        total := total + (item.getValor() * item.getQuantidade())
    FIM PARA
    RETORNA total

```

FIM METODO

FIM CLASSE

O que se pode observar com essa abordagem é que:

- a) ela resolve o problema e está correta;
- b) ela não produz nenhum elemento de software reusável, a não ser a consulta de sistema em si.

A abordagem de delegação pode resolver o problema e, ao mesmo tempo, criar estruturas reusáveis e menos acopladas do que no caso anterior.

Em primeiro lugar, deve-se impedir que a classe *Livir* tenha acesso a quaisquer instâncias de *Item*, uma vez que não existe conexão entre essas classes no modelo conceitual. Para fazer isso, *Livir* vai ter de delegar para a classe *Venda* o cálculo do resultado. Como se está tratando de consultas e o valor de retorno é escalar, isso equivale a criar um atributo derivado na classe *Venda* com a seguinte definição OCL:

```
Context Venda::valorTotal:Moeda
  derive:
    itens→sum(quantidade*valor)
```

Além da criação de um atributo derivado na venda, que permite calcular o seu valor total independentemente da operação de sistema que se ocupa disso, poderia ser criado um atributo derivado na classe *Item* definindo o subtotal, conforme a seguir:

```
Context Item::subtotal:Moeda
  derive:
    quantidade*valor
```

Assim, o atributo *valorTotal* de *Venda* passa a ser definido como:

```
Context Venda::valorTotal:Moeda
  derive:
    itens→sum(subtotal)
```

Agora, o pseudocódigo poderia ser definido com responsabilidades distribuídas entre as classes:

```
CLASSE Livir
VAR compradores : MAP[String->Pessoa]
VAR livros : MAP[String->Livro]
```

```
VAR vendaCorrente : Venda
MÉTODO getTotalVendaCorrente() : Moeda
    RETORNA vendaCorrente.getValorTotal()
FIM MÉTODO
FIM CLASSE

CLASSE Venda
VAR itens : SET[Item]
MÉTODO getValorTotal() : Moeda
    VAR total : Moeda
    total := 0
    PARA CADA item EM itens FAÇA
        total := total + item.getSubtotal()
    FIM PARA
    RETORNA total
FIM MÉTODO
FIM CLASSE

CLASSE Item
VAR quantidade : Número
VAR valor : Moeda
MÉTODO getSubtotal () : Moeda
    RETORNA quantidade * valor
FIM MÉTODO
FIM CLASSE
```

Agora, há duas estruturas altamente reusáveis: o subtotal da classe Item e o total da classe Venda, que não existiriam com a estratégia concentradora.

No caso de operações de sistema, o padrão de acoplamento baixo também se realiza quando o projetista opta por delegar em vez de retornar o objeto. Considere o modelo conceitual parcial da Figura 9.27a. O diagrama de comunicação da Figura 9.27b mostra a execução da operação mudaData(umaData) da forma concentradora. Já a Figura 9.27c mostra a mesma operação sendo

será determinada a direção de navegação das associações em função da direção das mensagens nos diagramas de comunicação;

- c) *possível detalhamento dos atributos e associações*. É possível que, na atividade de análise, nem todos os atributos tenham seus tipos definidos. Nesse caso, esses elementos poderão ser adicionados na atividade de projeto, na medida do necessário. Além disso, os tipos abstratos de dados definidos nos papéis de associações poderão ser substituídos por tipos concretos (por exemplo, trocar *lista* por *array* ou *lista encadeada*);
- d) *possível alteração na estrutura das classes e associações*. Pode ser necessário criar novas classes para implementar certas estruturas de projeto, como estratégias, por exemplo. Assim, é possível que a estrutura de classes do DCP não corresponda exatamente à mesma estrutura do modelo conceitual em alguns casos;
- e) *possível criação de atributos privados ou protegidos*. No modelo conceitual, todos os atributos devem ser públicos porque ali se está representando a informação disponível. Assim, não faz sentido que seja modelada alguma informação que não possa ser acessível fora da classe. Porém, quando se inicia a descrição dos aspectos dinâmicos e de representação interna dos objetos, pode ser necessário trabalhar com atributos privados ou protegidos para encapsular estados internos que determinarão o funcionamento de alguns métodos.

A rigor, o modelo conceitual poderá ser modificado durante a atividade de projeto, mas apenas quando se identificar, nessa atividade, que a modelagem original precisa ser corrigida. Caso contrário, essas modificações são feitas sobre o DCP.

Algumas informações aprendidas durante a construção dos diagramas de comunicação são imediatamente passadas ao DCP. Essas informações são de dois tipos:

- a) *métodos delegados*: sempre que um objeto receber uma mensagem delegada, a classe correspondente ao objeto deve registrar a implementação desse método (Figura 9.28);
- b) *direção das associações*: a direção das associações no DCP corresponderá à direção do envio das mensagens sobre as ligações de visibilidade baseadas em associações.

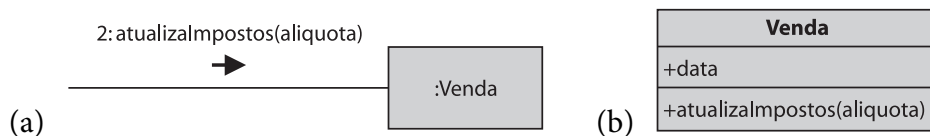


Figura 9.28: (a) Uma instância de Venda recebendo uma mensagem delegada. (b) Consequência: a classe Venda deve implementar um método para responder a essa mensagem.

Não é necessário colocar no diagrama de classes as operações básicas e consultas a atributos ou associações, visto que elas podem ser deduzidas pela própria existência das classes, associações e atributos. Basicamente, cada classe tem predefinidas as seguintes operações:

- uma operação de criação de instâncias: *create*;
- uma operação de destruição de instâncias: *destroy*;
- para cada atributo:
 - uma operação de atualização: *set*;
 - uma consulta: *get*;
- para cada associação:
 - uma operação de adição: *add*;
 - uma operação de remoção: *remove*;
 - uma consulta: *get*.

Assim, por exemplo, uma classe com seis associações e 15 atributos teria, só de operações e consultas básicas:

- uma operação de criação de instâncias;
- uma operação de destruição de instâncias;
- seis operações de adição de associação;
- seis operações de remoção de associação;
- seis operações de consulta de associação;
- quinze operações de atualização de atributo;
- quinze operações de consulta de atributo.

Assim, só de operações e consultas básicas essa classe teria 50 métodos declarados! É mais simples assumir que cada classe implementará as operações e consultas citadas para os atributos e associações por padrão e declarar no diagrama de classe apenas os métodos que não podem ser deduzidos pela existência desses elementos, ou seja, os *métodos delegados*.

Deve-se também adicionar ao DCP a *direção* das associações à medida que se verificar a necessidade de um objeto enviar mensagem a outro. As-

sim, as associações sempre terão a direção do envio das mensagens entre as instâncias das respectivas classes. Quando as mensagens trafegarem nas duas direções (mesmo que em diagramas de comunicação distintos), as associações serão bidirecionais (Figura 9.29b).

As Figuras 9.29 e 9.30 mostram como a direção das mensagens pode determinar a direção de navegação das associações no DCP.

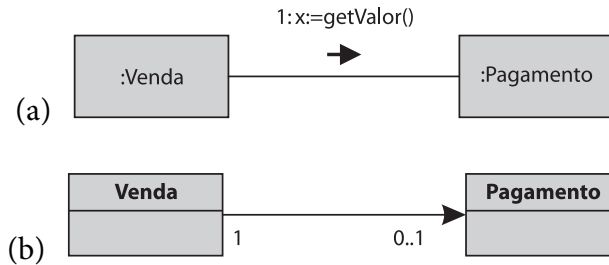


Figura 9.29: (a) Se apenas instâncias de Venda enviam mensagens a instâncias de Pagamento, então a associação entre Venda e Pagamento é unidirecional (b).

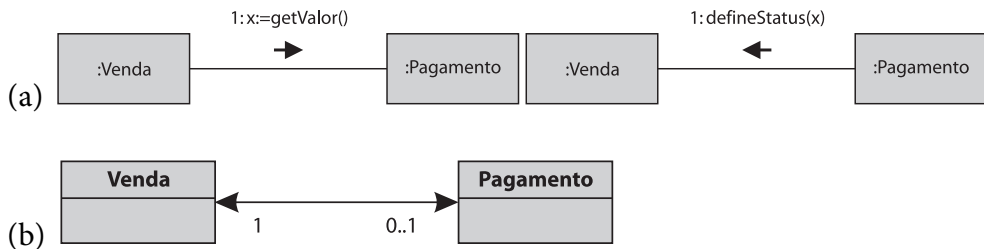


Figura 9.30: (a) Se mensagens são enviadas nas duas direções, então (b) a associação deve ser bidirecional.

A atividade de projeto lógico termina quando o DCP tem informações suficientes para implementar as classes da camada de domínio, isto é, as classes que realizam toda a lógica de transformação e apresentação de dados do sistema. Isso normalmente acontece quando todos os contratos de operação e consulta de sistema foram examinados e seus modelos dinâmicos incorporados ao projeto na forma de diagramas ou algoritmos.

Restam, ainda, os projetos tecnológicos, dentre os quais interface e persistência, que serão tratados respectivamente nos Capítulos 10 e 11, e a fase de construção, incluindo a programação ou geração de código e os testes, a serem tratados no Capítulo 12. Em especial no Capítulo 12, essas estruturas de projeto (DCP e diagramas de comunicação) serão retomadas para mostrar as regras de transformação delas em estruturas de programação.

Projeto da Camada de Interface (Web)

O projeto da camada de interface de um sistema depende de alguns artefatos da análise, especificamente dos casos de uso expandidos ou diagramas de sequência de sistema. Será necessário construir um projeto de interfaces que permitam que as operações especificadas possam ser executadas por um usuário que estiver seguindo os fluxos.

Também se deve ter em mente, ao fazer esse projeto, os requisitos não funcionais e suplementares de interface que eventualmente tenham sido levantados.

Há vários tipos de interface: Web, baseada em janelas, baseada em texto, realidade virtual etc. Tendo em vista que muitos sistemas de informação são baseados em interfaces Web, este capítulo dará ênfase à apresentação de uma linguagem de modelagem para esse tipo de interface. Essa linguagem é conhecida como WebML, e consiste em uma extensão UML para modelagem de interfaces Web (Ceri *et al.*, 2003).

O capítulo tem como objetivo apenas apresentar os conceitos fundamentais da WebML mostrando seu potencial de modelagem. Mais informações podem ser encontradas no site oficial www.webml.org. Sugere-se também o uso da ferramenta *WebRatio* (disponível gratuitamente no site para

uso não comercial), que permite a criação de modelos WebML e a geração automática das páginas Web com código executável.

10.1. WebML

WebML, ou *Web Modeling Language*, é uma linguagem de modelagem para aplicações *Web* complexas, especialmente aquelas que fazem uso intensivo de dados, como sistemas de informações acessíveis via Web.

O método de modelagem associado à linguagem WebML propõe que cinco modelos sejam construídos para definir de forma completa uma interface:

- a) *modelo estrutural*, que trata da organização dos dados. Ceri *et al.* (2003) utilizam o diagrama entidade-relacionamento para definir o modelo estrutural. Porém, o diagrama de classes da UML (modelo conceitual do Capítulo 7 ou mesmo o DCP do Capítulo 9) pode ser usado para o mesmo fim;
- b) *modelo de derivação*, originalmente proposto para a definição de dados que podem ser calculados a partir de outros. No contexto deste livro, o modelo de derivação pode ser entendido como as definições de atributos e associações derivadas, que usualmente pertencem à especificação do modelo conceitual ou DCP;
- c) *modelo de composição*, incluindo a definição de páginas Web como um agregado de *unidades* básicas (Seção 10.2) de publicação de informação e subpáginas (Seção 10.3);
- d) *modelo de navegação*, no qual são definidos os links entre as páginas e unidades de publicação (Seção 10.4);
- e) *modelo de apresentação*, no qual se define o posicionamento de unidades em páginas e sua aparência.

Este capítulo vai tratar em detalhes os modelos de composição e navegação, visto que os modelos estrutural e de derivação já foram discutidos no Capítulo 7.

Quanto ao modelo de apresentação, a ferramenta *WebRatio* prevê o uso de *XSL Style Sheets* (<http://www.w3.org/Style/XSL/>). Essas regras de apresentação são usadas pelo gerador de código para produzir páginas de acordo com o *design* possivelmente produzido por um artista gráfico ou projetista de interfaces.

10.2. Unidades

As unidades de publicação de informação ou simplesmente *units* são os elementos básicos de uma especificação WebML. As *units* podem ser diretamente associadas a classes do modelo conceitual e, por conseguinte, às suas instâncias.

Há cinco tipos de *units* em WebML:

- data units*, que gerenciam informação sobre um único objeto;
- multidata units*, que gerenciam informação sobre uma coleção de objetos;
- index units*, que listam atributos de uma coleção de objetos;
- scroller units*, que permitem a operação típica de *browse* sobre uma coleção de objetos;
- entry units*, que permitem entrada de dados.

As subseções seguintes detalham cada um desses tipos. Para cada tipo de *unit*, será apresentada uma definição textual e gráfica, bem como a aparência provável de uma renderização de página Web baseada na definição. Todos os exemplos são baseados no DCP da Figura 10.1.

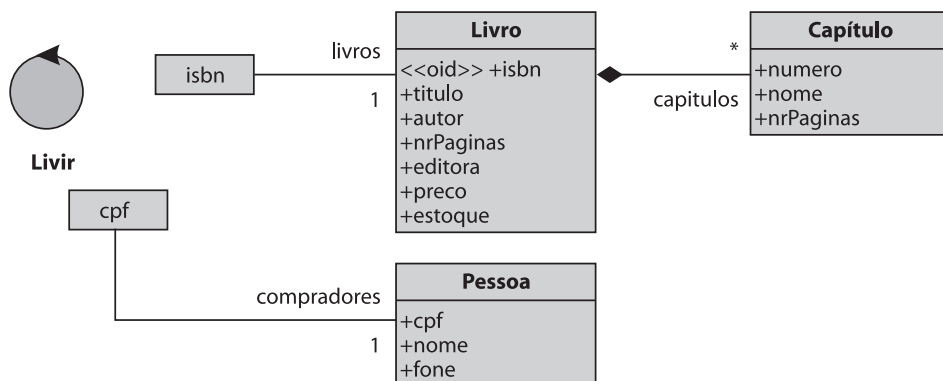


Figura 10.1: DCP de referência para os exemplos.

10.2.1. Data Units

Uma *data unit* é definida em função de quatro propriedades:

- o *nome* da unit, conforme especificado pelo projetista;
- a *fonte dos dados*, usualmente o nome de uma classe do DCP;
- um *seletor* (opcional), que define quais instâncias da classe são gerenciadas pela *unit*. O seletor deve ser especificado como uma expressão lógica (sugere-se usar OCL);

- d) o conjunto de *atributos* incluídos na *unit*. Usa-se o símbolo * caso se queira incluir todos os atributos da classe na *unit*.

Textualmente, para gerenciar uma instância de Livro cujo ISBN é dado, uma *data unit* poderia ser descrita assim:

```
DataUnit DLivro (
    source Livro;
    selector isbn = "12345";
    attributes *
)
```

A Figura 10.2a apresenta a representação gráfica WebML dessa definição. A propriedade *attributes* usualmente não aparece nessas representações gráficas.

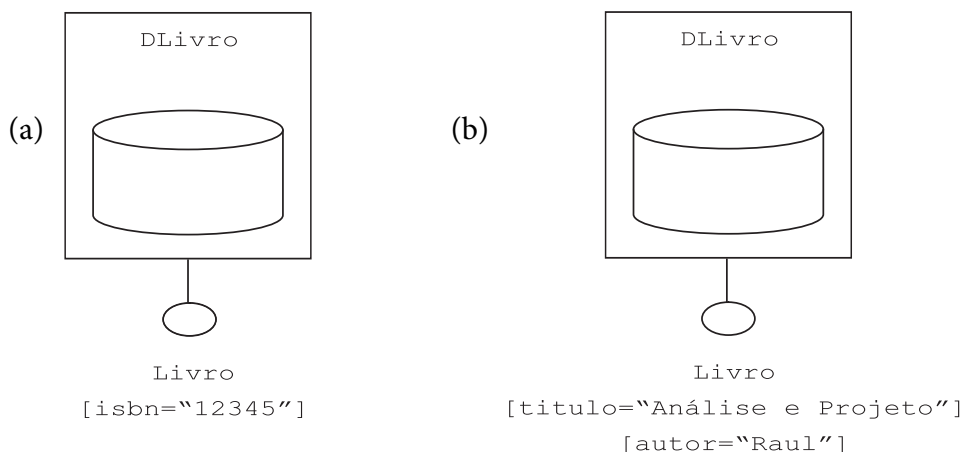


Figura 10.2: Representação gráfica de *data unit*: (a) com seletor simples e (b) com seletor composto.

Caso seja necessário usar mais de um atributo para determinar o seletor de forma conjuntiva, é possível definir o *selector* da seguinte forma:

```
selector titulo = "Análise e Projeto", autor = "Raul";
```

Nesse caso, a instância deve ter título e autor conforme definidos. A Figura 10.2b mostra como seria a representação gráfica dessa variação. A Figura 10.3 mostra uma possível renderização Web para essa definição.



isbn	12345
titulo	Análise e Projeto
autor	Raul
nrPaginas	302
editora	Campus
preco	75,00
estoque	190

Concluído

Figura 10.3: Uma possível renderização para uma *data unit* com todos os atributos da classe Livro.

Uma versão dessa unit apenas com os atributos titulo e autor deveria ter a propriedade *attributes* definida assim:

```
attributes titulo, autor;
```

A Figura 10.4 mostra uma possível renderização para essa versão da unit.



titulo	Análise e Projeto
autor	Raul

Concluído

Figura 10.4: Uma possível renderização para uma *data unit* com apenas alguns atributos selecionados da classe Livro.

A renderização final pode variar, pois é possível adicionar definições de estilos para que todas as renderizações produzam interfaces homogêneas de acordo com um projeto de *look and feel*.

Além da forma *conjuntiva* do seletor, já mostrada, seria possível usar uma forma *disjuntiva*, por exemplo, estabelecendo que o autor pode ser “Raul” ou “Rui”:

```
selector autor = “Raul” | “Rui”;
```

A Figura 10.5 mostra a forma gráfica dessa disjunção.

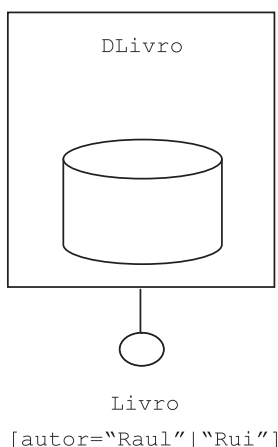


Figura 10.5: Seletor com disjunção.

10.2.2. Multidata Units

As *multidata units* gerenciam coleções de instâncias de uma única classe. Além das propriedades já apresentadas para *data units*, elas acrescentam uma nova propriedade opcional que define os atributos usados na *ordenação* das instâncias.

A definição textual de uma *multidata unit* para gerenciar instâncias de livros poderia ser feita assim:

```
MultidataUnit MLivros (
    source Livro;
    selector autor = "Raul";
    attributes isbn, titulo, autor;
    orderBy titulo;
)
```

Agora, o *selector* funciona como um filtro opcional, definindo quais instâncias devem efetivamente aparecer na *multidata unit*. A Figura 10.6 apresenta a versão gráfica para essa definição. No caso, apenas os livros cujo autor chama-se “Raul” aparecem na *multidata unit*.

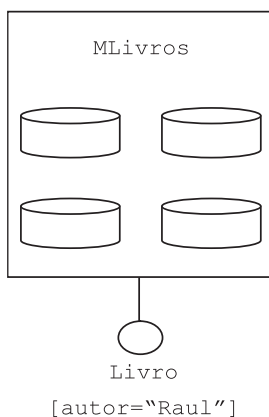


Figura 10.6: Representação gráfica de uma *multidata unit*.

Para usar mais de um parâmetro de ordenação, basta separar os atributos por vírgulas. Por exemplo:

```
orderBy sobrenome, nome, idade;
```

Nesse caso, a ordenação é feita primeiro pelo sobrenome, depois pelo nome e depois pela idade.

A Figura 10.7 apresenta uma possível renderização para a definição da Figura 10.6.



Figura 10.7: Uma possível renderização para uma *multidata unit*.

10.2.3. Index Units

As *index units* apresentam um ou mais atributos de uma classe na forma de uma lista de todas as instâncias que satisfizerem o seletor opcional. Enquanto as *multidata units* são usadas normalmente para editar vários objetos em uma mesma página, as *index units* funcionam mais como uma lista para que se selecione um ou mais objetos. As propriedades são exatamente as mesmas das *multidata units*.

Seria possível, por exemplo, definir uma *index unit* para selecionar livros a partir de seu título (a ausência de seletor significa que todas as instâncias da classe seriam listadas):

```
IndexUnit ILivros (  
    source Livro;  
    attributes titulo;  
)
```

A Figura 10.8 apresenta a versão gráfica dessa definição (a) e uma possível renderização (b).

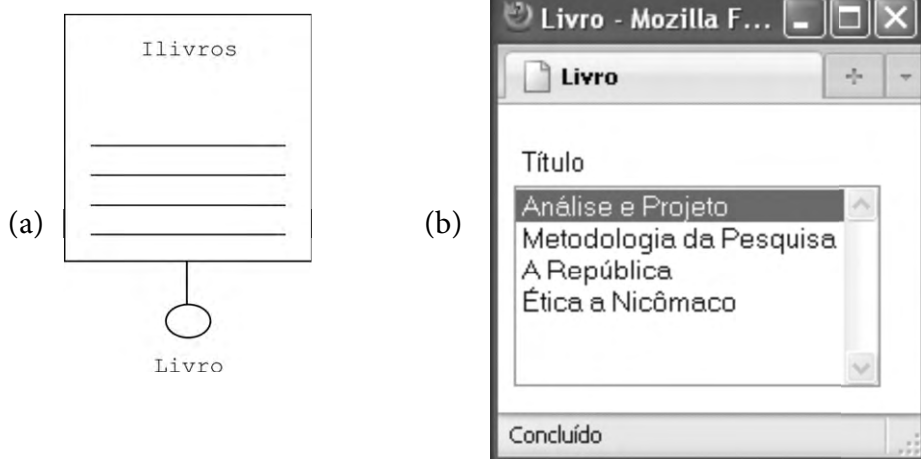


Figura 10.8: (a) Representação gráfica de uma *index unit*. (b) Possível renderização dessa *index unit*.

As *index units* ainda apresentam as seguintes variações:

- a) *multi-choice index unit* ou lista de escolha múltipla;
- b) *hierarchical index unit* ou lista hierárquica.

10.2.3.1. Multi-Choice Index Unit

A lista de escolha múltipla permite ao usuário, quando renderizada, selecionar mais de um elemento. Para definir uma lista de escolha múltipla, basta adicionar o termo multi-choice à definição da index unit, como a seguir:

```
IndexUnit MCILivros multi-choice (
    source Livro;
    attributes titulo;
)
```

A Figura 10.9a apresenta a versão gráfica dessa definição, e a Figura 10.9b, uma possível renderização dessa definição.

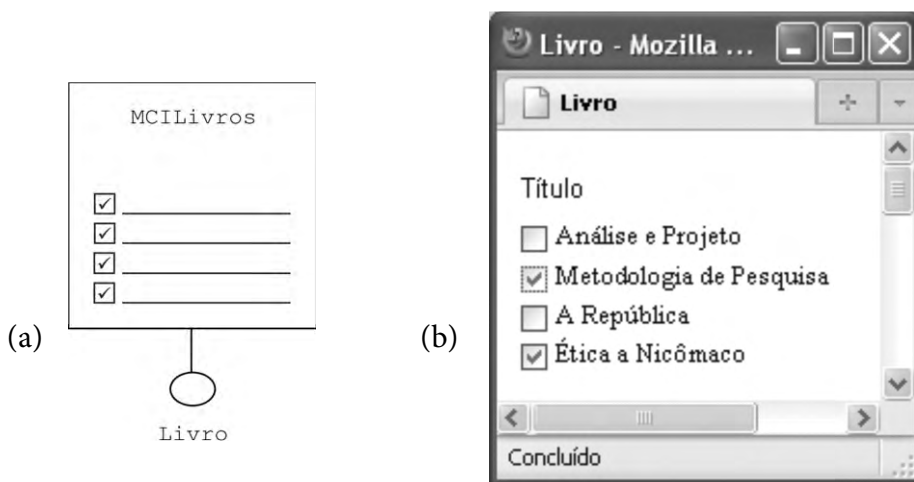


Figura 10.9: (a) Representação gráfica de uma *multi-choice index unit*. (b) Possível renderização dessa *multi-choice index unit*.

10.2.3.2. Hierarchical Index Unit

Uma *index unit* também pode ser definida como *hierárquica*, o que é bastante útil quando se tem classes que funcionam de acordo com o padrão Mestre/Detalhe, ou seja, há um conceito representando entidades como um todo e outro conceito, normalmente ligado ao primeiro por composição, representando os detalhes. Exemplos: CDs e suas músicas, livros e seus capítulos, vendas e seus itens, passagens aéreas e seus trechos etc.

Com uma *index unit* hierárquica, pode-se selecionar diretamente os conceitos que representam o todo, bem como os detalhes ou componentes.

Uma definição de *hierarchical index unit* para os conceitos Livro e Capítulo do modelo de referência poderia ser feita assim:

```
IndexUnit HILivros hierarchical (
  source Livro;
  attributes titulo
  NEST Capitulo (
    selector capitulos;
    attributes numero, nome;
    orderBy numero
  )
)
```

Nessa definição, os seguintes pontos são dignos de nota:

- o uso da palavra-chave *hierarchical*, logo após o nome da *unit*, define que se trata de uma *hierarchical index unit*;
- a expressão NEST introduz a definição do “detalhe”, ou seja, a *index unit* Capítulo subordinada ao mestre Livro;
- não se usa na *sub unit* a propriedade *source*, mas *selector* seguido do nome de papel de uma associação do mestre; no caso, de Livro para Capítulo.

A Figura 10.10 apresenta a versão gráfica e uma possível renderização para essa *unit*.

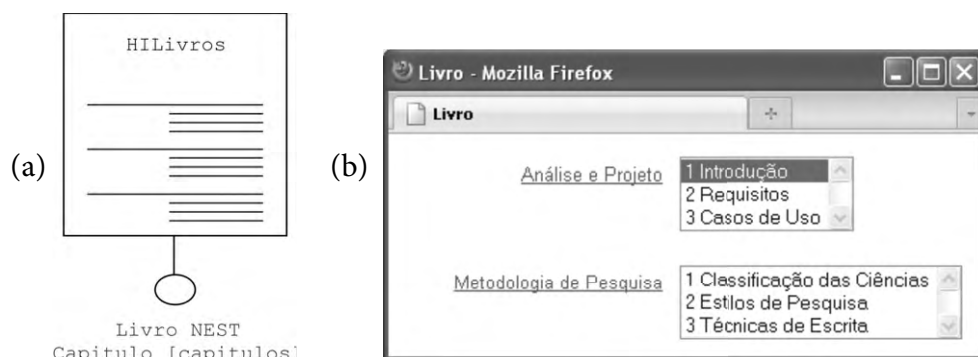


Figura 10.10: (a) Representação gráfica de uma hierarchical index unit. (b) Uma possível renderização para essa unit.

Observa-se que, na representação gráfica, em vez de um seletor composto por uma expressão booleana, como nos casos anteriores, aparece o nome de papel da associação de Livro para Capítulo. Isso significa que todos os capítulos ligados a cada livro aparecerão nos menus à direita (Figura 10.10b).

Porém, se a intenção for mostrar apenas alguns capítulos selecionados, pode-se ainda usar seletores adicionais. Por exemplo, para obter uma *hierarchical index unit* apenas com os primeiros cinco capítulos de cada livro, pode-se especificar a unit assim:

```
IndexUnit HILivros hierarchical (
  source Livro;
  attributes titulo
  NEST Capítulo (
    selector capitulos, numero <= 5;
    attributes numero, nome;
    orderBy numero
  )
)
```

10.2.3.3. Hierarchical Index Unit Recursiva

Certas situações semelhantes ao padrão Hierarquia Organizacional da Seção 7.6.5 podem ser modeladas em termos de interface como uma *hierarchical index unit* recursiva. A estrutura e a definição são semelhantes às da *hierarchical index unit*. A única diferença é que a classe mestre é a mesma classe do detalhe, o que faz com que a estrutura se torne recursiva. Por exemplo, a classe da Figura 10.11 poderia ter uma interface modelada assim:

```
IndexUnit HIUnidadesAdm hierarchical (
  source UnidadeAdm;
  attributes nome;
  RECURSIVE NEST UnidadeAdm (
    selector subunidades;
    attributes nome;
  )
)
```

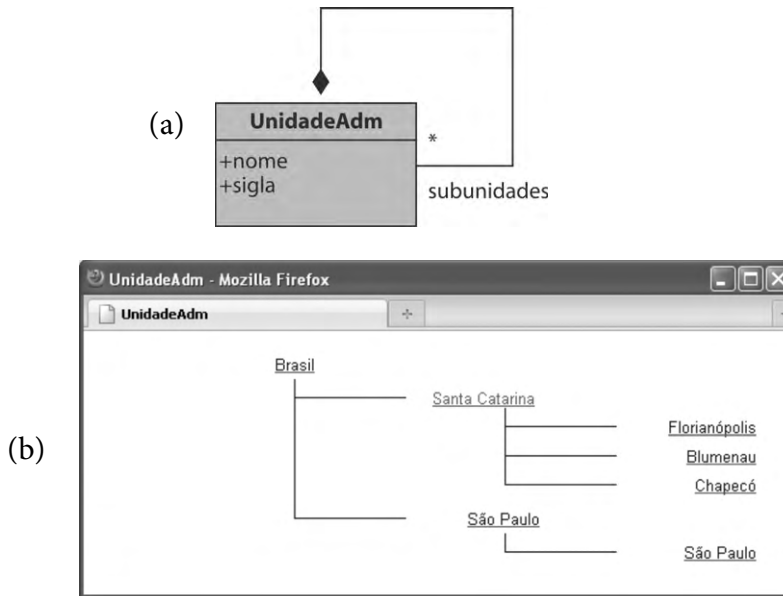


Figura 10.11: (a) Classe com associação recursiva. (b) Uma possível renderização para uma estrutura recursiva usando *hierarchical index unit*.

Não há limite preestabelecido para o número de níveis dessas estruturas.

10.2.4. Scroller Units

Scroller units são usadas em conjunto com outras *units*. Elas renderizam controles específicos para avançar para a próxima instância, retornar, avançar para o fim e retornar ao início da série, além de mostrar a posição do elemento atual em relação à série completa.

Além do nome e das propriedades *source*, *selector* e *orderBy*, já presentes nas estruturas anteriores, a *scroller unit* possui a propriedade *blockFactor*, que corresponde ao número de instâncias que são avançadas a cada vez que a operação *scroll* for realizada. O valor default do *blockFactor* é 1.

A *scroller unit* é usada *sempre* em conjunto com outra *unit*, como *data*, *multidata* ou *index*. A *unit* associada define os atributos que serão visualizados, enquanto a *scroller unit* provê o mecanismo de navegação que permite ir de uma instância a outra.

A *scroller unit* individualmente pode ser representada como na Figura 10.12 ou através de uma definição textual:

```
ScrollerUnit SLivros (
    source Livro;
    blockFactor 1;
    orderBy titulo
)
```

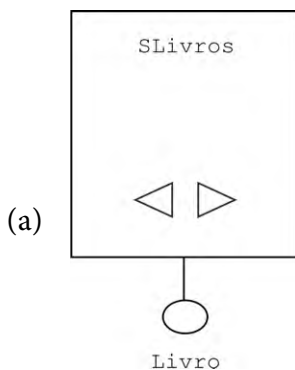


Figura 10.12: Uma *scroller unit* isolada (a) e sua possível renderização (b).

Na Figura 10.12b não aparecem os atributos do livro na renderização porque, para isso, seria necessário que a *scroller unit* estivesse associada a uma *data unit*. A Seção 10.3.1.2 vai mostrar como ficaria a junção dessas duas *units*.

10.2.5. Entry Units

A *entry unit* é usada para entrada de dados baseada em *forms*. É bastante útil para a criação de novas instâncias de objetos e também para fornecer parâmetros para pesquisas, consultas ou operações.

Uma *entry unit*, além de seu nome, tem apenas uma lista de campos. Ela não é diretamente ligada a uma *source*, como as demais *units*. Mais adiante, será explicado como usá-la em conjunto com outras *units*.

Os campos, ou *fields*, têm as seguintes propriedades:

- a) *nome*;
- b) *tipo de dados do campo* (*string*, inteiro, data, moeda etc.);
- c) *valor inicial* (opcional);
- d) *modificabilidade*, que define se o valor inicial do campo pode ser ou não modificado (o *default* é que qualquer campo seja modificável);
- e) *predicado de validade*, que consiste em uma expressão booleana que define quais valores são admitidos.

Se não for definido um predicado de validade, todos os valores entrados serão válidos. Os predicados são uma das formas de implementar a tipagem de campos em nível de interface (restrições sintáticas de parâmetros que não devem ser especificadas como condições), conforme explicado na Seção 8.1.1.

Além de expressões comparativas com os sinais <, >, = e <>, é possível utilizar a expressão *notNull* para indicar que um determinado campo não pode ser deixado de preencher.

A Figura 10.13 apresenta a versão gráfica da *entry unit* a seguir, bem como sua renderização.:

```
EntryUnit ELivro ( fields
    ISBN String notNull;
    Título String notNull;
    Autor String;
    NrPaginas Integer, nrPaginas > 10;
    Editora String;
    Preço Money;
    Estoque Integer;
)
```

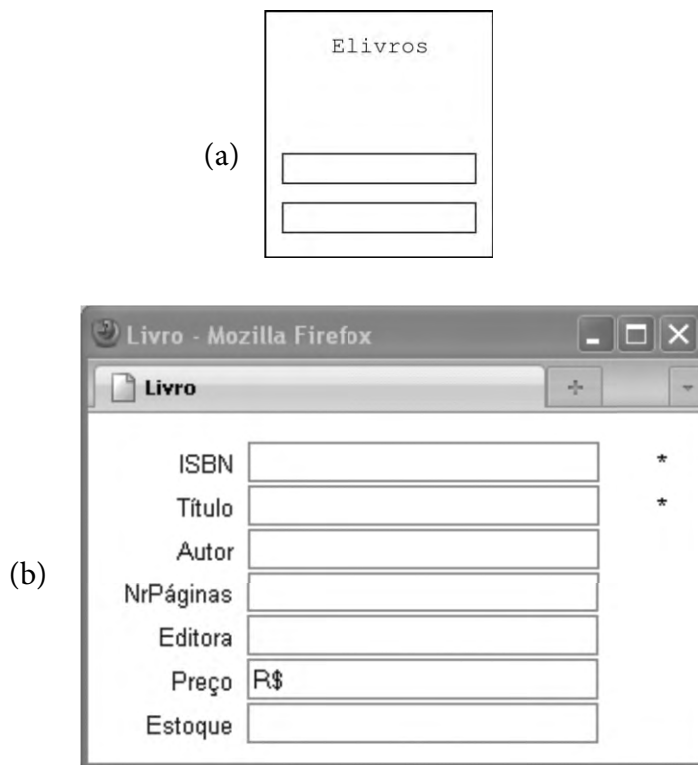


Figura 10.13: Uma *entry unit* (a) e sua possível renderização (b).

10.3. Páginas

Páginas são os elementos de interface efetivamente acessados pelo usuário. Tipicamente, uma página contém uma ou mais *units* e possivelmente subpáginas. Uma página em WebML é representada por um retângulo simples, possivelmente com um nome e opcionalmente um conjunto de *units* e subpáginas incluídas.

Por exemplo, a Figura 10.14 apresenta uma página que contém uma listagem de livros e uma listagem de pessoas.

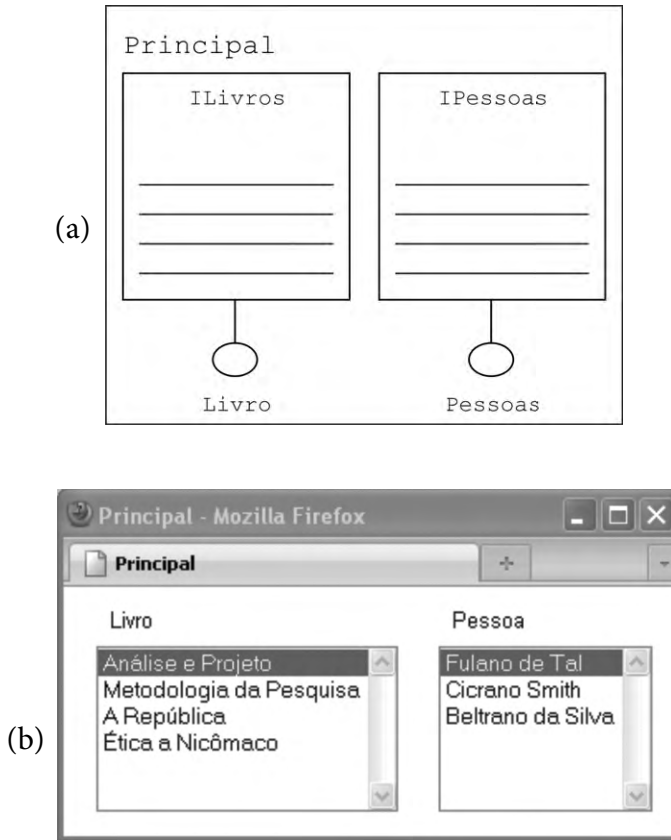


Figura 10.14: (a) Definição e uma página com duas units; (b) uma possível renderização para essa página.

As duas units que aparecem na definição de página da Figura 10.14, cuja definição textual é dada a seguir, são independentes, isto é, não há qualquer tipo de ligação semântica entre elas:

```
Page Principal (
  units ILivros, IPessoas
)
```

Para que existam dependências entre *units* (por exemplo, uma *index unit* determinar o que vai aparecer em uma *data unit*), é necessário estabelecer *links* entre as *units*.

10.3.1. Links

Um *link* é uma conexão orientada entre duas páginas ou *units*. *Links* podem ser usados não apenas para definir possibilidades de navegação entre páginas, mas também para ligar *units* entre si, definindo dependências e relações de causa e efeito. Por exemplo, selecionar um elemento em uma *index unit* pode fazer aparecer a descrição dele em uma *data unit* associada por um *link* à *index unit*.

Um *parâmetro de link* é uma especificação de uma informação que é transmitida da origem ao destino do *link*.

Algumas *units* anteriormente vistas podiam ter seletores (*selector*). Um *link selector* é um valor de seletor que faz referência a um parâmetro de *link*. Por exemplo, se uma *index unit* de lista de livros envia um ISBN para uma *data unit* de Livro, então a *data unit* terá um seletor baseado no *link*, ou *link selector*, como na Figura 10.15 ou na definição textual a seguir:

```
Link ILivrosParaDLivro (
  from ILivros to DLivro;
  parameters livroSelecionado : Livro.isbn
)
```

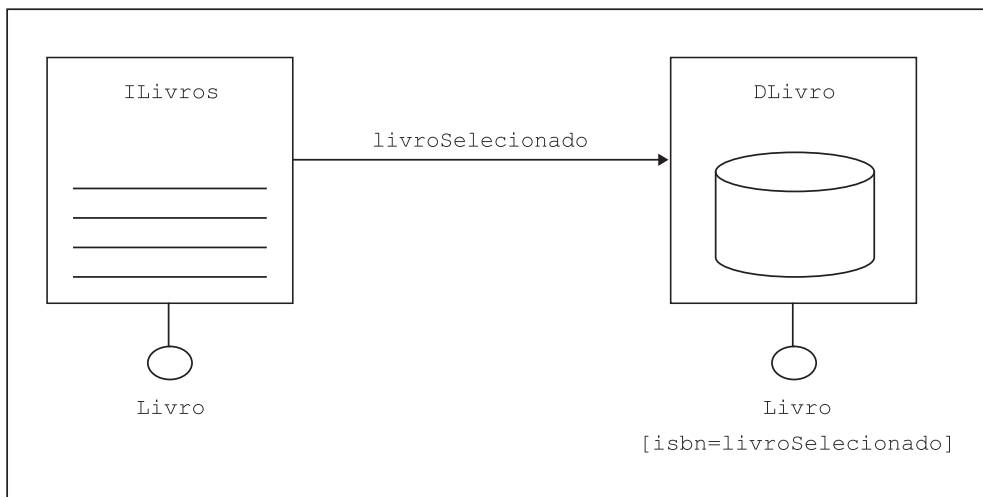


Figura 10.15: Um *link* entre duas *units*.

Observa-se, especialmente na definição textual, que um parâmetro de *link* tem duas partes:

- a) um *nome*, que é uma *string* definida ao gosto do usuário (livroSelecionado, no exemplo anterior);
- b) um *valor*, que, no exemplo anterior, corresponde a um atributo da classe fonte (Livro.isbn).

A definição da Figura 10.15 é renderizada em uma única página com dois controles: uma lista de livros conforme a definição de *ILivros* e um campo onde aparecem os atributos do livro correntemente selecionado nessa lista, de acordo com as definições de *DLivro*.

Links podem ser classificados como:

- a) *interpáginas* quando ligam duas páginas diferentes;
- b) *intrapáginas* quando ligam units dentro da mesma página, como na Figura 10.15.

Outra distinção de *links* é considerá-los contextuais ou não contextuais:

- a) *links contextuais* são aqueles que transmitem informação, na forma de parâmetros de *link*, da origem para o destino;
- b) *links não contextuais* não transmitem informação, mas apenas indicam navegação.

A Figura 10.16 apresenta uma definição de *link* não contextual interpáginas, cuja finalidade é indicar que, na página que apresenta livros, é possível navegar para outra página onde se pode verificar cadastros de pessoas.

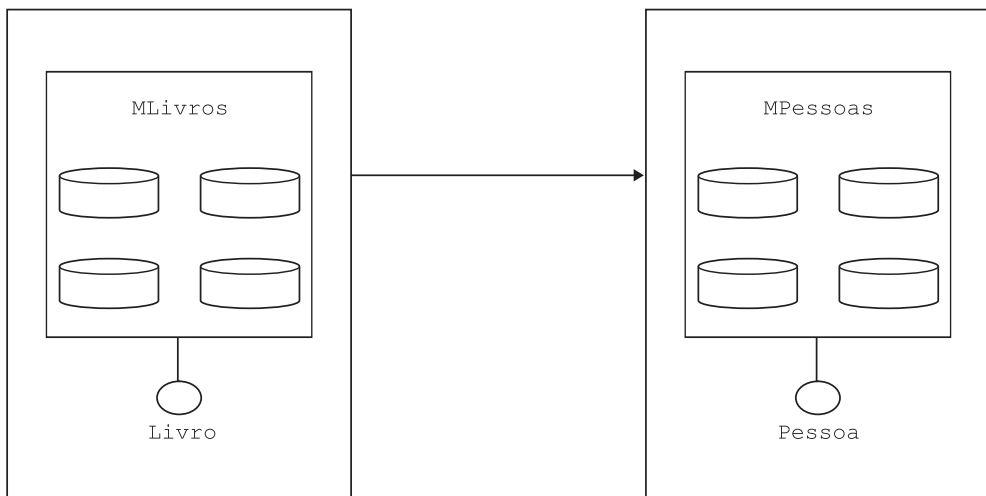


Figura 10.16: Exemplo de *link* interpáginas não contextual.

Observa-se que, na Figura 10.16, o *link* não une as *units*, mas as páginas. Portanto, o *hiperlink* de navegação será renderizado na mesma página onde os livros são vistos, mas sem estar relacionado com as informações sobre livros.

É possível definir *links* com múltiplos valores, por exemplo, uma *multi-choice index unit* associada a uma *multidata unit*. Por exemplo, pode-se querer selecionar um conjunto de livros em uma lista e depois visualizar os detalhes de todos os livros selecionados numa *multidata unit*. Nesse caso, o valor do parâmetro deve ser indicado entre chaves:

```
Link MCILivrosParaMLivros (
    from MCILivros to MLivros;
    parameters selecoes : {Livro.isbn}
)
```

A Figura 10.17 apresenta graficamente o link e as *units* originais. Nota-se que, em vez de afirmar que o ISBN é igual a um valor, o seletor afirma que o ISBN está *em* um conjunto.

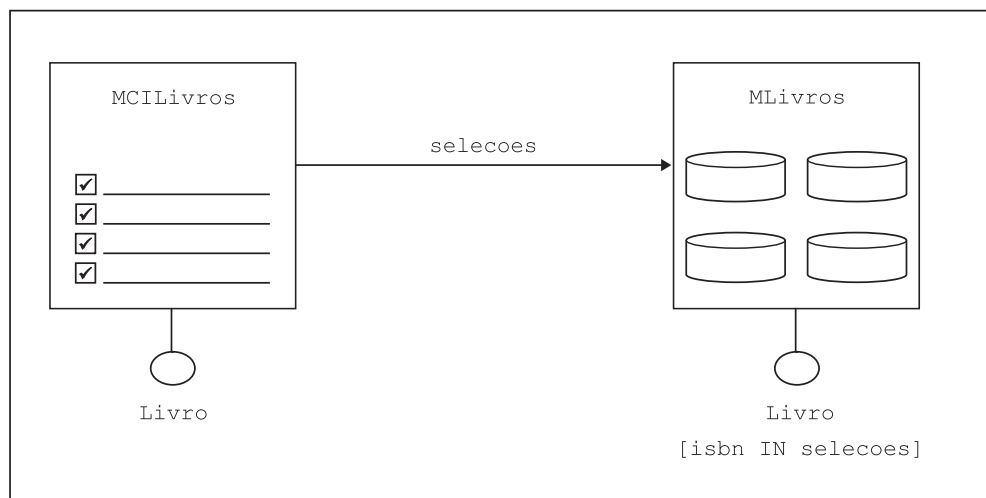


Figura 10.17: Um *link* que transfere um conjunto de dados entre duas *units*.

Um *link* contextual pode ser usado também para associar uma *entry unit* a uma *data* ou *multidata unit*, de forma que a *entry unit* funcione como um campo de pesquisa ou filtragem para os dados da *data* ou *multidata unit* (Figura 10.18):

```
EntryUnit Pesquisa ( fields
    campoPesquisa String
```

```

)
Link PesquisaParaMLivros (
  from Pesquisa to MLivros;
  parameters palavraChave : campoPesquisa
)

```

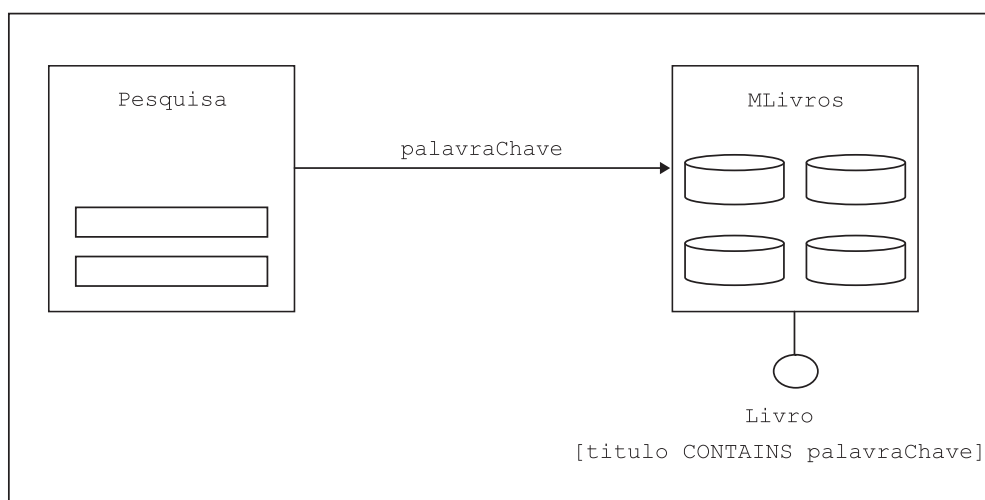


Figura 10.18: Um link contextual passando parâmetros de uma pesquisa sobre um conjunto de objetos.

10.3.1.1. Link de Transporte

Links de transporte não definem navegação, mas apenas a dependência entre duas *units*. Graficamente, são representados por setas tracejadas e, textualmente, adiciona-se o termo *transport* à definição do *link*. Uma possível aplicação seria exibir, em uma mesma janela, dados de um livro e a lista de seus capítulos (Figura 10.19):

```

Link DLivroParaICapitulos transport (
  from DLivro to ICapitulos
)

```

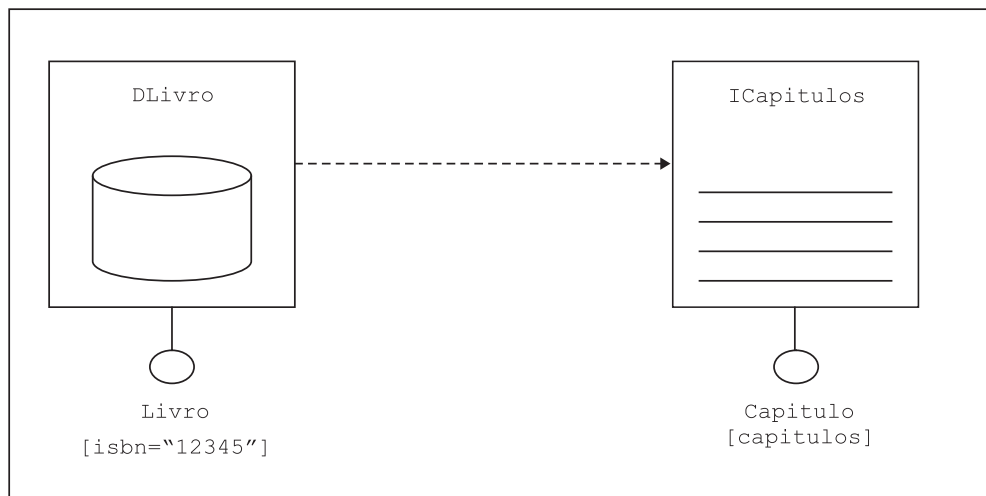


Figura 10.19: Um exemplo de *link* de transporte.

10.3.1.2. Link Automático

Um *link* automático é navegado independentemente da intervenção do usuário. Ele é definido textualmente pela adição do termo “automatic” à definição do *link* e graficamente com um quadrado com a letra “A” inscrita na seta que representa o *link*.

Uma das aplicações do *link* automático é fazer a ligação entre uma *scroller unit* e outras *units*. No exemplo da Figura 10.20, uma *scroller unit* é usada com uma *multidata unit* para disponibilizar operações de *scroll*.

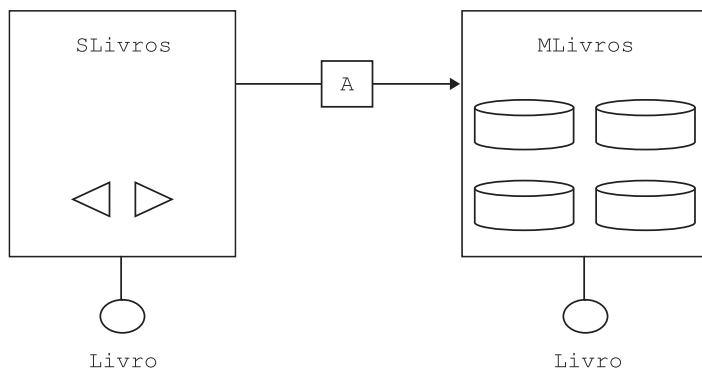


Figura 10.20: Link automático.

Seria possível definir um *blockFactor*, por exemplo, de 10 para que a operação de *scroll* apresente 10 livros de cada vez na *multidata unit*.

10.4. Organização de Hipertexto

As *units*, *links* e sua organização em páginas constituem a modelagem de interface em seus aspectos mais locais. Porém, também é necessário realizar uma modelagem mais ampla da interface, indicando grupos de páginas relacionadas, formas de navegação entre elas, regiões etc. Esse tipo de modelagem será descrito nesta seção e é referenciado como *organização do hipertexto*.

10.4.1. Visões de Site

Uma visão de *site* (*siteview*) é um pacote com várias páginas de uma mesma aplicação Web. O conceito é equivalente ao conceito de *pacote* da UML, e sua representação gráfica é a mesma (Figura 10.21).

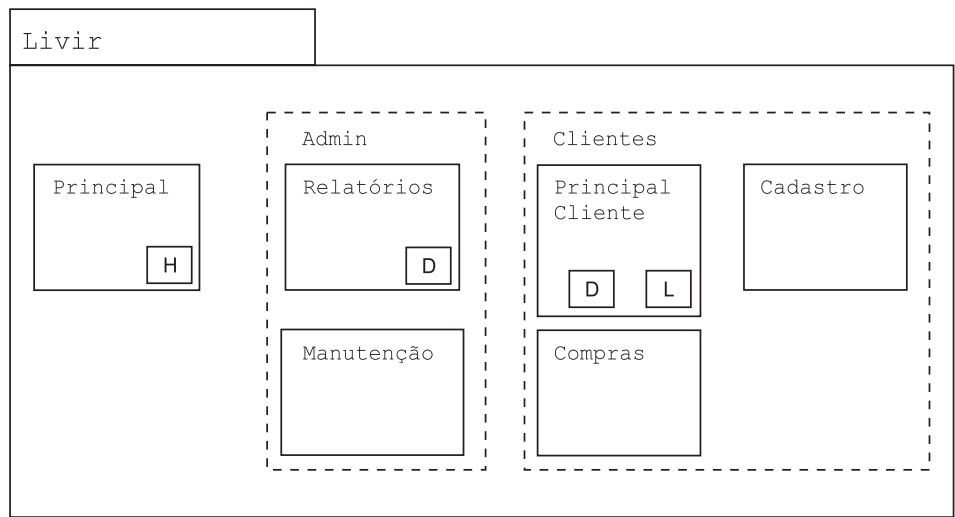


Figura 10.21: Uma visão de *site*.

A visão de *site* na Figura 10.21 é representada pelo retângulo mais externo, rotulado com *Livir*. Textualmente, um *siteview* é definido da seguinte forma:

```
Siteview Livir (...)
```

Dentro dos parênteses, serão definidas as áreas e páginas, conforme será visto a seguir.

10.4.2. Áreas

Uma visão de site pode ser organizada em áreas. *Áreas* são grupos de páginas ou recursivamente de outras áreas que têm afinidades, como, por exemplo, um conjunto de botões ou uma parte do site com anúncios diversos. Na notação gráfica (Figura 10.21), as áreas devem aparecer como grupos de páginas cercadas por um quadrado tracejado.

Áreas e páginas podem formar um *siteview*. A descrição a seguir mostra parcialmente a definição da Figura 10.21, incluindo o *siteview*, as áreas e a página principal:

```
Siteview Livir (  
  areas Admin, Clientes;  
  page Principal  
)
```

As áreas, por sua vez, são definidas assim:

```
Area Admin (  
  pages Relatórios, Manutenção  
)  
  
Area Clientes (  
  pages PrincipalCliente, Compras, Cadastro  
)
```

10.4.3. Tipos de Páginas

Na Figura 10.21, algumas páginas têm marcas especiais com os seguintes significados:

- a) H: representa a *homepage*. Na definição textual da página, adiciona-se a expressão “home”. Essa é a página inicial da aplicação;
- b) D: representa a página *default* de uma área. Na definição textual da página, adiciona-se a expressão “default”. Essa página será a página padrão para onde se navega sempre que se entrar em uma área, ou seja, é a página inicial de uma área;
- c) L: representa uma página *landmark*. Na definição textual da página, adiciona-se a expressão “landmark”. Essa página é acessível, sem necessidade de *links* explícitos, por qualquer outra página incluída da mesma área.

Uma visão também pode ser marcada com landmark, significando que a visão como um todo é acessível de todas as páginas incluídas na visão (ou *siteview*) que inclui a visão marcada.

10.5. Padrões de Interface Web

Ceri *et al.* (2003) apresentam vários padrões para publicação de conteúdo em páginas Web, alguns dos quais são resumidos aqui. Os exemplos seguintes são baseados no DCP da Figura 10.22.

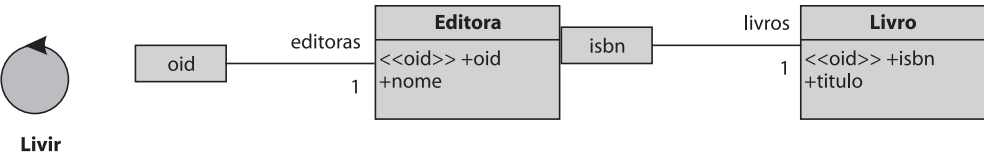


Figura 10.22: DCP de referência.

10.5.1. Índice em Cascata

O índice em cascata é uma sequência de menus baseados em *index units* até chegar a uma *data unit*. Por exemplo, pode-se inicialmente selecionar a editora e depois o título para chegar a visualizar os dados do livro. A Figura 10.23 ilustra esse padrão.

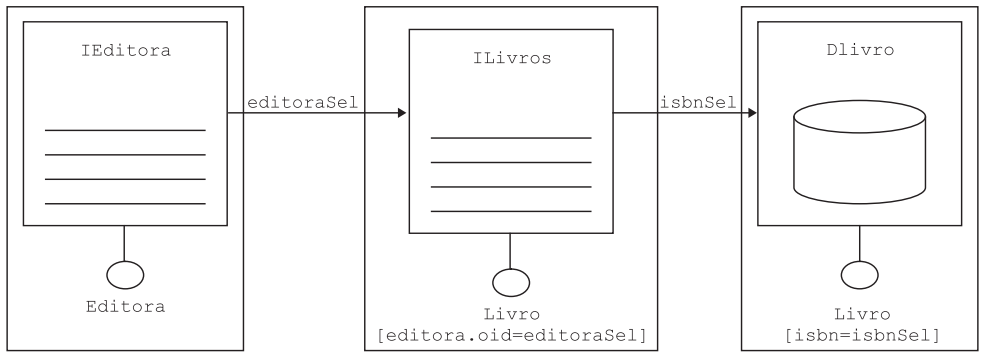


Figura 10.23: Exemplo de índice em cascata.

Uma variante desse padrão consiste em mostrar alguns dados do objeto selecionado em um dos índices intermediários. A Figura 10.24 mostra essa situação. Nela, ao selecionar uma editora, a interface mostrará detalhes da editora ao mesmo tempo em que apresenta a lista de livros para nova seleção.

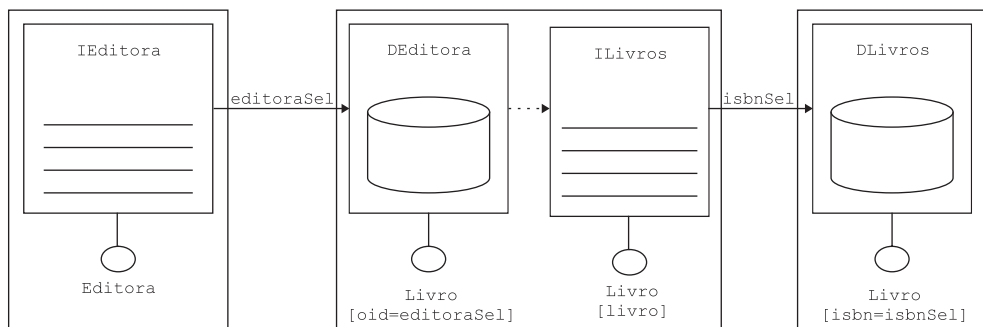


Figura 10.24: Índice em cascata com apresentação de dados intermediários.

10.5.2. Índice Filtrado

Em alguns casos, poderá ser interessante não exibir uma lista com todos os valores possíveis. Por exemplo, se a livraria possuir milhares de livros em seu catálogo, poderá ser impraticável simplesmente procurar um título nessa lista. Pode-se usar, em vez disso, uma lista já filtrada, solicitando ao usuário, por exemplo, que indique uma palavra-chave para obter uma lista de títulos reduzida.

Esse padrão se realiza pela conexão entre uma *entry unit*, uma *index unit* e uma *data unit*. A Figura 10.25 mostra um exemplo.

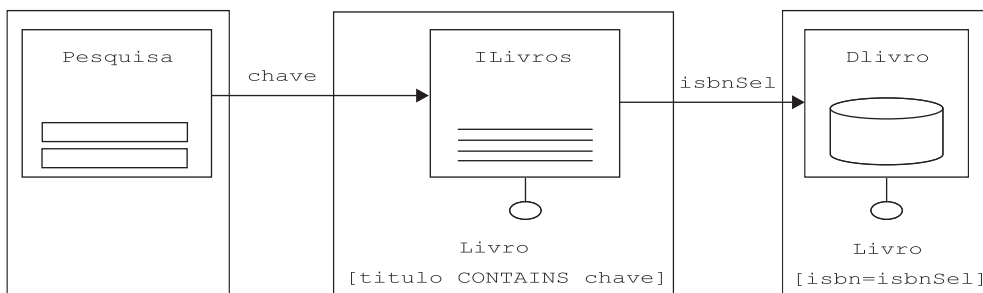


Figura 10.25: Um exemplo de índice filtrado.

Uma variação é o índice filtrado com *scroll*, típico de mecanismos de busca na Internet. Aplica-se uma chave de busca, e vários resultados são apresentados em listas de n itens. O usuário pode visualizar os próximos n itens ou os n anteriores, realizando operações de *scroll* com *blockFactor* n .

Esse padrão é implementado pela junção de uma *entry unit* com uma *scroller unit* ligada a uma *index unit* por um *link* automático. Por sua vez, a *index unit* se associa a uma *data unit* para visualização dos detalhes do elemento selecionado (Figura 10.26).

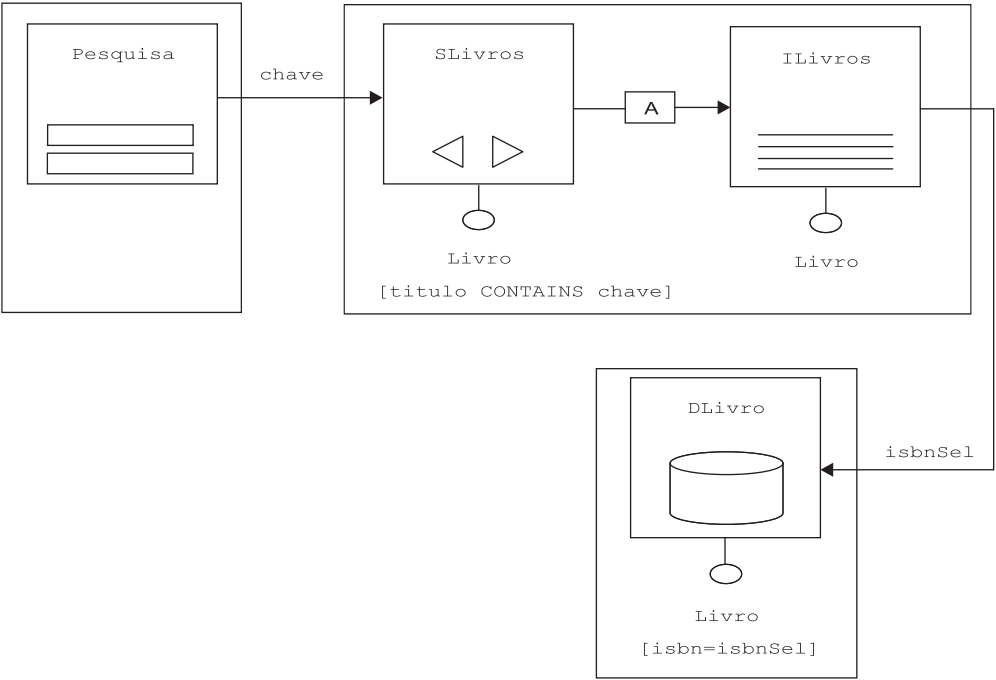


Figura 10.26: Um exemplo de índice filtrado com *scroll*.

10.5.3. Tour Guiado

Um *tour* guiado é um padrão pelo qual se visualizam detalhes de objetos um por um usando *scroll*. Usa-se uma *scroller unit* com *blockFactor* igual a 1 conectado por um *link* automático a uma *data unit*.

No exemplo da Figura 10.27, usa-se um *tour* guiado para visualizar todos os livros do autor “Raul”.

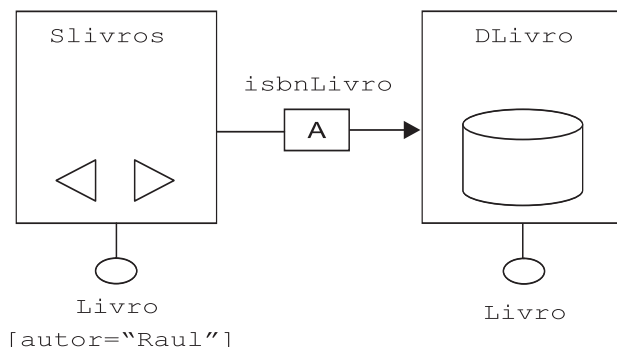


Figura 10.27: Exemplo do padrão *tour guiado*.

10.5.4. Pontos de Vista

Por vezes, pode ser interessante apresentar diferentes facetas de certos objetos. Por exemplo, apresentar dados resumidos sobre um livro, expandir esses dados visualizando dados completos e novamente visualizar os dados resumidos.

Para realizar esse padrão, basta definir duas *data units* com diferentes conjuntos de atributos de uma mesma classe e criar *links* entre as duas *units* (Figura 10.28).

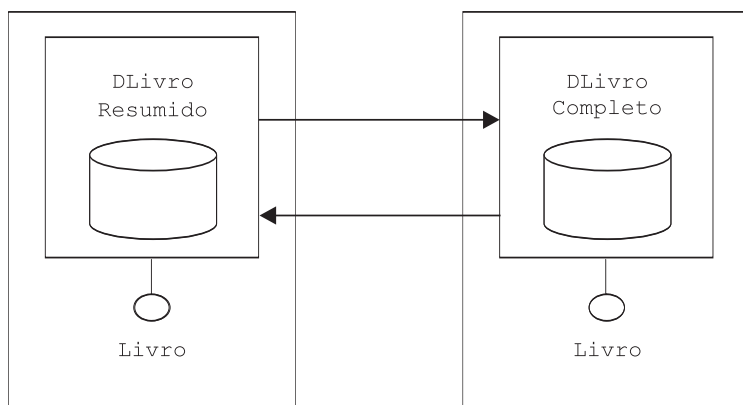


Figura 10.28: Exemplo do padrão *ponto de vista*.

10.6. Modelagem de Operações na Interface

A WebML permite ainda que se modelem operações de criação, exclusão e alteração de objetos, com o uso de *operation units*. As cinco *operation*

units básicas têm relação direta com as cinco operações básicas sobre objetos. A Figura 10.29 apresenta as cinco *operation units* de forma gráfica.

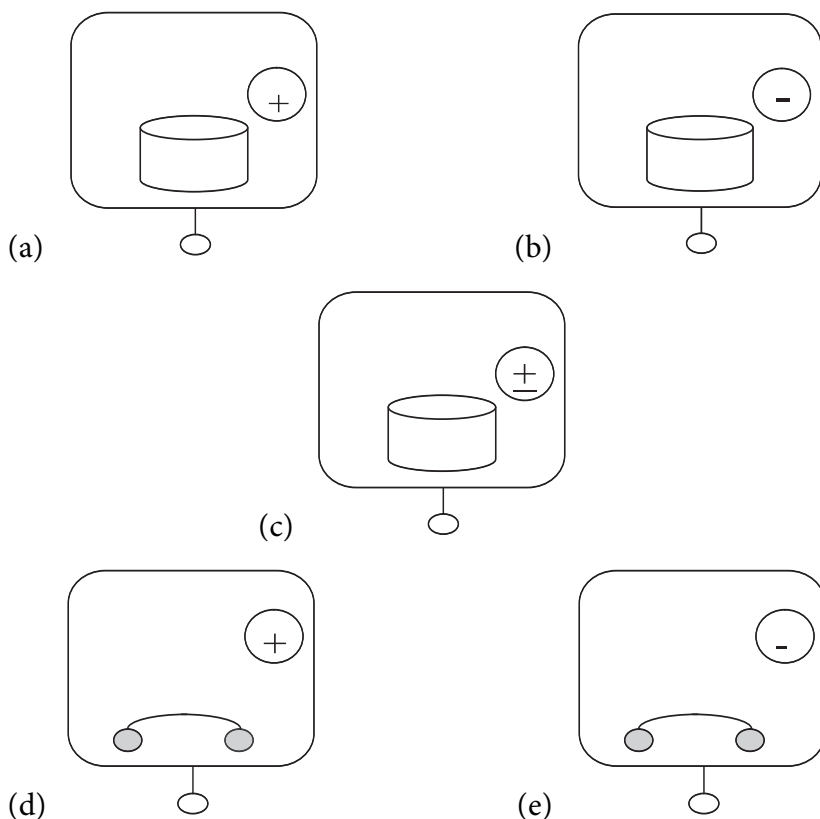


Figura 10.29: Operation units: (a) *create unit* – criação de objetos, (b) *delete unit* – exclusão de objeto, (c) *modify unit* – alteração de valor de atributo, (d) *connect unit* – adição de associação e (e) *disconnect unit* – remoção de associação.

As *operation unit* não contêm informação; elas apenas processam a informação. São representadas graficamente sempre fora das páginas.

Toda *operation unit* tem dois *links* de saída:

- link OK*: para onde vai o controle se a operação é executada com sucesso;
- link KO*: para onde vai o controle se a operação falha em executar.

As *operation units* também têm uma classe como “*source*”, sendo responsáveis pelas operações sobre instâncias dessa classe que satisfaçam o seletor definido na *operation unit*.

Uma *operation unit* que cria instâncias de Editora (Figura 10.22) poderia ser definida juntamente com uma *entry unit* e um *link*:

```
EntryUnit EEditora ( fields
    campoOid String,
    campoNome String
)
Link EEditoraParaCriaEditora (
    from EEditora to CriaEditora;
    parameters
        umOid : campoOid,
        umNome : campoNome
)
CreateUnit CriaEditora (
    source Editora;
    oid := umOid,
    nome := umNome
)
```

Se a operação ocorrer com sucesso, o controle pode seguir para uma *data unit* onde os dados da editora são visualizados:

```
DataUnit DEditora (
    source Editora;
    attributes *
)
OKLink sucesso (
    from CriaEditora to DEditora
)
```

Em caso de falha (exceção) na execução da operação, o controle deve retornar à *entry unit*:

```
KOLink falha (
    from CriaEditora to EEditora
)
```

A Figura 10.30 apresenta a representação gráfica completa dessa operação.

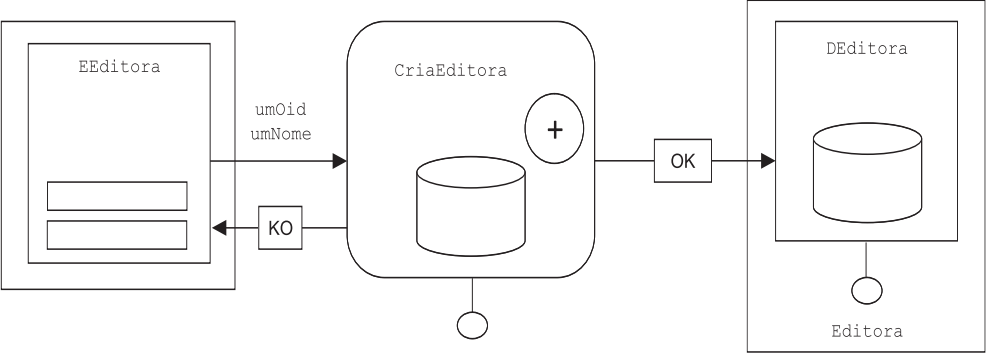


Figura 10.30: Exemplo de aplicação de *create unit*.

A *delete unit* opera sobre um ou mais objetos que satisfaçam a condição do seletor. Se um ou mais dos objetos não puder ser excluído, a *delete unit* segue o *link* de saída *KO* transportando como parâmetros os OIDs dos objetos que não foram excluídos.

Uma *modify unit* contém um seletor para um ou mais objetos e um conjunto de atribuições para alterar atributos desses objetos. Usualmente, os novos valores dos atributos são recebidos por *links* de transporte.

A *connect unit* e a *disconnect unit* têm dois seletores: um para o objeto origem e outro para o objeto destino da associação a ser criada ou destruída.

Outras operações que não se encaixam nos cinco tipos básicos podem ser representadas como operações genéricas, ou *operation units*. A Figura 10.31 apresenta o símbolo WebML para tais operações.

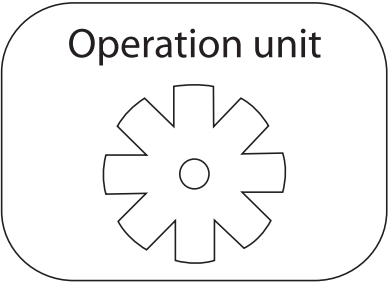


Figura 10.31: *Operation unit* genérica.

10.7. Construção de Modelos WebML a Partir de Diagramas de Sequência de Sistema

O projeto de interfaces necessita de pelo menos dois artefatos: o diagrama de sequência de sistema e o modelo conceitual.

O diagrama de sequência de sistema vai indicar o encadeamento das operações e das entradas e saídas de dados na interface. Ele também vai mencionar operações padrões como CRUD, listagem e relatórios. Conforme mencionado, tais operações padrões não precisam ser modeladas com contratos e diagramas de comunicação, pois a própria **WebML** já conta com os padrões necessários em sua definição.

Quando os contratos e diagramas de comunicação das operações padrões foram apresentadas nos capítulos anteriores, foi apenas à guisa de exemplo, para mostrar como funcionam tais operações. Mas, na prática, apenas as operações que não se encaixam nos padrões precisam ser modeladas na forma de contratos e diagramas de comunicação. Em WebML, tais operações podem ser representadas por *operation units* genéricas, conforme visto na seção anterior.

Esta seção vai mostrar alguns passos na concepção de um modelo WebML a partir de um diagrama de sequência (o diagrama original está na Figura 6.6).

Inicialmente, é necessário implementar a operação *criaCompra* que toma como parâmetro um *idComprador* informado pelo usuário (Figura 10.32).

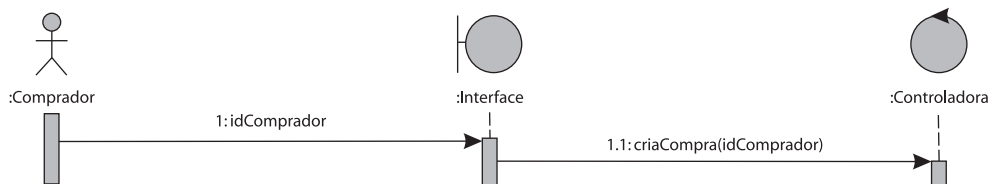


Figura 10.32: Primeiro passo do diagrama de sequência a ser modelado.

Pode ser usada uma *entry unit* para receber o *idComprador*. A operação *criaCompra* não é uma operação básica e, portanto, deve ser implementada por uma *operation unit* genérica. Um *link* contextual leva o parâmetro *idComprador* da *entry unit* para a *operation unit* (Figura 10.33). Se a operação falhar (se houver exceção), retorna-se o controle à *entry unit* (*link KO*). Caso contrário segue-se em frente (*link OK*).

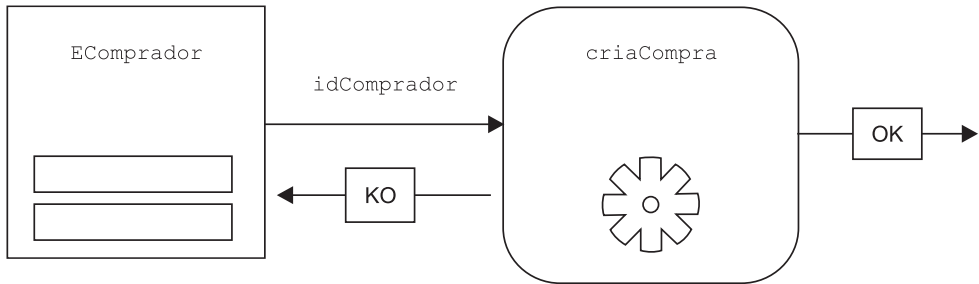


Figura 10.33: Modelo WebML parcial para o diagrama da Figura 10.32.

Continuando o exemplo, percebe-se na Figura 10.34 que o passo seguinte, caso a identificação do comprador tenha sido feita com sucesso, é apresentar a lista de livros disponíveis.

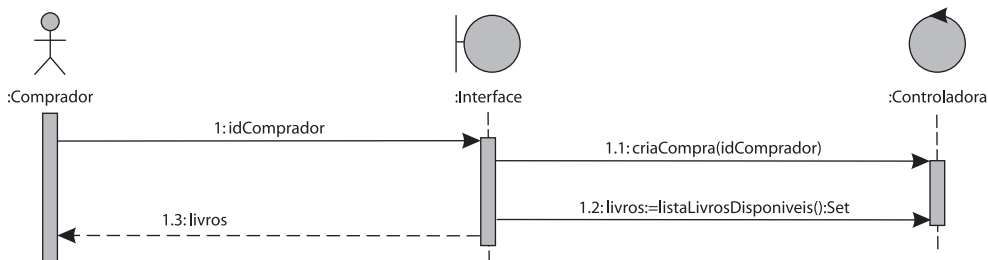


Figura 10.34: Continuação do diagrama de sequência.

Isso pode ser feito através de uma *index unit* seguindo o padrão “Listagem com Filtro”. São apresentados os principais dados dos livros disponíveis (Figura 10.35). Como o usuário poderá escolher mais de um livro, deve-se usar uma *multi-choice index unit*.

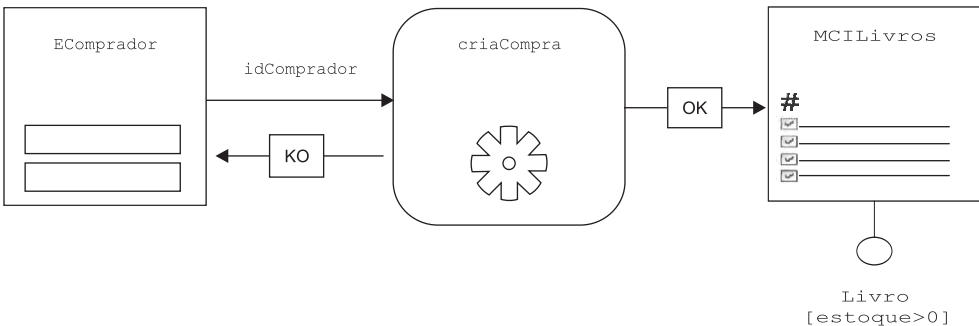


Figura 10.35: Continuação da modelagem introduzindo uma multi-choice index unit.

Possivelmente, tal lista será muito longa, e o projetista, nesse ponto, poderá pensar em usar o padrão “Índice Filtrado” para apresentar apenas alguns livros com base em uma escolha de palavra-chave por parte do comprador. Mas, por ora, ficará assim mesmo.

Além disso, a *multi-choice index unit* deverá passar, além do ISBN de cada livro escolhido, para a quantidade solicitada. O padrão *multi-choice index unit* não permite realizar diretamente essa operação. Deveriam ser definidas *entry units* para as quantidades sempre que uma opção fosse selecionada. Considerando-se que esse tipo de interação é bastante comum em sistemas Web, pode-se optar pela definição de um novo estereótipo de *unit*: uma *multi-choice index unit* com quantidade. Ela não será definida aqui, mas pode-se atribuir a ela uma representação simbólica como a da Figura 10.35, no qual aparece com a marca # para indicar que cada elemento selecionado é complementado com uma quantidade.

Na continuação (Figura 10.36), percebe-se que, após o usuário selecionar os livros e quantidades, deve ser executada a operação não padronizada *adicionaCompra*. E logo após, o total da compra será exibido.

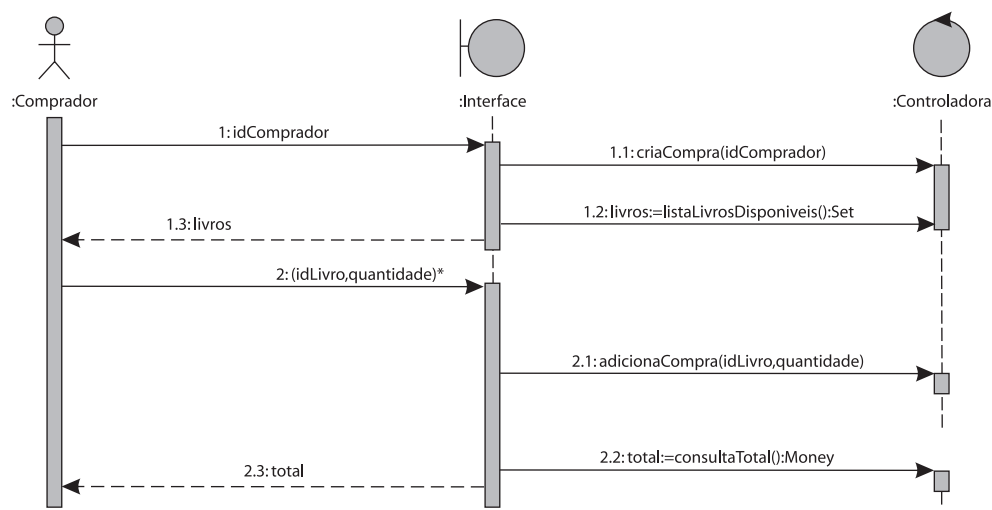


Figura 10.36: Continuação do diagrama de sequência.

Para exibir um atributo derivado da compra, é possível usar uma *data unit*, como na Figura 10.37.

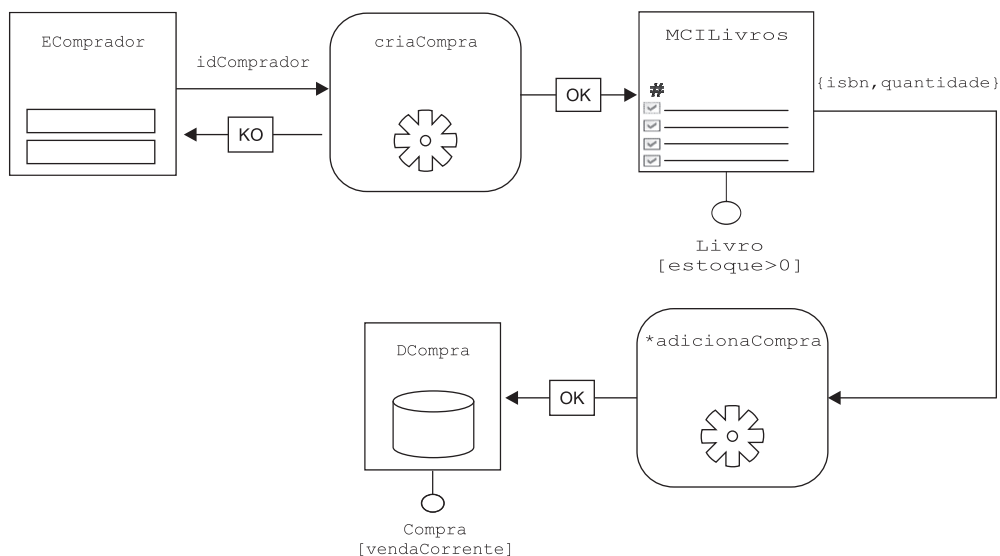


Figura 10.37: Continuação da modelagem introduzindo uma nova *operation unit* e uma *data unit*.

A *unit* DCompra deveria ser definida, a princípio, apenas com o atributo derivado *valorTotal*:

```

DataUnit DCompra (
    source Compra;
    selector CompraCorrente;
    attributes valorTotal
)
  
```

E assim por diante. A modelagem continua até que todas as operações e consultas definidas no diagrama de sequência do fluxo principal do caso de uso, bem como dos diagramas de sequência dos fluxos alternativos, estejam representadas.

Em relação à modelagem WebML “pura” e à forma apresentada neste livro, nota-se uma significativa vantagem na segunda: a modelagem WebML pura faz com que as operações básicas individuais (criação e destruição de instância, criação e destruição de associação e modificação de valor de atributo) sejam representadas nos diagramas WebML, o que poderia deixá-los bastante confusos. A abordagem usada aqui sugere que, em vez de representar essas operações básicas individuais no diagrama WebML, se usem as *operation units* genéricas a partir das operações e consultas de sistema identificadas no diagrama de sequência. Essas *operation units* estarão encapsulando todo um conjunto de operações básicas (de acordo com seus contratos) e, dessa forma, deixam os diagramas mais legíveis.

Persistência

A disponibilização de *frameworks* de persistência para linguagens comerciais (ver, por exemplo, http://docs.jboss.org/hibernate/stable/annotations/reference/en/pdf/hibernate_annotations.pdf) tornou praticamente secundária uma atividade de projeto de persistência que envolva o projeto especificamente de tabelas relacionais, formato de campos, restrições estruturais etc. Com o uso de ferramentas adequadas, é possível gerar a camada de persistência automaticamente a partir do projeto da camada de domínio. Eventualmente, será necessário efetuar alguns ajustes no mecanismo de persistência para acomodar objetos com características especiais ou para satisfazer requisitos de *performance* ou segurança de dados.

Assim, cabe ao projetista indicar qual a ferramenta de persistência a ser usada e apresentar as indicações necessárias para que essa ferramenta possa ser usada de forma profícua.

O objetivo deste capítulo é, então, indicar ao leitor o que acontece na camada de persistência quando se usa um *framework* desse tipo. Mas os aspectos discutidos aqui não precisam ser necessariamente redefinidos a cada projeto.

Em primeiro lugar, é interessante deixar claro que um mecanismo de persistência se baseia em uma ideia de separação entre interface, domínio e

persistência. Todas as operações e consultas da interface são realizadas através da camada de domínio, e não através de expressões SQL.

Cabe ao mecanismo de persistência, então, garantir que os objetos sejam salvos em um dispositivo de memória secundária e que de lá sejam recarregados quando necessário.

11.1. Equivalência entre Projeto Orientado a Objetos e Modelo Relacional

O DCP permite a geração automática de uma estrutura de banco de dados relacional que reflete, em memória secundária, a informação que os objetos representam em memória primária. Para isso é necessário seguir algumas regras de equivalência que são apresentadas nas próximas subseções.

11.1.1. Classes e Atributos

O primeiro conjunto de regras trata das classes e seus atributos. Cada classe do DCP equivale a uma *tabela relacional*. Cada atributo de uma classe equivale a uma *coluna* na tabela respectiva. Cada instância de uma classe equivale a uma *linha* na tabela respectiva. Identificadores de objetos são representados como *colunas indexadas* que não admitem repetição de elementos, sendo, portanto, marcadas com a expressão unique (Figura 11.1).

(a)

Livro
<<oid>> +isbn
+titulo
+editora
+autor
+nrPaginas

(b)

Tabela: Livro					
pkLivro<<pk>>	isbn <<unique>>	titulo	editora	autor	nrPaginas
10001	12345	análise e projeto	campus	raul	302
10002	54321	metologia de pesquisa	campus	raul	156
10003	11111	a república	acrópole	platão	205

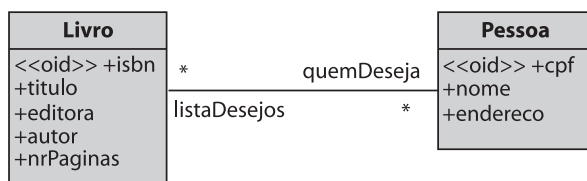
Figura 11.1: (a) Uma classe. (b) Tabela relacional equivalente a essa classe com três instâncias representadas.

A representação relacional, além dos atributos da classe, terá uma coluna consistindo em uma chave primária (pk, de *primary key*), tratando-se de um código inacessível e sem qualquer significado para a camada de domínio. O valor da chave primária deve ser conhecido, portanto, apenas na camada de persistência.

11.1.2. Associações de Muitos para Muitos

As associações entre as classes (exceto as temporárias) corresponderão a *tabelas associativas* no modelo relacional, ou seja, tabelas com uma chave primária composta pelas chaves primárias de duas outras tabelas.

Conforme o tipo de multiplicidade dos papéis da associação, algumas regras devem ser observadas. Se nenhum dos lados da associação tiver multiplicidade 1, nenhuma das colunas que formam a chave primária será marcada com unique (Figura 11.2). Suponha que, além dos três livros da Figura 11.1, houvesse três pessoas representadas na tabela apropriada (Figura 11.2c). A tabela da Figura 11.2b mostra como se relacionam algumas pessoas com alguns livros de sua lista de desejos.



(a)

(b)

Tabela: listaDesejos_quemDeseja	
pkLivro<<pk>>	pkPessoa <<pk>>
10001	20001
10001	20003
10003	20001

(c)

Tabela> Pessoa			
pkPessoa <<pk>>	cpf <<unique>>	nome	endereco
20001	3637283	joão	rua não
20002	3729109	miguel	av. das dores
20003	3938204	maria	rua talvez

Figura 11.2: (a) Exemplo de associação de muitos para muitos. (b) Tabela associativa que representa essa associação. (c) Tabela para a classe Pessoa.

Na Figura 11.2b, as duas colunas formam uma chave composta. Isso significa que, individualmente, elas podem repetir valores. Apenas não é possível repetir os pares de valores, pois cada par forma a chave primária da tabela.

Na figura, a tabela associativa, juntamente com as tabelas das Figuras 11.1b e 11.2c, estabelece que João deseja os livros “análise e projeto” e “a república”. Já Maria deseja apenas o livro “análise e projeto”, e Miguel não deseja livro algum.

Nota-se que é preferível nomear a tabela associativa com os nomes de papel da associação do que com os nomes das classes, pois pode haver mais de uma associação entre as mesmas classes e, usando os nomes de papel, garante-se que não haverá ambiguidade. Usam-se os nomes das classes apenas na falta do nome de papel explícito, como no caso de expressões OCL.

11.1.3. Associações de Um para Muitos

No caso de associações de um para muitos ou de muitos para um, a tabela associativa terá a condição unique na coluna correspondente à classe do lado “muitos”. Isso significa que a coluna correspondente ao lado “muitos” da associação não pode ter seus valores individuais repetidos.

(a)



(b)

Tabela: livro_capitulos	
pkLivro<<pk>>	pkCapitulo <<pk>> <<unique>>
10001	30001
10001	30002
10001	30003
10002	30004
10002	30005
10003	30006
10003	30007

Figura 11.3: (a) Associação de um para muitos. (b) Tabela associativa correspondente.

Na Figura 11.3b, a restrição unique na coluna da direita impede que um mesmo capítulo apareça associado a mais do que um livro.

Mais algumas observações:

- se a associação for estritamente de um para muitos, todos os elementos da tabela do lado “muitos” *devem* aparecer na tabela associativa. No caso, todos os capítulos existentes aparecem na tabela associativa da Figura 10.3b, pois é obrigatório que um capítulo esteja associado a um livro;
- se a associação fosse de 0..1 para muitos, nem todos os elementos da tabela Capítulo precisariam aparecer na tabela associativa;
- se a associação fosse de 1 para 1..*, ela seria obrigatória nas duas direções. Logo, todos os livros e todos os capítulos deveriam aparecer na tabela associativa.

Genericamente, considerado que A tem uma associação para B e que o limite mínimo do papel de A para B é n , enquanto o limite máximo é m (onde m pode ser $*$ ou infinito), o número de vezes que cada instância de A aparece na tabela associativa é limitado inferiormente por n e superiormente por m . Por exemplo, se a multiplicidade de papel de A para B for 2..5, cada instância de A deve aparecer na tabela associativa no mínimo duas e no máximo cinco vezes.

11.1.4. Associações de Um para Um

No caso de associações de um para um, um para 0..1, 0..1 para um e 0..1 para 0..1, a tabela associativa terá unique nas duas colunas da chave primária, impedindo, com isso, que qualquer dos elementos associe-se com mais de um elemento da outra classe (Figura 11.4).

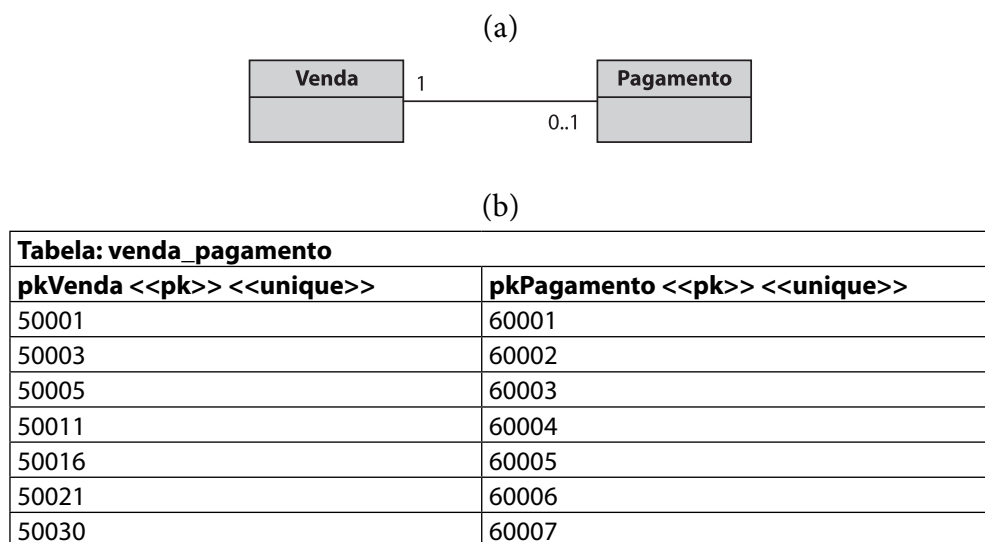


Figura 11.4: (a) Associação um para 0..1. (b) Tabela associativa correspondente.

Como o papel é obrigatório para o pagamento, todas as instâncias de Pagamento aparecem na tabela associativa, mas nem todas as instâncias de Venda precisam aparecer, pois o papel não é obrigatório para elas.

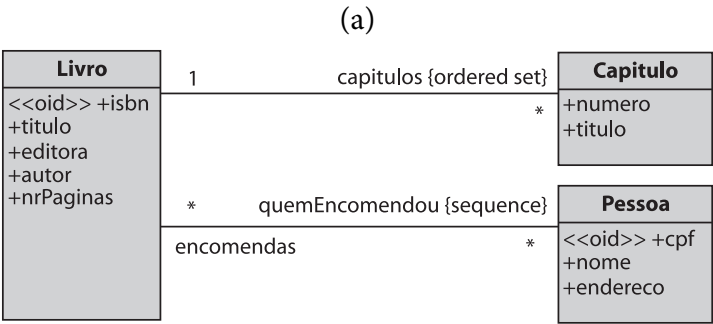
É possível também representar associações para um ou para 0..1 como chaves estrangeiras na tabela que representa a classe de origem. Embora isso possa parecer interessante em um primeiro momento, por evitar a criação de uma nova tabela para representar uma associação, pode tornar-se um problema quando for necessário dar manutenção ao sistema, criando ou substituindo associações ou, ainda, se for necessário, mudar a direção ou a multiplicidade de uma associação.

Outro problema é que o uso de chave estrangeira nesse caso deixa o acesso à associação praticamente unidirecional do lado “muitos” para o lado “um”. Mas, na prática, essas associações acabam quase sempre sendo navegadas na direção oposta, isto é, de “um” para “muitos”. Pode haver, então, se não forem tomadas medidas, uma degradação de *performance* no sistema final.

É mais cômodo, assim, representar as associações como tabelas associativas. Especialmente quando for necessário fazer alterações na estrutura da informação, a vantagem das tabelas associativas é evidente.

11.1.5. Associações Ordenadas

Uma associação que tenha um papel ordenado (*sequence* ou *ordered set*) em um dos lados deverá implementar na tabela relacional uma coluna adicional que representa a ordem de ocorrência dos elementos (Figura 11.5). Caso se trate de um conjunto ordenado (portanto, sem repetição de elementos), a coluna de ordem não deve fazer parte da chave primária (Figura 11.5b). Caso se trate de uma *sequence*, ou seja, com repetição de elementos na lista, a coluna de ordem deve fazer parte da chave primária da tabela (Figura 11.5c).



(b)

Tabela: livro_capitulos		
pkLivro <<pk>>	pkCapitulo <<pk>> <<unique>>	ordem
10001	30001	1
10001	30002	2
10001	30003	3
10002	30004	1
10002	30005	2
10003	30006	1
10003	30007	2

(c)

Tabela: encomendas_quemEncomendou		
pkLivro <<pk>>	pkPessoa <<pk>>	ordem <<pk>>
10001	20001	1
10001	20003	2
10001	20002	3
10001	20001	4
10002	20003	1
10003	20001	1
10003	20002	2

Figura 11.5: (a) Modelo com associações ordenadas. (b) Tabela associativa para *ordered set*. (c) Tabela associativa para *sequence*.

Na Figura 11.5c, o fato de que a coluna de ordem pertence à chave primária permite que a mesma pessoa (20001 • João) encomende o mesmo livro (10001 • análise e projeto) mais de uma vez, aparecendo na primeira e na quarta posições da fila. Isso não seria possível na tabela da Figura 10.5b porque não se pode repetir pares livro/capítulo.

11.1.6. Associações Representando Multiconjuntos

No caso de multiconjuntos, ou *bags*, que são conjuntos que admitem repetição de elementos mas que não estabelecem ordem entre eles, a solução usual é adicionar uma coluna extra com um contador do número de vezes que uma instância aparece na associação. A Figura 11.6 retoma o exemplo da Figura 7.22 mostrando uma possível implementação como tabela relacional.

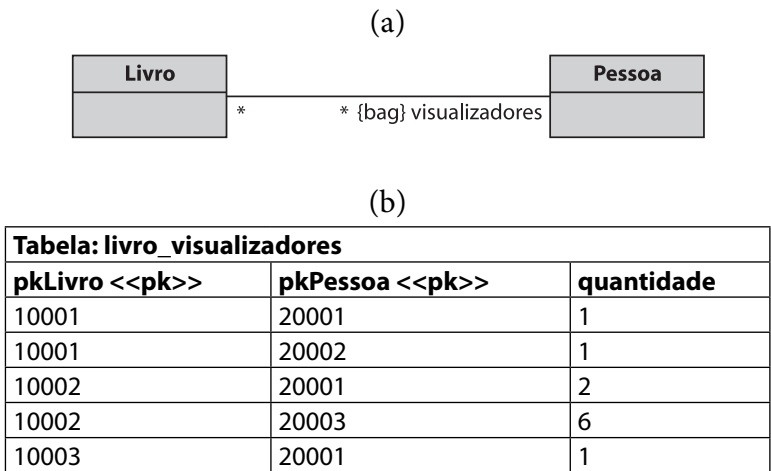


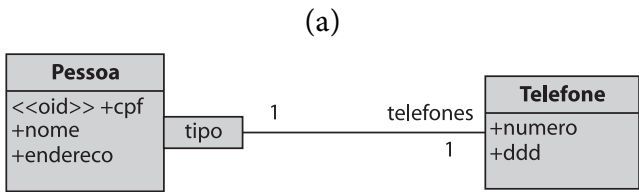
Figura 11.6: (a) Associação multiconjunto (bag). (b) Representação dessa associação como tabela relacional.

Na Figura 11.6b, Miguel (10002) visualizou o livro “a república” (20003) seis vezes. Já João (10001) visualizou o livro “análise e projeto” (20001) apenas uma vez. Não é necessário representar a quantidade zero. Por exemplo, Maria (20003) nunca visualizou “a república” (10003); então essa combinação simplesmente não deve aparecer na tabela.

11.1.7. Associações Qualificadas

No caso de associação qualificada *para um* com *qualificador interno* (o qualificador é atributo da classe), basta implementar a associação como mera associação para muitos (Figura 11.3), tomando o cuidado de fazer com que a coluna que contém o atributo qualificador seja marcada com unique na tabela que contém o conceito original. Se a ferramenta de banco de dados permitir, pode-se ainda indexar o campo com o atributo qualificador para que o acesso a este seja mais rápido (Date, 1982).

Porém, quando o qualificador for *externo*, é necessário adicionar uma terceira coluna à tabela associativa que permita mapear elementos da classe destino a partir do valor do qualificador (Figura 11.7).



(b)

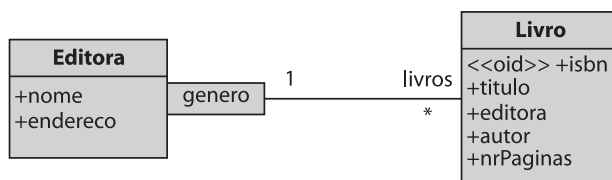
Tabela: pessoa_telefones		
pkPessoa <<pk>>	tipo <<pk>>	pkTelefone <<unique>>
20001	casa	70001
20001	celular	70002
20002	casa	70003

Figura 11.7: (a) Associação com qualificador externo. (b) Sua representação como tabela relacional.

Nesse caso, a tabela relacional associativa deve ter uma chave primária dupla, formada pelas colunas como na Figura 11.7. A pkPessoa e tipo a coluna pkTelefone não pertence à chave, mas deve ser unique visto que um telefone só pode se associar a uma única pessoa.

A situação de um qualificador externo que define uma partição, como na Figura 11.8, é implementada da mesma maneira que a associação qualificada da Figura 11.7. Porém, no caso da Figura 11.8, a chave primária deve ser tripla e a restrição que impede a repetição de pares de instâncias da classe de origem e qualificador não deve existir.

(a)



(b)

Tabela: editora_livros		
pkEditora <<pk>>	genero <<pk>>	pkLivro <<pk>> <<unique>>
60001	computação	10001
60001	computação	10002
60002	filosofia	10003

Figura 11.8: (a) Associação qualificada representando uma partição e (b) sua representação como tabela relacional.

No caso de *relações*, como na Figura 7.26, faz-se a mesma implementação, mas elimina-se o unique na coluna que representa a classe destino.

11.1.8. Classes de Associação

Uma classe de associação e sua associação correspondente são representadas em uma única tabela no banco de dados. Em primeiro lugar, cria-se uma tabela associativa para a associação (normalmente é “muitos para muitos”). Os atributos da classe de associação são acrescentados como colunas extras nessa tabela associativa. Porém, como a classe de associação pode ter suas próprias associações, pode ser inconveniente manter uma chave primária dupla para ela. Nesse caso, deve-se criar uma nova chave primária para representar as instâncias da classe de associação. Assim, a tabela associativa terá três tipos de colunas:

- a) a sua própria chave primária simples;
- b) duas colunas com valores tomados das chaves primárias das tabelas associadas, correspondendo a uma chave candidata;
- c) os atributos da classe de associação.

A Figura 11.9 apresenta esquematicamente a equivalência entre uma classe de associação e uma tabela relacional.

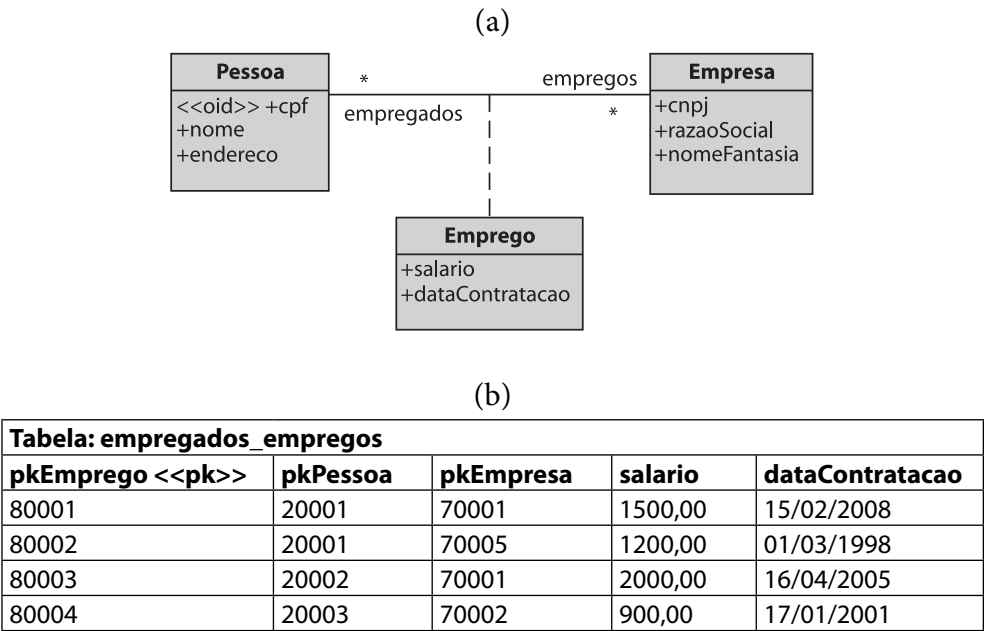


Figura 11.9: (a) Classe de associação e (b) sua representação como tabela associativa.

Na Figura 11.9 observa-se que o par pkPessoa/pkEmpresa é uma chave candidata da tabela, pois seus pares não devem se repetir. Mas a chave primária

ria efetiva é pkEmprego, para que seja uma chave simples e dessa forma facilite a representação de associações entre a classe de associação Emprego e outras classes.

11.1.9. Associações Temporárias e Associações da Controladora

As associações temporárias *não são* transformadas em tabelas relacionais porque, por sua própria definição, existem apenas em memória primária, não sendo necessário nem desejável torná-las persistentes ao longo do tempo.

Já algumas associações ligadas à controladora não precisam ser persistentes porque a controladora é *singleton*. As associações de *singletons*, caso fossem transformadas em tabelas associativas, teriam sempre o mesmo valor na coluna da chave primária do lado do *singleton*, já que só existe uma única instância dele. Assim, essas tabelas são, a princípio, desnecessárias, e pode-se usar simplesmente a chave primária da tabela que representa a outra classe da associação quando se quer, a partir da controladora, localizar uma de suas instâncias.

Porém, isso só vale para as associações *um para muitos* da controladora para as classes independentes que ligam a controladora a todas as instâncias da classe. Caso se trate de associações opcionais do lado da controladora, elas devem ser implementadas, pois trazem informação nova. Por exemplo, na Figura 11.10, a associação clientes não precisa ser representada como tabela associativa, pois acessa todas as instâncias da classe Cliente e para isso basta acessar diretamente a chave primária da tabela Cliente. Já a associação clientesPremium deve ser implementada, pois nem todos os clientes pertencem a ela. Embora todas as colunas do lado Livir da tabela associativa repitam o mesmo valor (pk de Livir), nem todas as instâncias de Pessoa estarão na tabela, o que justifica que se trata de informação nova não acessível por outros meios.

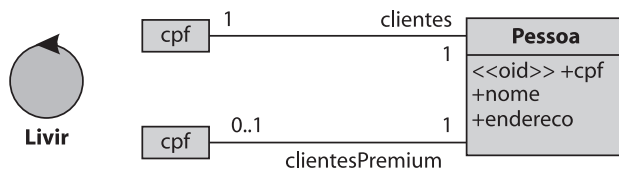


Figura 11.10: Uma associação obrigatória (clientes) e uma associação opcional (clientesPremium) para a controladora.

Além disso, conforme foi dito, as associações obrigatórias da controladora não *precisam* ser implementadas, mas isso não quer dizer que seja proibido implementá-las. Por uma questão de uniformidade de tratamento, o projetista pode optar por implementar todas as associações, inclusive essas, se julgar que será adequado para o projeto.

11.1.10. Herança

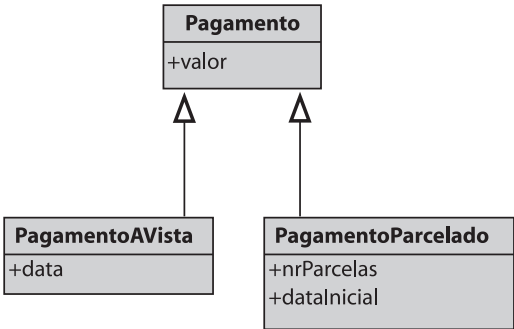
Visto que a herança é principalmente uma maneira de fatorar propriedades na especificação das classes, pode-se implementar tabelas relacionais para as subclasses de forma não fatorada, ou seja, criando cada tabela com todos os atributos da classe e atributos herdados.

Mas essa forma de representação pode ser inconveniente caso se queira fazer muitas operações com o conjunto das instâncias do ponto de vista da superclasse porque, nesse caso, seria necessário unir tabelas heterogêneas. Além disso, poderia ser complicado implementar um mecanismo de controle de tabelas associativas quando existem associações para a superclasse e para as subclasses.

Então, outra opção que surge é a decomposição das instâncias de subclasses em tabelas com os atributos próprios das subclasses e tabelas representando as superclasses com os atributos generalizados. Essa situação (Figura 11.11a) pode ser representada por um equivalente conceitual como o modelo da Figura 11.11b. Adiciona-se ainda, ao modelo da Figura 11.11b, uma invariante na classe Pagamento:

```
Context Pagamento inv:  
pagamentoAVista→size() + pagamentoParcelado→size() = 1
```

(a)



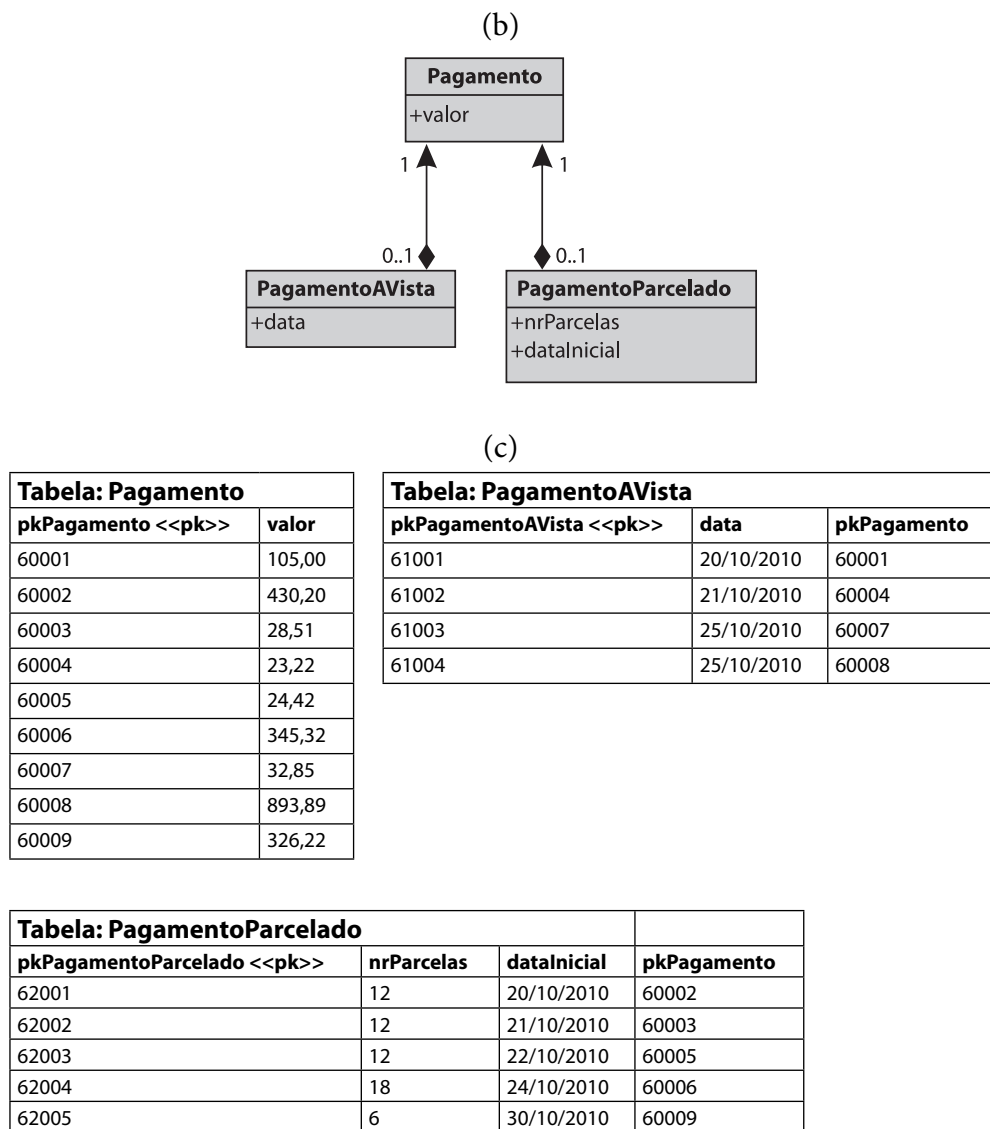


Figura 11.11: (a) Situação conceitual em que existe herança. (b) Equivalente de projeto. (c) Tabelas relacionais equivalentes.

A implementação da associação unidirecional das subclasses para a superclasse *nesse caso* pode ser implementada como uma chave estrangeira na tabela que representa a subclasse porque:

- não existe nenhuma *associação* efetiva entre as instâncias que tivesse que ser representada à parte;

- b) as propriedades da subclasse e da superclasse se complementam. São propriedades de um único objeto representadas em dois lugares diferentes.

A interpretação é que a tabela Pagamento apresenta a *continuação* da definição das tabelas PagamentoAVista e PagamentoParcelado.

11.2. Proxy Virtual

A equivalência em termos de representação entre classes e tabelas é apenas parte do problema de compatibilizar um projeto orientado a objetos com um banco de dados. É preciso ainda decidir como e quando os objetos serão salvos e carregados do banco. O projetista poderá optar por um projeto em que ele insira nos pontos adequados do código de carregamento e salvamento dos objetos à medida que a lógica da aplicação determine essa necessidade. Porém, essa abordagem, por assim dizer, *manual* de salvamento e carregamento, introduz uma carga extra no tempo de projeto, além de possibilitar a introdução de mais erros de lógica além daqueles que já são inerentes ao projeto da camada de domínio.

Além disso, se o projetista é que decide o momento de carregar e salvar objetos, muitas vezes essas operações poderão acabar sendo executadas sem necessidade, por exemplo, carregando objetos que já estão na memória e salvando objetos que não foram alterados. Controlar mais essas características caso a caso, método a método, não é a maneira mais produtiva de se proceder.

O ideal é que as tarefas de salvamento e carregamento de objetos sejam totalmente controladas pela própria arquitetura do sistema, ou seja, o projetista apenas teria de definir que um objeto é persistente, e todo um conjunto de métodos e estruturas de dados seria automaticamente criado em torno dele para permitir que o carregamento e o salvamento ocorram nos momentos mais apropriados.

Para implementar esse esquema, pode ser usado um padrão de projeto denominado *proxy virtual* (Larman, 2001). Um *proxy* virtual é um objeto muito simples que implementa apenas duas responsabilidades:

- a) conhecer o valor da chave primária do objeto real. Isso não é problema, pois o *proxy* virtual é uma classe da camada de persistência e por isso pode ter acesso a esse valor;
- b) repassar ao objeto real todas as mensagens que receber em nome dele.

O algoritmo da Figura 11.12 representa, de forma geral, o funcionamento de um *proxy* virtual.

Classe *Proxy Virtual*

Para qualquer mensagem recebida faça:

Solicite ao BrokerManager o objeto real a partir de sua *pk*.

Repasse a mensagem recebida ao objeto real.

Fim

Figura 11.12: Funcionamento geral de um *proxy* virtual.

Mais adiante, será explicado o que é e como funciona o BrokerManager.

Assim, o projeto poderá determinar que, em vez de os objetos se associarem uns aos outros, eles se associam com seus *proxies*. Dessa forma, será possível trazer para a memória uma instância de Editora sem trazer com ela as instâncias de Livro associadas. Basta associar a instância de Editora aos *proxies* dos livros. Essa atitude econômica, também chamada de *carregamento preguiçoso* (*lazy load*), permite grandes ganhos de eficiência de tempo e memória no sistema implementado.

O carregamento preguiçoso fará com que as instâncias de Livro só sejam carregadas para a memória se forem realmente necessárias. Por exemplo, se a instância de Editora vai apenas alterar seu endereço, não será necessário carregar as instâncias de Livro associadas. Porém, se a instância de Editora quiser saber qual o livro mais caro associado a ela, será necessário carregar as instâncias de livro associadas também.

Para que o projetista não tenha de decidir caso a caso quando os objetos devem ser carregados, o mecanismo de *proxy* virtual deve se interpor a todas as associações persistentes entre objetos. Os objetos reais simplesmente mandam mensagens uns aos outros como se todos estivessem em memória principal. Os *proxies* cuidam do carregamento quando ele se fizer necessário.

A Figura 11.13 exemplifica o funcionamento do mecanismo de *lazy load*. Inicialmente (a) apenas uma instância de Editora está em memória. Ela possui associação para três livros. Porém, em vez de os livros estarem em memória, apenas seus *proxies* estão. Quando a editora precisa solicitar alguma operação ou consulta de um dos livros (b), ela envia a mensagem pela asso-

ciação, que é interceptada pelo *proxy*, que providencia para que o livro seja trazido à memória. O livro teria associação com dois capítulos, mas apenas os proxies dos capítulos são carregados.

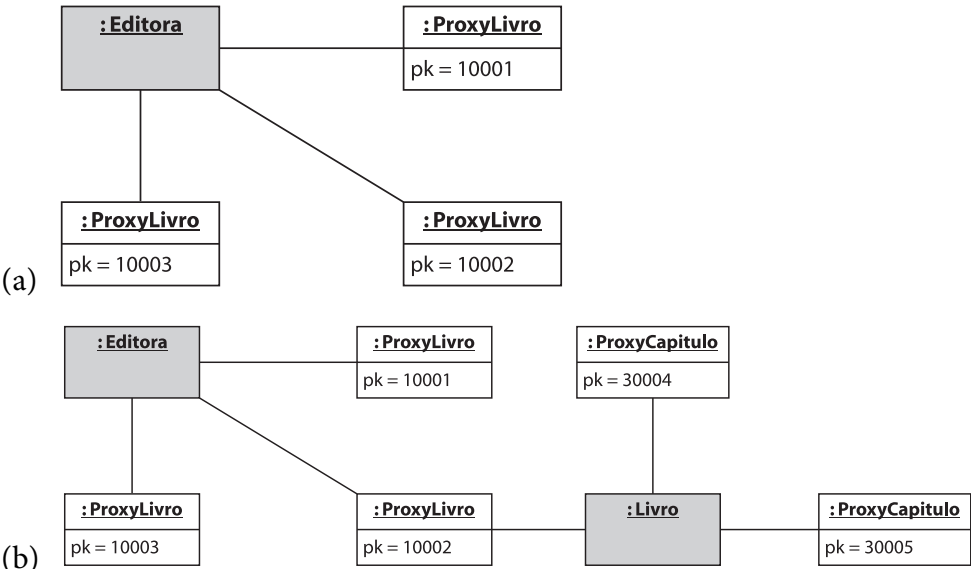


Figura 11.13: Ilustração do método *lazy load*: (a) apenas uma editora e seus proxies em memória e (b) um livro que se tornou necessário é carregado juntamente com seus proxies.

Se, agora, um dos capítulos fosse acessado por uma mensagem enviada do livro, então apenas o capítulo seria trazido à memória, já que não existiriam mais associações a partir dele.

11.2.1. Estruturas de Dados Virtuais

A implementação de *proxies* virtuais para cada objeto pode ser bastante ineficiente quando se trata de mapear coleções de objetos. Por exemplo, uma editora associada a 1.000 títulos de livro teria de ter 1.000 *proxies* instanciados associados a ela quando fosse trazida à memória? A resposta, felizmente, é *não*. A solução para evitar a instanciação indiscriminada de *proxies* em memória é a implementação de estruturas de dados virtuais pra substituir a implementação das associações.

Assim, uma editora não conteria um Set com 1.000 instâncias de *proxies* de livros, mas uma estrutura *VirtualSet*, que implementa os mesmos métodos de adição, remoção e consulta de um Set normal. Só que o *VirtualSet* não traz

seus elementos para a memória imediatamente, mas apenas sob demanda. O `VirtualSet` e seus congêneres, `VirtualSequence`, `VirtualOrderedSet`, `VirtualBag` e `VirtualMap`, em vez de implementarem uma coleção de objetos, implementam uma chamada a um mecanismo de persistência, ou `BrokerManager`. Esse mecanismo é que fará o carregamento do objeto caso ele não esteja em memória.

Assim, um `VirtualSet` teria em sua implementação uma estrutura de lista simples, contendo apenas números das chaves primárias dos elementos. A execução de uma consulta no `VirtualSet` corresponderia a tomar a pk do elemento em questão e solicitar o objeto ao `BrokerManager`, que o retorna. O `BrokerManager` é que vai verificar, como será visto mais adiante, se o objeto real se encontra em memória ou não.

Uma estrutura de dados virtual deve, então, funcionar da seguinte forma:

- a) em vez de uma representação física de um conjunto de objetos, terá uma representação física de um conjunto de números inteiros: as pk dos objetos;
- b) o método que adiciona um elemento na estrutura deve adicionar apenas a pk do elemento na representação física;
- c) o método que remove um elemento da estrutura deve apenas remover a pk do elemento na representação física;
- d) qualquer método que consulte a estrutura para obter um objeto deve tomar a pk do objeto da representação física e solicitar ao (até agora misterioso) `BrokerManager` que retorne o objeto real.

Assim, a adição e a remoção de objetos de associações podem ser feitas sem que os objetos sequer estejam em memória. O objeto só é trazido quando informações internas dele se tornam necessárias.

11.3. Materialização

O processo de carregamento de um objeto do banco de dados para a memória principal é denominado *materialização*. A materialização é solicitada pelo `BrokerManager` a um *broker* especializado sempre que necessário, ou seja, sempre que um *proxy* solicitar acesso a um objeto que ainda não esteja materializado.

Poderá existir um *broker* especializado para cada classe persistente ou um único *broker* genérico. Um *broker* deve implementar um método *materializa* que faz o seguinte:

- a) cria uma instância da classe persistente;
- b) inicializa os valores dos atributos da nova instância com valores da respectiva linha e coluna do banco de dados;
- c) inicializa as estruturas de dados virtuais que implementam as associações do objeto com as chaves primárias dos respectivos objetos associados.

Para obter os valores das chaves primárias dos objetos associados, o *broker* deve saber quais são as associações que saem do objeto em questão e, em seguida, deve buscar nas tabelas associativas correspondentes ocorrências da *pk* do objeto em questão. A *pk* associada na tabela será adicionada na estrutura de dados virtual que vai implementar a associação.

Para exemplificar, um *BrokerLivro* deve materializar instâncias da classe *Livro*, definida conforme a Figura 11.3. De acordo com essas definições, esse *BrokerLivro* deverá implementar um método *materializa*, executando as seguintes operações:

- a) criar uma instância de *Livro*;
- b) preencher os atributos *isbn*, *titulo*, *editora*, *autor* e *nrPaginas* da nova instância com os valores armazenados nas respectivas colunas da tabela *Livro* no banco de dados;
- c) buscar na tabela *livro_capitulos* ocorrências da *pk* do livro na coluna *pkLivros*. Para todas as ocorrências, adicionar no *VirtualSet* *capitulos* da nova instância de *Livro* os valores da coluna correspondente *pkCapitulo*.

Não se deve confundir a materialização feita pelo *Broker* com a criação de instância definida nos contratos de operação de sistema. Nos contratos, a criação de uma instância refere-se à inserção de nova informação, independentemente do meio físico no qual ela esteja armazenada (memória principal ou secundária). A materialização feita pelo *Broker* apenas representa o ato de trazer para a memória principal um objeto que está em memória secundária. A materialização é, pois, uma operação exclusiva da camada de persistência, nada tendo a ver com as regras de negócio.

11.4. Caches

Os objetos em memória principal podem ser classificados em:

- a) *limpos* ou *sujos*, dependendo de estarem ou não consistentes com o banco de dados;
- b) *novos* ou *velhos*, dependendo de já existirem ou não no banco de dados;

- c) *excluídos*, dependendo de terem sido excluídos da memória, mas ainda não do banco de dados.

Uma *cache* é uma estrutura de dados na forma de um dicionário (ou *Map*), que associa valores de pk com objetos reais.

Embora existam oito combinações possíveis, e Larman (2001) trabalhe com seis delas, a prática indica que apenas quatro caches são suficientes para gerenciar objetos em memória primária:

- a) *old clean cache*: onde ficam os objetos que estão consistentes com o banco de dados, ou seja, velhos e limpos;
- b) *old dirty cache*: onde ficam os objetos que existem no banco de dados, mas foram modificados em memória, isto é, velhos e sujos;
- c) *new cache*: onde ficam os objetos que foram criados em memória, mas ainda não existem no banco de dados;
- d) *delete cache*: onde ficam os objetos deletados em memória, mas que ainda existem no banco de dados.

Quando se diz que o *BrokerManager* verifica se um objeto está em memória, ele faz uma consulta às caches existentes e, caso encontre uma referência ao objeto cuja pk foi passada, retorna o objeto ao *Proxy*. Se o *BrokerManager* procurar todas as caches e não encontrar o objeto, ele deverá solicitar ao *broker* especializado na classe do objeto que o materialize. O objeto assim materializado é inserido na *OldCleanCache*.

Se, em algum momento, esse objeto for alterado em memória, ou seja, se algum de seus atributos for mudado ou se alguma associação partindo dele for criada ou destruída, ele será movido da *OldCleanCache* para a *OldDirtyCache*.

Para se ter um bom controle do estado de cada objeto, é importante que as variáveis de instância que representam atributos e associações do objeto só possam ser alteradas pelos métodos *set*, *add* e *remove*. Dessa forma, em cada um desses métodos, poderá ser adicionado um comando do tipo *BrokerManager.instance().ficouSujo(self)*. Essa mensagem enviada ao *BrokerManager* vai fazer com que ele mova o objeto de uma *OldCleanCache* para uma *OldDirtyCache* ou que o mantenha na *OldDirtyCache*, se ele já estiver lá.

Objetos criados em memória, como resultado de uma pós-condição de contrato, devem ser armazenados em uma *NewCache*. Um objeto em uma *NewCache* não pode ser movido para a *OldDirtyCache*, mesmo se tiver seus atributos alterados. Ele só será movido para a *OldCleanCache* depois do *commit*

sobre o banco de dados. Antes disso ele permanece na NewCache, mesmo que o seus atributos sejam alterados.

Quando for solicitada a destruição de um objeto em memória, o resultado dependerá de onde ele está. Se ele estiver na OldCleanCache ou na OldDirtyCache será movido para a DeleteCache. Porém, se ele estiver na NewCache, pode ser simplesmente deletado sem maior cerimônia.

11.5. Commit e Rollback

As operações de *commit* e *rollback* são normalmente ativadas pela camada de aplicação para indicar, respectivamente, que uma transação foi bem-sucedida e confirmada ou que foi cancelada. Essas operações são implementadas pelo BrokerManager. No caso do commit, o BrokerManager deve executar o seguinte:

- a) efetuar um update no banco de dados para os objetos da OldDirtyCache e mover esses objetos para a OldCleanCache;
- b) efetuar um insert no banco de dados para os objetos da NewCache e mover esses objetos para a OldCleanCache;
- c) efetuar um remove no banco de dados para os objetos da DeleteCache e remover esses objetos da *cache*.

No caso de um rollback, o BrokerManager deve apenas remover todos os objetos de todas as *caches*, exceto os da OldCleanCache.

Como a OldCleanCache pode crescer indefinidamente, é necessário implementar algum mecanismo para remover dela os objetos mais antigos sempre que seu tamanho atingir algum limite preestabelecido.

As outras *caches* crescem apenas até o momento do commit ou rollback, quando são esvaziadas.

11.6. Controle das Caches em um Servidor Multiusuário

Se mais de um usuário conecta-se ao sistema, é necessário determinar como vai funcionar o compartilhamento de dados em memória. Considerando uma arquitetura cliente/servidor com camadas de interface, domínio e persistência, pelo menos duas abordagens são possíveis:

- a) na primeira abordagem, as três camadas são executadas no cliente. Não existe compartilhamento de memória no servidor, o qual serve apenas

para armazenar os dados no momento em que a transação efetuar um commit. Nesse caso, o que trafega pela rede são registros do banco de dados e instruções SQL apenas nos momentos da materialização de objetos ou commit. A desvantagem dessa forma de definir a arquitetura é que o nó cliente fica sobrecarregado, e mecanismos de controle de segurança adicionais devem ser implementados no próprio banco de dados para impedir acessos não autorizados;

- b) outra possibilidade é implementar no nó cliente apenas a camada de interface e deixar no servidor as camadas de domínio e persistência. Nesse caso, os objetos existirão em memória apenas no servidor, e a comunicação na rede consistirá no envio de mensagens e recebimento de dados.

Estando os objetos fisicamente no servidor, existem duas possibilidades ainda. No primeiro caso, todos os usuários compartilham as quatro caches, com a desvantagem de que um usuário poderá ter acesso a objetos modificados que ainda não foram confirmados por commit pela aplicação de outro usuário. Essa opção parece ser desaconselhável na maioria das aplicações.

A outra opção é permitir a cada usuário apenas a visualização dos objetos cuja informação é confirmada, ou seja, objetos que estejam na OldCleanCache. Objetos em outras caches são, portanto, objetos que estão em processo de modificação por algum usuário e não devem ser acessíveis.

Assim, o mecanismo de persistência em sistemas multiusuário pode ser implementado da seguinte forma:

- a) uma OldCleanCache compartilhada por todos os usuários;
- b) cada usuário possuirá individualmente sua própria OldDirtyCache, DeleteCache e NewCache.

Procedendo assim, é possível garantir que nenhum usuário tenha acesso a objetos sendo modificados por outro usuário. É possível, portanto, usar as caches para implementar um mecanismo de lock, ou seja, quando um usuário usa um objeto, outros não podem ter acesso a ele. Quando o usuário que está de posse do objeto efetua um commit ou rollback, o lock é desfeito e outros usuários podem acessar novamente o objeto.

Uma grande vantagem desse método está no uso otimizado da memória do servidor. Os objetos na OldCleanCache, que é a única que cresce de forma indeterminada, são compartilhados por todos os usuários. As outras quatro caches, que são específicas para cada usuário, só crescem durante uma transação. Quando ocorrer commit ou rollback essas caches são esvaziadas.

Geração de Código e Testes

A fase de construção do UP prevê a geração e código e testes do sistema. É necessário gerar código tanto para a camada de domínio, resultante do projeto lógico, quanto para as demais camadas, resultantes do projeto tecnológico.

Uma vez definidos os diagramas de comunicação e o DCP, a *geração de código* é uma tarefa passível de automatização. Trata-se neste capítulo da geração de código das classes correspondentes à camada de domínio da aplicação, ou seja, as classes que realizam toda a lógica do sistema a partir das operações e consultas de sistema.

Este capítulo apresenta regras para geração de código a partir do DCP e dos diagramas de comunicação. Os exemplos são apresentados em pseudocódigo, que pode ser traduzido para qualquer linguagem de programação (preferencialmente orientada a objetos).

12.1. Classes e Atributos

Classes do DCP são imediatamente convertidas em classes na linguagem de programação. Os atributos das classes são convertidos em variáveis de instância (privadas) da respectiva classe. Atributos sempre terão tipos alfanu-

méricos ou tipos primitivos. Para cada atributo devem ser implementadas as operações `get` e `set` correspondentes.

CLASSE Livro

```
VAR PRIVADA
    isbn : String
    titulo : String
    autor : String
    nrPaginas : Inteiro
```

```
MÉTODO getIsbn():String
    RETORNA isbn
FIM MÉTODO
```

```
MÉTODO setIsbn(umIsbn:String)
    isbn := umIsbn
FIM Método
```

... - `getter` e `setter` similares para `titulo`, `autor` e `nrPaginas`

Livro
<<oid>> +isbn
+titulo
+autor
+nrPaginas

Figura 12.1: Uma classe e seu pseudocódigo correspondente.

Por uma questão de controle e consistência, é recomendável que apenas os métodos `get` e `set` acessem diretamente os valores das variáveis de instância, sendo vedado o acesso a outros métodos, mesmo que sejam da mesma classe.

12.2. Associações Unidirecionais

Associações unidirecionais do DCP podem ser transformadas em variáveis de instância (da mesma forma que os atributos), e terão métodos para alteração e consulta. Há, porém, algumas diferenças a considerar.

Em primeiro lugar, os atributos são sempre implementados por variáveis cujos tipos são primitivos (alfanuméricos) e as associações são implementadas por variáveis cujos tipos são classes (no caso de associações para um) ou estruturas de dados (no caso de associações para muitos).

Além disso, considerando as diferentes multiplicidades de papel e outras características das associações, haverá algumas distinções a fazer quanto aos métodos associados.

Na geração de código da camada de domínio, não se diferencia associações temporárias e persistentes, pois sua implementação é a mesma.

De forma geral, cada associação deverá implementar pelo menos três métodos:

- a) `add`, tendo como parâmetro o objeto a ser associado;
- b) `remove`, tendo como parâmetro o objeto a ser desassociado;
- c) `get`, retornando uma cópia da coleção de objetos associados, sobre a qual é possível realizar iterações.

As associações derivadas implementam apenas o método `get`, de acordo com sua definição.

Em relação a essas operações básicas, ainda pode ser possível implementar variações:

- a) se a associação for *qualificada*, pode-se ter um `get` que recebe como parâmetro a chave do qualificador e retorna apenas o objeto qualificado, e não o conjunto todo. Da mesma forma, o `add` poderá passar como parâmetro adicionalmente o valor do qualificador, especialmente se for um qualificador externo. O método `remove` poderá remover a associação a partir do valor do qualificador;
- b) no caso de associações *ordenadas*, pode-se ter um método `get` que retorna um elemento conforme sua posição. Da mesma forma, o `add` poderá adicionar elementos diretamente em uma posição indicada como parâmetro e o `remove` remover da posição indicada;
- c) associações ordenadas também podem ter métodos especiais para acessar, adicionar e remover elementos no início ou no fim da lista;
- d) *pilhas* e *filas* terão métodos específicos como *push* e *pop*, que seguem as regras específicas dessas estruturas.

12.2.1. Associação Unidirecional para Um

A associação unidirecional *para um* ou *para 0..1* pode ser armazenada em uma variável de instância na classe de origem da associação e seu tipo deve ser a classe de destino. Assim, uma associação unidirecional *para um* de

Pagamento para Venda corresponderá a uma variável de instância na classe Pagamento declarada com tipo Venda.

Em relação aos métodos, observa-se que o método que remove a associação não precisa de parâmetros, pois existe um único objeto a ser removido.

A Figura 12.2 mostra um exemplo de classe com associação unidirecional para um e o respectivo pseudocódigo a ser implementado. Atributos foram suprimidos, pois já foram explicados na seção anterior.

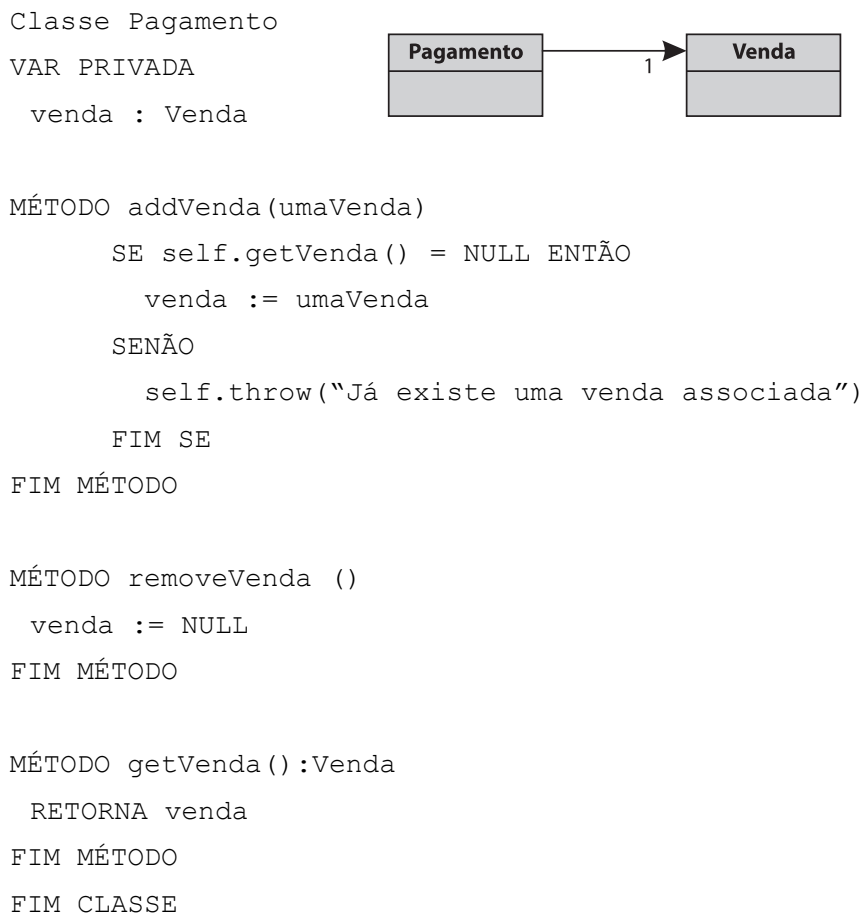


Figura 12.2: Classe com associação unidirecional para um e respectivo pseudocódigo.

Quando a multiplicidade for estritamente para um, a associação pode ser removida, como mencionado no Capítulo 8, mas o objeto ficará temporariamente inconsistente, devendo a associação removida ser substituída por outra ainda no contexto da mesma operação de sistema.

12.2.2. Associação Unidirecional para Muitos

A associação unidirecional para * (ou qualquer outra multiplicidade diferente de um ou 0..1) corresponde à implementação de uma estrutura de dados. Sendo uma associação simples, será implementada como um conjunto (Set).

A Figura 12.3 apresenta um exemplo desse tipo de construção.

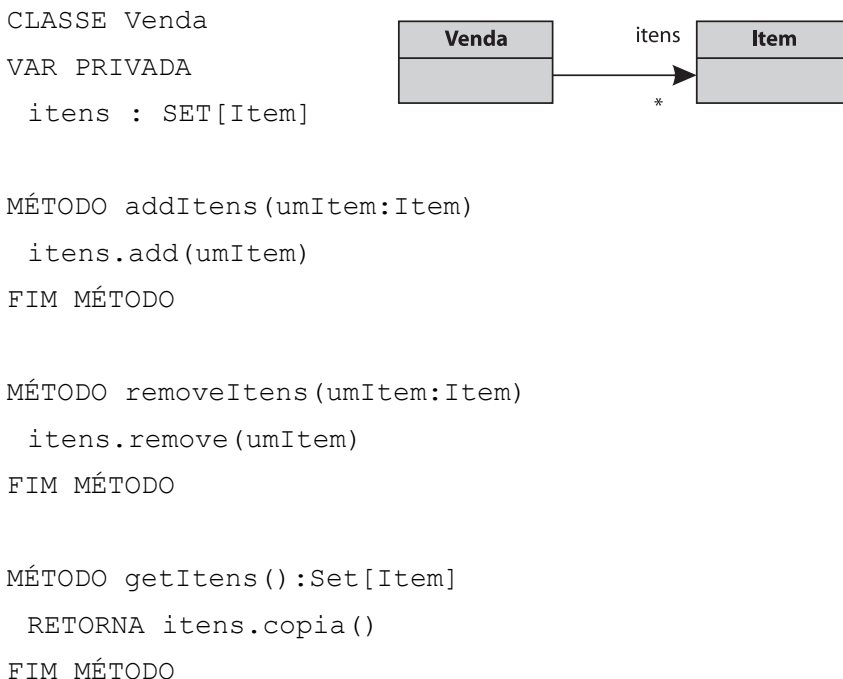


Figura 12.3: Classe com associação unidirecional simples para muitos e respectivo código.

Se a associação for rotulada com {sequence}, {ordered set} ou {bag}, deve-se substituir o tipo de dados da variável de instância Set pelo tipo apropriado de acordo com a linguagem. Podem ser usados também, conforme o caso, tipos concretos de dados como *array* ou árvore binária, por exemplo. No caso de associações para muitos com limite inferior e superior idênticos, inclusive, recomenda-se a implementação como *array*. Por exemplo, uma associação com multiplicidade de papel 5 (ou 5..5) deve ser implementada como um *array* de cinco posições.

Adicionalmente podem ser implementados métodos específicos dessas estruturas, conforme mencionado no início da Seção 12.2.

12.2.3. Associação Unidirecional Qualificada

A associação unidirecional qualificada é implementada de forma semelhante à associação com multiplicidade para muitos. Porém, em vez do tipo de dados Set, usa-se uma estrutura de dicionário ou mapeamento (Map), que associa o atributo qualificador a um objeto ou objetos dependendo da multiplicidade do papel. Nesse caso, será possível implementar um método de consulta que retorne um objeto da coleção a partir de um valor para o qualificador.

A Figura 12.4 apresenta a implementação de uma associação qualificada definindo um mapeamento (para um) com qualificador interno. A Figura 12.5 apresenta o caso de qualificador externo. A Figura 12.6 apresenta o caso de implementação de uma associação qualificada como partição (para *).

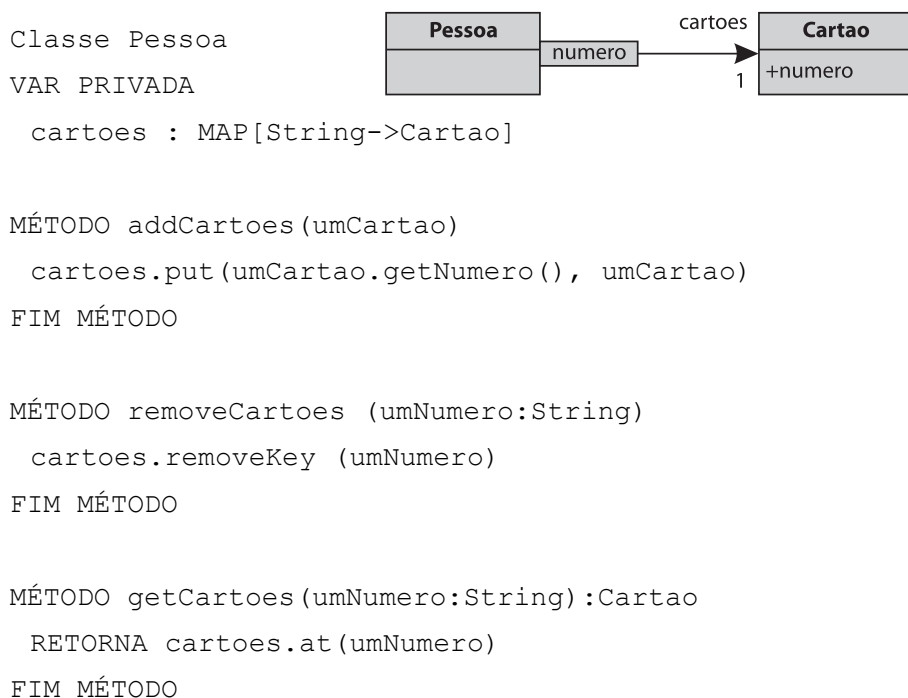


Figura 12.4: Associação qualificada como mapeamento (com qualificador interno) e seu código correspondente.

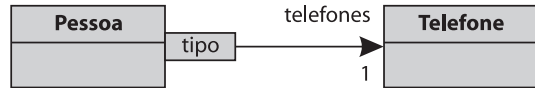
O atributo numero do Cartao é tipado como String, conforme discutido no Capítulo 7, porque se comporta como tal. Dificilmente serão executadas operações matemáticas com números de cartão.

Na Figura 12.4, pode-se adicionar, ainda, a implementação dos métodos get, add e remove da associação para muitos conforme a Seção 12.2.2.

Classe Pessoa

VAR PRIVADA

telefones : MAP[String->Telefone]



MÉTODO addTelefones (umTipo:String, umTelefone: Telefone)

telefones.put (umTipo, umTelefone)

FIM MÉTODO

MÉTODO removeTelefones (umTipo:String)

telefones.removeKey (umTipo)

FIM MÉTODO

MÉTODO getTelefones (umTipo:String) :Telefone

RETORNA cartoes.at (umTipo)

FIM MÉTODO

Figura 12.5: Associação qualificada como mapeamento (com qualificador externo) e seu código correspondente.

Classe Editora

VAR PRIVADA

livros MAP[String, SET[Livro]]



MÉTODO addLivros (umGenero:String, umLivro:Livro)

SE SET livros.at (umGenero) = NULL ENTÃO

livros.put (umGenero, SET.new ())

FIM SE

livros.at (umGenero).add (umLivro)

FIM MÉTODO

MÉTODO removeLivros (umGenero:String, umLivro:String)

livros.at (umGenero).remove (umLivro)

FIM MÉTODO

```
MÉTODO removeGenero (umGenero:String)
    livros.removeKey(umGenero)
FIM MÉTODO

MÉTODO getLivros(umGenero:String):SET[Livro]
    RETORNA livros.at(umGenero)
FIM MÉTODO
```

Figura 12.6: Associação qualificada como partição e seu código correspondente.

Aqui foram definidas duas operações de remoção, uma baseada no livro, que remove uma única associação, e outra baseada no gênero, que remove todos os livros do gênero.

O método getLivros, baseado no gênero, retorna um conjunto de livros.

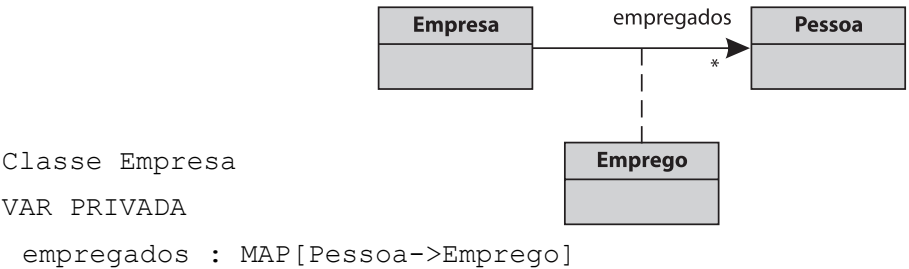
12.2.4. Associação Unidirecional com Classe de Associação

Quando a associação contém uma classe de associação, é necessário implementar a criação e destruição de instâncias dessa classe de associação cada vez que uma associação for adicionada e removida.

Classes de associação podem existir em associações com qualquer multiplicidade. Entretanto, o mais comum é que classes de associação sejam usadas em associações de * para *.

A implementação desse tipo de associação consiste em um Map, associando instâncias do destino da associação com instâncias da classe de associação.

O exemplo da Figura 12.7 mostra uma classe de associação e a implementação da associação unidirecional na classe de origem.




```
MÉTODO addEmpregados (umaPessoa:Pessoa)
    empregados.put (umaPessoa, Emprego.newInstance ())
FIM MÉTODO
```

```
MÉTODO removeEmpregados (umaPessoa)
    empregados.removeKey (umaPessoa)
FIM MÉTODO
```

```
MÉTODO getEmpregados () :SET[Pessoa]
    RETORNA empregados.copia ()
FIM MÉTODO
```

```
MÉTODO getEmpregados (umaPessoa) :Emprego
    RETORNA empregados.at (umaPessoa)
FIM MÉTODO
```

Figura 12.7: Classe de associação e respectivo código.

Na Figura 12.7, nota-se que, ao adicionar uma nova associação de Empresa com Pessoa, é automaticamente criado um novo Emprego.

Há dois métodos de acesso implementados: um que retorna todos os empregados (instâncias de Pessoa associadas à empresa) e outro, parametrizado, que retorna um emprego a partir de uma pessoa.

12.3. Associação Bidirecional

Existem pelo menos três padrões para implementação de associações bidirecionais (Fowler, 2003):

- implementar a associação como duas associações unidirecionais nas duas classes participantes;
- implementar a associação como unidirecional apenas em uma das classes. A outra classe pode acessar os elementos da associação fazendo uma pesquisa;
- implementar um objeto intermediário que representa a associação e pode ser identificado através de métodos de localização rápida como *hash*.

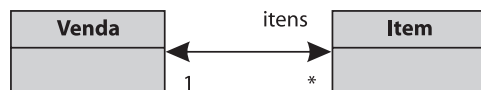
O método `get`, em todos os casos de associação bidirecional, deve ser implementado nas duas classes para permitir a navegação nas duas direções. Os métodos `add` e `remove` podem ser implementados em apenas uma das duas classes porque, se existissem em ambas as classes, seriam operações perfeitamente simétricas e, portanto, desnecessárias.

12.3.1. Implementação nas Duas Direções

A opção de implementação das associações bidirecionais nas duas direções é a mais eficiente em termos de tempo de processamento, mas produz código mais complexo e gasta mais espaço de armazenamento, pois a informação sobre a associação é representada de forma duplicada, ou seja, nas duas classes que participam dela.

Via de regra, seguem-se os padrões indicados na Seção 12.2, mas é necessário considerar que os métodos que criam e removem as associações unidirecionais componentes serão aqui apenas métodos auxiliares que não podem ser usados em nenhum outro lugar, a não ser nos métodos de adição e remoção efetivos. A Figura 12.8 apresenta um exemplo de implementação em duas direções de uma associação bidirecional de um para muitos.

```
CLASSE Venda
VAR PRIVADA
    itens : SET[Item]
```



```
MÉTODO addItensPrivado (umItem:Item)
    itens.add (umItem)
FIM MÉTODO
```

```
MÉTODO removeItensPrivado (umItem:Item)
    itens.remove (umItem)
FIM MÉTODO
```

```
MÉTODO addItens (umItem:Item)
    self.addItensPrivado (umItem)
    umItem.addVendaPrivado (self)
FIM MÉTODO
```

```
MÉTODO removeItens (umItem:Item)
    self.removeItensPrivado (umItem)
    umItem.removeVendaPrivado ()
FIM MÉTODO
```

```
MÉTODO getItens():Set[Item]
    RETORNA itens.copia()
FIM MÉTODO
```

```
CLASSE Item
VAR PRIVADA
    venda : Venda
```

```
MÉTODO addVendaPrivado (umaVenda)
    venda := umaVenda
FIM MÉTODO
```

```
MÉTODO removeVendaPrivado ()
    venda := NULL
FIM MÉTODO
```

```
MÉTODO getVenda():Venda
    RETORNA venda
FIM MÉTODO
FIM CLASSE
```

Figura 12.8: Uma associação bidirecional e sua implementação nas duas direções.

Nesse exemplo, optou-se por implementar os métodos `add` e `remove` apenas na classe `Venda`. Mas os métodos `get`, `addPrivado` e `removePrivado` devem ser implementados nas duas classes.

12.3.2. Implementação Unidirecional

Mesmo que a associação seja bidirecional pode acontecer que a navegação seja muito mais frequente ou mais crítica em uma direção do que em outra. Se isso acontecer, pode ser uma opção realizar a implementação apenas em uma

direção. A vantagem é o código mais simples e a economia de espaço. A desvantagem é que a navegação na direção oposta será uma operação bem mais lenta do que na direção implementada. A Figura 12.9 apresenta um exemplo no qual a associação bidirecional é implementada apenas na classe Venda.

CLASSE Venda

VAR PRIVADA

itens : SET[Item]

MÉTODO addItens(umItem:Item)

itens.add(umItem)

FIM MÉTODO

MÉTODO removeItens(umItem:Item)

itens.remove(umItem)

FIM MÉTODO

MÉTODO getItens():Set[Item]

RETORNA itens.copia()

FIM MÉTODO

CLASSE Item

-- não se declara aqui a variável como nos casos anteriores

MÉTODO getVenda():Venda

PARA TODA venda EM Venda.allInstances() FAÇA

SE venda.itens().includes(self) ENTÃO

RETORNA venda

FIM SE

FIM PARA

FIM MÉTODO

FIM CLASSE

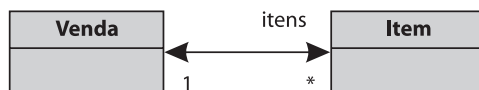


Figura 12.9: Uma associação bidirecional e sua implementação unidirecional.

O método `get` da classe `Item` é, portanto, implementado como uma consulta que pesquisa as instâncias da classe `Venda` até encontrar aquela que está associada ao `Item`. Caso a associação fosse para muitos nesse sentido, a implementação do método `get` retornaria um conjunto e não apenas um elemento. Seria algo como:

```
MÉTODO getVendas(): SET[Venda]
  VAR venda : SET[Venda]
  PARA TODA venda EM Venda.allInstances() FAÇA
    SE venda.itens().includes(self) ENTÃO
      vendas.add(venda)
  FIM SE
FIM PARA
RETORNA vendas
FIM MÉTODO
FIM CLASSE
```

Em relação à complexidade de `getVenda`, a implementação da Figura 12.8 é constante e a da Figura 12.9 é em média $n/2$, e, no pior caso n , onde n é o número de instâncias de `Venda`. Portanto, a segunda implementação é bem menos eficiente.

12.3.3. Implementação de um Objeto Associação

Uma associação bidirecional também pode ser implementada através de um objeto intermediário representando a associação. O objeto intermediário consistirá em uma tabela com os pares de instância associadas e cada uma das classes participantes terá acesso a esse objeto. A Figura 12.10 apresenta uma possível implementação para a associação bidirecional através de um objeto intermediário.

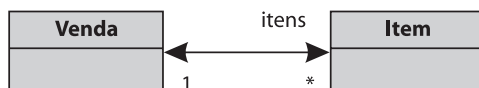
```
VAR GLOBAL venda_itens : RELATION[Venda, Item]
```

```
CLASSE Venda
```

```
MÉTODO addItens(umItem:Item)
```

```
  venda_itens.add(self,umItem)
```

```
FIM MÉTODO
```



```
MÉTODO removeItens (umItem:Item)
    venda_itens.remove (self,umItem)
FIM MÉTODO

MÉTODO getItens():Set[Item]
    RETORNA venda_itens.getKey(self)
FIM MÉTODO

CLASSE Item
MÉTODO getVenda():Venda
    RETORNA venda_itens.getValue(self)
FIM MÉTODO
FIM CLASSE
```

Figura 12.10: Uma associação bidirecional e sua implementação com um objeto intermediário.

Se a linguagem de programação permitir, em vez de declarar a associação como global, pode ser preferível declará-la como visível apenas nas classes participantes da associação.

O tipo RELATION usado aqui é semelhante ao mapeamento (MAP), mas permite que uma mesma chave seja associada a vários objetos, enquanto o mapeamento só permite um objeto por chave.

12.4. Métodos Delegados e Operações de Sistema

Até aqui, foi mostrado como gerar código não só para classes, atributos e associações, mas também para as operações básicas set, get, add e remove.

As operações básicas create e destroy devem ser implementadas de acordo com as características da linguagem, lembrando que, neste livro, adotou-se como padrão que o create apenas cria a instância e inicializa os atributos com valores iniciais, devendo os demais atributos e associações ser inicializados por outras operações básicas dentro da mesma operação de sistema.

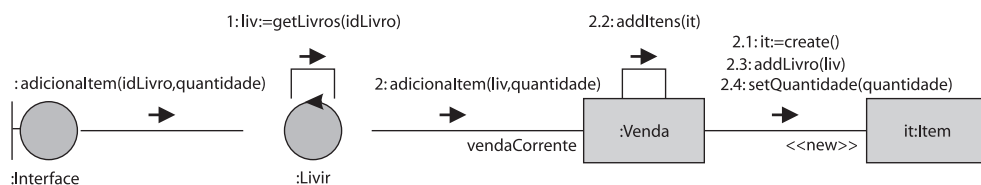
As demais operações (métodos delegados e operações de sistema) serão implementadas de acordo com as especificações dos diagramas de comunicação apresentados no Capítulo 9.

Para implementar um método desse tipo, deve-se observar o diagrama de comunicação onde ele apareceu:

- toda mensagem delegada com número x que chega a uma instância da classe A deve ser implementada como a sequência das mensagens $x.1$, $x.2$, ..., $x.n$, que saem da instância de A e são enviadas a outros objetos;
- no caso das operações de sistema que não têm número, implementa-se a operação como a sequência das mensagens 1, 2, 3, 4..., de acordo com o diagrama de comunicação que a define.

Por exemplo, a operação de sistema adicionaltem da Figura 12.11 seria implementada na controladora de sistema Livir através da sequência de operações 1 e 2. Já o método delegado adicionaltem deverá ser implementado na classe Venda através da sequência de operações 2.1, 2.2, 2.3 e 2.4.

Observa-se que cada implementação considera apenas um nível da árvore, não devendo envolver outros níveis. Por exemplo, no caso da Figura 12.11 seria errado implementar a operação de sistema adicionaltem como a sequência 1, 2, 2.1, 2.2, 2.3 e 2.4 porque ela deve implementar apenas as mensagens do primeiro nível: 1 e 2. As mensagens do segundo nível (2.1, 2.2, 2.3 e 2.4) são parte da implementação da mensagem 2.



CLASSE Livir

...

MÉTODO adicionaItem(idLivro:String, quantidade: Inteiro)

VAR liv:Livro

liv := self.getLivros(idLivro) --1

self.getVendaCorrente().adicionaItem (liv,quantidade) --2

FIM MÉTODO

FIM CLASSE

```

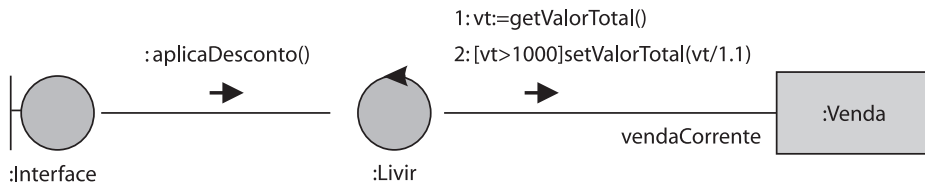
CLASSE Venda
...
MÉTODO adicionaItem(liv:Livro, quantidade: Inteiro) --2
    VAR it:Item
    it:=Item.newInstance() --2.1
    self.addItens(it) --2.2
    it.addLivro(liv) --2.3
    it.setQuantidade(quantidade) --2.4
FIM MÉTODO
FIM CLASSE

```

Figura 12.11: Implementação de uma operação de sistema e um método delegado.

Na figura foram omitidas as declarações de atributos, associações e métodos básicos relacionados. Apenas a operação de sistema e o método delegado foram mostrados.

A Figura 12.12 mostra a implementação de uma operação de sistema com mensagem condicionada.



```

CLASSE Livir
...
MÉTODO aplicaDesconto()
    VAR vt:MOEDA
    vt := self.getValorTotal() --1
    SE vt>1000 ENTÃO
        self.getVendaCorrente().setValorTotal (vt/1.1) --2
    FIM SE
FIM MÉTODO
FIM CLASSE

```

Figura 12.12: Implementação de uma operação de sistema com mensagem condicionada.

Finalmente, a Figura 12.13 mostra como seria a implementação de uma operação de sistema com mensagens iterativas.

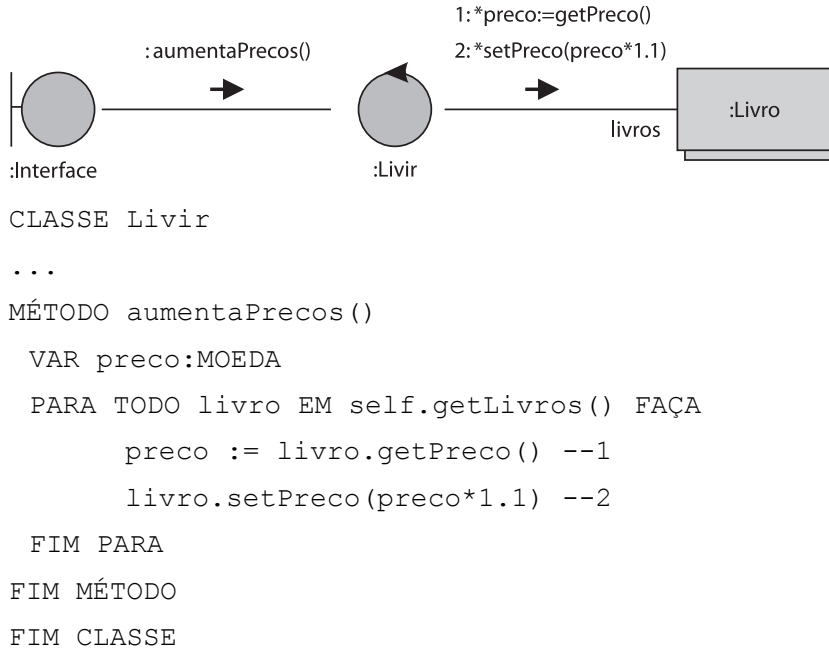


Figura 12.13: Implementação de uma operação de sistema com mensagem condicionada.

O procedimento de geração de código, então, pode ser implementado assim:

- geração de código para as classes, atributos e associações, conforme definições deste capítulo;
- geração de código para as operações básicas, também segundo os padrões descritos neste capítulo;
- geração de código para as operações e consultas de sistema e métodos delegados de acordo com os diagramas de comunicação definidos na atividade de projeto ou seus equivalentes diagramas de sequência ou algoritmos.

12.5. Testes

Por melhores que sejam as técnicas de projeto e por mais ferramentas de geração de código que se tenha, o teste do software continuará a ser sempre uma necessidade devido ao fator “erro humano”. Especificações malfeitas, es-

quecimentos, inconsistências podem fazer o software falhar. Cabe ao analista de testes prover as ferramentas para solucionar tais problemas.

Foge ao escopo deste livro apresentar um método completo de teste de software. Existem trabalhos específicos sobre esse assunto, como o livro de Maldonado, Delamaro e Jino (2007). A intenção desta seção é mostrar como o teste de software pode ser concebido quando um conjunto de técnicas como as apresentadas aqui forem usadas.

Embora o teste fosse relegado a segundo plano até os anos 1990, quando o plano de software normalmente se resumia a chamar um programador e dizer “testa aí!”, o assunto hoje se reveste de grande importância devido a fatores como qualidade do produto de software, cláusulas contratuais que punem erros e funcionalidades incorretas e a tendência da grande indústria de software a realizar testes independentes.

Sem entrar no mérito de definir famílias técnicas, como testes caixa-branca ou caixa-preta, pode-se classificar os testes de software em relação aos seus *objetivos* nos seguintes grupos:

- a) teste de unidade • verificar o funcionamento dos métodos implementados;
- b) teste de integração • verificar se a comunicação entre objetos funciona;
- c) teste de sistema • verificar execução do sistema do ponto de vista do usuário final;
- d) teste de aceitação • teste conduzido pelos usuários finais;
- e) teste de operação • teste conduzido pelos administradores do sistema no ambiente final;
- f) teste de regressão • aplicado quando novas versões do software são liberadas.

12.5.1. Teste de Unidade

O teste de unidade tem como objetivo verificar o funcionamento dos métodos mais básicos. Já foi mostrado que a camada de domínio do software (que realiza toda a lógica de acesso e transformação de dados) é projetada por diagramas de comunicação nos quais as mensagens se estruturam como em uma árvore, estando a operação de sistema na raiz, os métodos delegados nos ramos e as operações básicas nas folhas.

Como os métodos delegados implementam a comunicação entre objetos, o teste de unidade deverá apenas verificar se as operações básicas realizam o que devem realizar.

Existem poucos tipos de operações básicas e relativamente poucas variantes. Como todas são bastante simples e implementadas de maneira padrão, o teste de unidade resume-se a testar cada nova variante quando for definida pela primeira vez. A partir daí, caso sejam usados mecanismos de geração automática de código, não será mais necessário fazer o teste de unidade, pois geradores de código não costumam cometer erros humanos.

Então, a ideia aqui é testar cada padrão da primeira vez que ele for definido e, a partir daí, gerar os métodos apenas com o gerador de código, evitando editá-los manualmente.

12.5.2. Teste de Integração

Conforme mencionado, o objetivo do teste de integração é verificar se os objetos se comunicam adequadamente. Isso pode ser feito de forma modular e sistemática. Estando os métodos básicos resolvidos cabe verificar se os métodos delegados e operações de sistema funcionam conforme esperado.

Sugere-se organizar os testes dessa fase a partir dos contratos de operação e consulta de sistema. Pode-se aplicar testes à implementação de cada operação e consulta de sistema e verificar se o contrato especificado para elas é obedecido, isto é, se cada uma das pós-condições é obtida.

Assim, os contratos de operação e consulta de sistema serão uma excelente base para a preparação do conjunto de testes de integração. A validação dos métodos delegados intermediários segue como subproduto dos testes das operações e consultas de sistema, já que estão sempre subordinados a estes.

A sistematização oferecida pela orientação a objetos é bastante útil para organizar testes que efetivamente cubram toda a funcionalidade observável do sistema, o que é exatamente o caso do conjunto das operações e consultas de sistema.

12.5.3. Teste de Sistema

O teste de sistema tem como objetivo avaliar o sistema do ponto de vista do usuário. Então, se até esse ponto não foram avaliadas, por exemplo, as in-

interfaces, já que as operações e consultas de sistema são chamadas por elas, o teste de sistema vai incluir não só a avaliação das interfaces como também do encadeamento das operações e consultas de sistema nos casos de uso.

Um teste de sistema em um sistema orientado a objetos desenvolvido de acordo com as técnicas apresentadas neste livro vai consistir em um estudo sistemático dos casos de uso verificando se um usuário conseguiria executar, sem provocar erros, o fluxo principal e todos os fluxos alternativos de cada caso de uso.

Novamente, o que se tem é um procedimento sistemático para programar e executar baterias de teste que vão avaliar, entre outras coisas:

- a) a qualidade da interface gráfica;
- b) o correto encadeamento das operações e consultas de sistema de acordo com a lógica do caso de uso;
- c) se as precondições de cada operação e consulta de sistema são atendidas no projeto. Deve-se programar os testes de forma a verificar se é possível fazer alguma precondição falhar ao longo do fluxo de execução.

O teste de sistema é feito do ponto de vista do usuário, mas não necessariamente pelo usuário, devendo ser conduzido preferencialmente por uma equipe de analistas devidamente preparados nas técnicas de teste, especialmente, nesse caso, das técnicas de teste caixa-preta.

12.5.4. Testes de Aceitação, Operação e Regressão

Os *testes de aceitação* são feitos pelos usuários para verificar se o sistema atende aos requisitos solicitados. Se os casos de uso tiverem sido validados pelos usuários, esses testes podem seguir a mesma lógica dos testes de sistema.

Os usuários podem receber um documento que consiste nas versões reais dos casos de uso no qual terão a lista completa de todas as funcionalidades do sistema e dos possíveis fluxos normais e alternativos de trabalho. Esse documento é a base para o manual de operação do sistema e pode não só ser usado como base para que os usuários organizem suas atividades de teste como também para aprimorar o documento em si, lançando luz sobre pontos obscuros.

Os *testes de operação* são conduzidos no ambiente final, com dados finais, e podem também ser organizados a partir dos casos de uso, even-

tualmente com uma atenção maior aos requisitos não funcionais associados, como segurança e *performance* esperada.

Já os *testes de regressão* consistem em testar novamente funcionalidades que foram alteradas em uma nova versão do sistema. Pode ser necessário, portanto, repetir os testes de unidade, integração, sistema etc. para determinadas funcionalidades do sistema.

Conforme mencionado, esta seção apenas orientou a forma como as atividades de teste podem ser organizadas quando se utilizam as técnicas deste livro. Mais detalhes sobre como executar as atividades de teste devem ser buscados em livros específicos sobre testes ou em livros de Engenharia de Software.

Apêndice: Sumário OCL

Apenas expressões e comandos usados neste livro são apresentados. Para uma definição mais completa de OCL sugere-se consultar Warmer e Keppe (1998) ou OMG (2009). Há também um bom guia de referência rápida em http://www.eoinwoods.info/doc/ocl_quick_reference.pdf.

.	1. Referencia um atributo (à direita) de um objeto (à esquerda). 2. Referencia uma coleção de objetos associados por um papel (à direita) a outro objeto. 3. Referencia o retorno de um método (à direita) enviado a um objeto (à esquerda). <i>Obs.</i> Quando aplicado a uma coleção de objetos (à esquerda), referencia uma coleção da aplicação do mesmo operador a cada um dos objetos. Exemplos: <pre>pessoa.idade --atributo pessoa.automoveis --associação pessoa.getEndereco() --método compradores.nome --aplicado a uma coleção</pre>
---	--

::	1. Indica que um método (à direita) está implementado em uma classe (à esquerda). 2. Indica que um valor (à direita) pertence a uma enumeração (à esquerda). 3. Indica envio de uma mensagem a uma classe. Exemplo: <pre>Venda::getValorTotal():Moeda -- método EstadoPagto::pendente -- enumeração Livro::newInstance() -- método de classe</pre>
→	Indica que a mensagem (à direita) é enviada a uma coleção (à esquerda). Exemplo: <pre>clientes→size()</pre>
^	A expressão é verdadeira se a mensagem indicada à direita com seus parâmetros foi enviada ao objeto ou coleção denotado pela expressão à esquerda. Usada especialmente em pós-condições para indicar que uma mensagem foi enviada a um objeto. Exemplo: <pre>pessoa^setData(novaData)</pre>
[]	Notação para acessar um elemento diretamente em uma associação qualificada. Exemplo: <pre>compradores[cpf]</pre>
@pre	Usada em pós-condições de operações para indicar o valor de um atributo, objeto ou associação antes de a operação ter sido executada porque por <i>default</i> qualquer valor referenciado em uma pós-condição é posterior à execução da operação. Exemplo: <pre>post: if self.saldo@pre = 0 then self^setSaldo(1) endIf</pre>
AND	Conector de duas expressões lógicas. A expressão resultante é verdadeira se as expressões à direita e à esquerda são verdadeiras. Exemplo: <pre>x=1 AND y<3</pre>
body:	Indica que a expressão à direita é a definição (retorno) de uma consulta (método) do contexto definido à esquerda. Exemplo: <pre>Context Livir::saldoCompradorCorrente():Moeda body: compradorCorrente.saldo</pre>

collect:	Retorna uma coleção cujos elementos consistem na avaliação da expressão entre parênteses aplicada a cada elemento da coleção original (à esquerda). Em algumas situações pode ser substituída pela notação ".". Exemplo: <pre>compradores→collect(c Tuple { cpf = c.cpf, nome = c.nome, telefone = c.telefone }</pre>
Context	Indica o contexto de uma expressão: classe, método, associação ou atributo. Exemplos: <pre>Context Venda -- classe Context Venda::getValorTotal():Moeda -- método Context Pessoa::nome -- atributo Context Venda::itens -- associação</pre>
def:	Usado para definir um termo que passa a valer como resultado de uma expressão. Exemplo: <pre>def: comprador = compradores[cpfComprador]</pre>
derive:	Usado para definir um atributo derivado. À esquerda, deve constar o atributo como contexto e, à direita, uma expressão. Exemplo: <pre>Context Produto::lucroBruto:Moeda derive: precoVenda - precoCompra</pre>
exception:	Indica que a expressão a seguir é avaliada se ocorrer uma exceção durante a execução de um método definido no contexto à esquerda: <pre>Context Livir::identificaComprador(umCpf) def: comprador = compradores→select(cpf = umCpf) post: self^addCompradorCorrente(comprador) exception: comprador→size() = 0 IMPLIES self^throw("cpf inválido")</pre>
exists()	Retorna <i>true</i> se a coleção (à esquerda) possuir pelo menos um elemento para o qual a expressão entre parênteses é verdadeira. Exemplo: <pre>compradores→exists(saldo = 0)</pre>

<code>forall()</code>	No contexto de uma invariante ou pós-condição, indica que a expressão entre parênteses é verdadeira para todos os elementos da coleção à esquerda. Exemplo: <pre>Context Aluno inv: self.disciplinas → forall(d d.cursos → includes(self.curso))</pre>
<code>if then else endif</code>	Se a condição após o <code>if</code> for verdadeira, a expressão como um todo vale a expressão entre o <code>then</code> e o <code>else</code>. Caso contrário, a expressão como um todo consiste na avaliação da expressão entre o <code>else</code> e o <code>endif</code>.
<code>IMPLIES</code>	Conector de duas expressões lógicas. A expressão resultante é verdadeira se a primeira for falsa ou ambas verdadeiras. Pode ser substituído por uma estrutura <code>if...then...endif</code>. Exemplo: <pre>x=1 IMPLIES y<3</pre>
<code>includes()</code>	Mensagem enviada a uma coleção. Retorna <i>true</i> se o parâmetro pertence ao conjunto e <i>false</i> caso contrário. Exemplo: <pre>clientes → includes(joao)</pre>
<code>init:</code>	Usado para definir um valor inicial para um atributo. À esquerda, deve constar o atributo como contexto e, à direita, uma expressão. Exemplo: <pre>Context Venda::valorTotal:Moeda init: 0.0</pre>
<code>inv:</code>	Indica que a expressão à direita é uma invariante para a classe que aparece como contexto (à esquerda). Exemplo: <pre>Context Transacao inv: self.movimentos.valor → sum() = 0</pre>
<code>isEmpty()</code>	Retorna <i>true</i> se a coleção à esquerda é vazia e <i>false</i> caso contrário. Exemplo: <pre>clientes → isEmpty()</pre>
<code>isNull()</code>	Retorna <i>true</i> se a expressão à esquerda é indefinida e <i>false</i> caso contrário. Exemplo: <pre>self.liquidacao.isNull()</pre>
<code>notEmpty()</code>	Retorna <i>true</i> se a coleção (à esquerda) for vazia e <i>false</i> caso contrário. Exemplo: <pre>compradores → notEmpty()</pre>
<code>OR</code>	Conector de duas expressões lógicas. A expressão resultante é verdadeira se pelo menos uma das expressões à direita ou à esquerda é verdadeira. Exemplo: <pre>x=1 OR y<3</pre>

post:	Indica que a expressão à direita é uma pós-condição para o método indicado no contexto à esquerda. Exemplo: <pre>Context Livir::criaLivro(umIsbn, umTitulo, umAutor) def: novoLivro = Livro::newInstance() post: self^addLivro(novoLivro) AND novoLivro^setIsbn(umIsbn) AND novoLivro^setTitulo(umTitulo) AND novoLivro^setAutor(umAutor)</pre>
pre:	Indica que a expressão à direita é uma pré-condição para o método indicado no contexto à esquerda. Exemplo: <pre>Context Livir::operacaoQualquer() pre: compradorCorrente→notEmpty()</pre>
return:	Pode ser usado em operações de sistema quando se deseja que retornem algum valor. Exemplo: <pre>Context Livir::criaCompra(idComprador):LongInt def: novaCompra = Compra::newInstance() def: comprador = compradores[idComprador] post: novaCompra^setNumero(novoNumeroAutomatico()) AND novaCompra^setData(dataAtual()) AND novaCompra^addComprador(comprador) AND return: novaCompra.numero()</pre>
select()	Mensagem enviada a uma coleção (à esquerda). Retorna uma coleção com os elementos para os quais a expressão entre parênteses é verdadeira. Exemplo: <pre>pessoas→select(idade>18)</pre>
self	Denota uma instância da classe do contexto. Se o contexto for uma associação, método ou atributo, então é a instância da classe à qual a associação, método ou atributo pertencem.
size()	Retorna o número de elementos da coleção à esquerda. Exemplo: <pre>livros→size()</pre>

<code>sum()</code>	Mensagem aplicável apenas a coleções de valores numéricos. Retorna o somatório dos elementos. Pode ser aplicada diretamente a uma coleção de números (sem parâmetros) ou a uma coleção de objetos (com um parâmetro que indica como obter valores numéricos a partir da coleção de objetos). Exemplos: <pre>self.movimentos.valor→sum() self.movimentos→sum(valor)</pre>
<code>Tuple{}</code>	Construtor de tuplas. Entre as chaves devem aparecer as definições de campos separadas por vírgula. Cada definição de campo tem um nome, um sinal de igual e um valor. Exemplo: <pre>Tuple{ nome = compradores[cpfComprador].nome, telefone = compradores[cpfComprador].telefone }</pre>

Bibliografia

Adams, D.N. *O guia do mochileiro das galáxias*. Sextante, 2004. (Tradução de *The hitchhiker's guide to the galaxy*, Completely Unexpected Productions Ltd., 1979.)

Adams, D.N. *Vida, universo e tudo o mais*. Sextante, 2005. (Tradução de *Life, universe and everything*, Completely Unexpected Productions Ltd., 1982.)

Alford, M. *Requirements-driven software design*. McGraw Hill, 1991.

Ambler, S. *Process patterns*. Cambridge University Press, 1998.

Ambler, S., Constantine, L., Smith, R. *The Unified Process elaboration phase: best practices in implementing the UP*. CMP Books, 2000.

Arlow, J., Neustadt, I. *UML and the Unified Process: practical object-oriented analysis and design*. Pearson Education, 2001.

Beck, K. *Programação extrema – XP explicada: acolha as mudanças*. Porto Alegre: Bookman, 2004. (Tradução de *Extreme programming explained: embrace change*.)

Bezerra, E. *Princípios de análise e projeto de sistemas com UML*. Campus–Elsevier, 2003.

Boehm, B.W. Software engineering. *IEEE. Transactions on Computers*, vol. 25, nº 12, 1976.

- Booch, G. *Object-oriented analysis and design with applications*. Addison-Wesley, 1993.
- Booch, G. *Object solutions – managing the object-oriented project*. Addison-Wesley, 1996.
- Brown, A.W. *Large-scale component-based development*. Prentice-Hall, 2000.
- Ceri, S., Fraternali, P., Bongio, A., Brambilla, M., Comai, S., Matera, M. *Designing data-intensive Web applications*. Morgan Kaufmann Publishers, 2003.
- Cox, B. *Object-oriented programming: an evolutionary approach*. Addison-Wesley, 1986.
- D'Souza, D.F., Wils A.C. *Objects, components, and frameworks with UML*. Addison-Wesley, 1999.
- Date, C.J. *An introduction to database systems*. Addison-Wesley, 1982.
- Emam, K., Drouin, J.N., Melo, W. *Spice: the theory and practice of software process improvement and capability determination*. IEEE Computer Society, 1998.
- Embley, D.W., Kurtz, B.D., Woodfield, S.N. *Object-oriented systems analysis: a model-driven approach*. Prentice-Hall, 1992.
- Erickson, H.E., Penker, M. *UML toolkit*. John Wiley and Sons Inc., 1998.
- Fayad, M.E., Schmidt, D.C., Johnson, R.E. *Implementing application frameworks*. John Wiley & Sons, Inc., 1999.
- Fowler, M., Scott, K. *UML distilled*. Addison-Wesley, 1997.
- Fowler, M. *Patterns of enterprise application architecture*. Addison-Wesley, 2003.
- Gamma, E., Helm, R., Johnson, R., Vlissides, J. *Design patterns. Elements of reusable object-oriented software*. Addison-Wesley, 1995.
- Garmus, D., Herron, D. *Function point analysis: measurement practices for successful software projects*. Addison-Wesley Information Technology Series, 2000.
- Goldberg, A., Robson, D. *Smalltalk 80: the language*. Addison-Wesley Pub Co., 1989.
- Jacobson, I., Christerson, M., Jonsson, P., Övergaard, G. *Object-oriented software engineering – a use CASE driven approach*. Addison-Wesley, 1992.
- Jacobson, I. *The object advantage – business process reengineering with object technology*. Addison-Wesley, 1994.
- Jacobson, I., Booch, G., Rumbaugh, J. *The unified software development process*. Addison-Wesley, 1999.

- Kehoe, R., Jarvis, A., Shah-Jarvis, A. *Iso 9000-3: a tool for software product and process improvement*. Springer Verlag, 1996.
- Larman, C. *Applying UML and patterns: an introduction to object-oriented analysis and design and the unified process*. Prentice Hall, 2001.
- Lieberherr, Karl, Holland, I. Assuring good style for object-oriented programs. *IEEE Software*: 38–48, September 1989.
- Karner, G. *Use CASE points – resource estimation for objectory projects*. Objective Systems, 1993.
- Kruchten, P. *The rational unified process: an introduction*. Addison-Wesley, 2000.
- Kruchten, P. *The rational unified process made easy: a practitioner's guide to rational unified process*. Addison-Wesley, 2003.
- Maldonado, J.C., Delamaro, M.E., Jino, M. *Introdução ao teste de software*. Campus-Elsevier, 2007.
- Martin, R.C. *Agile software development, principles, patterns, and practices*. Prentice-Hall, 2002.
- Meyer, B. *Object-oriented software construction*. Prentice Hall, 1988.
- Meyer, B. *Eiffel: the language*. Prentice-Hall, 1992.
- Mitchel, R., McKim, J. *Design by contract by example*. Addison-Wesley, 2001.
- Object Management Group (OMG) *Object Constraint Language* OMG available specification version 2.0. Disponível em <http://www.omg.org/technology/documents/formal/ocl.htm>. Consultado em 26 de agosto de 2009.
- Object Management Group, *OMG Unified Modeling Language – UML*. Disponível em http://www.omg.org/technology/documents/modeling_spec_catalog.htm#UML. Consultado em 23 de setembro de 2009.
- Page-Jones, M. *Fundamentos do desenho orientado a objeto com UML*. Makron Books, 2001. (Tradução de *Fundamentals of object-oriented design in UML*.)
- Paula Filho, W.P. *Engenharia de software: fundamentos, métodos e padrões*. LTC, 2003).
- Pereira e Silva, R. *UML 2 – Modelagem orientada a objetos*. Visual Books, 2007.
- Pereira e Silva, R. *Como modelar com UML 2*. Visual Books, 2009.
- Pressman, R.S. *Software engineering: a practitioner's approach*. McGraw Hill, 2010.
- Riel, A. J. *Object-oriented design heuristics*. Addison-Wesley, 1996.

Rocha, A.R.C., Maldonado, J.C., Weber, K.C. *Qualidade de software: teoria e prática*. Pearson–Prentice-Hall, 2001.

Royce, W. Managing the development of large software Systems. *Proceedings of IEEE WESCON*, 1970.

Rumbaugh, J., Blaha, M.R., Lorensen, W., Eddy, F., Premerlani, W. *Object-oriented modeling and design*. Prentice Hall, 1990.

Rumbaugh, J., Jacobson, I., Booch, G. *The Unified Modeling Language reference manual*. Addison-Wesley, 1999.

Santos, C.C. *Geração automática de diagramas de comunicação a partir de contratos OCL*. Dissertação de Mestrado, UFSC-PPGCC, 2007.

Scott, K. *The Unified Process explained*. Addison-Wesley Pub Co., 2001 (O processo unificado explicado. Bookman, 2003.)

Shalloway, A., Trott, J.R. *Explicando padrões de projeto: uma nova perspectiva em projeto orientado a objeto*. Bookman. (Tradução de *Design patterns explained: a new perspective on object-oriented design*.)

Silva, A.A., Gomide, C.F., Petrillo, F. *Metodologia e projeto de software orientados a objetos: modelando, projetando e desenvolvendo sistemas com UML e componentes distribuídos*. Érica, 2003.

Warmer, J., Keppe, A. *The Object Constraint Language: precise modeling with UML*. Addison-Wesley Pub Co., 1998.

Wirfs-Brock, R., McKean, A. *Object design: roles, responsibilities, and collaborations*. Addison-Wesley, 2002.

Índice Remissivo

A

ações corretivas, 58, 59

agregação, 115, 116, 210, 216

alfanumérico, 70, 91, 94, 96, 97, 100, 105, 109, 214, 222, 223, 292

alteração, operação de, 65, 66, 141, 154, 173, 174, 175, 192, 198, 215, 232, 261, 262, 292

análise de domínio, 5, 43, 89, 90, 196

análise de requisitos, 5, 21, 25, 29, 43

arquitetura do sistema, 5, 24, 282

associação histórica, 146, 147

associação ordenada, 99, 202

associação ternária, 116, 117

associação unidirecional, 107, 281, 293, 294, 295, 296, 298

associações, 82, 86, 92, 93, 96, 98, 102, 102, 103, 104, 105, 107, 108, 109, 110, 115, 116, 117, 118, 119, 120, 121, 125, 126, 127, 129, 130, 131, 133, 138, 141, 144, 147, 149, 150, 154, 159, 163, 165, 166, 167, 168, 170, 174, 176, 177, 179, 187, 194, 196, 197, 198, 199,

201, 202, 204, 206, 209, 215, 216, 224, 231, 232, 233, 234, 236, 271, 272, 173, 274, 275, 276, 278, 279, 280, 283, 284, 285, 286, 287, 292, 293, 295, 298, 299, 300, 304, 306, 307

associações derivadas, 108, 236, 193

associações temporárias, 82, 159, 179, 187, 279, 293

ator, 11, 12, 15, 35, 37, 38, 40, 41, 4, 46, 47, 48, 49, 50, 51, 52, 55, 59, 63, 64, 68, 69, 74, 75, 76, 77, 78, 79, 80, 83, 86, 102

atributo, 31, 86, 87, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100, 101, 102, 103, 105, 108, 109, 113, 114, 116, 118, 119, 120, 121, 122, 124, 125, 126, 127, 128, 129, 130, 131, 132, 133, 134, 135, 136, 137, 141, 142, 144, 145, 148, 149, 154, 155, 159, 162, 163, 164, 165, 166, 167, 169, 170, 171, 173, 174, 175, 177, 178, 179, 197, 198, 199, 209, 214, 215, 222, 223, 224, 225, 226, 227, 229, 232, 233, 236, 237, 238, 239, 240, 241, 242, 246, 247, 252, 261, 262, 264, 267, 268, 270, 271, 276, 278, 280, 286, 287, 288, 291, 292, 294, 296, 304, 306, 307, 313, 314, 315, 316, 317

atributo com valor inicial, 94, 108, 169
 atributo derivado, 94, 95, 108, 109, 128, 142, 145, 165, 166, 169, 198, 199, 222, 223, 229, 267, 268, 315
 atributo privado, 232
 atributo temporário, 159
 atributo tipado, 93, 94, 95, 96
 atualizar, 39

B

baixa coesão, 127, 132, 133, 134
 banco de dados, 12, 23, 26, 47, 53, 69, 80, 87, 91, 97, 98, 140, 143, 144, 151, 270, 276, 278, 282, 285, 286, 187, 288, 289
 branch, 13, 14, 15

C

cadastro, 25, 55, 56, 58, 59, 62, 65, 76, 99, 113, 118, 122, 154, 156, 174, 252, 257
 camada de aplicação, 74, 288
 camada de domínio, 76, 77, 86, 193, 194, 195, 197, 199, 201, 203, 205, 207, 209, 211, 213, 215, 217, 219, 221, 223, 225, 227, 229, 231, 233, 234, 269, 270, 271, 282, 291, 293, 308
 caminho, 12, 13, 14, 15, 44, 45, 55, 65, 99, 133, 170
 CASE, 3, 27, 37, 76, 84, 170, 220, 320, 321
 caso de uso, 2, 4, 5, 710, 21, 22, 30, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 67, 68, 69, 70, 71, 73, 74, 75, 76, 77, 81, 83, 84, 85, 91, 99, 100, 101, 102, 103, 105, 159, 173, 179, 180, 193, 235, 268, 310
 caso de uso de alto nível, 5, 7, 35, 37, 39, 41, 43
 caso de uso essencial, 41, 44, 49, 53
 caso de uso expandido, 30, 45, 46, 47, 54, 63, 68, 71, 75, 76, 173, 179
 cenário, 63, 74
 centrado na arquitetura, 5, 40

chave candidata, 98, 278
 chave estrangeira, 105, 274, 281
 chave primária, 97, 271, 273, 274, 275, 277, 278, 279, 282
 ciclos iterativos, 5, 22, 35
 classe, 3, 4, 5, 7, 21, 24, 63, 68, 69, 76, 78, 86, 87, 90, 91, 92, 93, 94, 95, 96, 98, 99, 101, 103, 104, 109, 110, 113, 114, 116, 118, 119, 120, 121, 122, 123, 124, 125, 127, 128, 130, 132, 133, 134, 135, 136, 138, 139, 142, 144, 145, 147, 148, 149, 157, 159, 163, 164, 165, 166, 167, 169, 173, 174, 177, 178, 186, 193, 194, 195, 196, 197, 198, 199, 200, 201, 202, 203, 204, 205, 206, 208, 210, 214, 215, 223, 224, 225, 226, 227, 228, 229, 230, 231, 232, 233, 234, 235, 236, 237, 238, 239, 240, 241, 242, 243, 245, 246, 252, 261, 262, 270, 271, 272, 273, 274, 276, 277, 278, 279, 280, 281, 282, 283, 285, 286, 287, 291, 292, 293, 294, 295, 296, 297, 298, 299, 300, 301, 302, 303, 304, 305, 306, 307, 314, 315, 316, 317
 classe abstrata, 120, 130, 138
 classe de associação, 122, 123, 124, 142, 186, 200, 204, 205, 278, 298, 299
 classe de especificação, 134, 135
 classe estereotipada, 87, 95, 98,
 classe modal, 124
 coesão, 80, 105, 127, 132, 133, 134, 148
 coleção, 110, 101, 178, 201, 220, 221, 237, 285, 293, 296, 313, 314, 315, 316, 317, 318
 commit, 287, 288, 289
 composição, 115, 116, 139, 210, 216, 236, 243, 280
 conceito, 2, 17, 25, 39, 46, 68, 73, 86, 91, 92, 93, 95, 96, 97, 98, 99, 100, 101, 102, 103, 104, 105, 107, 110, 113, 115, 116, 118, 119, 122, 123, 124, 125, 126, 127, 128, 129, 130, 131, 132, 133, 134, 136, 139, 140, 141, 144, 154, 163, 178, 198, 208, 235, 243, 244, 256, 276
 concentração, 227, 228

concepção, fase de, 5, 6, 7, 9, 10, 21, 22, 32, 35, 37, 43, 70, 89, 129

condição de guarda, 17, 19, 218

conjunto, 8, 11, 16, 17, 20, 26, 27, 29, 31, 33, 37, 40, 59, 63, 74, 78, 86, 92, 94, 95, 98, 108, 109, 110, 111, 112, 123, 114, 115, 120, 129, 132, 142, 143, 145, 147, 151, 155, 157, 159, 161, 162, 164, 166, 168, 169, 171, 178, 179, 182, 193, 197, 200, 201, 202, 203, 204, 209, 213, 214, 215, 222, 224, 238, 246, 248, 249, 253, 254, 257, 261, 264, 268, 270, 274, 275, 276, 280, 282, 285, 293, 295, 298, 303, 308, 309, 316

conjunto ordenado, 111, 274

connect unit, 262, 264

construção, fase de, 5, 7, 153, 234, 291

consulta, 2, 4, 6, 7, 28, 30, 32, 36, 38, 39, 43, 49, 50, 65, 63, 67, 73, 74, 76, 77, 78, 79, 80, 83, 85, 89, 91, 95, 109, 147, 150, 153, 154, 155, 156, 157, 158, 159, 160, 173, 174, 178, 179, 180, 182, 183, 184, 186, 187, 188, 189, 190, 192, 193, 194, 196, 197, 199, 200, 201, 205, 207, 213, 114, 115, 121, 222, 223, 224, 225, 226, 227, 229, 231, 233, 234, 247, 268, 270, 283, 284, 285, 287, 291, 292, 296, 303, 307, 309, 310, 313, 314, 321

consulta de sistema, 2, 7, 43, 49, 50, 73, 74, 76, 77, 78, 79, 80, 89, 91, 153, 154, 155, 160, 173, 180, 182, 187, 188, 193, 194, 196, 205, 221, 222, 223, 227, 223, 231, 234, 268, 291, 307, 309, 310

Conta/Transação, 144

Context, 94, 95, 109, 128, 145, 149, 150, 151, 155, 158, 160, 161, 162, 163, 166, 168, 171, 172, 174, 175, 176, 177, 179, 181, 182, 183, 184, 185, 186, 188, 189, 190, 191, 192, 205, 210, 212, 213, 214, 216, 217, 219, 220, 221, 227, 229, 280, 314, 315, 316, 317

contexto, 6, 24, 44, 52, 94, 109, 110, 151, 155, 160, 162, 169, 180, 205, 230, 294, 314, 315, 316, 317

contrato, 2, 7, 10, 31, 32, 43, 44, 48, 76, 84, 89, 141, 153, 154, 155, 156, 157, 159, 160, 161, 162, 163, 165, 166, 167, 168, 169, 170, 171, 172, 173, 174, 175, 176, 177, 178, 179, 181, 183, 185, 186, 187, 188, 189, 191, 193, 194, 196, 200, 209, 211, 212, 213, 214, 216, 217, 219, 220, 221, 222, 223, 226, 227, 231, 234, 265, 268, 286, 287, 309, 322,

controladora de sistema, 76, 98, 99, 170, 209, 305

controladora-fachada. Consulte controladora de sistema

copiar e substituir, 141, 142, 143

copy and replace. Consulte copiar e substituir

create, 39, 80, 85, 174, 233, 262, 263, 206, 304, Criador, 209, 210, 216

CRUD, 39, 40, 62, 64, 65, 66, 67, 80, 84, 85, 99, 173, 178, 180, 222, 265

D

Data Transfer Object. Consulte DTO

data unit, 237, 238, 239, 240, 241, 242, 247, 250, 251, 253, 255, 258, 259, 260, 261, 263, 267, 268

DCP, 7, 90, 194, 195, 200, 231, 232, 233, 234, 236, 237, 258, 270, 291, 292

delegação, 3, 7, 226, 227, 229, 231,

delete, 39, 175, 262, 264, 287, 288, 289

delete unit, 262, 264

dependências, 12, 13, 37, 195, 250, 251

derive, 95, 109, 128, 166, 229, 315

destroy, 198, 176, 177, 216, 233, 304

diagrama de atividades, 11, 12, 15, 20

Diagrama de Classes de Projeto. Consulte DCP

diagrama de comunicação, 195, 200, 201, 203, 209, 211, 212, 213, 215, 217, 218, 219, 220, 221, 222, 230, 305

diagrama de máquina de estados, 15, 17, 18, 19, 20, 129
 diagrama de requisitos, 23, 27
 diagrama de sequência, 73, 74, 75, 76, 77, 79, 80, 81, 82, 83, 84, 85, 86, 180, 218, 222, 265, 266, 267, 268
 dirigido por casos de uso, 4, 40
 disconnect unit, 262, 264
 documento de requisitos, 5, 21, 23, 26, 27, 43, 54, 60, 150
 domínio da solução, 23, 90
 DTO, 81, 86, 87, 160, 222

E

efeito colateral, 39, 79
 elaboração, fase de, 5, 7, 21, 22, 26, 40, 43, 70, 89, 193
 empacotamento, 33
 entry unit, 237, 247, 248, 249, 253, 259, 260, 263, 265, 267
 enumeração, 95, 96, 114, 121, 125, 128, 136, 314
 erro, 32, 38, 40, 44, 83, 116, 125, 127, 140, 150, 169, 172, 196, 182, 307, 308, 309, 310, 320
 especialização, 119
 Essência/Aparência, 142, 143
 estado, 4, 7, 15, 16, 17, 18, 19, 20, 21, 38, 53, 103, 118, 124, 125, 126, 127, 128, 129, 130, 131, 133, 134, 139, 156, 164, 169, 170, 172, 218, 232, 287, 314
 estados paralelos, 18, 19
 estilo de escrita, 55
 Estratégia, 24, 80, 81, 82, 83, 137, 138, 139, 141, 142, 159, 180, 186, 187, 192, 230, 232
 estrutura de seleção, 13
 estruturas de dados, 87, 92, 94, 132, 282, 284, 286, 292
 evento, 15, 16, 17, 43, 49, 50, 51, 53, 55, 56, 57, 58, 73, 74, 75, 77, 78, 79, 83, 142, 154

evento de sistema, 49, 50, 55, 58, 73, 75, 77, 78, 79, 83
 exceção, 44, 46, 55, 56, 57, 58, 59, 60, 61, 62, 63, 65, 66, 67, 68, 70, 79, 83, 84, 85, 86, 101, 154, 171, 172, 174, 175, 176, 177, 182, 183, 186, 263, 265, 315
 exceções genéricas, 59
 exclusão, 65, 66, 175, 176, 177, 261, 262
 expansão, 7, 43, 64,
 Extreme Programming, 1, 319

F

façade controller. Consulte controladora
 falha, 216, 32, 140, 171, 193, 262, 263, 265, 308, 310
 filtro, 182, 223, 224, 225, 226, 266
 fluxo, 12, 46, 60
 fluxo principal, 46, 60
 fluxo principal, variantes do, 60, 61
 fork, 13, 14, 15, 18, 19,
 fragmento, 2, 39, 84, 210, 231
 fronteira do sistema, 37, 41, 53
 funções, 21, 22, 25, 26, 27, 30, 31, 32, 37, 37, 65, 69, 74, 74, 182, 222

G

garantia de parâmetros, 156, 15
 generalização, 118, 119, 120, 121
 gerenciar, 10, 23, 24, 32, 39, 62, 65, 67, 84, 90, 638, 240, 287
 get, 87, 147, 148, 197, 198, 199, 213, 214, 215, 22, 225, 226, 228, 230, 233, 292, 293, 294, 295, 296, 297, 298, 299, 300, 301, 302, 303, 304, 305, 306, 307, 313, 314, 315

H

herança, 63, 118, 119, 120, 121, 123, 126, 127, 280, 201
 Hierarchical, 242, 243, 244, 245, 246

hierarquia organizacional, 139, 245

I

identificador, 113

implementação, 5, 6, 31, 33, 69, 76, 82, 86, 95

inception. Consulte concepção

includes, 117, 148, 150, 232, 275, 277, 281, 284, 285, 293, 295, 6, 298, 299, 300, 301, 302, 303, 304, 305, 306, 307, 309

inclusão, 55, 65, 66, 141

index unit, 237, 242, 242, 244, 245, 246, 250, 251, 253, 258, 259, 260, 266, 267

Índice Filtrado, 259, 267

init, 94, 165, 169, 316

inserção, 62, 174, 286

interativo, 38, 40

interessados, 41, 68, 69

interface, 2, 7, 21, 26, 27, 28, 30, 31, 32, 33, 41, 44, 45, 49, 50, 53, 56, 62, 64, 68, 69, 73, 74, 75, 76, 77, 78, 79, 80, 83, 85, 86, 87, 91, 140, 143, 158, 172, 178, 193, 194, 209, 211, 234, 235, 263, 239241, 243, 245, 247, 248, 249, 251, 253, 255, 256, 257, 258, 259, 261, 263, 265, 267, 269, 288, 209, 310

Intervalo, 36, 92, 148, 149, 164, 225, 226

invariante, 127, 131, 132, 145, 149, 150, 151, 158, 159, 170, 280, 316

iterativo e incremental, 5

J

join, 13, 14, 15, 18, 19

junção, 140, 141, 143, 240, 247, 260

L

levantamento de requisitos, 12, 21, 22

Link KO, 262, 265

Link OK, 262, 265

lista, 23, 28, 33, 49, 50, 51, 54, 55, 57, 61, 66, 67, 68, 77, 85, 87, 92, 94, 101, 111, 112, 132,

147, 158, 160, 161, 162, 177, 178, 179, 180, 182, 183, 184, 188, 189, 222, 142, 148, 251, 252, 253, 254, 258, 259, 266, 267, 271, 274, 285, 293, 310

Listagem com Filtro, 266

login, 15, 16, 39

look and feel, 26, 64, 239

M

manter, 23, 39, 100, 132, 141, 142, 144, 194, 225, 178

manutenção, 29, 30, 150, 164, 174, 257, 274

medida, 106, 112, 125, 129, 135, 136, 137, 145, 232, 233, 274, 282

mensagem, 50, 75, 78, 83, 85, 151, 164, 168, 176, 201, 203, 207, 209, 211, 212, 213, 214, 216, 219, 220, 223, 227, 232, 233, 283, 284, 287, 305, 306, 307, 314, 316, 317, 318

merge, 13, 14, 15

Mestre/Detalhe, 243

método, 1, 3, 7, 44, 49, 73, 76, 79, 80, 86, 87, 91, 109, 124, 127, 138, 147, 149, 150, 155, 194, 195, 196, 197, 198, 199, 206, 208, 209, 224, 225, 226, 227, 228, 229, 230, 231, 232, 233, 236, 282, 284, 285, 286, 287, 289, 292, 293, 294, 295, 296, 297, 298, 299, 300, 301, 302, 303, 304, 305, 306, 307, 308, 309, 313, 314, 315, 317, 321, 322

método básico, 306, 309

método delegado, 198, 199, 231, 232, 233, 304, 305, 306, 307, 308, 309

modelo conceitual, 5, 7, 43, 49, 73, 82, 86, 87, 89, 90, 91, 92, 94, 95, 98, 99, 100, 102, 104, 107, 110, 111, 118, 119, 120, 131, 139, 149, 150, 151, 153, 154, 155, 156, 157, 158, 159, 160, 162, 163, 165, 166, 169, 173, 175, 181, 184, 187, 193, 194, 195, 196, 199, 200, 204, 205, 208, 209, 210, 218, 227, 228, 229, 230, 231, 232, 236, 237, 265

modelo dinâmico, 89, 99, 219,

modify unit, 262, 264
 monossessão, 38
 Multi-choice, 242, 243, 266, 267
 multiconjunto, 110, 111, 112, 275, 276
 multidata unit, 237, 240, 241, 242, 253, 255
 multiplicidade, 106, 107, 109, 110, 113, 114, 116, 128, 131, 156, 167, 169, 176, 184, 200, 201, 202, 203, 204, 205, 209, 271, 273, 274, 295, 294, 295, 296, 298

N

navegação, 53, 62, 104, 232, 234, 236, 246, 251, 252, 253, 254, 256, 300, 301

O

Object Constraint Language. Consulte OCL
 objeto essencial, 143
 OCL, 2, 17, 84, 85, 94, 95, 96, 109, 128, 155, 156, 160, 162, 164, 168, 171, 173, 229, 237, 272, 313, 315, 317, 321, 322
 operação de sistema, 2, 62, 68, 78, 80, 83, 84, 85, 153, 154, 155, 156, 162, 163, 164, 165, 166, 167, 168, 169, 170, 172, 181, 199, 209, 210, 211, 214, 216, 217, 218, 220, 229, 286, 294, 304, 305, 306, 307, 308
 operation unit, 261, 262, 263, 264, 265, 268
 ordered set. Consulte conjunto ordenado
 orientação a objetos, 1, 2, 4, 8, 119, 163, 194, 196, 309
 otimização, 95

P

padrão, 24, 26, 39, 46, 55, 64, 65, 80, 86, 87, 98, 99, 125, 130, 131, 135, 136, 137, 138, 139, 140, 144, 147, 148, 151, 163, 166, 167, 1173178, 179, 180, 182, 198, 199, 208, 209, 210, 216, 224, 225, 230, 233, 243, 245, 257, 258, 259, 260, 261, 266, 267, 382, 304, 309
 padrões de análise, 131

página, 10, 77, 113, 114, 115, 236, 237, 242, 248, 249, 250, 251, 252, 253, 256, 257, 258, 262, 270, 286, 292
 pânico organizado, 59
 papel, 37, 45, 91, 103, 104, 106, 107, 109, 110, 111, 113, 114, 116, 118, 128, 142, 155, 156, 160, 167, 159, 176, 198, 199, 200, 204, 205, 209, 213, 215, 217, 244, 245, 272, 273, 274, 293, 295, 296, 313
 partição, 113, 114, 203, 277, 296, 298
 passo, 2, 30, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 65, 66, 67, 68, 71, 74, 75, 76, 77, 78, 83, 84, 101, 171, 265, 266
 passos complementares, 52, 55, 75
 passos impróprios, 53, 55
 passos obrigatórios, 46, 47, 49, 55, 63, 76
 performance, 269, 274, 311
 permanente, 29, 30, 208
 pilha, 5, 111, 112, 193
 pós-condição, 68, 70, 153, 154, 162, 163, 164, 165, 166, 167, 168, 169, 170, 171, 172, 173, 176, 177, 182, 183, 209, 211, 212, 216, 218, 220, 287, 309, 314, 316, 317
 pré-condição, 71, 171, 200
 Princípio Aberto-Fechado, 24
 Processo Unificado. Consulte UP
 projeto lógico, 193, 291, 231, 234, 291
 projeto tecnológico, 193, 291
 Protection Proxy, 87
 proxy virtual, 282, 283
 pseudoatividade, 12

Q

qualificador, 114, 115
 Quantidade, 27, 40, 54, 55, 60, 64, 65, 69, 71, 94, 106, 108, 111, 112, 135, 136, 137, 144, 180, 182, 183, 188, 189, 204, 211, 214, 224, 227, 228, 229, 230, 267, 276, 305, 306
 queue, 111, 112

R

raias, 11, 15
 rastreabilidade, 23, 30, 37, 71
 razões de conversão, 136
 regiões concorrentes, 18
 regras de negócio, 34, 25, 38, 39, 54, 58, 154, 155, 186
 relação, 114, 115, 118, 119, 120, 146, 210
 relacional, 90, 97, 210, 271, 274, 275, 276, 277, 278
 relatório, 9, 11, 22, 25, 27, 28, 30, 39, 40, 41, 64, 65, 178, 179, 257, 265
 relatórios, 9, 11, 22, 39, 41, 65, 178, 257, 265
 Remove, 39, 73, 168, 169, 198, 285, 288, 293
 renderização, 237, 238, 239, 241, 242, 243, 244, 245, 246, 247, 248, 249, 250
 rep, 40, 64, 65, 178, 180
 report, 40
 requisito, 2, 5, 6, 7, 10, 11, 12, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 35, 36, 37, 39, 40, 43, 54, 58, 60, 65, 68, 70, 137, 139, 150, 235, 269, 310, 311
 requisito desejado, 31
 requisito evidente, 30
 requisito funcional, 25, 26
 requisito não-funcional, 31
 requisito obrigatório, 31
 requisito oculto, 30
 requisito permanente, 29, 30
 requisito suplementar, 26, 27, 29, 60
 requisito transitório, 29, 30
 requisitos correlacionados, 68, 70
 requisitos suplementares, 26, 27
 requisitos transitórios, 137
 responsabilidade, 3, 7, 50, 100, 195, 196, 197, 198, 199, 209, 211, 223, 224, 226, 227, 229, 282
 resposta de sistema, 77, 78
 restrição complementar, 156, 157

restrição tecnológica, 26
 restrições, 21, 22, 23, 24, 25, 26, 28, 31, 58, 86, 132, 149, 150, 156, 157, 170, 248, 269
 restrições lógicas, 24, 25, 28, 58
 resultado consistente, 36, 38, 39
 Retrieve, 39, 178
 reusabilidade, 223
 risco, 5, 10, 32, 39, 40, 201
 rollback, 59, 288, 289

S

scroller unit, 237, 246, 247, 255, 260
 segurança, 26, 32, 37, 86, 91, 193, 194, 269, 289, 311
 sequence, 111, 112, 147, 201, 274, 275, 285, 295
 sequência, 2, 4, 7, 12, 16, 26, 29, 30, 44, 46, 49, 53, 55, 56, 58, 59, 60, 62, 63, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 99, 122, 129, 147, 153, 179, 180, 193, 194, 195, 196, 209, 211, 212, 1213, 215, 217, 218, 219, 220, 221, 222, 233, 235, 258, 265, 266, 267, 268, 266, 305, 307
 set, 87, 95, 110, 161, 162, 164, 166, 167, 168, 169, 171, 173, 174, 175, 178, 179, 180, 181, 182, 183, 184, 186, 188, 189, 190, 191, 198, 199, 201, 14, 219, 220, 221, 225, 226, 228, 230, 233, 274, 275, 284, 285, 286, 287, 292, 293, 294, 295, 296, 297, 298, 299, 300, 301, 301, 302, 303, 304, 306, 307, 314, 317
 Singleton, 208, 279
 stack, 111, 112
 statefull, 80, 81, 82, 83, 159, 180, 186, 187, 192
 stateless, 80, 81, 82, 180, 186, 192
 subclasse, 118, 119, 120, 121, 123, 124, 138, 139, 280, 281, 282
 subestados, 18, 19
 subsistemas, 27, 37

sucessor, 140, 141, 142, 143
 sumário executivo. Consulte visão geral do sistema
 superclasse, 63, 119, 120, 122, 280, 281
 superestado, 17, 18
 superseding. Consulte sucessor
 swimlanes. Consulte raias

T

testes, 5, 7, 55, 234, 291, 293, 295, 297, 299, 301, 303, 305, 307, 308, 309, 310, 311
 tipagem, 92, 157, 248,
 tipo abstrato de dados, 110, 111
 tipo concreto, 110, 111
 tipo primitivo, 93, 96, 148
 tipos, 25, 32, 33, 40, 49, 58, 64, 68, 75, 79, 91, 92, 93, 94, 96, 103, 110, 111, 118, 121, 122, 123, 124, 139, 154, 157, 163, 197, 198, 200, 211, 221, 232, 235, 237, 257, 264, 278, 291, 292, 295, 309
 tipos alfanuméricos, 91, 96
 tipos escalares, 92
 tipos primitivos, 96, 292
 top-down, 3
 transação, 59, 60, 81, 104, 116, 124, 144, 145, 146, 180, 288, 289, 316,
 transição estável, 125
 transição monotônica, 125, 126, 127, 129

transição não-monotônica, 129
 transição, fase de, 5
 tupla, 86, 160, 162, 178, 179, 222, 223, 318

U

UML, , 3, 4, 7, 11, 27, 36, 37, 40, 63, 74, 91, 95, 101, 10004, 120, 194, 235, 236, 256, 297, 319, 320, 321, 322
 Unified Modeling Language. Consulte UML
 Unified Process. Consulte UP
 UP, 1, 4, 5, 6, 9, 21, 24, 35, 40, 43, 291, 319,
 Update, 39, 174, 288
 usabilidade, 32

V

valor inicial, 93, 94, 108, 169, 248, 316
 variações tecnológicas, 68, 71
 variante, 60, 61, 65, 66, 67, 85, 101, 258, 309
 visão de site, 256, 257
 visão geral do sistema, 9, 10, 11, 13, 15, 17, 19
 visibilidade global, 208
 visibilidade local, 207, 208, 214
 visibilidade por associação, 200, 205, 208
 visibilidade por parâmetro, 206, 207, 208

W

WebML, 2, 7, 235, 236, 237, 238, 249, 261, 264, 265, 166, 168